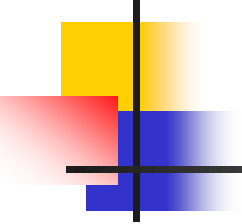
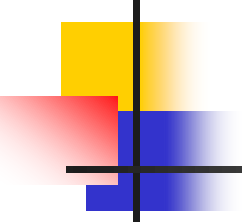
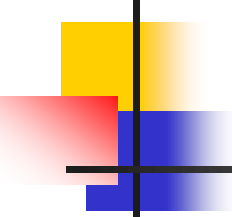


Inginerie Software

- 
-
- Notiunea de **inginerie software** (sau **ingineria programarii**) a fost introdusa in 1968 la o conferinta la care s-a discutat despre '**criza software**', rezultata in urma primelor incercari de dezvoltare a unor sisteme software de dimensiuni netriviale.
 - Dupa peste 50 ani insa, se pare ca aceasta **criza software exista inca...**

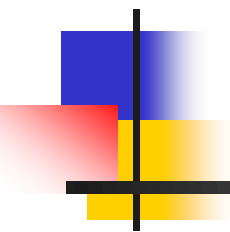
- 
-
- Conform [IEEE90], **ingineria software** este disciplina care se ocupa cu
 1. Aplicarea de abordari sistematice, disciplinate si cuantificabile in dezvoltarea, intretinerea si operarea sistemelor software.
 2. Studiul abordarilor de la punctul 1.
 - Aceasta disciplina se ocupa in special cu dezvoltarea de **sisteme software de dimensiuni mari**, care necesita tehnici, metode si instrumente specifice.

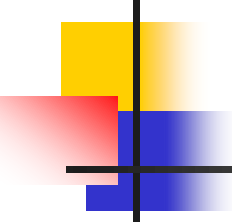


Referinte principale si bibliografie recomandata

- [Som15] I. Sommerville, *Software Engineering*, (10th edition), Addison-Wesley, 2016 (and previous editions).
- [LL05] T. Lethbridge and R. Laganieri, *Object-Oriented Software Engineering: Practical Software Development using UML and Java* (2nd edition), McGraw-Hill, 2005. <http://www.site.uottawa.ca/school/research/lloseng/>
- [FP20] A. Fox and D. Patterson, *Engineering Software as a Service: An Agile Approach using Cloud Computing*, (1st, 2nd editions) Strawberry Canyon, (2016, 2020) <http://www.saasbook.info/>
- [GoF94] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison Wesley, 1994.
- [CHVB18] E.M. Clarke, T.A. Henzinger, H. Veith and R. Bloem, Editors, *Handbook of Model Checking*, Springer, 2018.
- [Pie20] B. Pierce et al, *Software Foundations*, 2020.
<https://softwarefoundations.cis.upenn.edu/>
- [IEEE90] *IEEE Standard Glossary of Software Engineering Terminology*, IEEE Standard 610.12-1990, 1990.

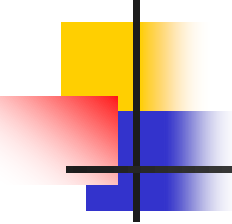
Paradigme de dezvoltare software





Paradigme

- In acest capitol sunt prezentate paradigme software de baza si evolutive.
- Prezentarea este adaptata (in special) dupa [Pre97, Pre10, Pre15, Som89, Som01, Som05, Som07, Som11, Som16].
- O paradigma software furnizeaza o reprezentare abstracta a unui model de proces software.



Paradigme

- Notiunea de **process software** denota activitatile necesare obtinerii unui produs software care satisface cerinte date.
- O **paradigma de dezvoltare software** este un model foarte general de proces software, reprezentat de obicei din perspectiva arhitecturala.



Paradigme

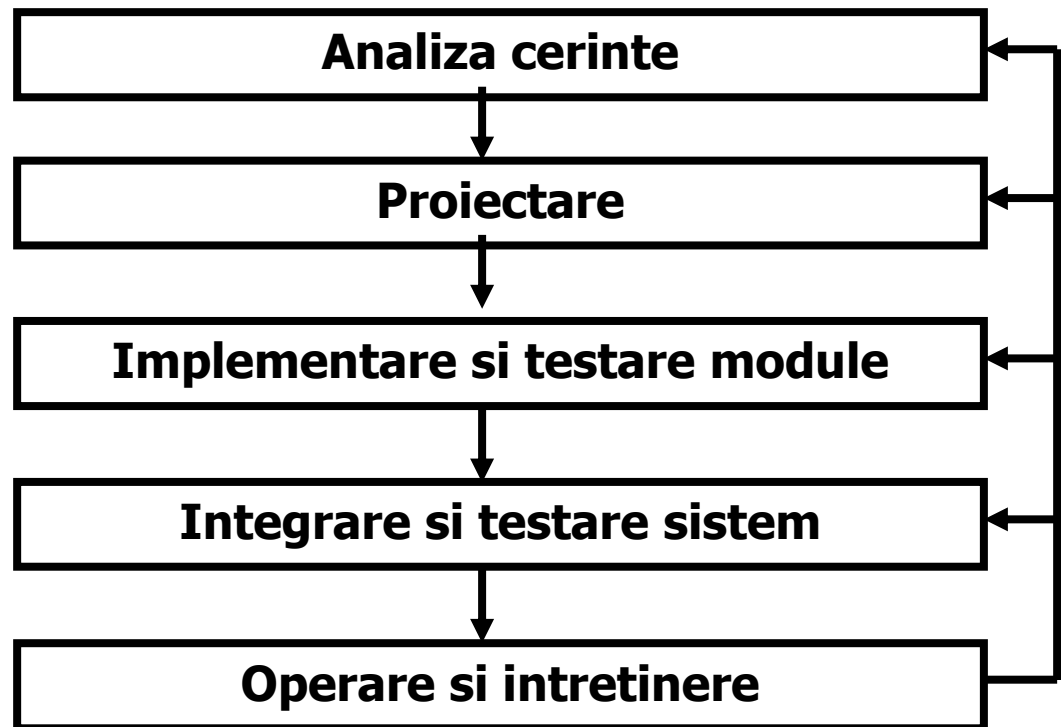
- Sunt prezentate patru **pardigme de baza**:
 - modelul linear ('cascada'),
 - prototipizare,
 - componente reutilizabile si
 - metode formale.
- Sunt prezenate urmatoarele paradigme evolutive care furnizeaza suport pentru **processe iterative**:
 - modelul incremental,
 - modelul spirala si
 - modelul ingineriei concurente.

Paradigme de baza – modelul linear

Modelul linear (numit uneori 'cascada') [Roy70]

■ Dezvoltarea software in acest model cuprinde urmatoarele stadii:

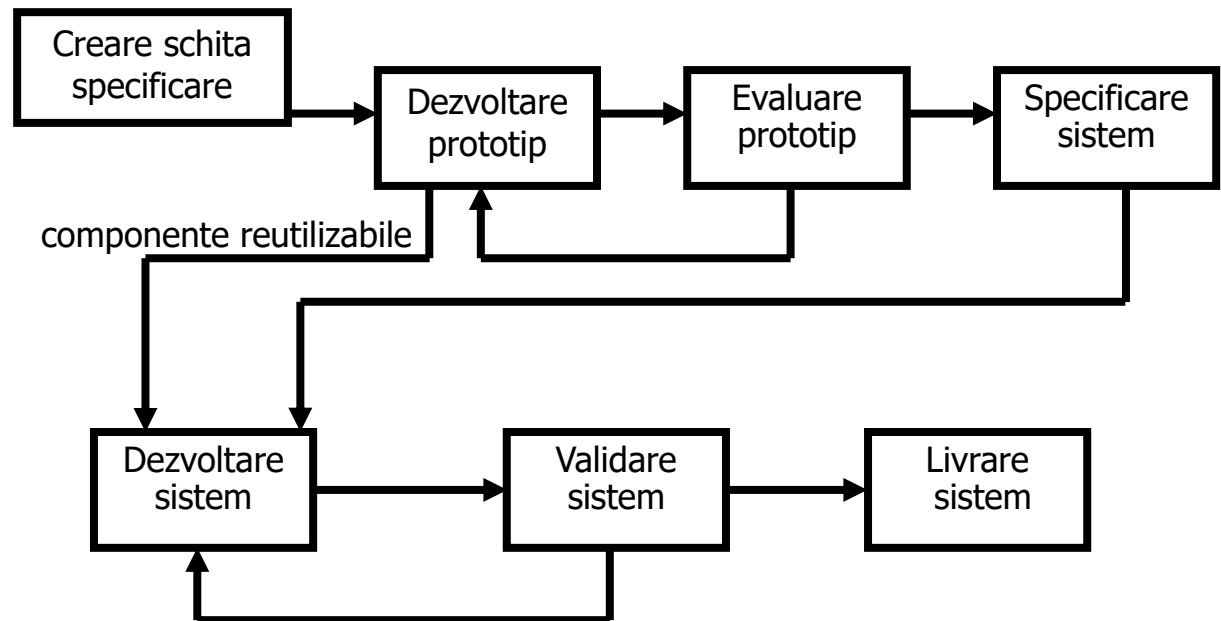
- Acest model este util in proiectele in care cerintele sunt bine intelese de la inceput.
- In acest model insa, este dificil sa se reactioneze la modificarea cerintelor (la solicitarea clientului) deoarece toate deciziile de analiza si design sunt luate la inceputul proiectului.
- Modelul linear este utilizat in multe proiecte mici si medii. In proiecte mari este de obicei utilizat ca parte a unui proces software iterativ.

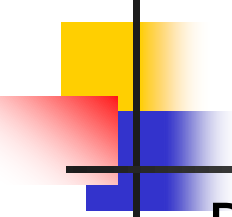


Paradigme de baza – prototipizare

Dezvoltare prin prototipizare

- Un **prototip** este o implementare initiala a unui sistem, care serveste ca mecanism de identificare a cerintelor software.
- Dupa stabilirea cerintelor se poate renunta la prototip (o parte dintre componentele prototipului se pot reutiliza in sistemul final).
- Urmeaza o reimplementare a sistemului software cu scopul de a se realiza un produs ce satisface criteriile de performanta date.





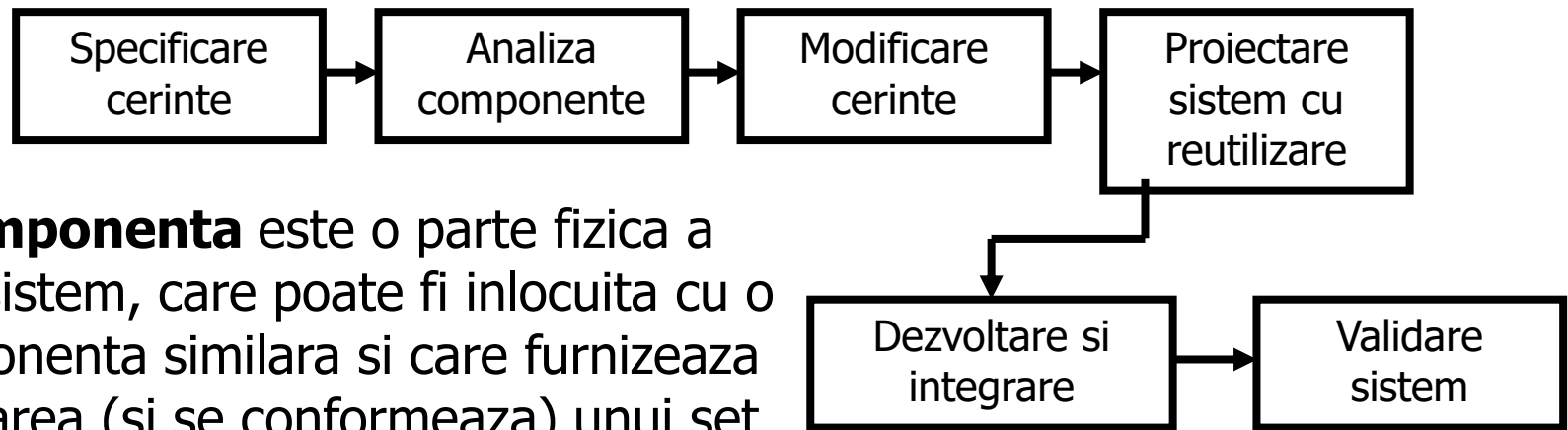
Paradigme de baza – prototipizare

- Prototipul are numai rolul de a demonstra aspectele functionale ale sistemului (nu si pe cele nonfunctionale).
- Este necesar ca prototipul sa poata fi dezvoltat la un cost semnificativ mai mic decat costul sistemului final. Pentru reducerea costurilor aferente construirii prototipului [Som16,Som07,Som89]:
 - se poate renunta la o parte din functionalitate si se pot relaxa cerintele nonfunctionale;
 - in realizarea prototipului se pot utiliza limbaje de programare de nivel foarte inalt (Haskell, Prolog, Scala);
 - se ignora tratarea conditiilor de eroare / exceptie.
 - se pot folosi medii vizuale si generatoare automate de cod;
 - se pot reutiliza componente existente.

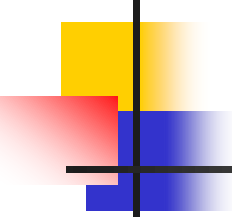
Paradigme de baza – componente reutilizabile

Modelul de **dezvoltare orientata pe componente** are drept caracteristica principala **reutilizabilitatea**.

- Aceasta abordare presupune existenta unor biblioteci de componente si a unui cadru de dezvoltare, reutilizare si integrare.



O **componenta** este o parte fizica a unui sistem, care poate fi inlocuita cu o componenta similara si care furnizeaza realizarea (si se conformeaza) unui set de interfete [JBR99].



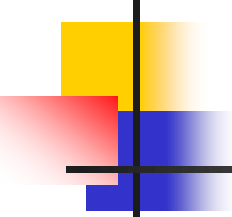
Paradigme de baza – componente reutilizabile

- Stadiile procesului de dezvoltare bazata pe componente reutilizabile sunt urmatoarele [Som07,Som11]:
 - Data fiind o **specificare a cerintelor**, in faza de **analiza componente** se cauta componente care sa implementeze respectiva specificare. Adesea, nu exista o potrivire exacta iar componentele disponibile furnizeaza numai o parte din functionalitatea ceruta.
 - **Modificare cerinte** este o faza in care cerintele sunt analizate si, daca este necesar, sunt modificate pentru a reflecta functionalitatea componentelor disponibile. Acolo unde cerintele nu se pot modifica, se reia faza de analiza componente in scopul de obtinere a unor solutii alternative.
 - In faza de **proiectare sistem prin reutilizare**, se proiecteaza sistemul tinand cont de componentele reutilizabile.
 - In faza de **dezvoltare si integrare**, se dezvolta modulele software care nu pot fi cumparate si componentele sunt integrate pentru a se crea sistemul.

Paradigme de baza – metode formale

- In **modelul de dezvoltare prin metode formale** sistemul software este construit pe baza unei specificari matematice (formale).
- O **specificare formală** furnizează o descriere precisă și independentă de implementare a comportamentului sistemului, putând fi privită drept (un prototip matematic sau) o 'schita matematică' a sistemului.
- Trecerea de la specificarea formală la implementare se poate realiza prin transformări formale (care păstrează semantica matematică, dar un asemenea proces poate fi costisitor) sau se poate realiza în mod informal.
- Pentru sisteme mari, pasul de specificare formală este precedat de o fază de proiectare arhitecturală, iar pasul de implementare este urmat de o fază de testare și integrare la nivel de sistem.





Paradigme de baza – metode formale

- Specificarea matematica face posibila **verificarea formală** a corectitudinii software, ceea ce conduce la o reducere semnificativa a costurilor aferente testarii. In principiu, prin verificare formală se poate elimina testarea la nivel de modul, aspect important deoarece
- Testarea este mare consumatoare de resurse:
 - 30%-40% in proiecte obisnuite;
 - pana la de 5 ori resursele necesare tuturor celorlalte activitati de dezvoltare in proiecte care necesita fiabilitate foarte ridicata (cf. [Pre97]).
- Metodele formale sunt potrivite pentru proiecte cu cerinte stringente in ceea ce priveste **siguranta**, **securitatea** sau **fiabilitatea**, cum sunt sistemele de control trafic aerian, sau sistemele medicale.
- Exemple de metode si abordari formale:
 - Specificare bazata pe model in notatia Z [Crr99,Jac97,Spi92]
 - Model-checking [BK08, CHVB18]
 - Specificare algebrica [EM85, EM90]



Paradigme evolutive

- Sistemele software de dimensiuni mari evolueaza in timp si cerintele se modifica adesea pe parcursul procesului de dezvoltare.
- Inginerii software au nevoie de modele de procese care sa ofere un cadru de dezvoltare pentru sistemele care evolueaza in timp.
- Dar paradigmele de baza nu iau in considerare natura evolutiva a sistemelor software.



Paradigme evolutive

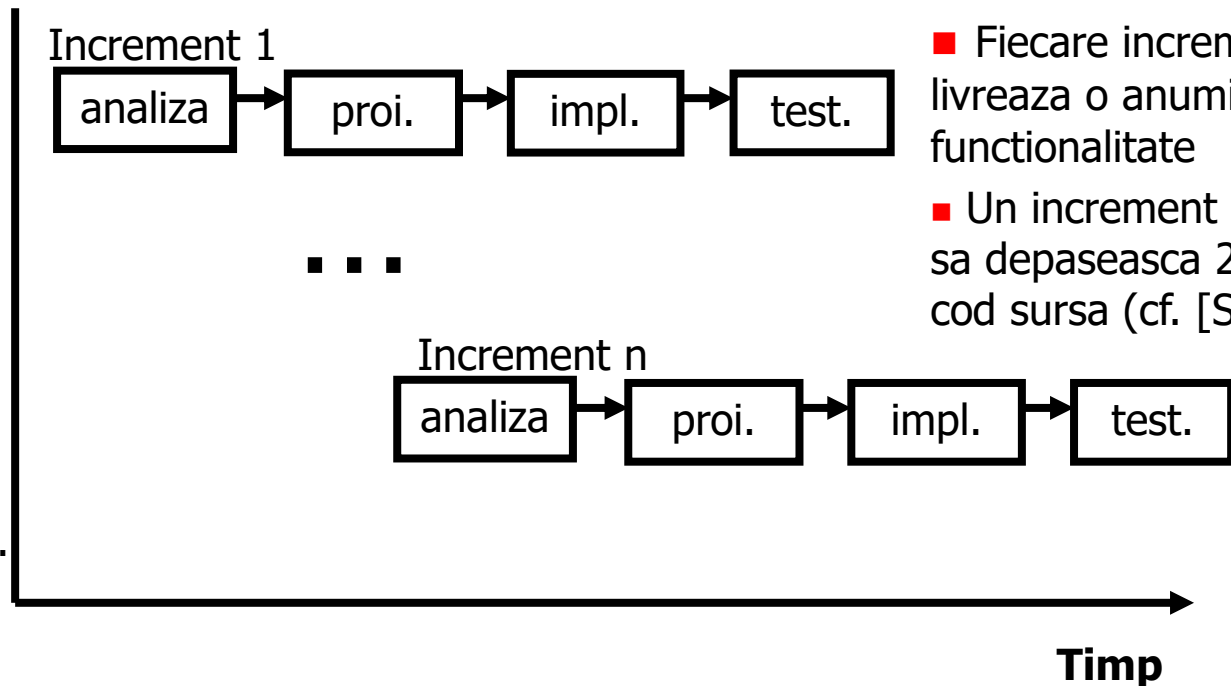
- In cele ce urmeaza prezentam trei **modele evolutive**:
 - modelul incremental,
 - modelul spirala si
 - modelul ingineriei concurente.
- Esenta modelelor evolutive:
 - specificarea se dezvolta in conjunctie cu sistemul software [Som01,Som07,Som11]
 - paradigmele evolutive sunt iterative.

Paradigme evolutive – modelul incremental

Modelul incremental

- Utilizat in RUP [JBR99,Kru03] si XP [Bec99]
- Imparte proiectul in parti de dimensiuni controlabile, numite *incremente* (figura este din [Pre97,Pre15]):

- Conceptual, fiecare iteratie este baza pentru iteratia urmatoare.
- Totusi, iteratiile se pot suprapune in timp (o iteratie poate incepe inainte de finalizarea iteratiei precedente).



■ Fiecare increment livreaza o anumita functionalitate

■ Un increment nu ar trebui sa depaseasca 20.000 linii cod sursa (cf. [Som01]).



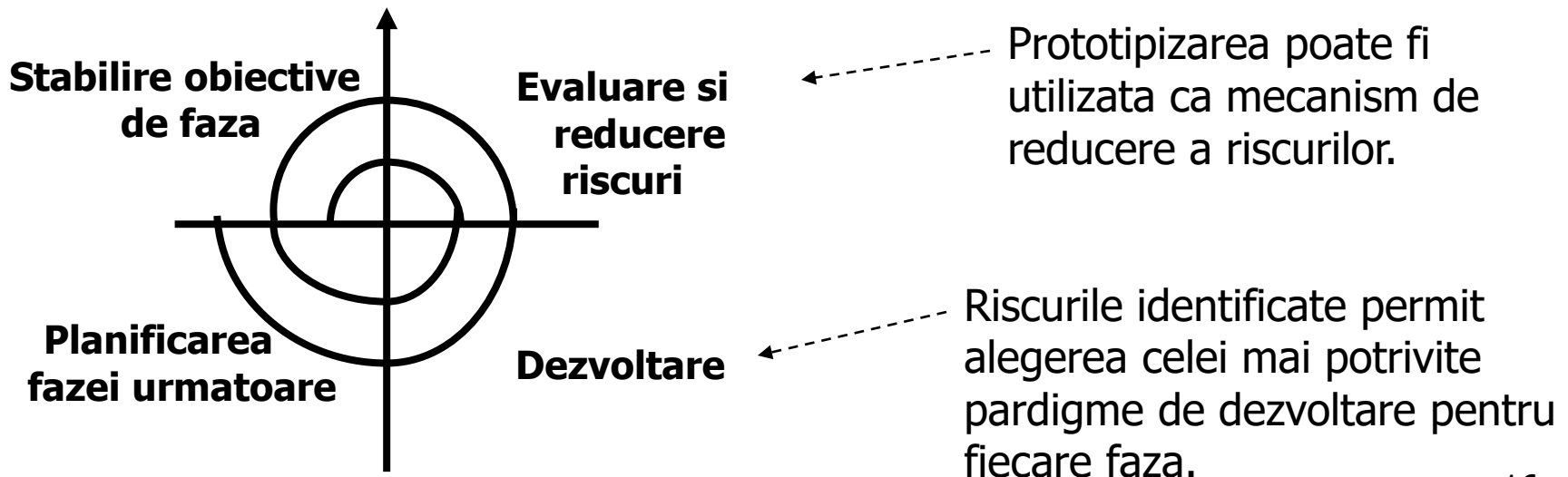
Paradigme evolutive – modelul incremental

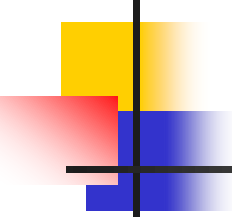
- In figura s-a presupus ca in fiecare increment se utilizeaza modelul linear (cascada). In general insa, fiecare increment poate incorpora orice paradigma potrivita [Som01].
- Avantaje ale dezvoltarii incrementale [Som07,Som16,JBR99]:
 - clientii nu trebuie sa astepte pana cand intregul sistem este livrat; primele incremente satisfac cele mai critice cerinte, deci produsul software poate fi deja utilizat;
 - riscurile sunt reduse la costul unui singur increment.
- Principala dificultate in dezvoltarea incrementală [Som01,Som07]:
 - poate fi dificil sa se mapeze cerintele clientului la incremente de dimensiuni controlabile.

Paradigme evolutive – modelul spirala

In **modelul spirala** [Boe88] (vezi si [Pre15,Som07]) produsul software este dezvoltat intr-o serie de versiuni incrementale, iar procesul este reprezentat ca o spirala evolutiva. **Riscurile** sunt considerate si tratate in fiecare faza (bucla) a spiralei.

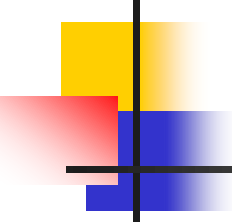
- Fiecare faza (bucla) a spiralei este impartita in zone:





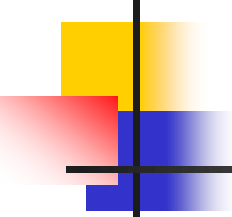
Paradigme evolutive – inginerie concurenta

- Modelul de **dezvoltare concurenta**, numit uneori **inginerie concurenta** [DP94,She94,Pre15], furnizeaza o descriere precisa a starii curente a unui proiect; proiectul este reprezentat ca o retea de activitati concurente si comunicante.
 - Un eveniment (de exemplu o modificare tarzie a cerintelor) din cadrul unei activitati (de exemplu **analiza**) poate cauza o tranzitie de la o stare la alta a unei alte activitati (de exemplu activitate de **design** trece din starea de dezvoltare intr-o starea de asteptare schimbari).
- De obicei, pentru reprezentarea activitatilor concurente in acest model se utilizeaza **diagrame de tranzitie de stare**. Acest model descrie procesul software ca o colectie de automate concurente si comunicante.



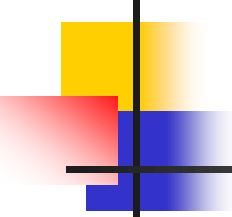
Bibliografie

- [BK08] C. Baier, J.-K. Katoen. Principles of Model Checking. The MIT Press Cambridge, 2008.
- [Bec99] K. Beck. Embracing Change with Extreme Programming. *IEEE Computer* 32(10):70-78, 1999.
- [Boe88] B. Boehm. A spiral model of software development and enhancement. *IEEE Computer*, 21(5):61-72, 1988.
- [CHVB18] E.M. Clarke, T.A. Henzinger, H. Veith and R. Bloem, Editors, *Handbook of Model Checking*, Springer, 2018.
- [Crr99] E. Currie. *The Essence of Z*. Prentice Hall, 1999.



Bibliografie

- [DP94] A. Davis, P. Pitaram. A concurrent process model for software development. *Software Engineering Notes*, ACM Press, 19(2):38-51, 1994.
- [EM85] H. Ehrig and B. Mahr. *Fundamentals of Algebraic Specification, part 1*. Springer, 1985.
- [EM90] H. Ehrig and B. Mahr. *Fundamentals of Algebraic Specification, part 2*. Springer, 1990.
- [Jac97] J. Jacky. *The Way of Z: Practical Programming with Formal Methods*. Cambridge University Press, 1997.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.



Bibliografie

- [Kru03] P. Kruchten. *The Rational Unified Process: An Introduction* (3rd edition). Addison-Wesley, 2003.
- [Pre97] R. Pressman. *Software Engineering: A Practitioner's Approach*, (4th edition). McGraw-Hill, 1997.
- [Pre10] R. Pressman. *Software Engineering: A Practitioner's Approach*, (7th edition). McGraw-Hill, 2010.
- [Pre15] R. Pressman. *Software Engineering: A Practitioner's Approach*, (8th edition). McGraw-Hill, 2015.
- [Roy70] W. Royce. Managing the development of large software systems: concepts and techniques. In *Proc. IEEE WESTCON*, 1970.



Bibliografie

- [She94] W. Sheleg. Concurrent Engineering: A New Paradigm for C/S Development. *Application Development Trends*, 1(6):28-33, 1994.
- [Spi92] J.M. Spivey. *The Z Notation: A Reference Manual*. Prentice-Hall, 1992.
- [Som89] I. Sommerville. *Software Engineering*, (3rd edition). Addison-Wesley, 1989.
- [Som01] I. Sommerville. *Software Engineering*, (6th edition). Addison-Wesley, 2001.
- [Som05] I. Sommerville. *Software Engineering*, (7th edition). Addison-Wesley, 2005.



Bibliografie

- [Som07] I. Sommerville. *Software Engineering*, (8th edition). Addison-Wesley, 2007.
- [Som11] I. Sommerville. *Software Engineering*, (9th edition). Addison-Wesley, 2011.
- [Som16] I. Sommerville. *Software Engineering*, (10th edition). Addison-Wesley, 2016.



Procesul Unificat

- **Procesul Unificat** (engl. *Unified Proces* (UP)) este procesul software atasat UML [BRJ99,JBR99,Kru03].
- UP angajeaza o **abordare incrementală** in dezvoltarea software.
- UP incorporeaza multe dintre elementele caracteristice ale *Rational Unified Process* (RUP) [Kru03].

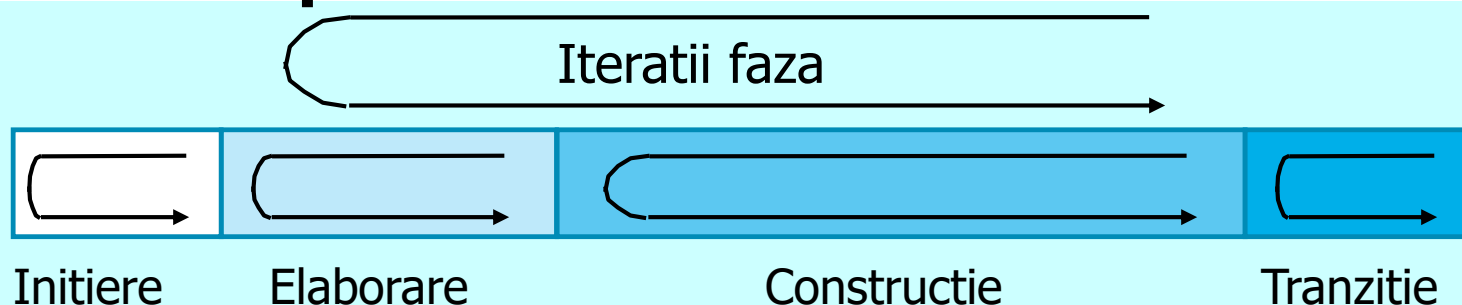


Procesul Unificat

- UP [JBR99,Kru03]
 - este orientat pe componente,
 - utilizeaza UML pentru crearea modelelor OO si
 - are trei caracteristici distinctive:
 - este ghidat de cazurile de utilizare,
 - este centrat pe arhitectura,
 - este iterativ si incremental.

Procesul Unificat

Exista patru faze in UP



- Fiecare faza consta dintr-un numar de **iteratii**. Fiecare iteratie produce un nou **increment**.

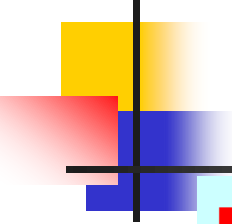
- Dezvoltarea este iterativa si incrementală.

- Cele mai multe riscuri sunt tratate in fazele de initiere si elaborare.

- Majoritatea resurselor sunt consumate in faza de constructie.

In UP o iteratie consta din urmatoarele **fluxuri de baza**:

- Captare cerinte
- Analiza
- Proiectare
- Implementare
- Testare



Procesul Unificat

■ **Initiere**

- Se realizeaza o estimare initiala a resurselor necesare.
- Resursele utilizate in iteratiile din faza de initiere se consuma in special in fluxul de captare a cerintelor.

■ **Elaborare**

- Se dezvoltă arhitectura sistem. Prin definiție, faza de elaborare se încheie atunci când arhitectura se stabilizează.
- Iteratiile din această fază implică toate fluxurile de lucru.

■ **Constructie**

- Proiectare, implementare și testare sistem.

■ **Tranzitie**

- Se realizeaza tranzitia spre mediul utilizator.
- Rolul fluxurilor de baza este mai mic in aceasta faza.



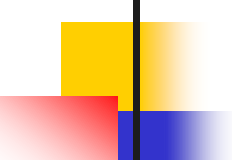
Procesul Unificat

- UP este ghidat de cazurile de utilizare.
 - Cazurile de utilizare ghideaza intregul proces de dezvoltare, de la captarea cerintelor pana la testarea sistemului.
 - Un caz de utilizare este o descriere a unui set de secvente de actiuni, incluzand variante, ce pot fi efectuate de un sistem si care produc un efect observabil si semnificativ pentru un actor particular [BRJ99,JBR99].
- UP este centrat pe arhitectura.
 - In UP, arhitectura software este o realizare completa (de la specificare pana la testare) a cazurilor de utilizare relevante din punct de vedere arhitectural (arhitectul decide ce este *relevant*).
- UP este iterativ si incremental.
 - Un increment este o realizare a unui set de cazuri de utilizare.
 - Abordarea este **ghidata de riscuri**.



Metode flexibile

- **Metodele flexibile** (engl. *Agile methods*) reprezinta o evolutie recenta a **modelului incremental** [Som01,Som07,Som11,Som16].
- Exemple de abordari flexibile includ: XP [Bec99], DSDM [Sta97], SCRUM [SB01], Crystal [Coc01] and FDD [PF02].
- Metodele flexibile se potrivesc cel mai bine pentru aplicatii de dimensiuni mici / medii destinate mediului de afaceri [Som07,Som11,Som16].
 - Daca arhitectura software furnizeaza mecanisme de dezvoltare compositionala, abordarea se poate repeta pentru proiecte de dimensiuni mai mari [FP16]
 - De exemplu utilizand compunerea serviciilor in SOA (Service Oriented Architectures)



Metode flexibile

Principiile metodelor flexibile

Principiu	Descriere
Implicare client	Clientii ar trebui sa fie implicati direct in intregul proces de dezvoltare. Rolul lor este de a furniza si ordona cerintele dupa prioritati si de a evalua iteratiile sistemului.
Livrare incrementală	Sistemul software este dezvoltat intr-o maniera incrementală. Clientul specifica cerintele pentru fiecare increment.
Oameni nu procese	Calitatile profesionale si de comunicare (intre persoane) ale membrilor echipei de dezvoltatori ar trebui sa fie recunoscute si exploatate. Membrii echipei pot sa dezvolte propriile metode de lucru, fara procese prestabilite. Abordarile flexibile incurajeaza comunicarea (preferabil fata catre fata, nu prin documente) intre toti participantii la proiect (clienti si dezvoltatori).
Cerinte dinamice	Procesul de dezvoltare ar trebui sa fie ghidat de modificarea cerintelor, iar nu de planuri inflexibile prestabilite.
Simplitate	Complexitatea inutila ar trebui sa fie eliminata din sistemul software si din procesul de dezvoltare. In abordarile flexibile accentul este pe cod, nu pe documentatie.



Metode flexibile

- **Dificultati in abordarile flexibile:**
 - Poate fi dificil sa se mentina interesul si implicarea clientului in intregul proces de dezvoltare.
 - Membrii echipei de dezvoltatori pot sa nu fie potriviti pentru implicarea si comunicarea intensa ce caracterizeaza metodele flexibile.
 - Ordonarea cerintelor (si a modificarilor) dupa prioritati poate fi dificila, in conditiile implicarii mai multor participanti din partea dezvoltatorului si a clientului.
 - Mentinerea simplitatii necesita munca suplimentara.
 - Ca si in alte abordari iterative contractarea este dificila.



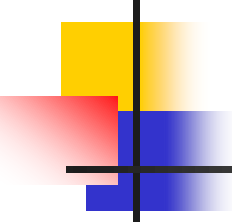
Programare extrema

- **Programarea extrema** (engl. *Extreme Programming* (XP)) este cea mai cunoscuta dintre metodele flexibile.
- XP angajeaza o abordare 'extrema' in dezvoltarea iterativa.
 - Incrementele sunt livrate spre client la fiecare cateva saptamani (in abordarile incrementale conventionale timpul alocat unei iteratii se masoara in luni).
 - Fiecare nou increment este acceptat numai daca toate testele sunt trecute cu succes. Testele (automate) sunt construite inainte de scrierea codului program.



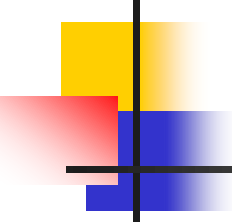
Programare extrema

- Valori in abordarea XP [Bec99]:
 - **Comunicare**
 - XP incurajeaza colaborarea directa intre client si programatori. Un reprezentant al clientului ar trebui sa fie angajat permanent in echipa XP. In XP comunicarea directa frecventa intre participantii la proiect este preferata documentelor scrise.
 - **Simplitate**
 - XP recomanda alegerea celei mai simple solutii la inceput si imbunatatirea acesteia prin refactorizare. Se recunoaste insa ca simplitatea nu se obtine cu usurinta.
 - **Feedback**
 - Aceasta valoare este legata direct de comunicare si simplitate. Programatorii si clientul obtin feedback din partea sistemului prin teste de acceptanta si teste la nivel de modul. Clientul obtine feedback din partea echipei de dezvoltatori atunci cand aceasta (echipa de dezvoltatori) estimeaza timpul necesar pentru implementarea noilor cerinte.
 - **Curaj**
 - Programatorii trebuie sa aiba curajul sa refactorizeze codul, sau chiar sa renunte la parti din cod si sa reprojeteze arhitectura software daca acest lucru devine necesar.



Programare extrema

- In XP toate cerintele sunt exprimate sub forma de **scenarii XP** (engl. *user stories*).
 - **Scenariile** traditionale (**cazuri de utilizare** in UP) sunt descrieri de exemple de sesiuni interactive; fiecare scenariu cuprinde una sau mai multe interactiuni posibile [Som05]. In practica, scenariile XP pot fi mai fragmentare decat scenariile traditionale.
- Scenariile XP sunt scrise de client in limbaj natural, utilizand concepte din domeniul de aplicatie (nu concepte tehnice).
 - In mod tipic, un scenariu XP nu depaseste 3-4 propozitii.



Programare extrema

- Scenariile XP sunt elementele de baza in planificarea proiectelor XP.
 - Daca dezvoltatorul estimeaza ca implementarea unui scenariu necesita mai mult de 2-3 saptamani atunci acesta trebuie descompus in scenarii XP mai mici.
 - Pe baza prioritatilor de afaceri si a estimarilor realizate de dezvoltator, clientul decide care scenarii XP urmeaza sa fie implementate in urmatorul increment al sistemului.
- In abordarea XP dezvoltarea este ghidata de testare. Testele sunt dezvoltate din scenarii XP inainte de design sau implementare.
 - Testele (black box) de acceptanta sunt specificate de client. Programatorii scriu in mod continuu teste la nivel de modul.
 - Se utilizeaza componente de testare automata pentru fiecare nou increment al sistemului.



Programare extrema

Practici XP

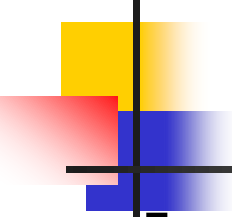
Principiu sau practica	Descriere
Planificare incrementală	Cerintele sunt înregistrate pe carduri de scenarii XP. Scenariile XP care urmează a fi incluse în următorul increment sunt determinate pe baza priorităților relative și a timpului disponibil.
Incremente de dimensiuni mici	Se dezvoltă mai întâi o funcționalitate minimală dar relevantă pentru client. Iterațiile care urmează sunt frecvente și adaugă în mod incremental funcționalitate la sistem.
Design simplu	Se proiectează numai cerintele planificate pentru iteratia curentă.
Dezvoltare ghidată de testare	Se utilizează componente de testare automată. Testele sunt dezvoltate din scenarii XP înainte de design sau implementare.
Perechi de programatori	Programatorii lucrează în perechi, verificându-se reciproc.
Proprietate comună asupra codului	Perechile de programatori acționează în toate aspectele proiectului, fiecare putând modifica orice. Se poate vorbi despre o formă de proprietate comună asupra codului.



Programare extrema

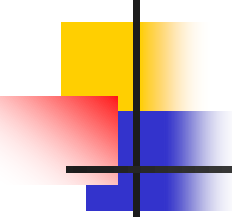
Practici XP

Principiu sau practica	Description
Refactorizare	Este important ca toti dezvoltatorii sa refactorizeze codul in mod continuu, de indata ce acest lucru este posibil. Exemple de refactorizare includ: adaugarea unui sablon Façade la un pachet pentru reducerea gradului de cuplare, adaugarea unei superclase care reprezinta comportamentul comun al unei set de clase existente, inlocuirea unei instructiuni conditionale cu polimorfism, sau chiar modificarea numelui unei variabile (sau a unei functii) cu un nume mai sugestiv. Refactorizarea ar trebui sa pastreze codul simplu si mentenabil.
Integrare continua	De indata ce un increment este finalizat, este integrat in sistem. Dupa fiecare asemenea integrare, toate testele la nivel de modul ar trebui sa fie trecute cu succes.
Participare client	Un reprezentant al clientului ar trebui sa fie in mod continuu la dispozitia echipei XP. Intr-un proces XP clientul este membru al echipei de dezvoltatori, fiind direct responsabil cu stabilirea si ordonarea cerintelor care urmeaza sa fie implementate.
Ritm sustinut, dar fara excese	In abordarea XP un volum mare de ore suplimentare este considerat inacceptabil, deoarece efectul net consta in reducerea calitatii si a productivitatii pe termen mediu.



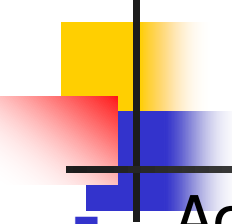
Programare extrema

- In XP, refactorizarea constanta este preferata abordarii traditionale in care se proiecteaza anticipandu-se posibile modificari ale cerintelor.
 - In ingineria software traditionala se considera ca merita sa se investeasca timp si efort de proiectare a unor solutii care anticipeaza posibile modificari ale cerintelor, deoarece aceasta abordare poate ulterior reduce costurile de dezvoltare.
 - In XP accentul este pe proiectarea cerintelor imediate, considerandu-se ca, de fapt, modificarile nu pot fi anticipate.
 - Pentru a facilita implementarea schimbarilor de cerinte la momentul la care acestea apar in mod efectiv, XP incurajeaza imbunatatirea constanta a codului prin refactorizare.



Programare extrema

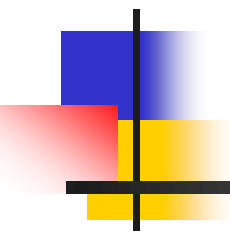
- **Dezvoltarea ghidata de testare** este una dintre cele mai importante inovatii ale XP [Som07,Som11,Som16].
- In [LL05] aceasta abordare este descrisa astfel:
 - In dezvoltarea ghidata de testare programatorul scrie mai intai un set de teste automate. Apoi, proiecteaza si se scrie codul astfel incat testele sa fie trecute. In esenta, testele reprezinta o reformulare a cerintelor intr-o forma automata. In abordarile flexibile bazate pe dezvoltare ghidata de testare, testele pot sa fie unica forma de reprezentare detaliata a cerintelor. ... Prin proiectarea testelor automate se garanteaza faptul ca cerintele sunt bine intelese.

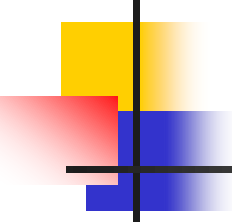


Programare extrema

- Activitatea in **perechi de programatori** reprezinta o alta practica inovativa a XP.
 - In XP, se lucreaza in perechi de (cate 2) programatori asezati in fata aceluiasi terminal. Perechile sunt formate in mod dinamic.
 - Aceasta practica
 - contribuie la o forma de proprietate comuna asupra codului si faciliteaza raspandirea cunostintelor in cadrul echipei XP,
 - serveste ca proces informal de recenzie a codului, fiecare linie de cod fiind vazuta de cel putin doua persoane si
 - incurajeaza refactorizarea in beneficiul intregii echipe XP.
 - Masuratorile efectuate sugereaza ca productivitatea unei perechi de programatori este similara cu cea a doi programatori care lucreaza independent.

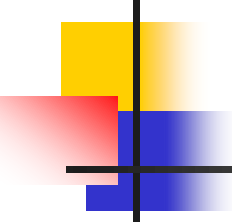
Activitati de baza in dezvoltarea software





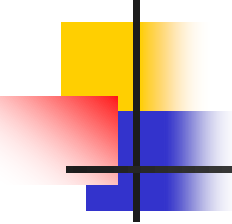
Activitati de baza

- In acest capitol sunt discutate activitatile de baza in ingineria software:
 - specificare,
 - proiectare,
 - validare, si
 - evolutie.
- Prezentarea este adaptata in special dupa [JBR99,Pre97,Pre00,Pre14,Som01,Som05,Som07, Som11,Som16].



Activitati de baza – analiza si specificare

- Activitatea de **analiza** are drept scop stabilirea cerintelor software (functionale si nonfunctionale).
- Cerintele rezultate din analiza trebuie sa fie documentate, aceasta activitate se numeste **specificare** [Som89,Som11]. Specificarea este un proces de reprezentare [Pre97].



Activitati de baza – analiza si specificare

- **Principiile activitatilor de analiza si specificare**
[Dav95] (cf. [Pre97]):
 1. Domeniul informational al problemei trebuie sa fie reprezentat si inteles.
 2. Functiile care vor fi implementate in sistemul software trebuie sa fie definite. Comportamentul sistemului software (descrie ca o consecinta a evenimentelor externe) trebuie sa fie reprezentat.
 3. In procesul de analiza se creeaza un **model cognitiv** (iar nu un model de proiectare sau de implementare) care descrie sistemul asa cum este perceput de utilizatori.
 4. Specificarea trebuie sa tolereze incompletitudini putand fi completata atunci cand exista informatia necesara. Continutul si structura specificarii trebuie sa permita **schimbări**.



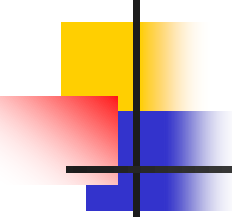
Activitati de baza – analiza si specificare

- Cerintele software pot fi impartite in doua mari categorii [Som89,Som11,Som16]:
 - **Functionale** Acestea sunt serviciile pe care utilizatorii le asteapta de la sistem. Cerintele functionale pot fi validate prin prototipizare.
 - **Nonfunctionale** Acestea stabilesc constrangerile sub care trebuie sa opereze sistemul, standardele pe care trebuie sa le satisfaca, etc.



Activitati de baza – analiza si specificare

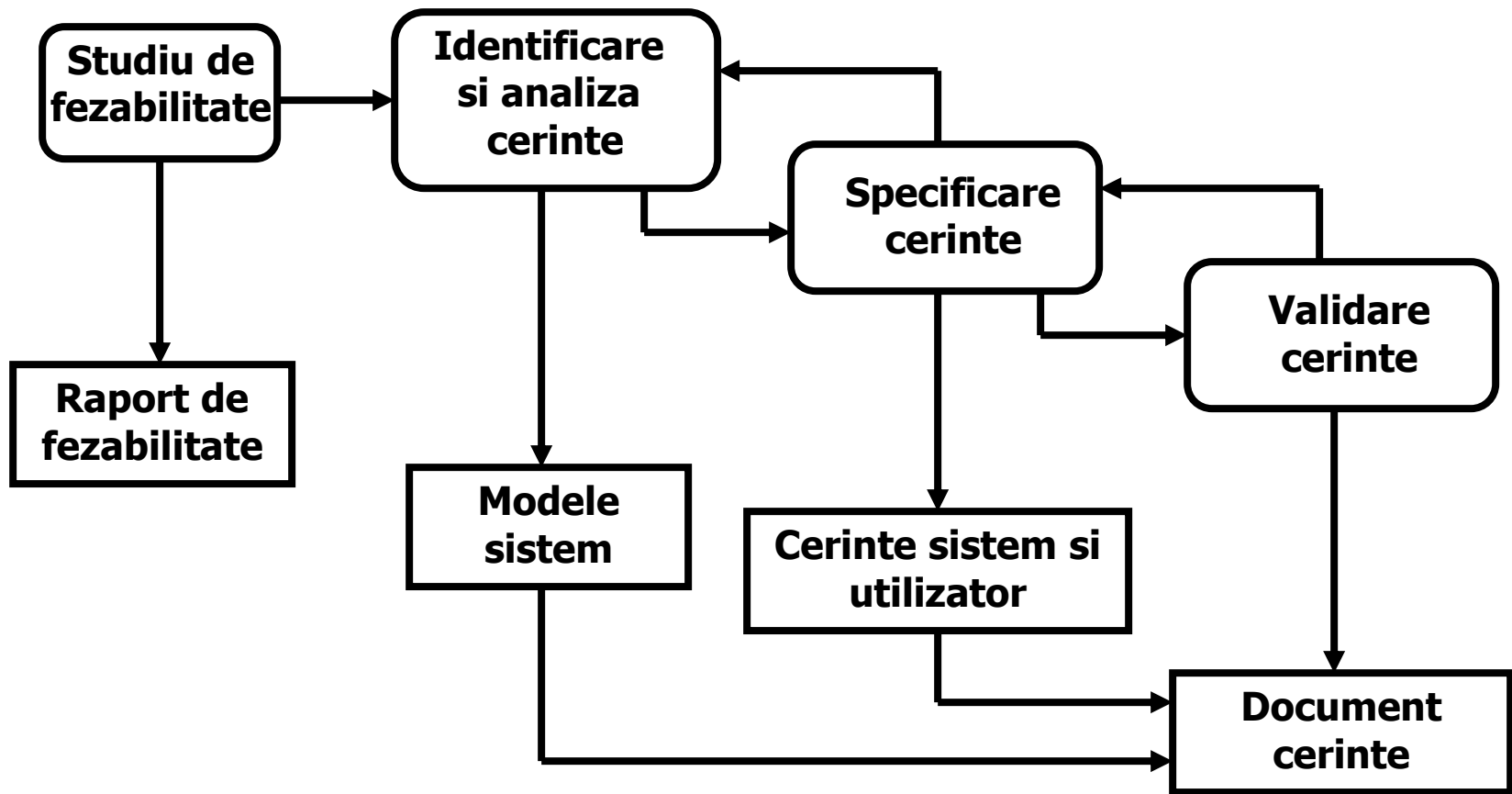
- Produsele software furnizeaza
 - **functionalitate** (servicii), si
 - **attribute nonfunctionale** care reflecta **calitatea**.
- **Atributele de calitate** (atributele nonfunctionale) reflecta [Som01,Som11,Som16]
 - Comportamentul produsului la executie
 - Structura si organizarea software
- Exemple de **attribute de calitate**: fiabilitate, eficienta, mentenabilitate (usurinta in intretinere), viteza (timpul de raspuns), siguranta, securitate, etc.



Activitati de baza – analiza si specificare

- Ingineria cerintelor se refera la toate activitatile legate de crearea si intretinerea unui **document de specificare a cerintelor**.
- In organizatii mari, structura documentului de specificare a cerintelor poate fi standardizata. IEEE propune propriul standard pentru asemenea documente [IEEE93b].
- Figura de mai jos [Som01,Som07,Som11] prezinta un **proces de ingineria cerintelor** (avand drept rezultat principal un document de specificare a cerintelor).

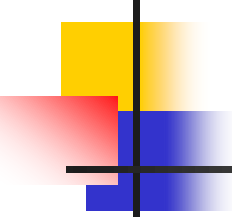
Activitati de baza – analiza si specificare





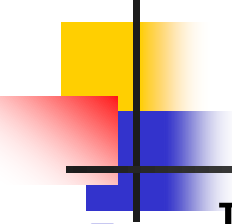
Activitati de baza – analiza si specificare

- **Faze ale procesului de inginerie a cerintelor [Som07,Som11]:**
 - **Studiu fezabilitate** Studiul decide daca sistemul propus poate fi economic in exploatare si daca poate fi dezvoltat utilizand tehnologia disponibila in limitele bugetului alocat proiectului. Studiul de fezabilitate ar trebui sa consume relativ putine resurse (inclusiv timp).
 - **Identificare si analiza cerinte** Acesta este procesul de captare a cerintelor sistem prin observarea comportamentului sistemelor existente, discutii cu clientul, analiza sarcini, etc. Poate implica dezvoltarea de modele sistem si prototipuri.
 - **Specificare cerinte** Cerintele colectate in faza de analiza sunt organizate in vederea alcatuirii documentului de specificare a cerintelor. Utilizatorii si clientii au nevoie de o descriere de nivel inalt a cerintelor; dezvoltatorii sistem opereaza cu o specificare sistem mai detaliata.
 - **Validare cerinte** Sunt studiate realismul, consistenta si completitudinea cerintelor obtinute. Se corecteaza defectele descoperite.



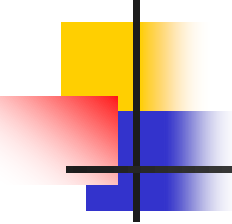
Activitati de baza – analiza si specificare

- Metode de identificare si analiza a cerintelor:
 - *Analiza structurata* [Mar79]
 - *Prototipizare*
 - *Scenarii* (**cazuri de utilizare** in OOSE [Jac92,JBR99])
 - *Analiza vederi* Aceasta abordare are la baza ideea ca un sistem poate fi descris prin vederi multiple, care permit structurarea cerintelor si organizarea procesului de analiza. CORE [Mul79] si VORD [KS96] sunt metode care utilizeaza analiza vederi.
 - Ex. – vederi mai importante pentru un sistem ATM: client, automat bancar, baza de date.
 - *Analiza si specificare formala (matematica)* (utilizate in subdomenii specializate).



Activitati de baza – proiectare

- Intrarea principala pentru activitatea de proiectare este documentul de specificare a cerintelor software.
- **O specificare este un model cognitiv**, bazat pe concepte din domeniul problemei (asa cum sunt percepute de comunitatea de utilizatori).
- **Un design este o schita de implementare** [JBR99], care utilizeaza concepte din domeniul solutiei tehnice.
- Deosebirea intre specificare si proiect (design) este uneori exprimata astfel:
 - **specificarea** spune **CE** anume trebuie implementat;
 - **designul** este o reprezentare a modului **CUM** se poate construi implementarea.



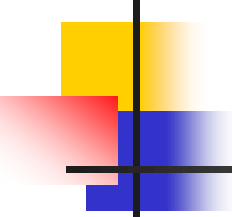
Activitati de baza – proiectare

- **Principii si ghid de proiectare [Dav95] (cf. [Pre97]):**
 1. Un bun proiectant ar trebui sa ia in considerare abordari (solutii tehnice) alternative, comparate pe baza cerintelor, a resurselor disponibile si a conceptelor de proiectare.
 2. Designul nu ar trebui sa reinventeze roata. Resursele sunt limitate. Timpul alocat proiectarii ar trebui sa fie investit in reprezentarea ideilor cu adevarat noi precum si in integrarea sabloanelor de design existente.
 3. Designul ar trebui sa fie uniform (stil unitar) si integrat (interfete de calitate).
 4. Designul ar trebui sa fie flexibil la schimbari.
 5. Pe baza designului ar trebui sa se poata obtine inapoi cerintele.
 6. Designul ar trebui sa trateze aspectele functionale, comportamentale si structurale din perspectiva implementarii.
 7. Designul nu este identic cu implementarea. Nivelul de abstractizare a designului este mai inalt decat cel din codul sursa.



Activitati de baza – proiectare

8. Designul ar trebui sa minimizeze distanta intelectuala intre produsul software si problema, asa cum exista ea in lumea reala (structura designului ar trebui sa mimeze structura problemei).
9. Designul ar trebui sa fie alcatuit din module conectate (intre ele si cu mediul extern si utilizatorii) prin interfete simple.
10. Designul ar trebui sa cuprinda module software cu inalt grad de coeziune si grad redus de cuplare (intre module).



Activitati de baza – proiectare

■ **Concepte:**

- *Architectura* se refera la structura de ansamblu a sistemului software si la modul in care aceasta structura furnizeaza sistemului integritate semantica [SG96].
 - In unele abordari numai o parte dintre module sunt considerate a fi relevante pentru vederea arhitecturala a sistemului [JBR99].
 - Notiunea de *arhitectura* are la baza conceptul de *modularitate*.
- *Modulul* reprezinta 'caramida' de baza in modularizarea sistemelor software. In general, un modul nu este considerat a fi un sistem independent.
- Un *subsistem* este o parte a unui sistem care apare de sine statator intr-un proces.
 - Un (sub) sistem este alcatuit din module.



Activitati de baza – proiectare

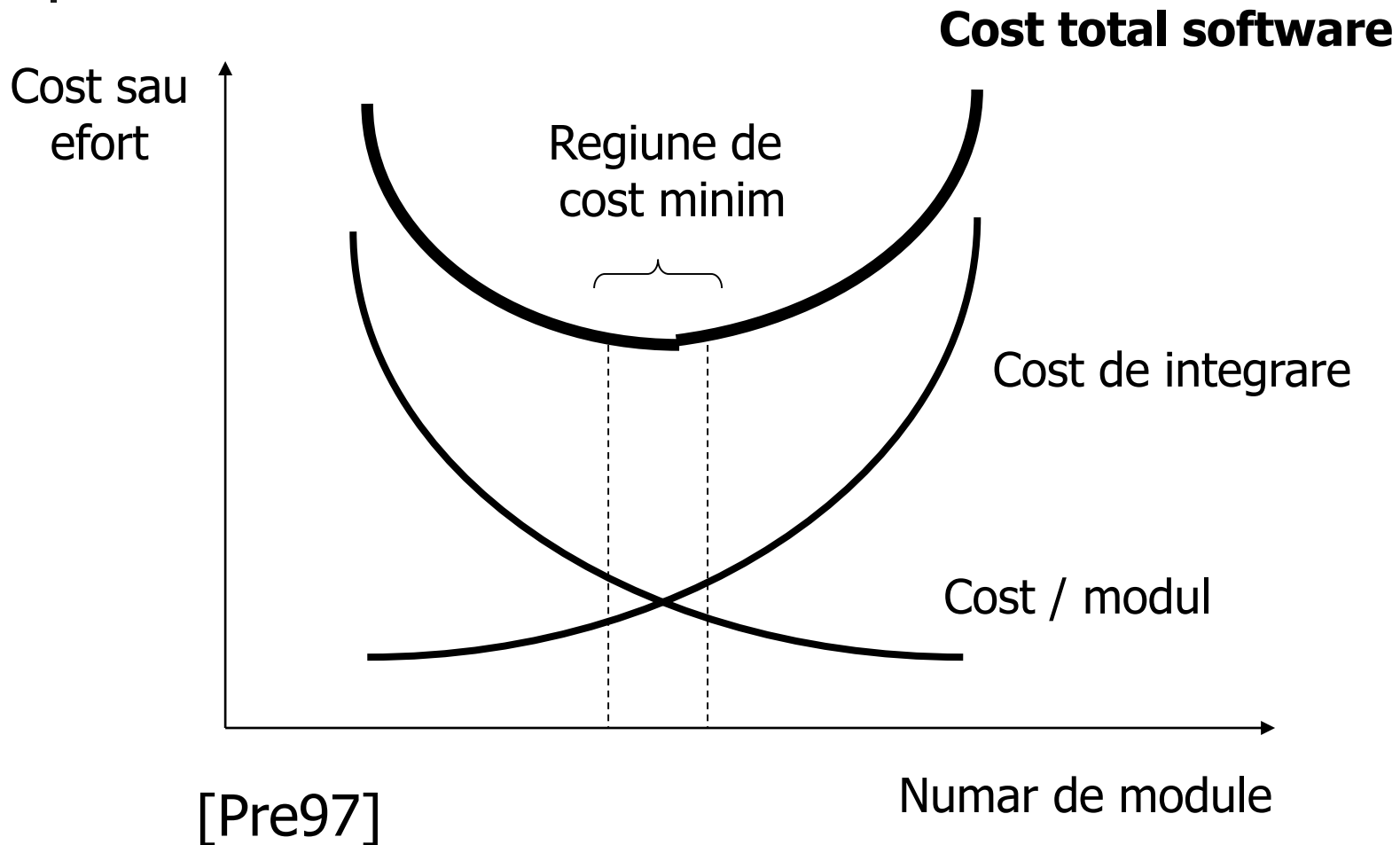
- Un argument in favoarea modularizarii este dat de urmatoarea 'lege' (empirica)

[Pre97]:

$$\mathbf{E(p1 + ... + pn) > E(p1) + ... + E(pn)}$$

- $p_1, ..., p_n$ este o descompunere a unei probleme p in subprobleme.
 - $E(x)$ este efortul necesar rezolvarii problemei x
- In practica atat *submodularizarea* cat si *supramodularizarea* ar trebui evitate.

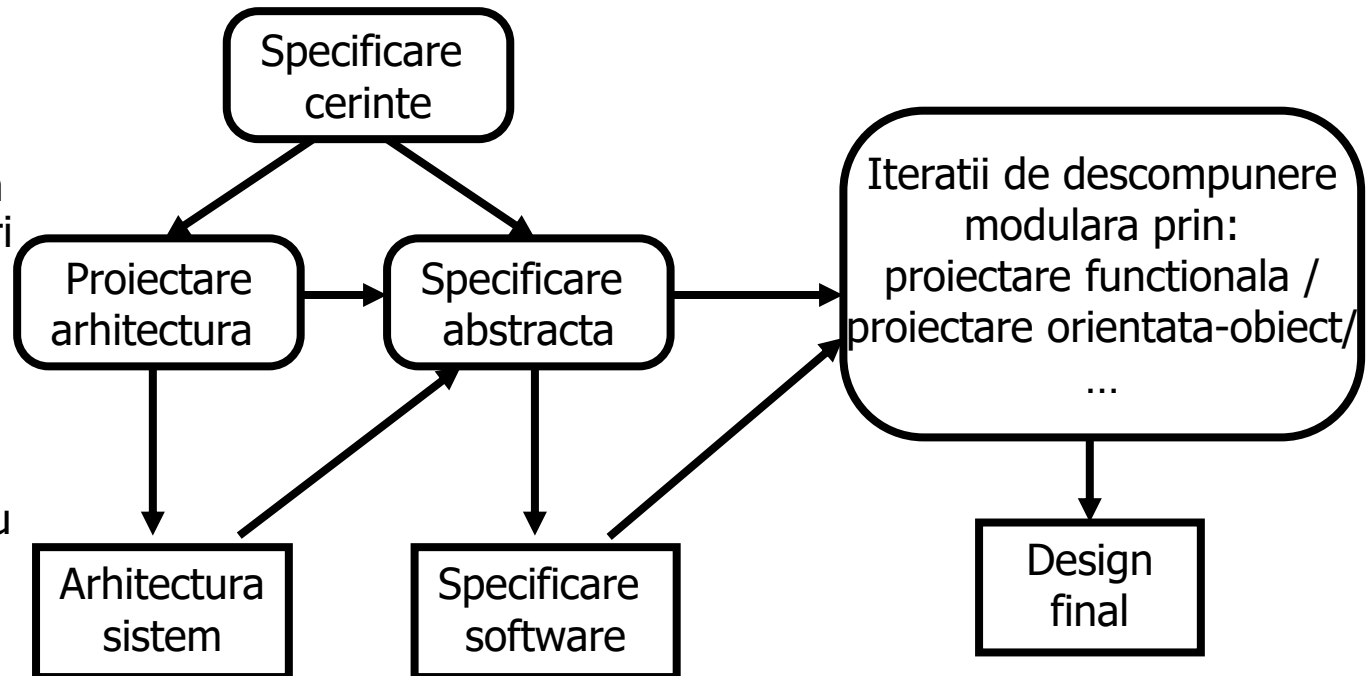
Activitati de baza – proiectare

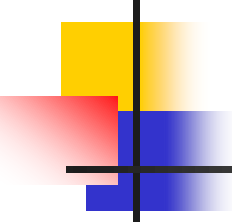


Activitati de baza – proiectare

Un model foarte general al procesului de proiectare:

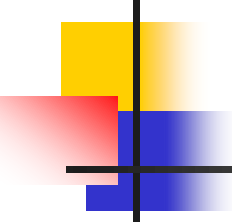
- Activitatea de proiectare necesita experienta si creativitate din partea inginerului software.
- Forma finala a designului este obtinuta prin iteratii din designuri preliminare.
- Activitatea de proiectare este un proces de formalizare si detaliere, in care pot fi necesare reveniri pentru corectarea unor designuri anterioare.





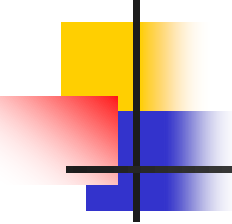
Activitati de baza – proiectare

- Fazele activitatii de proiectare [Som16,Som11,Som07,Som89]:
 - *Proiectare arhitectura* Sistemul este descompus intr-o structura de subsisteme. Rezultatul acestei activitati este o descriere a *arhitecturii sistem*.
 - *Specificare abstracta* Pentru fiecare subsistem, se realizeaza specificarea abstracta a serviciilor si constrangerilor sub care trebuie sa opereze.
 - *Descompunere modulara* In aceasta faza, se utilizeaza o metoda potrivita de proiectare (proiectare functionala / proiectare orientata obiect) si dupa un numar de iteratii se produce un design final.



Activitati de baza – proiectare

- Pentru descompunerea in module a subsistemelor vom considera:
 - *Modele data-flow* (proiectare functionala)
 - *Modele orientate obiect* (proiectarea OO)
- Strategii de baza in descompunerea modulara:
 - ***Proiectare top-down*** Aceasta strategie se bazeaza pe ideea ca structura problemei ar trebui sa determine structura solutiei software.
 - ***Proiectare bottom-up*** In practica, strategia top-down este combinata cu strategia bottom-up (proiectantii gasesc metode de reutilizare pe masura ce rafinarea top-down progreseaza).



Activitati de baza – testare

- In activitatea de **testare** scopul inginerului software este de a verifica si valida (V & V) implementarea.
 - Toate activitatile de dezvoltare – de la specificare la elaborare cod – implica activitati V & V, cum ar fi inspectii si recenzii.
 - Totusi, cea mai importanta activitate V & V este testarea software, care este realizata dupa implementare.
 - **Verificarea** stabileste daca sistemul se conformeaza cerintelor (specificarii).
 - **Validarea** stabileste daca produsul software satisface asteptarile clientului.
 - In limba engleza se utilizeaza urmatorul joc de cuvinte [Boe79]:
 - Verification: are we building the product right?
 - Validation: are we building the right product?



Activitati de baza – testare

- Activitatea de testare este o mare consumatoare de resurse:
 - 30%-40% in proiecte 'obisnuite';
 - pana la de 5 ori resursele tuturor celorlalte activitati de dezvoltare in proiecte cu cerinte stringente in ceea ce priveste fiabilitatea, securitatea, etc. (cf. [Pre97]).
- In proiecte dezvoltate prin metode formale testarea la nivel de modul poate fi inlocuita cu o verificare formala a specificatiei matematice. Acest proces are loc inainte de implementare.
 - Dar metodele formale sunt utilizate numai in domenii specializate.
- In majoritatea proiectelor, activitatea V & V care consuma cele mai multe resurse se desfasoara dupa implementare, atunci cand se realizeaza testarea sistemului.



Activitati de baza – testare

- Doua abordari complementare:
 - *Testare functionala sau black-box*
 - In aceasta abordare se testeaza comportamentul functional al produsului software (relatia intrari-iesiri).
 - Testarea functionala se bazeaza pe cerinte. Nu se utilizeaza informatie legata de structura interna a sistemului software.
 - *Testare white-box (sau glass-box)*
 - Este o modalitate de a testa structura sistemului software utilizand cunostinte despre structura sa interna.
 - Se utilizeaza informatii din fazele de proiectare si implementare.



Activitati de baza – testare

- Obiective ale testarii [Mye79]:
 - Testarea este procesul de executare a unui program cu scopul de a detecta defecte.
 - Un caz de test bun este acela care are o mare probabilitate de a detecta un defect care nu a fost descoperit pana in acel moment.
 - Un test este reusit daca descopera un defect care nu a fost descoperita inainte.
- Se subliniaza insa ca [Dij72]
 - Testarea nu poate detecta absenta defectelor, ci doar prezenta defectelor intr-un sistem software.



Activitati de baza – testare

- **Principiile activitatii de testare [Dav95] (cf. [Pre97]):**
 1. Testele trebuie sa fie ghidate de cerinte, deoarece cele mai severe defecte sunt cele care conduc la nesatisfacerea cerintelor.
 2. Testele ar trebui sa fie planificate (mult) inainte de inceperea activitatii de testare. Testele black-box pot fi planificate pe baza cerintelor. Testele de tip white-box pot fi planificate pe baza modelului de design.
 3. Testarea trebuie realizata de la simplu (testare module) spre complex (testare de integrare si testare de acceptanta).
 4. Testarea exhaustiva nu este posibila.
 5. Pentru a fi eficace, testarea ar trebui sa fie condusa de un grup independent (care nu este subordonat dezvoltatorului, ci organizatiei care se ocupa cu asigurarea calitatii).



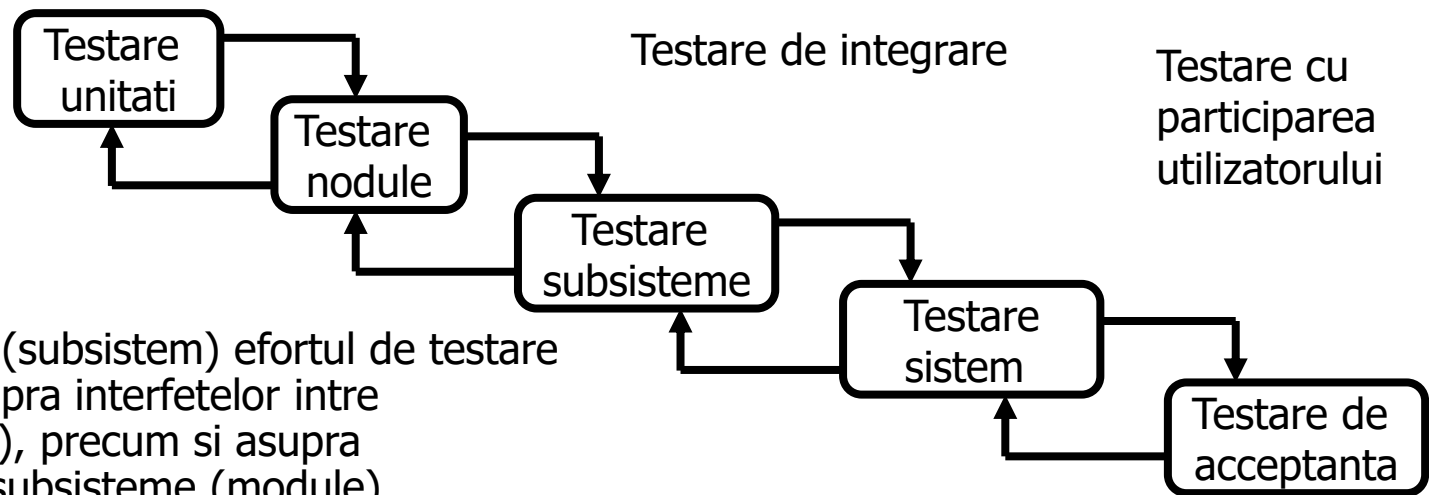
Activitati de baza – testare

- Testarea software se realizeaza in urmatoarele stadii:
 - testare unitati program (proceduri, functii),
 - testare la nivel de modul (clasa, ADT, modul fisier),
 - testare la nivel de subsistem,
 - testare la nivel de sistem,
 - testare de acceptanta (pe date furnizate de client, numita uneori *testare alfa*).
 - (Poate urma o faza de *testare beta*, in care sistemul este livrat spre utilizare si testare catre un numar de clienti si utilizatori potentiali).

Activitati de baza – testare

Procesul de testare:

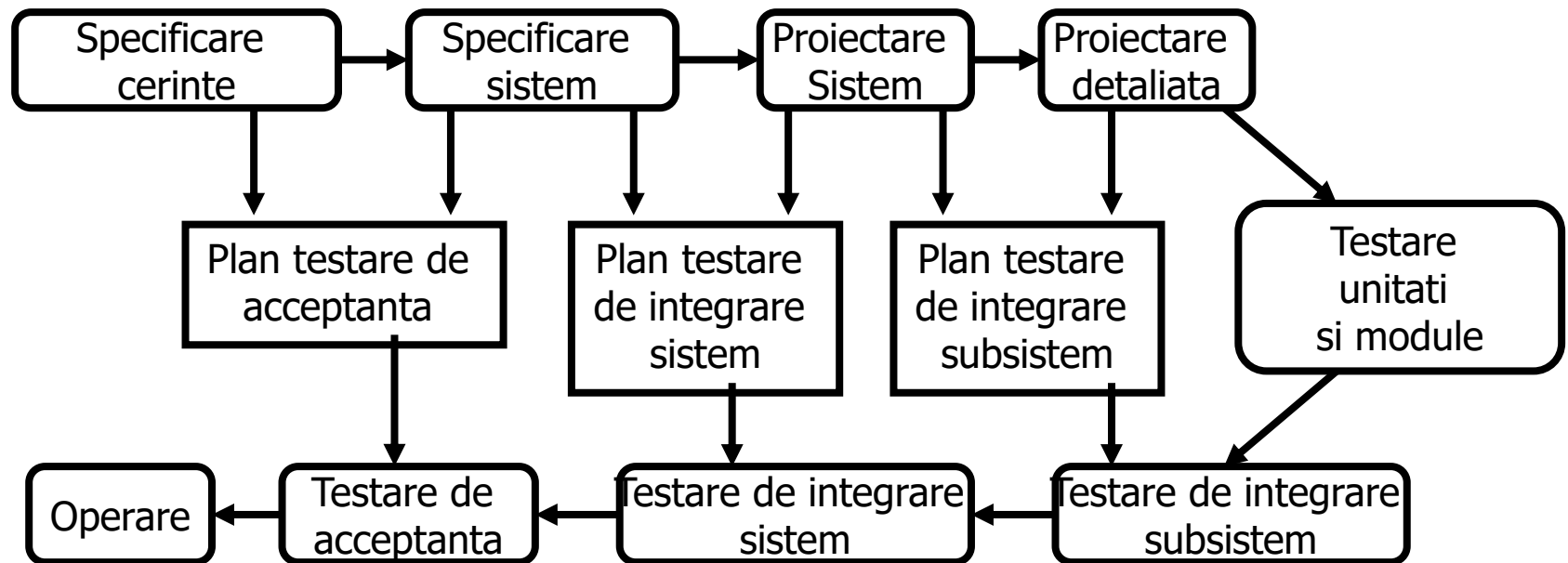
Testare unitati & module

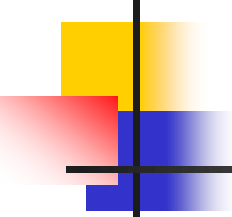


- La nivel de sistem (subsistem) efortul de testare se concentreaza asupra interfetelor intre subsisteme (module), precum si asupra interactiunilor intre subsisteme (module).
- In faza de testare de acceptanta sistemul este testat cu date furnizate de client (iar nu cu date de test simulate). In acest stadiu se pot valida atat cerinte functionale cat si cerinte nonfunctionale.

Activitati de baza – testare

- Figura de mai jos prezinta relatia intre activitatile de dezvoltare si cele de testare prin planuri de testare.





Activitati de baza – evolutie software

- Majoritatea sistemelor software trebuie sa evolueze (apar cerinte noi iar cerintele mai vechi se pot modifica).
 - Demarcatia care se realiza in mod traditional intre procesele de dezvoltare si evolutie devine tot mai putin relevanta [Som11].
 - Au aparut noi tehnologii, in particular inginerie software orientata pe reutilizare, si
 - In prezent putine sisteme software sunt sisteme complet noi.
 - Costurile legate de modificarea (evolutia) produselor software pot fi mai ridicate decat costurile de dezvoltare. Tehnologia software ofera insa beneficii semnificative [Som05,Som16]:
 - Sistemele software sunt mai flexibile decat cele hardware (si decat alte sisteme ingineresti).
 - Costurile de modificare sunt mai reduse in cazul produsele software decat in cazul unor produse hardware corespunzatoare.
 - Produsele ingineresti incorporeaza o cantitate tot mai mare de componente software.



Activitati de baza – evolutie software

- Strategii de evolutie software
[Som01,Som05,Som11,Som16]:
 - *Intretinere software* (engl. *software maintenance*):
 - corectare defecte,
 - adaptare software la noi medii de operare,
 - modificare sau adaugare functionalitate.
 - *Evolutie arhitecturala* (de exemplu, transformarea unei arhitecturi centralizate intr-o arhitectura distribuita client-server pentru un produs software mai vechi)

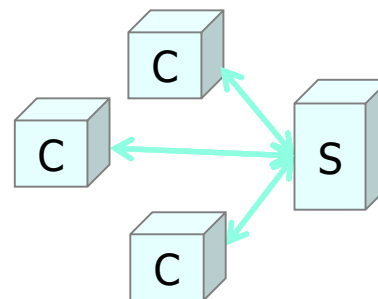
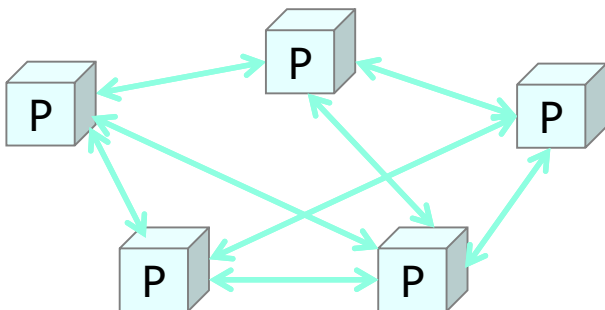


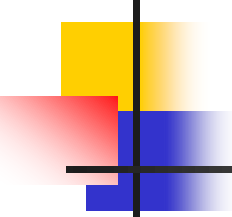
Activitati de baza – evolutie software

- *Reinginerie software* (engl. *re-engineering*)
 - Sistemul software este modificat cu scopul de a deveni mai usor de inteles si modificat.
 - Nu se adauga functionalitate noua (in mod normal nici arhitectura nu se modifica).
 - Modificarile pot include:
 - redocumentarea sistemului,
 - reorganizarea si restructurarea sistemului,
 - translatarea implementarii intr-un limbaj de programare nou,
 - In acest proces este important (sau chiar esential) sa se utilizeze instrumente CASE, care pot furniza suport pentru: translatare automata cod sursa, inginerie directa (forward engineering), inginerie inversa (reverse engineering), etc.

Specificare si Arhitectura

- Revenind la punctul de pornire al unui proiect software
 - Ce se realizeaza mai intai: specificarea sau arhitectura? Nu exista un raspuns simplu
 - Specificatiile si arhitectura se dezvoltă impreuna
 - Adesea, cea mai buna solutie se obtine prin iterare [JBR99]
- Sabloanele arhitecturale (ex.: Peer-to-Peer, Client-Server) furnizeaza solutii generale, reutilizabile





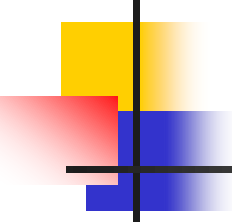
Specificare si Arhitectura

- **Abordarile flexibile** (engl. *agile*), cum este XP, utilizeaza practici si instrumente specifice [Som16,FP16]
 - Implicare client, **scenarii XP** (*user stories*), dezvoltare ghidata de testare (test driven development TDD)
 - Instrumente (frameworks, generatoare automate de cod) pentru accelerarea procesului de dezvoltare
 - Un posibil dezavantaj: un proiect dezvoltat prin metode flexibile ar putea sa raspunda cerintelor clientului dar sa nu conduca la cea mai buna arhitectura software [FP16]
- Traditional, in **abordarea orientata pe planificare si documentare** (engl. *plan-and-document*) arhitectura software este construita combinand [JBR99,LL05]
 - Sabloane arhitecturale
 - **Cazuri de utilizare** (*use cases*) relevante arhitectural



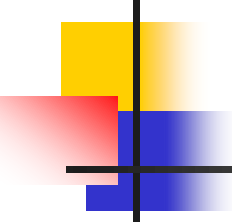
Bibliografie

- [Bec99] K. Beck. Embracing Change with Extreme Programming. *IEEE Computer* 32(10):70-78, 1999.
- [Boe79] B. Boehm. Software engineering; R & D trends and defense needs. In *Research Directions in Software Technology*, pages 1-9, MIT Press, 1979.
- [BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [Coc01] A. Cockburn. *Agile software development*. Addison-Wesley, 2001.



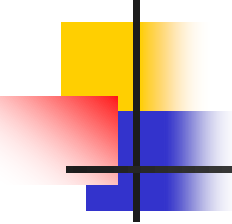
Bibliografie

- [Dav95] A. Davis. *201 Principles of Software Development*. McGraw-Hill, 1995.
- [Dij72] E. Dijkstra, et al. *Structured programming*. London: Academic Press, 1972.
- [FP16] A. Fox, D. Patterson. *Engineering Software as a Service: An Agile Approach using Cloud Computing*. Strawberry Canyon Publisher, 2016.
- [IEEE93b] IEEE recommended practice for software requirements specifications. *Software Engineering Requirements Engineering*, R.H. Thayer, M. Dorfman, editors, Los Alamitos, CA: IEEE Computer Society Press.



Bibliografie

- [Jac92] I. Jacobson. *Object-Oriented Software Engineering*. Addison-Wesley, 1992.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [KS96] G. Katonya, I. Sommerville. Requirements engineering with viewpoints. *BCS/IEEE Software Engineering J.*, 11(1):5-18, 1996.
- [Kru03] P. Kruchten. *The Rational Unified Process: An Introduction* (3rd edition). Addison-Wesley, 2003.
- [LL05] T. Lethbridge and R. Laganriere. *Object-Oriented Software Engineering* (2nd edition). McGraw-Hill, 2005.



Bibliografie

- [Mar79] T. DeMarco. *Structured Analysis and System Specification*. Prentice-Hall, 1979.
- [Mul79] G. Mullery. CORE – a method for controlled requirements specification. In *Proc. 4th Int. Conf. On Software Engineering*, IEEE Press, 1979.
- [Mye79] G. Myers. *The Art of Software Testing*. Wiley, 1979.
- [Pre97] R. Pressman. *Software Engineering: A Practitioner's Approach*, (4th edition). McGraw-Hill, 1997.
- [Pre00] R. Pressman. *Software Engineering: A Practitioner's Approach*, (5th edition). McGraw-Hill, 2000.
- [Pre14] R. Pressman. *Software Engineering: A Practitioner's Approach*, (8th edition). McGraw-Hill, 2014.



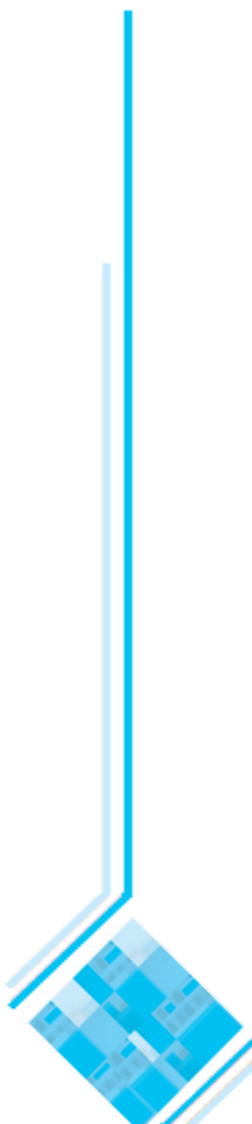
Bibliografie

- [SB01] K. Schwaber, M. Beedle. *Agile software development with SCRUM*. Prentice Hall, 2001.
- [SG96] M. Shaw and D. Garlan. *Software Architecture*. Prentice-Hall, 1996.
- [Som89] I. Sommerville. *Software Engineering*, (3rd edition). Addison-Wesley, 1989.
- [Som01] I. Sommerville. *Software Engineering*, (6th edition). Addison-Wesley, 2001.
- [Som05] I. Sommerville. *Software Engineering*, (7th edition). Addison-Wesley, 2004.



Bibliografie

- [Som07] I. Sommerville. *Software Engineering*, (8th edition). Addison-Wesley, 2006.
- [Som11] I. Sommerville. *Software Engineering*, (9th edition). Addison-Wesley, 2010.
- [Som16] I. Sommerville. *Software Engineering*, (10th edition). Addison-Wesley, 2015.
- [Sta97] J. Stapleton. *DSDM Dynamic Systems Development Method*. Addison-Wesley, 1997.



Dezvoltare cerinte

Adapted from:

Timothy Lethbridge and Robert Laganieri,

Object-Oriented Software Engineering –

Practical Software Development using UML and Java, 2005

(chapter 4)

4.1 Analiza de domeniu

***Analiza de domeniu* este procesul prin care inginerul software invata cunostinte din domeniul aplicatiei cu scopul de a intelege mai bine problema de rezolvat.**

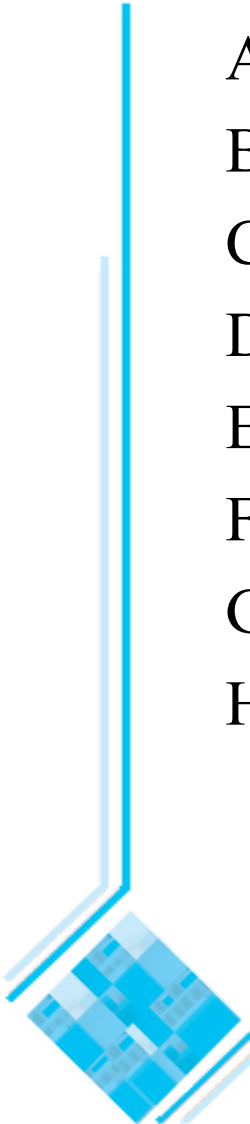
- *Domeniul* este sfera de activitate economica sau tehnologica in care va fi utilizat produsul software.
- Un *expert in domeniu* este o persoana care intelege foarte bine domeniul (de aplicatie)

Beneficiile analizei de domeniu:

- Dezvoltare mai rapida
- Un sistem mai bun
- Anticiparea extensiilor

Structura unui document de analiza a domeniului

- A. Introducere
- B. Glosar
- C. Cunostinte generale despre domeniu
- D. Clienti si utilizatori
- E. Sistemul si echipamentele utilizate
- F. Sarcini si proceduri efectuate
- G. Produse software similare existente
- H. Similaritati cu alte domenii



4.2 Punctul de pornire al proiectelor software

Patru mari categorii de proiecte software:

	Cerintele trebuie sa fie determinate	Cerintele au fost produse de clienti
Proiect complet nou	A	B
Evolutie a unui sistem existent	C	D

4.3 Definirea problemei

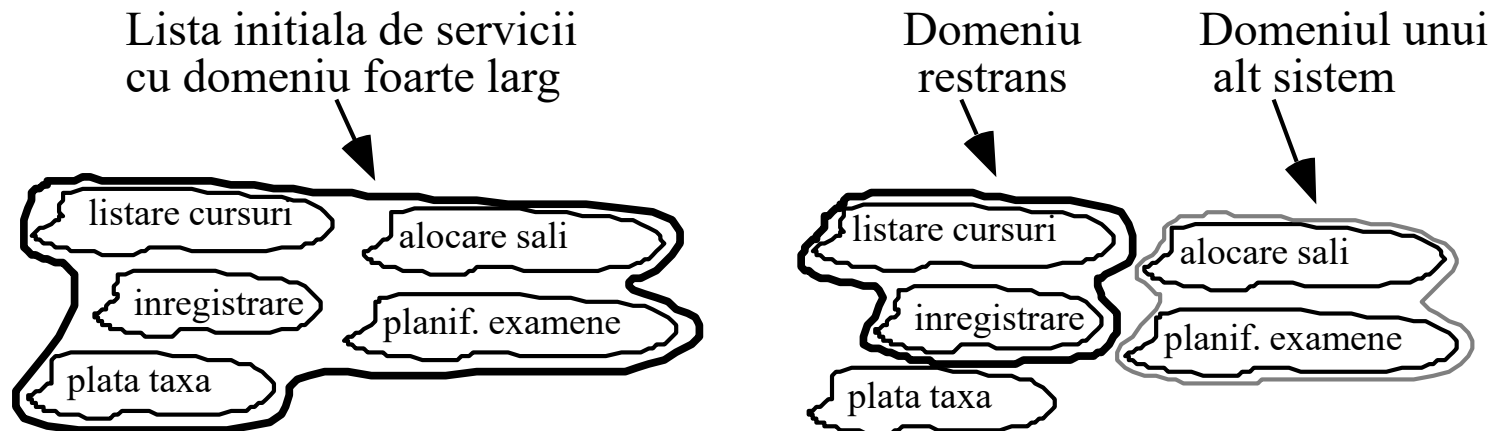
- **Procesul de determinare a cerintelor incepe prin definirea *problemei* de rezolvat.**
- **O problema poate fi exprimata ca:**
 - O *dificultate* intampinata de utilizatori sau clienti,
 - O *oportunitate* de imbunatatire a productivitatii sau a vanzarilor.
- **Solutia problemei implica dezvoltarea unui produs software.**
- **Este bine ca formularea problemei sa fie succinta – preferabil una sau doua fraze.**
 - Exemplu:
 - Sistemul va permite unui student sa se inregistreze la cursuri si sa isi modifice inregistrarea, simplu si rapid. De asemenea, va asista studentii in alegerea cursurilor pe care le considera a fi cele mai interesante si mai utile.

Limitarea domeniului problemei

Este important ca domeniul problemei sa fie definit cat mai precis

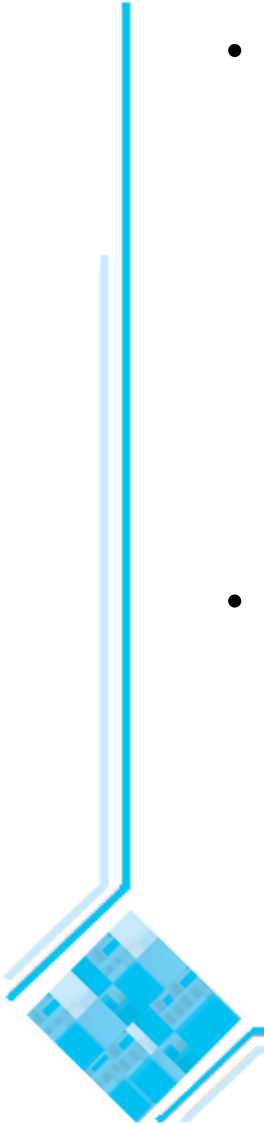
- Se alcatuieste o lista a serviciilor posibile
 - Unele se exclud daca lista initiala este prea larga
 - Se identifica scopurile principale (ale clientului / utilizatorului) daca lista initiala este prea ingusta

Exemplu: Sistem de gestiune date universitare



4.4 Ce este o cerinta

- O *cerinta* este 1) o descriere a unui serviciu pe care sistemul trebuie sa il furnizeze sau 2) o descriere a unei constrangeri pe care sistemul trebuie sa o satisfaca. Implementarea oricarei cerinta trebuie sa contribuie la rezolvarea problemei clientului / utilizatorului; pachetul de cerinte reprezinta rezultatul unui acord intre toti participantii la proiect.
- O colectie de cerinte este un *document de cerinte*.



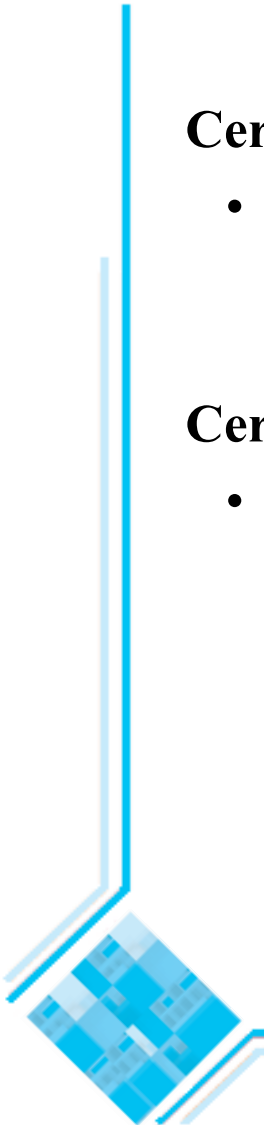
4.5 Tipuri de cerinte

Cerinte functionale

- Descriu serviciile sistem solicitate de client / utilizator (*ce ar trebui sa faca sistemul software*)

Cerinte nonfunctionale

- Descriu constrangeri sub care trebuie sa opereze sistemul sau standarde ce trebuie sa fie satisfacute de sistemul software



Cerinte functionale

Cerintele functionale descriu *servicii* furnizate catre *utilizator* sau catre *alte sisteme*.

Cerintele functionale pot sa descrie:

- Ce *intrari* trebuie sa accepte sistemul
- Ce *iesiri* trebuie sa produca sistemul
- Ce date (care pot fi utilizate de alte sisteme) trebuie sa fie *stocate* de sistem
- Ce *calcule* trebuie sa efectueze sistemul
- *Temporizarea sau sincronizarea* serviciilor (in special in cazul sistemelor de timp real)

Cerinte nonfunctionale

Trei mari categorii

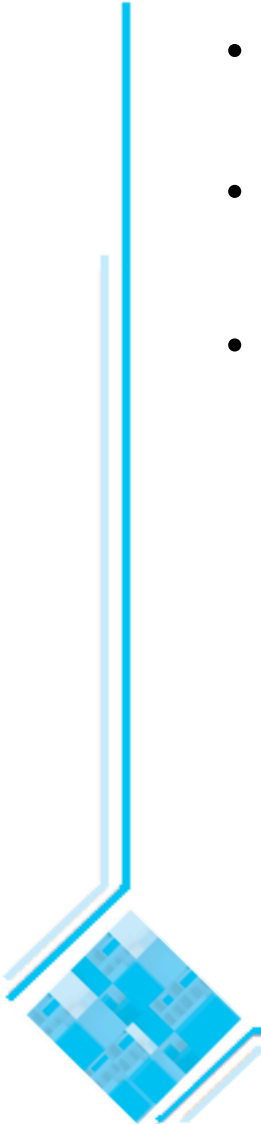
1. Cerinte care reflecta *attribute de calitate*:
 - Timp de raspuns
 - Utilizare resurse
 - Fiabilitate
 - Recuperare din erori
 - Mentenabilitate
 - Reutilizabilitate
2. Cerinte legate de platforma si tehnologia de calcul
3. Cerinte legate de metodologie (procesul de dezvoltare), costuri si date de livrare

4.6 Cazuri de utilizare

- Un *caz de utilizare* (engl. *use case*) este o descriere a unui set de secvente de actiuni pe care le efectueaza un sistem pentru a produce un rezultat observabil si semnificativ pentru un actor particular [BRJ99].
- Un *model de cazuri de utilizare* consta din
 - O colectie de cazuri de utilizare
 - O colectie de actori (un *actor* reprezinta un tip de utilizator (nu neaparat uman) care interactioneaza cu sistemul; in interactiunea cu sistemul, fiecare actor joaca un set coerent de roluri [BRJ99,JBR99])
 - O descriere (diagramatica) a modului in care aceste componente interactioneaza
- Pentru ca un model de cazuri de utilizare sa fie inteligibil este necesara gruparea secventelor de actiuni similare intr-un singur caz de utilizare.

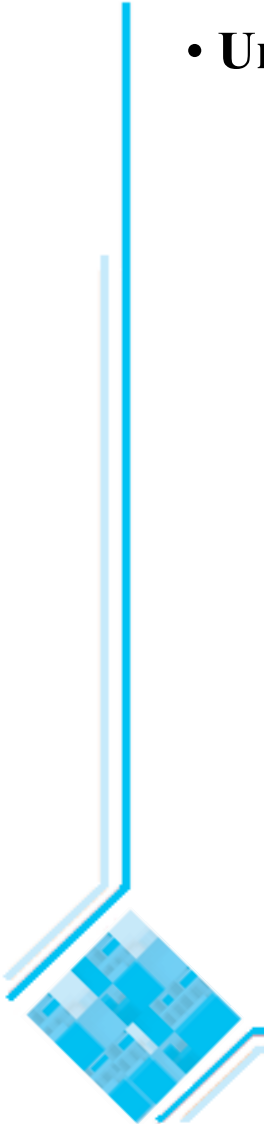
Cazuri de utilizare

- In general, un caz de utilizare acopera intreaga secventa de actiuni asociata unei sarcini de calcul (serviciu furnizat de sistem).
- Un caz de utilizare descrie *interactiunea utilizatorului* cu sistemul...
—nu calculele efectuate de sistem.
- Descrierea cazurilor de utilizare trebuie sa fie cat se poate de independenta de designul interfatei utilizator.



Scenarii

- Un **scenariu** este o *instanta* a unui caz de utilizare, care implica
 - o anumita instanta a unui actor
 - un anumit (moment / o anumita durata de) timp
 - date specifice.



Cum se descrie un caz de utilizare

A. Nume: se alege un nume scurt, descriptiv

B. Actori: Se listeaza actorii care utilizeaza cazul de utilizare.

C. Scopuri: Ce urmaresc actorii care utilizeaza cazul de utilizare.

D. Preconditii: Starea sistemului inainte de efectuarea cazului de utilizare.

E. Descriere: Scurta descriere informala a cazului de utilizare.

F. Relatii cu alte cazuri de utilizare

G. Actiuni / pasi: Se descrie fiecare actiune / pas (pe 2 coloane: actiuni actor, raspunsuri sistem).

H. Postconditii: Starea sistemului dupa terminarea cazului de utilizare.

- Numai numele (A) si actiunile atasate (G) sunt esentiale.
—Intr-o descriere simplificata se pot omite celelalte componente.

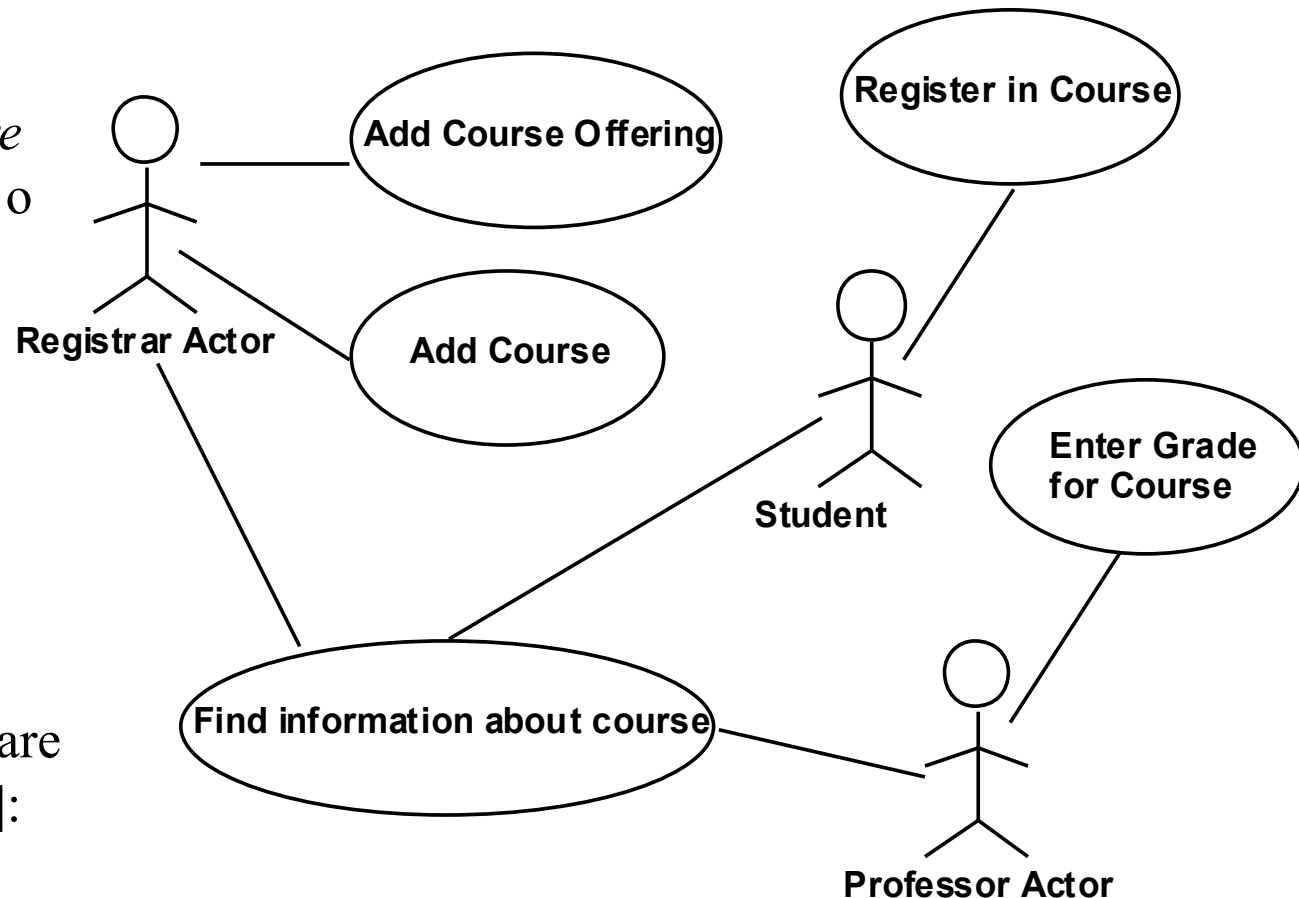
Un exemplu de diagrama de cazuri de utilizare

Grafic

- un *caz de utilizare* este reprezentat ca o elipsa,
- un *actor* este reprezentat sub forma unui omulet (din bete).

Diagramele de cazuri de utilizare contin [BRJ99]:

- actori,
- cazuri de utilizare,
- asocieri, generalizari si dependente.

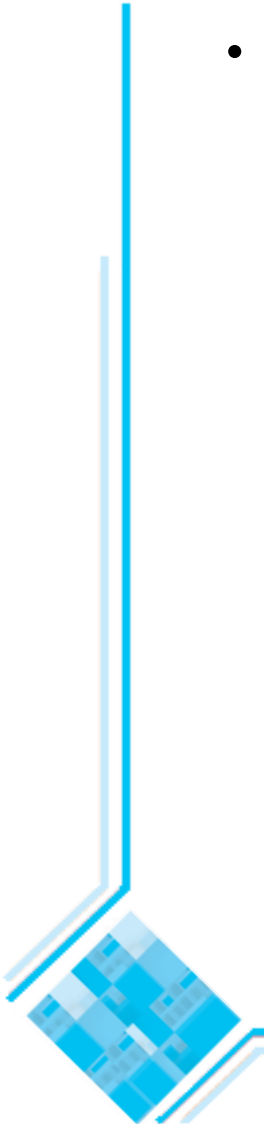


Extensii

- Utilizate pentru explicitarea interactiunilor *optionale* sau pentru tratarea cazurilor de *exceptie*.
 - Prin crearea de cazuri de utilizare care reprezinta extensii descrierea cazului de baza ramane simpla.
 - In extensie trebuie
 - fie sa se listeze toate actiunile (toti pasii) de la inceputul cazului de utilizare pana la sfarsit (inclusiv situatia optionala sau de exceptie),
 - fie sa se indice care este punctul de extensie (punctul in care extensia modifica secventa de baza).

Generalizari

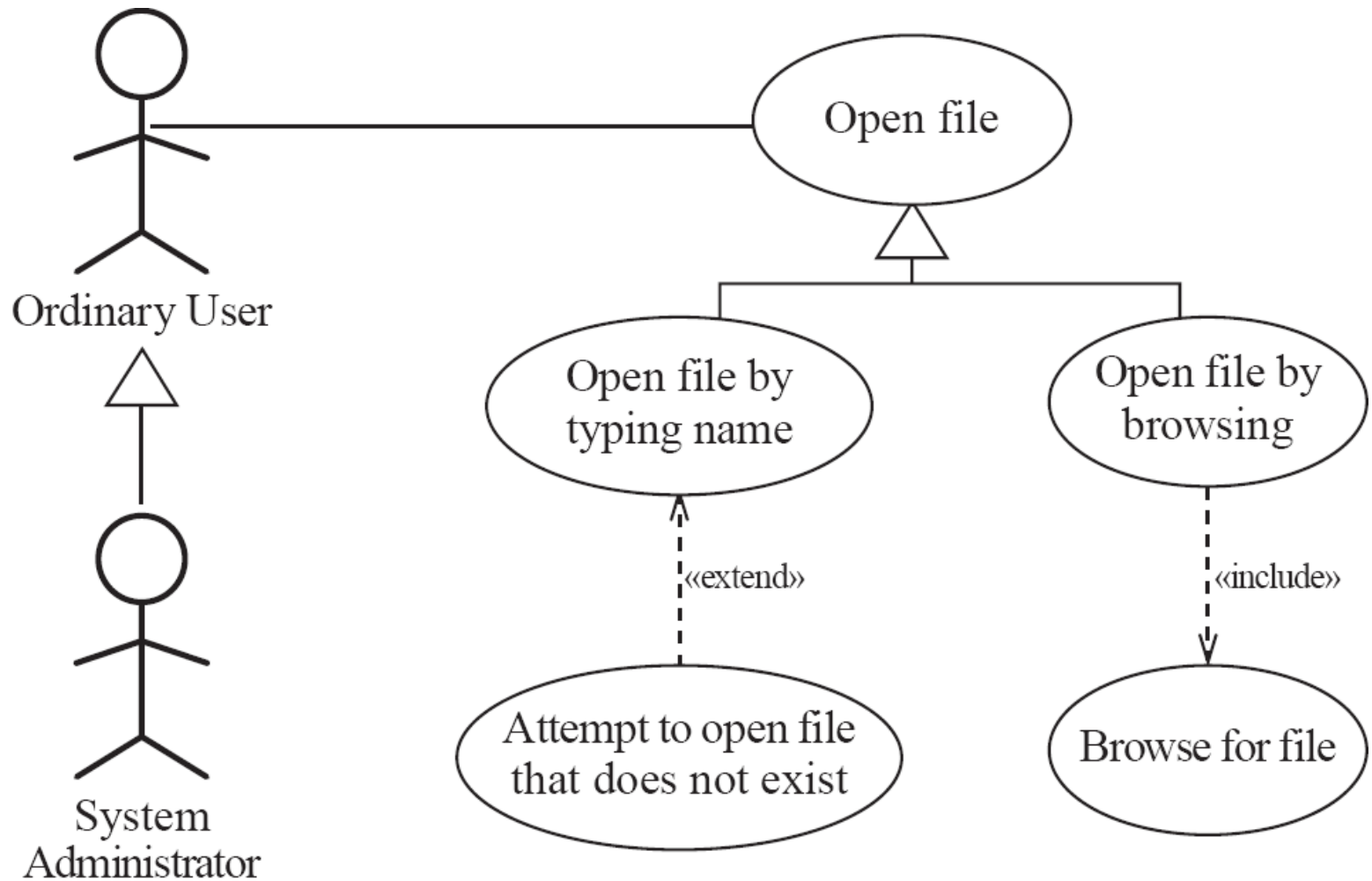
- *Generalizarile (/ specializarile)* se comporta la fel ca in diagramele de clase (si utilizeaza aceeaasi notatie grafica).
 - Un caz de utilizare mai general poate reprezenta cateva cazuri de utilizare similare.
 - Unul sau mai multe cazuri de utilizare pot oferi specializari ale comportamentului general.



Incluziuni

- Permit reprezentarea unui *comportament comun* pentru mai multe cazuri de utilizare.
 - Chiar si cazuri de utilizare foarte diferite pot partaja secvente de actiuni.
 - Permit evitarea repetarii unor detalii in mai multe cazuri de utilizare.
 - Reprezinta sarcini de calcul secundare (de nivel mai coborat).

Exemplu de diagrama de cazuri de utilizare cu generalizare, extensie si incluziune



Exemplu de descriere

Caz de utilizare: Open File

Relatii:

Generalizare pentru:

- Open file by typing name
- Open file by browsing

Actiuni:

Actiuni actor:

1. Alegere comanda 'Open...'
3. Specificare nume fisier
4. Confirmare selectie

Raspunsuri sistem:

2. Afisare caseta dialog 'File open'
5. Eliminare caseta dialog

Exemplu (continuare)

Caz de utilizare: Open file by typing name

Relatii:

Specializare pentru:

- Open file

Actiuni:

Actiuni actor

1. Alegere comanda 'Open...'
- 3a. Selectare camp text
- 3b. Introducere nume fisier
4. Selectare 'Open'

Raspunsuri sistem

2. Afisare caseta dialog 'File open'
5. Eliminare caseta dialog

Exemplu (continuare)

Caz de utilizare: Open file by browsing

Relatii:

Specializare pentru: Open file

Include: Browse for file

Actiuni:

Actiuni actor:

1. Alege comanda ‘Open...’
3. Browse for file (caz de utilizare inclus)
4. Confirmare selectie

Raspunsuri sistem:

2. Afisare caseta dialog ‘File open’
5. Eliminare caseta dialog



Exemplu (continuare)

Caz de utilizare: Attempt to open file that does not exist

Relatii:

Extensie pentru: Open file by typing name

Actiuni:

Actiuni actor:

1. Alege comanda 'Open...'
- 3a. Selectare camp text
- 3b. Introducere nume fisier
4. Selectare 'Open'
6. Corectare nume fisier
7. Selectare 'Open'

Raspunsuri sistem:

2. Afisare caseta dialog 'File open'
5. Se indica faptul ca fisierul nu exista
8. Eliminare caseta dialog

Exemplu (continuare)

Caz de utilizare: Browse for file (incluziune)

Actiuni:

Actiuni actor:

1. Daca fisierul dorit nu este afisat se selecteaza un director
3. Se repeta pasul 1 pana cand se afiseaza fisierul dorit
4. Selectare fisier

Raspunsuri sistem:

2. Afisare continut director

Cazuri de utilizare prioritare

- Adesea unul sau mai multe cazuri de utilizare pot fi selectate ca fiind principale (esentiale) pentru sistem
 - De exemplu, într-un sistem de rezervari pentru curse aeriene cazul de utilizare principal va fi ‘Rezervare loc pentru un anumit zbor’
 - » Intregul sistem poate fi construit în jurul acestui caz de utilizare.
- Exista și alte motive pentru tratarea cu prioritate a anumitor cazuri de utilizare:
 - Implementarea anumitor cazuri de utilizare poate implica un înalt grad de *risc* (tehnic sau managerial)
 - Anumite cazuri de utilizare pot avea o mare *valoare comercială sau politică*

Beneficii ale dezvoltarii ghidate de cazurile de utilizare

- Cazurile de utilizare ajuta in definirea *domeniului* sistemului.
- Cazurile de utilizare pot fi utilizate ca instrument de *planificare* a procesului de dezvoltare.
- Cazurile de utilizare permit atat dezvoltarea cat si validarea cerintelor.
- Pe baza cazurilor de utilizare se pot proiecta cazuri de test.
 - Un caz de test este o specificare a unui scenariu de testare a sistemului. Un caz de test specifica ce se testeaza, in ce conditii, cu ce intrari si care sunt rezultatele asteptate [JBR99].
- Cazurile de utilizare pot fi utilizate pentru structurarea manualelor utilizator.

Beneficii ale dezvoltarii ghidate de cazurile de utilizare

- Strategiile de baza pentru testarea de integrare (top-down si bottom-up) sunt aplicabile daca modulele software alcatuiesc o structura ierarhica.
 - Intr-un sistem OO modulele de baza ce urmeaza a fi testate sunt clasele de obiecte.
 - Problema este ca, in general, un sistem OO nu este structurat sub forma unei ierarhii de module sau subsisteme [Som01].
- **Testarea bazata pe scenarii sau cazuri de utilizare este adesea cea mai buna strategie pentru testarea de integrare a sistemelor OO [JBR99,Som01].**
 - In aceasta abordare inginerul software identifica scenarii de testare pe baza cazurilor de utilizare.
 - In general, fiecare caz de utilizare da nastere unui set de cazuri de testare.

4.7 Tehnici de colectare a cerintelor

Observare

- Lecturare documente si discutarea cerintelor cu utilizatorii
- Asistarea si urmarirea utilizatorilor potentiali in timpul lucrului
- Inregistrarea sesiunilor de lucru pe benzi video

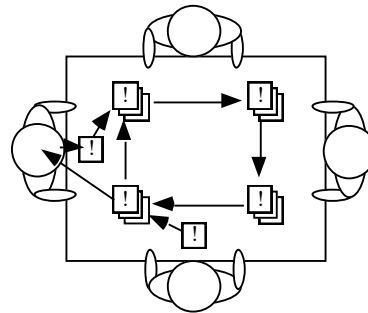
Intervievare

- Organizare interviuri cu intrebari / discutii despre
 - detalii specifice
 - viziunea participantilor asupra viitorului sistem
 - idei sau solutii alternative
 - alte surse de informatie
 - reprezentari diagramatice

Colectarea cerintelor...

Brainstorming (sesiuni ce favorizeaza idei si solutii inspirate)

- Se numeste un moderator experimentat
- Participantii sunt dispusi in jurul unei mese
- Se alege o intrebare suport, de exemplu:
 - Ce aspecte sunt importante pentru sistem?
 - Ce surse de date pot fi anticipate?
 - Ce ‘iesiri’ ar trebui sa produca sistemul?
 - Ce clase sunt necesare pentru reprezentarea domeniului?
- Se cere fiecarui participant sa scrie un raspuns si sa il inmaneze vecinului.

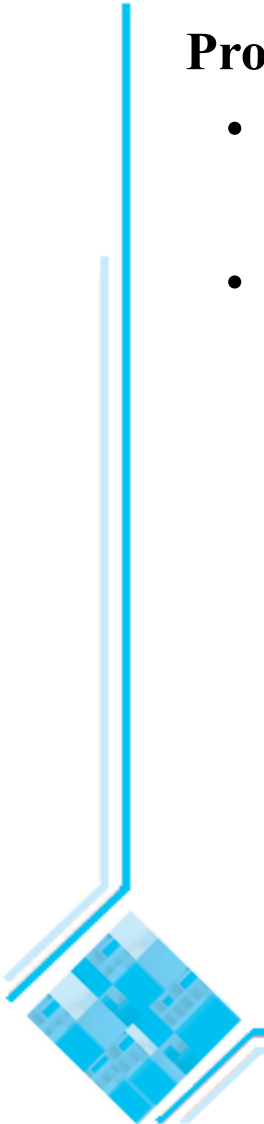


JAD (*Joint Application Development*) este o tehnica bazata pe sesiuni intense de brainstorming

Colectarea cerintelor...

Prototipizare

- Cea mai simpla varianta: prototip pe hartie a interfetei utilizator (IU)
 - Reprezentari grafice sugestive aratate si explicate utilizatorilor
- Cea mai uzuala abordare: un sistem ce simuleaza IU
 - Scris intr-un limbaj de prototipizare rapida
 - In mod normal nu efectueaza calcule si nu interactioneaza cu baze de date sau cu alte sisteme
 - Se pot prototipiza anumite aspecte ale sistemului (algoritmi specifici, operatii pe baze de date, etc.)

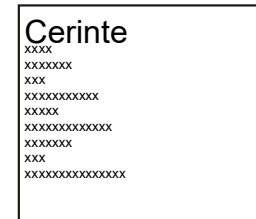


4.8 Tipuri de documente de cerinte

Extreme ce ar trebui sa fie evitate:

- O schita informala a cerintelor alcatuita din cateva paragrafe sau diagrame simple.
- O lista lunga de specificatii ce contine mii de pagini cu detalii complicate.

- Documentele de specificare a cerintelor pentru sistemele de dimensiuni mari sunt in mod normal structurate ierarhic



- In mod normal se utilizeaza urmatoarea terminologie:

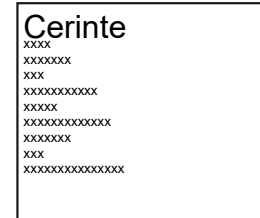
- *Definire cerinte*

- un document mai putin detaliat (de nivel inalt).

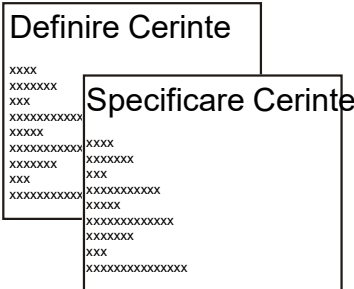
- *Specificare cerinte*

- un document mai precis si mai detaliat.

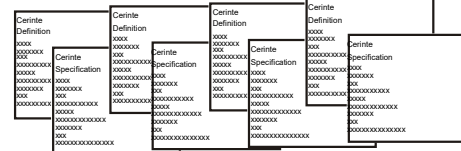
subsistem 1



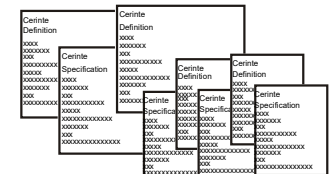
subsistem 2



sub-subsisteme

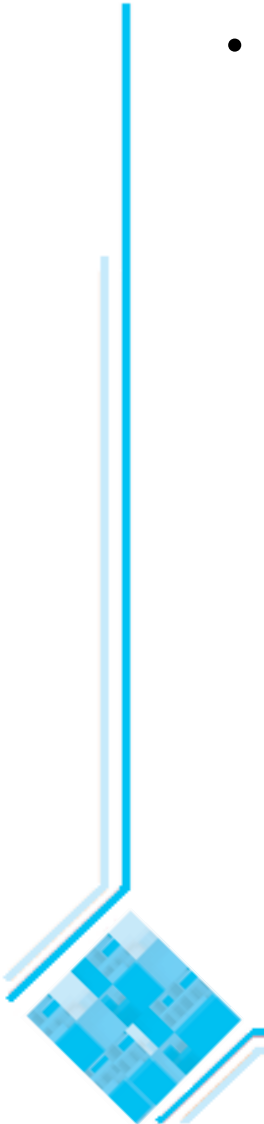


sub-subsisteme



Nivelul de detaliere in documentele de cerinte

- Nivelul de detaliere in documentele de cerinte depinde de:
 - Dimensiunea sistemului
 - Necesitatea de interfatare cu alte sisteme
 - Stadiul in colectarea cerintelor
 - Experienta in domeniul de aplicatie si in utilizarea tehnologiei
 - Costurile implicate de cerinte eronate

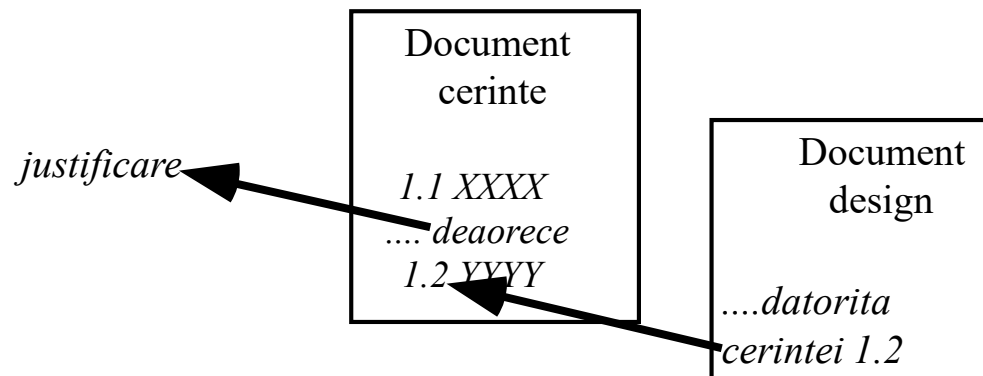


4.9 Recenzia documentelor de cerinte

- **Fiecare cerinta individuala trebuie sa fie verificata cu atentie. Fiecare cerinta individual ar trebui:**
 - Sa conduca la beneficii care sa depaseasca costurile de dezvoltare
 - Sa fie importanta pentru solutionarea problemei
 - Sa fie exprimata intr-o notatie clara si consistenta
 - Sa nu fie ambigua
 - Sa fie logic consistenta
 - Sa conduca la un sistem de calitate
 - Sa fie realista in conditiile resurselor disponibile
 - Sa fie verificabila
 - Sa fie identificabila in mod unic
 - Sa nu impuna constrangeri prea mari asupra procesului de proiectare

Documente de cerinte...

- Documentele de cerinte ar trebui sa fie:
 - Suficient de complete
 - Bine structurate
 - Clare
 - Acceptate de toti participantii (clienti si dezvoltatori)
- Intr-un document de design (proiectare) ar trebui sa fie intotdeauna posibil sa se precizeze care cerinta este implementata de fiecare aspect al designului (in limba engleza se utilizeaza termenul *traceability*).



Structura unui document de cerinte

A. Problema

- O descriere succinta a problemei de rezolvat

B. Domeniul

- Referinte la documente de analiza de domeniu, standarde, sisteme similare

C. Context si modele sistem

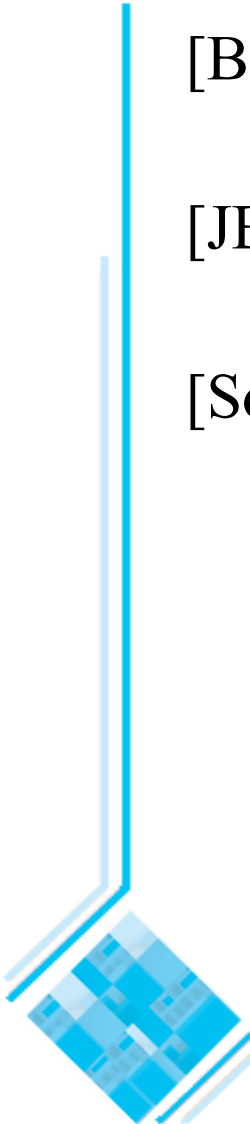
- Contextul in care opereaza sistemul, descriere globala subsisteme, suport hardware, etc.

D. Cerinte functionale

E. Cerinte nonfunctionale

Referinte suplimentare

- [BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [Som01] I. Sommerville. *Software Engineering* (6th edition). Addison-Wesley, 2001.



PRISM

Formal Modeling and Analysis

This material is adapted with permission from

- Dave Parker (DP), “Probabilistic Model Checking”, slides for a lecture course taught in the Department of Computer Science at Oxford University (2011) <http://www.prismmodelchecker.org/lectures/pmc/>
- Accompanying practical course (PRISM tutorial)
<http://www.prismmodelchecker.org/courses/pmc1112/>
<http://www.prismmodelchecker.org/tutorial/>

Formal Methods

- Software engineering methods can be divided into:
 - semi-formal methods (simple notations based on combinations of diagrams and text)
 - formal methods
- **Formal methods** are mathematically based techniques for specification, design and verification
- Formal methods are useful in software engineering when
 - The problem to be solved is novel or difficult
 - The focus is on **quality** attributes: **reliability, performance**

Probabilistic Model Checking

- Probabilistic model checking is formal technique for the modeling and analysis of stochastic systems
- PRISM is a leading tool in probabilistic model checking
 - It incorporates state-of-the art structures and algorithms
 - Application domains: distributed systems, communication protocols, randomized algorithms, biological systems
- PRISM is a state-oriented notation
 - The execution history does not influence the next state
 - The next state depends only on the current state and some current stimuli (or event)

Probabilistic Model Checking

- **PRISM** is based on the following formalisms (including extensions with costs and rewards):
 - **DTMC** (Discrete-Time Markov Chains)
 - **CTMC** (Continuous-Time Markov Chains)
 - **MDP** (Markov Decision Processes)
 - **PTA** (Probabilistic Timed Automata)
- Queries can be expressed in a **property specification language** incorporating the temporal logics **PCTL**, **CSL**, **LTL** and **PCTL*** (including extensions for costs/rewards)

Motivation

- Complexity of model checking
 - generally polynomial in model size (number of states)
- State space explosion problem
 - models for realistic case studies are typically huge
- Clearly (efficient) tool support is required
- Benefits:
 - fully automated process
 - high-level languages/formalisms for building models
 - visualisation of quantitative results

Probabilistic model checkers

- PRISM (this lecture): DTMCs, MDPs, CTMCs, PTAs + rewards
- Markov chain model checkers
 - MRMC: DTMCs, CTMCs + reward extensions
 - PEPA toolset: CTMCs + CSL
- Markov decision process (MDP) tools
 - LiQuor: LTL verification for MDPs (Probmela language)
 - RAPTURE: prototype for abstraction/refinement of MDPs
 - ProbDiVinE: parallel/distributed LTL model checking of MDPs
- Simulation-based probabilistic model checking:
 - APMC, Ymer (both based on PRISM language), VESTA
- And more
 - APNN-Toolbox, SMART, CADP, Möbius, PASS, PARAM, ...
 - see: <http://www.prismmodelchecker.org/other-tools.php>

The PRISM tool

- **PRISM: Probabilistic symbolic model checker**
 - developed at Birmingham and Oxford University, since 1999
 - free, open source (GPL)
 - versions for Linux, Unix, Mac OS X, Windows, 64-bit OSs
- **Modelling of:**
 - DTMCs, CTMCs, MDPs, PTAs + costs/rewards
- **Model checking of:**
 - PCTL, CSL, LTL, PCTL* + extensions + costs/rewards

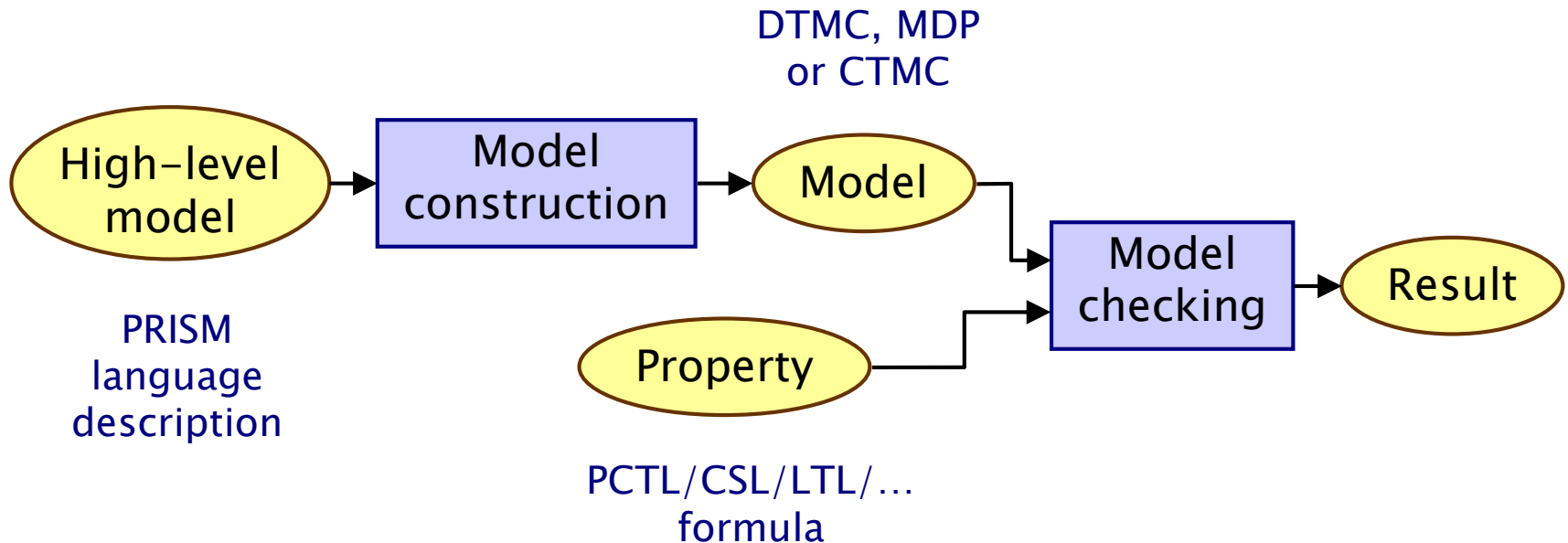


PRISM functionality

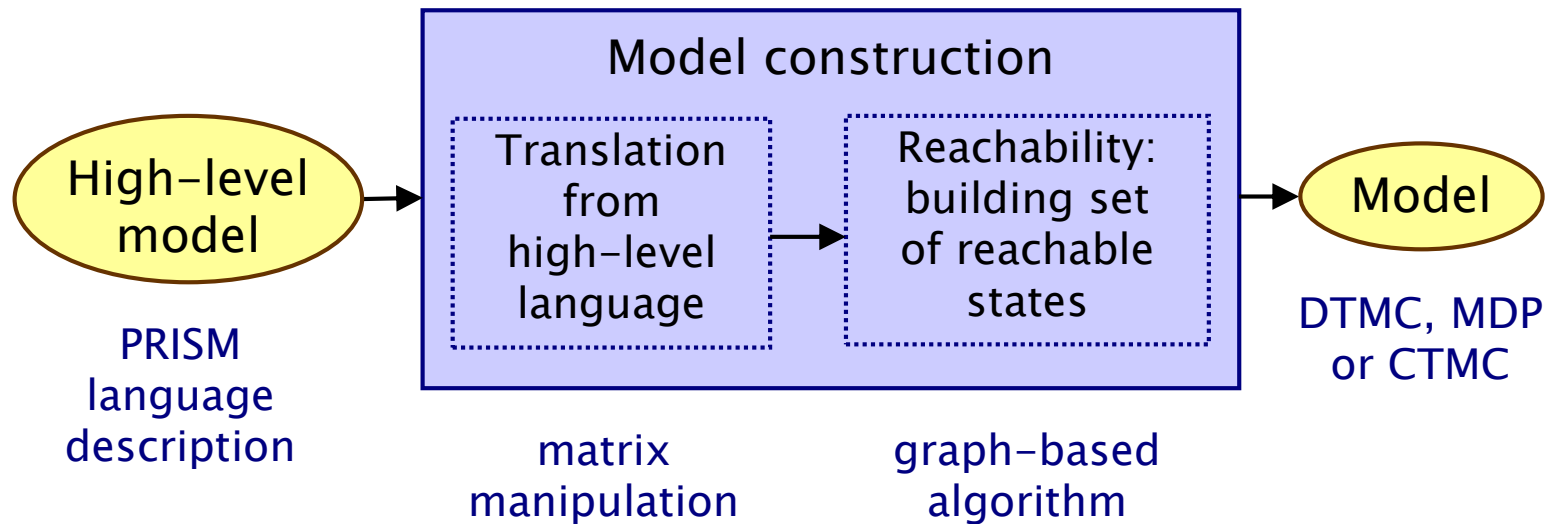
- High-level modelling language
- Wide range of model analysis methods
 - efficient symbolic implementation techniques
 - also: approximate verification using simulation + sampling
- Graphical user interface
 - model/property editor
 - discrete-event simulator – model traces for debugging, etc.
 - easy automation of verification experiments
 - graphical visualisation of results
- Command-line version
 - same underlying verification engines
 - useful for scripting, batch jobs

Probabilistic model checking

- Overview of the probabilistic model checking process
 - two distinct phases: **model construction**, **model checking**



Model construction



Modelling languages/formalisms

- Many high-level modelling languages, formalisms available
- For example:
 - probabilistic/stochastic process algebras (PEPA, Stochastic Pi)
 - stochastic Petri nets
 - stochastic activity networks
- Custom languages for tools, e.g.:
 - PRISM modelling language
 - Probmela (probabilistic variant of Promela, the input language for the model checker SPIN) – used in LiQuor

PRISM modelling language

- Simple, textual, state-based language
 - modelling of DTMCs, CTMCs, MDPs, ...
 - based on Reactive Modules [AH99]
- Basic components...
- Modules:
 - components of system being modelled
 - composed in parallel
- Variables
 - finite (integer ranges or Booleans)
 - local or global
 - all variables public: anyone can read, only owner can modify

PRISM modelling language

- Guarded commands
 - describe behaviour of each module
 - i.e. the changes in state that can occur
 - labelled with probabilities (or, for CTMCs, rates)
 - (optional) action labels

$[send] (s=2) \rightarrow p_{loss} : (s'=3) \& (lost'=lost+1) + (1-p_{loss}) : (s'=4);$



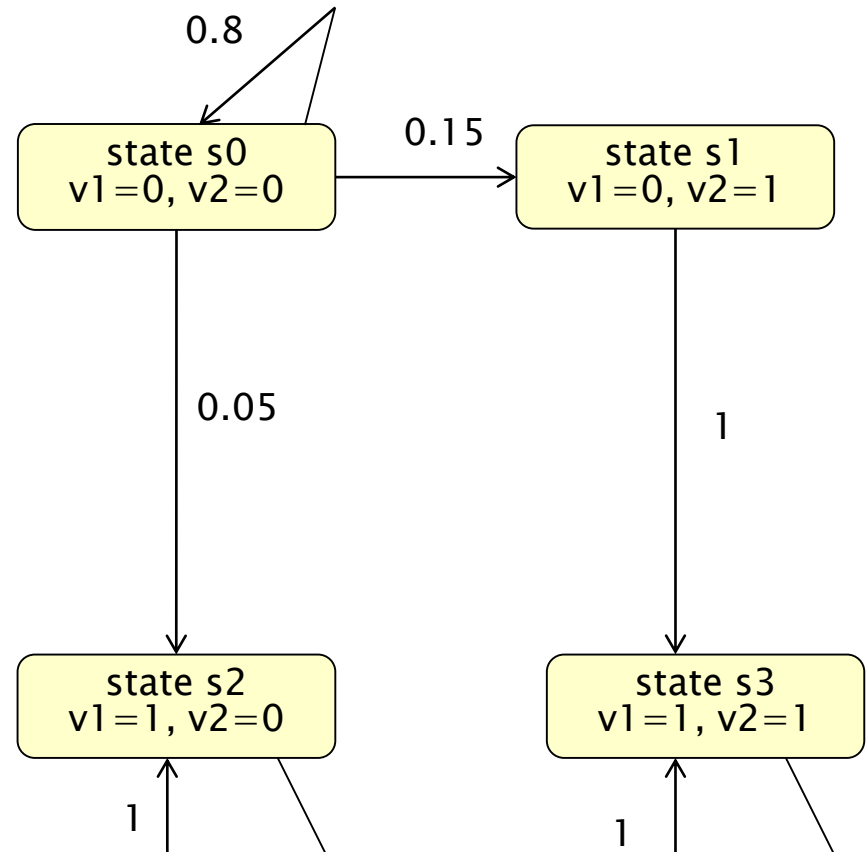
PRISM modelling language

- **Parallel composition**
 - model multiple components that can execute independently
 - for DTMC models, mostly assume components operate synchronously, i.e. move in lock-step
- **Synchronisation**
 - simultaneous transitions in more than one module
 - guarded commands with matching action-labels
 - probability of combined transition is product of individual probabilities for each component
- **More complex parallel compositions can be defined**
 - using process-algebraic operators
 - other types of parallel composition, action hiding/renaming
- **Required reading**
 - **The PRISM Language – Semantics** (available from <http://www.prismmodelchecker.org/doc/semantics.pdf>)

Example: DTMC described in PRISM

```
dtmc
module M1
  v1 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.9:(v1'=0) + 0.1:(v1'=1);
  [a] v1=0 & v2=1 -> 1:(v1'=1);
  [b] v1=1 -> 1:true;
endmodule
module M2
  v2 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.7:(v2'=0) + 0.3:(v2'=1);
  [a] v1=0 & v2=1 -> 1:true;
  [b] v1=1 -> 1:true;
endmodule
```

States and transitions generated by PRISM



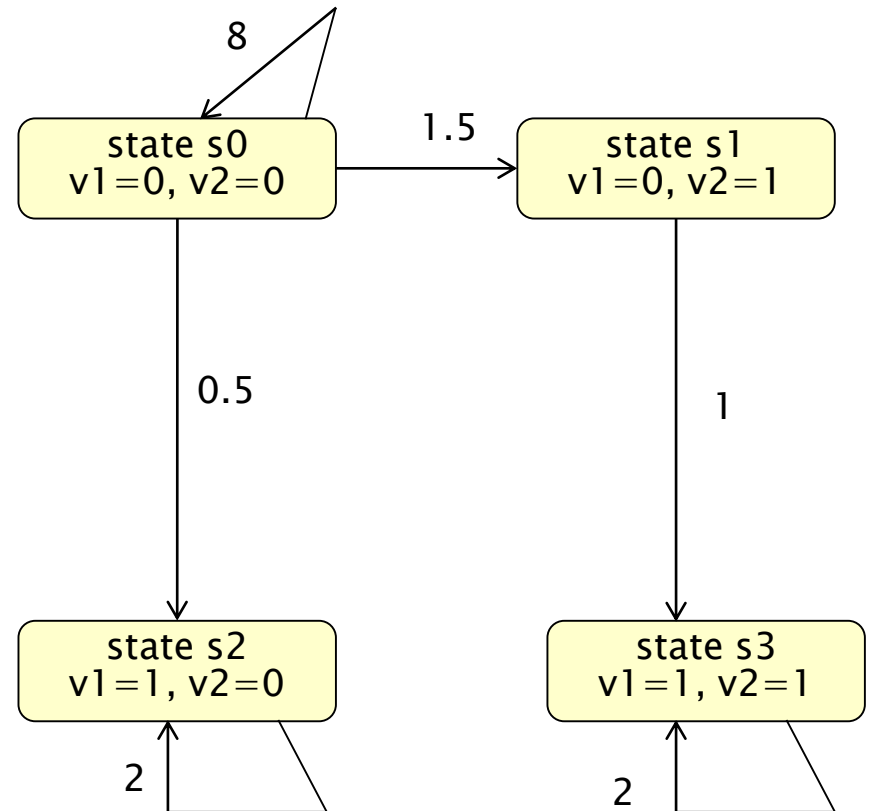
- **true** denotes an update where no variable's value changes
 - For example, in module M1 one could replace **true** with $(v1'=v1)$

- **Remark** Each state has a **probability distribution**. For example, in state s0:
$$0.8 + 0.15 + 0.05 = 1$$

Example: CTMC described in PRISM

```
ctmc
module M1
  v1 : [0..1] init 0;
  [] v1=0 & v2=0 -> 4.5:(v1'=0) + 0.5:(v1'=1);
  [a] v1=0 & v2=1 -> 1:(v1'=1);
  [b] v1=1 -> 1:true;
endmodule
module M2
  v2 : [0..1] init 0;
  [] v1=0 & v2=0 -> 3.5:(v2'=0) + 1.5:(v2'=1);
  [a] v1=0 & v2=1 -> 1:true;
  [b] v1=1 -> 2:true;
endmodule
```

States and transitions generated by PRISM



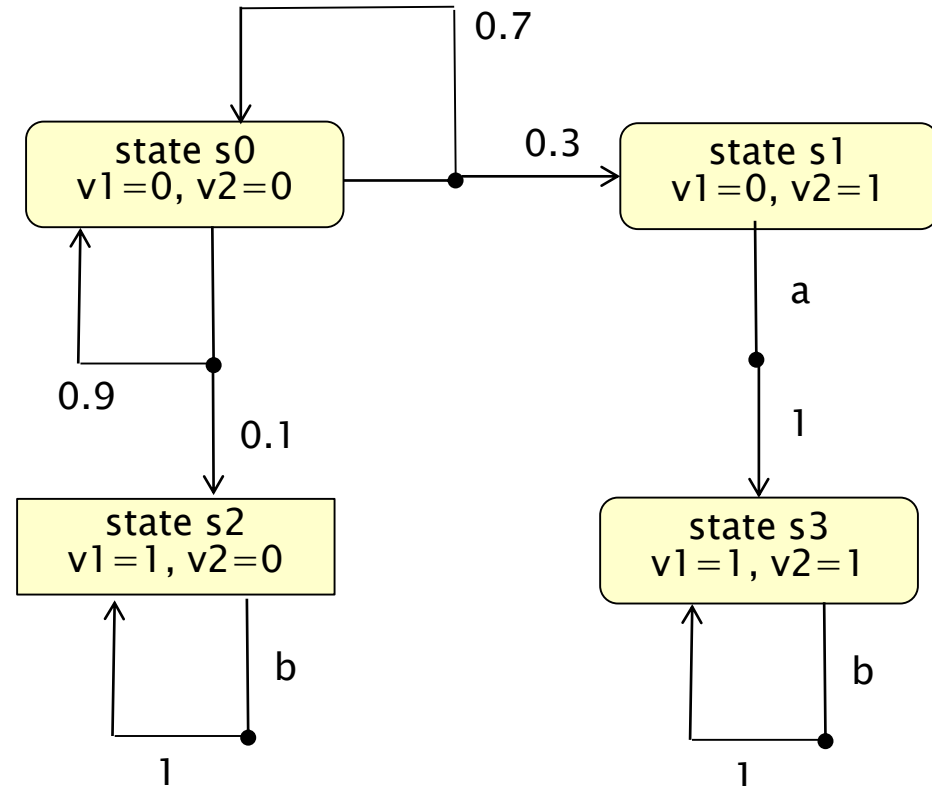
- In a CTMC updates are labeled with **rates** rather than probabilities
 - A rate is a positive real number ($\in \mathbb{R}_{\geq 0}$)
 - Computations proceed in continuous real time
 - Rates are used as parameters of the **exponential distribution**
 - The choice between transitions is modeled as a “race” between alternatives

Example: MDP described in PRISM

```
mdp
module M1
  v1 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.9:(v1'=0) + 0.1:(v1'=1);
  [a] v1=0 & v2=1 -> 1:(v1'=1);
  [b] v1=1 -> 1:true;
endmodule

module M2
  v2 : [0..1] init 0;
  [] v1=0 & v2=0 -> 0.7:(v2'=0) + 0.3:(v2'=1);
  [a] v1=0 & v2=1 -> 1:true;
  [b] v1=1 -> 1:true;
endmodule
```

States and transitions generated by PRISM



- A MDP can model both **probabilistic** and **nondeterministic** behavior
 - Example: the two unlabeled actions (that exit from state s0) are selected in a nondeterministic manner
 - Each action in a MDP has a probability distribution

Discrete-Time Markov Chains

- Formally, a DTMC D is a tuple $(S, s_{\text{init}}, P, L)$ where:
 - S is a set of states (“state space”)
 - $s_{\text{init}} \in S$ is the initial state
 - $P : S \times S \rightarrow [0,1]$ is the **transition probability matrix**
 where $\sum_{s' \in S} P(s, s') = 1$ for all $s \in S$
 - $L : S \rightarrow 2^{\text{AP}}$ is function labelling states with atomic propositions (taken from a set AP)

- Example:**

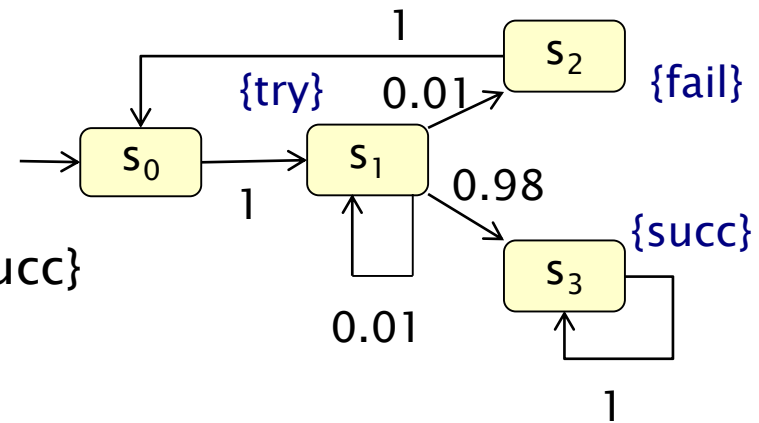
$$D = (S, s_{\text{init}}, P, L)$$

$$S = \{s_0, s_1, s_2, s_3\}$$

$$s_{\text{init}} = s_0$$

$$P = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0.01 & 0.01 & 0.98 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$\text{AP} = \{\text{try}, \text{fail}, \text{succ}\}$
 $L(s_0) = \emptyset,$
 $L(s_1) = \{\text{try}\},$
 $L(s_2) = \{\text{fail}\},$
 $L(s_3) = \{\text{succ}\}$



Remarks

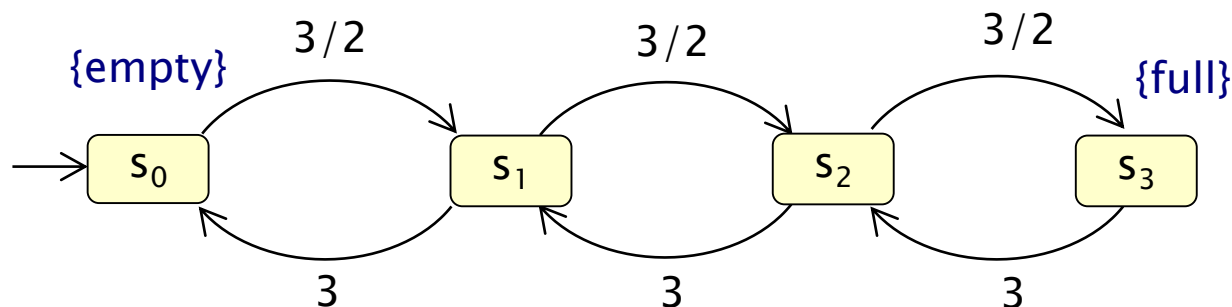
- We omitted atomic propositions in previous examples
- PRISM offers a keyword (**label**) for specifying atomic propositions

Continuous-Time Markov Chains

- Formally, a CTMC C is a tuple $(S, s_{\text{init}}, R, L)$ where:
 - S is a finite set of states (“state space”)
 - $s_{\text{init}} \in S$ is the initial state
 - $R : S \times S \rightarrow \mathbb{R}_{\geq 0}$ is the **transition rate matrix**
 - $L : S \rightarrow 2^{\text{AP}}$ is a labelling with atomic propositions
- Transition rate matrix assigns rates to each pair of states
 - used as a parameter to the **exponential distribution**

Simple CTMC example – modelling a queue of jobs:

- initially the queue is empty, maximum size of the queue is 3
- jobs **arrive** with rate $3/2$ (i.e. mean inter-arrival time is $2/3$)
- jobs are **served** with rate 3 (i.e. mean service time is $1/3$)
- state space: $S = \{s_i\}_{i=0..3}$ where s_i indicates i jobs in queue



Race conditions in CTMCs

- What happens when there exists **multiple** s' with $R(s,s') > 0$?
 - **Race condition**: first transition triggered determines next state. Two questions:
 1. How long is spent in a state s before a transition occurs?
 2. Which transition is eventually taken?

1. Time spent in a state before a transition

- Exponential distribution with parameter given by summation: $E(s) = \sum_{s' \in S} R(s,s')$
- Probability of leaving a state s within $[0,t]$ is: $1 - e^{-E(s) \cdot t}$
- $E(s)$ is the **exit rate** of state s ; s is called **absorbing** if $E(s)=0$ (no outgoing transitions)

2. Which transition is taken from state s ?

- Markov property: the choice is **independent** of the time at which it occurs
- Probability that next state from s is s' given by $P^{\text{emb}(C)}(s,s')$, computed based on
- The **embedded DTMC**: $\text{emb}(C) = (S, s_{\text{init}}, P^{\text{emb}(C)}, L)$

- state space, initial state and labelling as the CTMC

- for any $s, s' \in S$
$$P^{\text{emb}(C)}(s,s') = \begin{cases} R(s,s') / E(s) & \text{if } E(s) > 0 \\ 1 & \text{if } E(s) = 0 \text{ and } s = s' \\ 0 & \text{otherwise} \end{cases}$$

Two interpretations of a CTMC

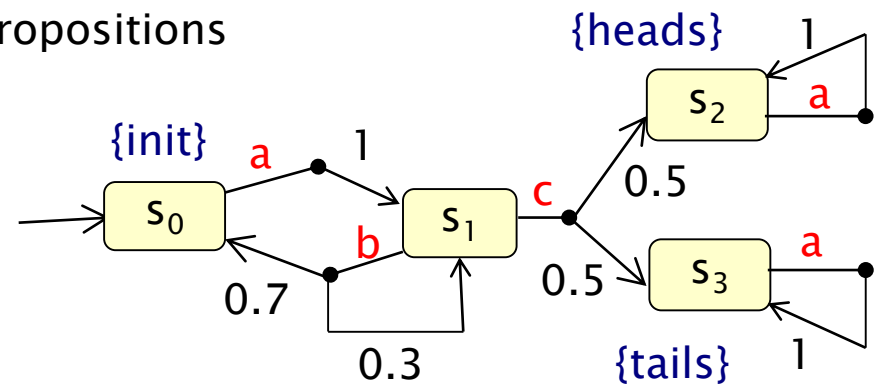
- Consider a (non-absorbing) state $s \in S$ with multiple outgoing transitions, i.e. multiple $s' \in S$ with $R(s,s') > 0$
- 1. Race condition
 - Each transition triggered after exponentially distributed delay
 - i.e. probability triggered before t time units: $1 - e^{-R(s,s') \cdot t}$
 - First transition triggered determines the next state
- 2. Separate delay/transition
 - Remain in s for delay exponentially distributed with rate $E(s)$
 - i.e. probability of taking an outgoing transition from s within $[0,t]$ is given by $1 - e^{-E(s) \cdot t}$
 - Probability that next state is s' is given by $P^{\text{emb}(C)}(s,s')$,
 - i.e., $R(s,s')/E(s) = R(s,s') / \sum_{s' \in S} R(s,s')$

Markov Decision Processes

- Formally, an MDP M is a tuple $(S, s_{\text{init}}, \text{Steps}, L)$ where:
 - S is a finite set of states (“state space”)
 - $s_{\text{init}} \in S$ is the initial state
 - Steps** : $S \rightarrow 2^{\text{Act} \times \text{Dist}(S)}$ is the **transition probability function**
 - Act is a set of actions and $\text{Dist}(S)$ is the set of discrete probability distributions over the set S
 - A (discrete) **probability distribution** over a countable set S is a function $\mu: S \rightarrow [0,1]$ satisfying $\sum_{s \in S} \mu(s) = 1$.
 - Notation:** We use $[s_0 \rightarrow p_0, \dots, s_n \rightarrow p_n]$ to denote the probability distribution that chooses s_i with probability p_i (for $1 \leq i \leq n$)
 - $L : S \rightarrow 2^{\text{AP}}$ is a labelling with atomic propositions

- Notes:**

- Steps(s) is always non-empty, i.e. no deadlocks
- The use of actions to label distributions is optional



PRISM property specification language

- Based on (probabilistic extensions of) temporal logic
 - Incorporates (subsumes) PCTL, CSL, LTL, PCTL* + costs/rewards
- Probabilistic operators P , S and R (see <http://www.prismmodelchecker.org/manual/>)
 - $P \sim p$ [pathprop] ($\in \text{Bool}$)
 - $\sim \in \{>=, >, <=, <\}$, $p \in [0,1]$ is a PRISM double
 - $>=p, >p / <=p, <p$ computes min./max. probabilities for MDPs and PTAs (over all adversaries, i.e. all possible resolutions of nondeterminism)
 - pathprop ::= X prop | prop1 U prop2 | F prop | ...
 - With (upper) “bounded” variants: prop1 $U \leq t$ prop2, $F \leq t$ prop
 - Lower bounds & time intervals: $F [t_1, t_2]$ prop ($t_1, t_2 \in \mathbb{R}_{\geq 0}$, only for CTMCs)
 - $S \sim p$ [prop] ($\in \text{Bool}$), for DTMC and CTMC
 - $R \sim r$ [rewardprop] ($\in \text{Bool}$), $r (\in \mathbb{R}_{\geq 0})$ is a PRISM double
 - rewardprop ::= F prop | $C \leq t$ | $I = t$ | S | ...
- Plus quantitative variants
 - $P=?$ [pathprop], $S=?$ [prop] ($\in [0,1]$) return the actual probability
 - $R=?$ [rewardprop] ($\in \mathbb{R}_{\geq 0}$) returns the actual expected reward
 - $=?$ replaced with $\min=?$ or $\max=?$ for MDPs or PTAs

Case Study: Dynamic Power Management

- Aims:
 - Low power consumption
 - Acceptable level of service
- We consider the PRISM solution available at
<http://www.prismmodelchecker.org/courses/pmc1112/>
<http://www.prismmodelchecker.org/tutorial/>
- PRISM modules:
 1. **Service (request) queue (SQ)**
 - q = current size of SQ
 - $q = 0 \Leftrightarrow$ empty queue
 2. **Service provider (SP)** – serves requests from SQ
 1. Power states: **sleep**, **idle**, **busy**
 3. **Power management (PM)** – simple PM policy:
 - $(q \geq q_trigger) \rightarrow \text{sleep2idle}$
 - $(q=0) \rightarrow \text{idle2sleep}$
- The technical solution is based on (a PM model for a Fujitsu disk drive given in)
 - Q. Qiu, Q. Wu, M. Pedram, “Stochastic modeling of a power-managed system: Construction and optimization”, Proc. International Symposium on Low Power Electronics and Design, pages 194--199, ACM Press, 1999

Case Study: Dynamic Power Management

- Properties specified by using the operators P and S:

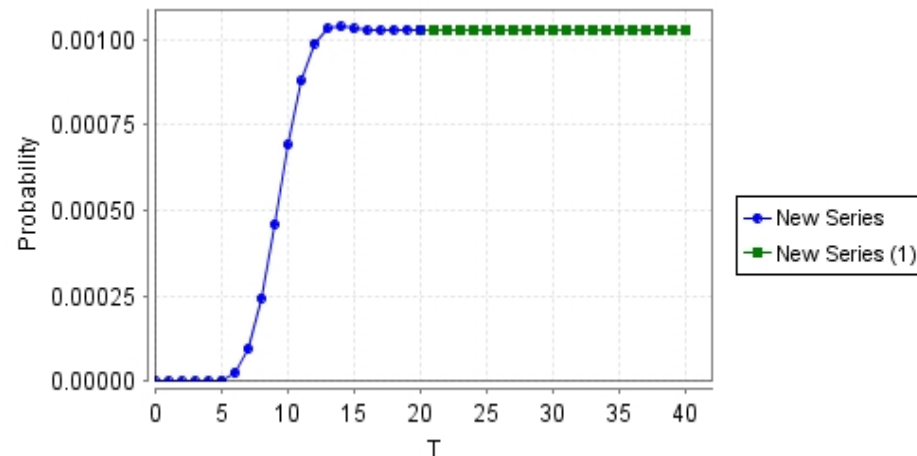
```
const double T;
```

```
P=? [F [T,T] q=q_max] // (transient) probability that q=q_max (queue is full) at time T  
                        // particular case of time interval [t1,t2] with t1=t2 (in general t1 ≤ t2)
```

```
S=? [q=q_max] // (steady state) long run probability that q=q_max
```

- **Experiments:** ranges of parameters, e.g.

- $P=? [F [T,T] q=q_max] \Rightarrow$
 - For $q_trigger=5$
 - $T=0,\dots,20$ (first 20 seconds)
 - Next $T=20,\dots,40$
- $S=? [q=q_max] \Rightarrow 0.001287\dots$
 - $0.001287\dots$ matches the results on the graph
 - Obtained with Gauss–Seidel
 - Jacobi (default method) doesn't converge here
- Identify **patterns**, **trends**, **anomalies** in **quantitative** results



Case Study: Dynamic Power Management

- Operator R can be used to compute expected values based on reward structures
- A reward structure is a list of reward items

```
rewards "reward_label"
```

```
// Assigns reward1 to all states that satisfy guard1
```

```
guard1 : reward1;
```

```
// Assigns reward2 to any transition with action label
```

```
// action2 from a state that satisfies guard2
```

```
[action2] guard2 : reward2;
```

```
endrewards
```

Simple examples:

```
rewards "queue_size"
```

```
true : q;
```

```
endrewards
```

```
rewards "lost"
```

```
[request] q = q_max : 1;
```

```
endrewards
```

- Expected queue size at time T:

```
R{"queue_size"}=? [I=T]
```

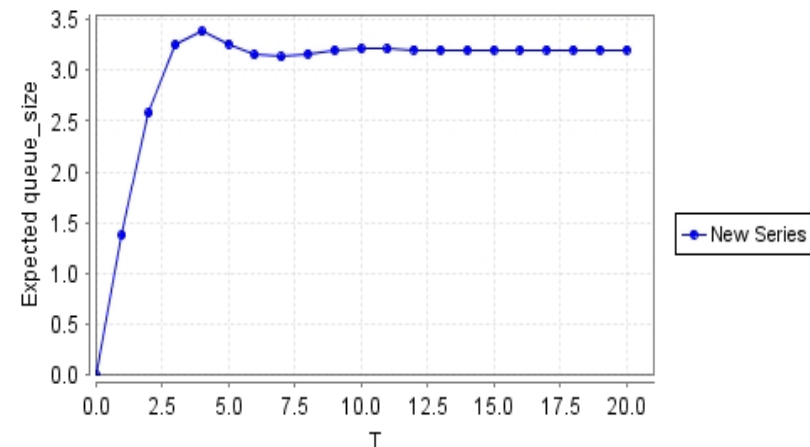
T = 0,...,20 ⇒

- Long run expected (average) queue size:

```
R{"queue_size"}=? [S]
```

⇒ 3.2038...

3.2038... matches the results on the graph



More info on PRISM

- PRISM website: <http://www.prismmodelchecker.org/>
 - tool download: binaries, source code (GPL)
 - on-line example repository (50+ case studies)
 - on-line documentation:
 - PRISM manual
 - <http://www.prismmodelchecker.org/manual/>
 - PRISM tutorial
 - <http://www.prismmodelchecker.org/tutorial/>
 - <http://www.prismmodelchecker.org/courses/pmc1112/>
 - support: help forum, bug tracking, feature requests
 - related publications, talks, tutorials, links

References and recommended reading

1. D. Parker (DP), “Probabilistic Model Checking” – slides for a lecture course taught in the Department of Computer Science at Oxford University (2011)
<http://www.prismmodelchecker.org/lectures/pmc/>
2. Practical course (PRISM tutorial) accompanying [1]
<http://www.prismmodelchecker.org/tutorial/>
3. PRISM manual <http://www.prismmodelchecker.org/manual/>
4. Semantics of the PRISM language, available from
<http://www.prismmodelchecker.org/doc/semantics.pdf>
5. M. Kwiatkowska, G. Norman, D. Parker, “Stochastic model checking”, 2007
6. V. Forejt, M. Kwiatkowska¹, G. Norman, D. Parker, “Automated Verification Techniques for Probabilistic Systems”, 2011
7. G. Norman, D. Parker, J. Sproston, “Model Checking for Probabilistic Timed Automata”, 2013
8. M. Kwiatkowska, G. Norman, R. Segala, J. Sproston, “Automatic Verification of Real-time Systems with Discrete Probability Distributions”, 2002.
9. M. Kwiatkowska, G. Norman, J. Sproston, F. Wang, “Symbolic Model Checking for Probabilistic Timed Automata”, 2007
10. R. Alur, D. Dill, “A Theory of Timed Automata”, Theoretical Computer Science 126:183–235, 1994.

Discrete-time Markov Chains

Dr. Dave Parker



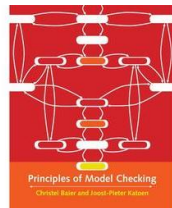
Department of Computer Science
University of Oxford

Selection of slides based on Lecture 2 (Discrete-time Markov Chains) from:

- Dave Parker (DP), “Probabilistic Model Checking”, slides for a lecture course taught in the Department of Computer Science at Oxford University (2011) <http://www.prismmodelchecker.org/lectures/pmc/>

Probabilistic Model Checking

- Formal verification of systems that exhibit probabilistic behaviour
 - e.g. systems with failures/unreliability or randomised algorithms/protocols
- PRISM model checker provides probabilistic operators that take as parameters state and path properties
- In this lecture the concept of a path is introduced formally in the context of Discrete-time Markov chains (DTMCs)
 - The concept of a **path** is employed in all types of model supported by PRISM: DTMCs, CTMCs, MDPs and PTAs
- **Overview** of this lecture
 - Probability basics
 - **Discrete-time Markov chains** (DTMCs: definition, properties, examples)
 - **Formalizing path-based properties** of DTMCs
 - Probability space over infinite paths
- Sources/further reading:



Principles of Model Checking
Christel Baier and Joost-Pieter Katoen
MIT Press, 2008

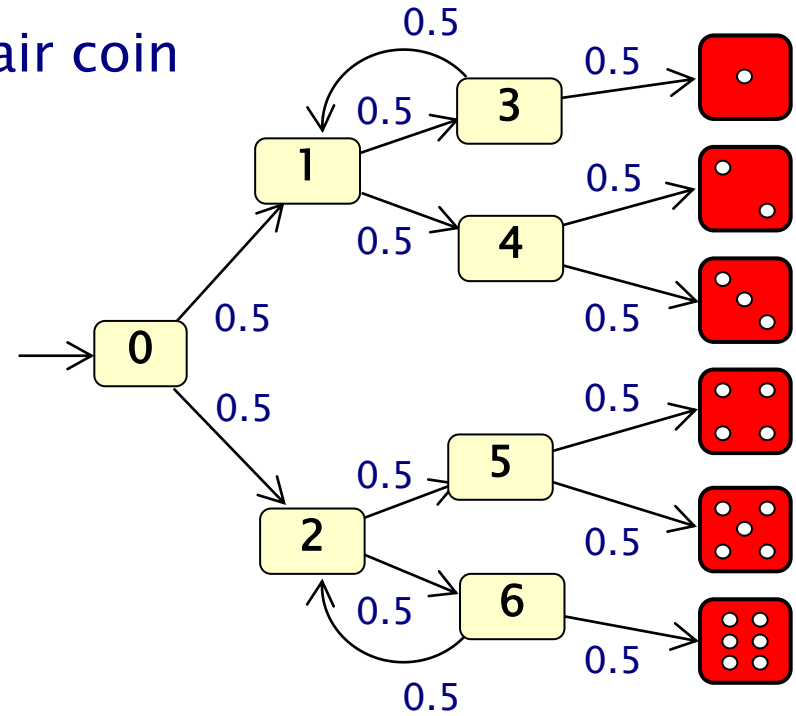
Probability basics

- First, need an experiment
 - The **sample space** Ω is the set of possible outcomes
 - An **event** is a subset of Ω , can form events $A \cap B$, $A \cup B$, $\Omega \setminus A$
- Examples:
 - toss a coin: $\Omega = \{H, T\}$, events: “H” (head), “T” (tail)
 - toss two coins: $\Omega = \{(H, H), (H, T), (T, H), (T, T)\}$,
event: “at least one H”
 - toss a coin ∞ -often: Ω is set of infinite sequences of H/T
event: “H in (either of) the first 3 throws”
- Probability is:
 - $\Pr(\text{“H”}) = \Pr(\text{“T”}) = 1/2$, $\Pr(\text{“at least one H”}) = 3/4$
 - $\Pr(\text{“H in the first 3 throws”}) = 1/2 + 1/4 + 1/8 = 7/8$

Probability example

- **Modelling a 6-sided die using a fair coin**

- algorithm due to Knuth/Yao:
- start at 0, toss a coin
- upper branch when H
- lower branch when T
- repeat until value chosen



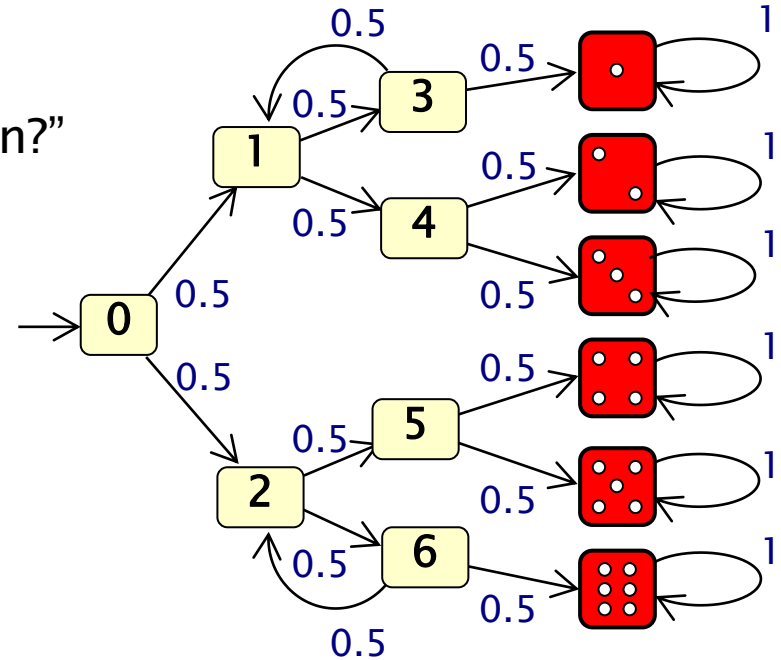
- **Is this algorithm correct?**

- e.g. probability of obtaining a 4?
- Obtain as disjoint union of events
- Execution paths for (event) “eventually 4”: THH, TTTHH, TTTTTHH, ...
- $\Pr(\text{“eventually 4”}) = (1/2)^3 + (1/2)^5 + (1/2)^7 + \dots = 1/6$

- PRISM specification available at <http://www.prismmodelchecker.org/tutorial/die.php>

Example...

- Other properties?
 - “what is the probability of termination?”
- e.g. efficiency?
 - “what is the probability of needing more than 4 coin tosses?”
 - “on average, how many coin tosses are needed?”
- Probabilistic model checking provides a framework for these kinds of properties...
 - modelling languages
 - property specification languages
 - model checking algorithms, techniques and tools



Discrete-time Markov chains

- State-transition systems augmented with probabilities

- States

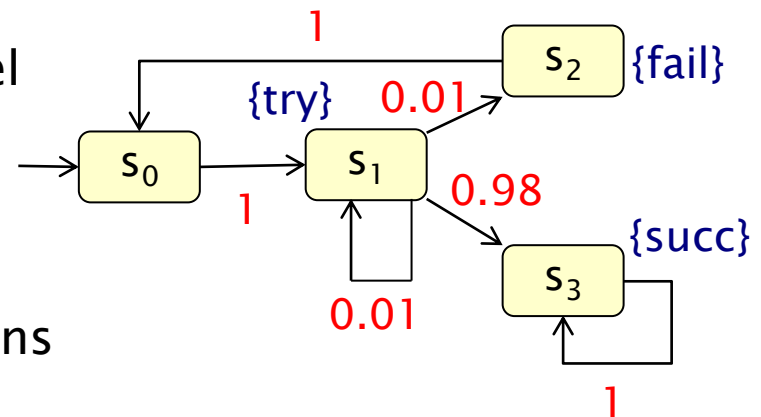
- **set of states** representing possible configurations of the system being modelled

- Transitions

- transitions between states model evolution of system's state; occur in **discrete time-steps**

- Probabilities

- probabilities of making transitions between states are given by **discrete probability distributions**

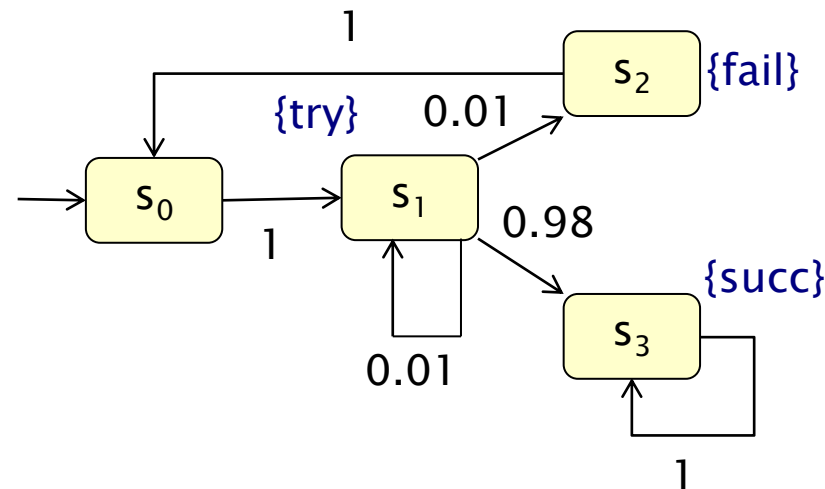


Markov property

- If the current state is known, then the future states of the system are independent of its past states
- i.e. the current state of the model contains all information that can influence the future evolution of the system
- also known as “memorylessness”

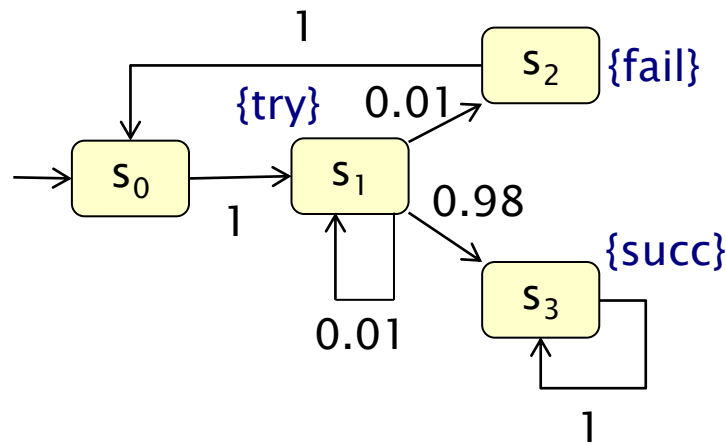
Simple DTMC example

- Modelling a very simple communication protocol
 - after one step, process starts **trying** to send a message
 - with probability 0.01, channel unready so wait a step
 - with probability 0.98, send message **successfully** and stop
 - with probability 0.01, message sending **fails**, restart



Discrete-time Markov chains

- Formally, a DTMC D is a tuple $(S, s_{\text{init}}, P, L)$ where:
 - S is a set of states (“state space”)
 - $s_{\text{init}} \in S$ is the initial state
 - $P : S \times S \rightarrow [0,1]$ is the **transition probability matrix** where $\sum_{s' \in S} P(s, s') = 1$ for all $s \in S$
 - $L : S \rightarrow 2^{\text{AP}}$ is function labelling states with atomic propositions (taken from a set AP)



Simple DTMC example

$$D = (S, s_{\text{init}}, P, L)$$

$$S = \{s_0, s_1, s_2, s_3\}$$

$$s_{\text{init}} = s_0$$

$$AP = \{\text{try}, \text{fail}, \text{succ}\}$$

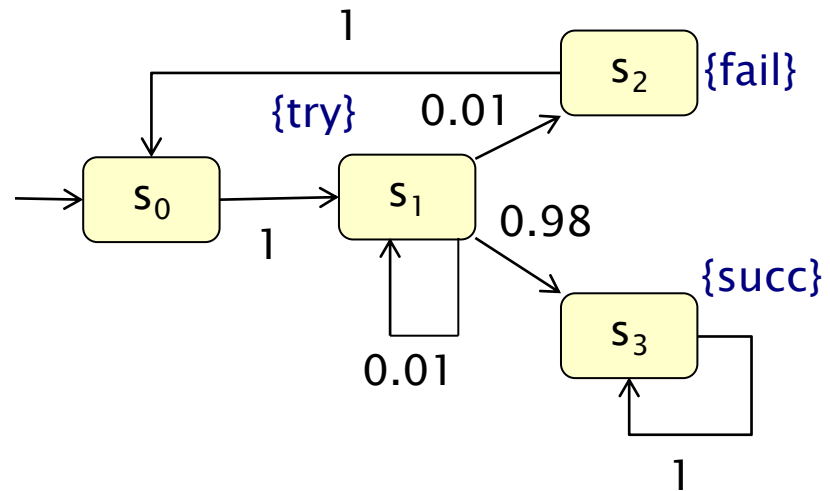
$$L(s_0) = \emptyset,$$

$$L(s_1) = \{\text{try}\},$$

$$L(s_2) = \{\text{fail}\},$$

$$L(s_3) = \{\text{succ}\}$$

$$P = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0.01 & 0.01 & 0.98 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Some more terminology

- **P** is a **stochastic** matrix, meaning it satisfies:
 - $P(s,s') \in [0,1]$ for all $s,s' \in S$ and $\sum_{s' \in S} P(s,s') = 1$ for all $s \in S$
- An **absorbing state** is a state s for which:
 - $P(s,s) = 1$ and $P(s,s') = 0$ for all $s \neq s'$
 - the transition from s to itself is sometimes called a **self-loop**
- **Note:** Since we assume **P** is stochastic...
 - every state has at least one outgoing transition
 - i.e. no **deadlocks** (in model checking terminology)

Other assumptions made here

- We consider **time-homogenous** DTMCs
 - transition probabilities are independent of time
- We will (mostly) assume that the state space S is **finite**
 - in general, S can be any countable set
- Initial state $s_{\text{init}} \in S$ can be generalised...
 - to an initial probability distribution $s_{\text{init}} : S \rightarrow [0,1]$
- Transition probabilities are reals: $P(s,s') \in [0,1]$
 - but for algorithmic purposes, are assumed to be rationals

Properties of DTMCs

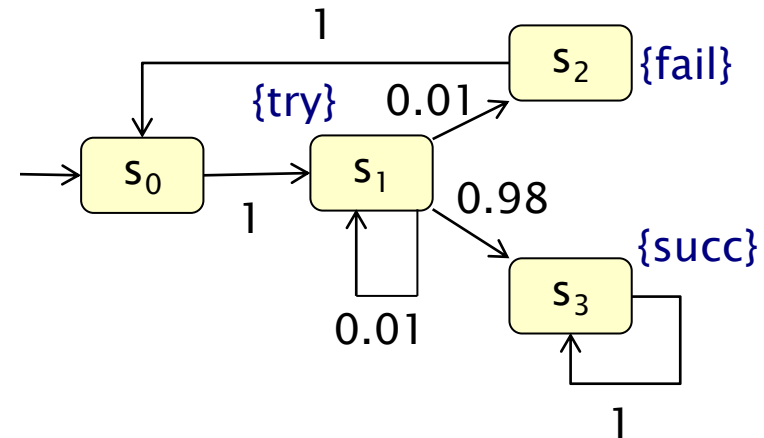
- Path-based properties
 - what is the probability of observing a particular behaviour (or class of behaviours)?
 - e.g. “what is the probability of throwing a 4?”
- Transient properties
 - probability of being in state s after t steps?
- Steady-state
 - long-run probability of being in each state
- Expectations
 - e.g. “what is the average number of coin tosses required?”

DTMCs and paths

- A **path** in a DTMC represents an **execution** (i.e. one possible behaviour) of the system being modelled

- **Formally:**

- infinite sequence of states $s_0s_1s_2s_3\dots$ such that $P(s_i, s_{i+1}) > 0 \ \forall i \geq 0$
- infinite unfolding of DTMC



- **Examples:**

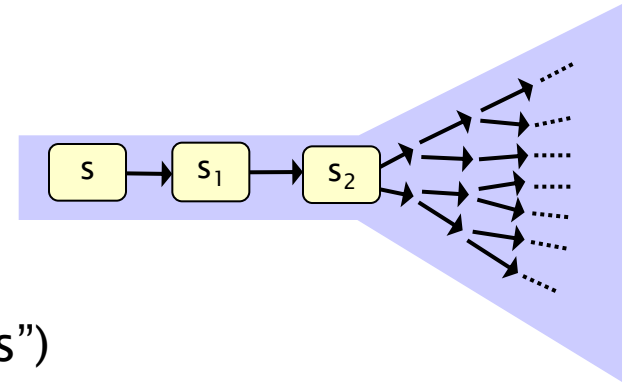
- never succeeds: $(s_0s_1s_2)^\omega$
- tries, waits, fails, retries, succeeds: $s_0s_1s_1s_2s_0s_1(s_3)^\omega$

- **Notation:**

- **Path**(s) = set of all infinite paths starting in state s
- also sometimes use finite (length) paths
- **Path_{fin}**(s) = set of all finite paths starting in state s

Paths and probabilities

- To reason (quantitatively) about this system
 - need to define a **probability space over paths**
- Intuitively:
 - sample space: $\text{Path}(s)$ = set of all infinite paths from a state s
 - events: sets of infinite paths from s
 - basic events: **cylinder sets** (or “cones”)
 - cylinder set $\text{Cyl}(\omega)$, for a finite path ω
= set of **infinite paths with the common finite prefix ω**
 - for example: $\text{Cyl}(ss_1s_2)$



Probability spaces

- Let Ω be an arbitrary non-empty set
- A σ -algebra (or σ -field) on Ω is a family Σ of subsets of Ω ($\Sigma \subseteq 2^\Omega$) closed under complementation and countable union, i.e.:
 - if $A \in \Sigma$, the complement $\Omega \setminus A$ is in Σ
 - if $A_i \in \Sigma$ for $i \in \mathbb{N}$, the union $\bigcup_i A_i$ is in Σ
 - the empty set $\emptyset$ is in Σ
- Elements of Σ are called **measurable sets** or **events**
- Theorem: For any family F of subsets of Ω , there exists a unique smallest σ -algebra on Ω containing F

Probability spaces

- A **probability space** is a triple $(\Omega, \Sigma, \text{Pr})$ where
 - Ω is the sample space
 - Σ is the set of events: $\Sigma (\subseteq 2^\Omega)$ is a σ -algebra on Ω
 - $\text{Pr} : \Sigma \rightarrow [0,1]$ is the **probability measure**:
 - $\text{Pr}(\Omega) = 1$, and
 - $\text{Pr}(\cup_i A_i) = \sum_i \text{Pr}(A_i)$, for all countable sequences of pairwise disjoint events $A_1, A_2, \dots$ ($A_i \in \Sigma, \forall i \in \mathbb{N}, A_i \cap A_j = \emptyset, \forall i \neq j$)

Probability space – Simple example

- Sample space Ω
 - $\Omega = \{1,2,3\}$
- Event set Σ
 - e.g. powerset of Ω
 - $\Sigma = \{ \emptyset, \{1\}, \{2\}, \{3\}, \{1,2\}, \{1,3\}, \{2,3\}, \{1,2,3\} \}$
 - (closed under complement/countable union, contains $\emptyset$)
- Probability measure $\Pr$
 - e.g. $\Pr(\{1\}) = \Pr(\{2\}) = \Pr(\{3\}) = 1/3$
 - $\Pr(\{1,2\}) = 1/3 + 1/3 = 2/3$, etc.

Probability space – Simple example 2

- Sample space Ω
 - $\Omega = \mathbb{N} = \{ 0, 1, 2, 3, 4, \dots \}$
- Event set Σ
 - e.g. $\Sigma = \{ \emptyset, \text{"odd"}, \text{"even"}, \mathbb{N} \}$
 - (closed under complement/countable union, contains $\emptyset$)
- Probability measure $\Pr$
 - e.g. $\Pr(\text{"odd"}) = 0.5, \Pr(\text{"even"}) = 0.5$

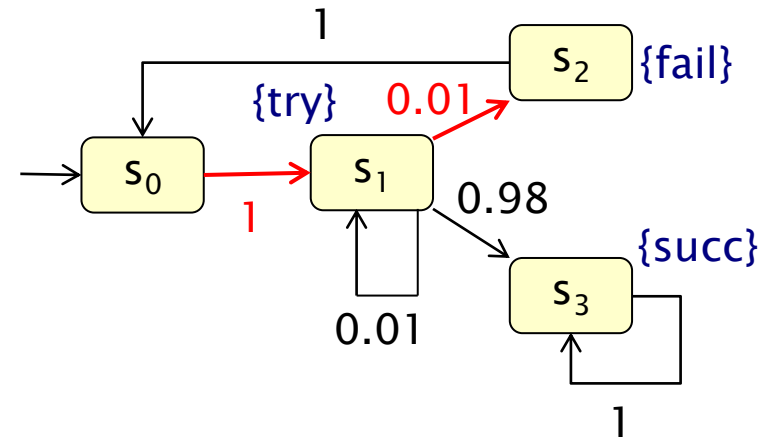
Probability space over paths

- Sample space $\Omega = \text{Path}(s)$ (=set of infinite paths with initial state s)
- Event set $\Sigma_{\text{Path}(s)}$
 - For any finite path ω the **cylinder set** $\text{Cyl}(\omega)$ is
$$\text{Cyl}(\omega) = \{ \omega' \in \text{Path}(s) \mid \omega \text{ is prefix of } \omega' \}$$
 - $\Sigma_{\text{Path}(s)}$ is the **least σ -algebra** on $\text{Path}(s)$ containing $\text{Cyl}(\omega)$ for all finite paths ω starting in s
- Probability measure Pr_s
 - define probability $\text{P}_s(\omega)$ for finite path $\omega = ss_1 \dots s_n$ as:
 - $\text{P}_s(\omega) = 1$ if ω has length one (i.e. $\omega = s$)
 - $\text{P}_s(\omega) = \text{P}(s, s_1) \cdot \dots \cdot \text{P}(s_{n-1}, s_n)$ otherwise
 - define $\text{Pr}_s(\text{Cyl}(\omega)) = \text{P}_s(\omega)$ for any finite path ω
 - Pr_s extends **uniquely** to a probability measure $\text{Pr}_s: \Sigma_{\text{Path}(s)} \rightarrow [0, 1]$
- For further details see
J. Kemeny, J. Snell, A. Knapp. *Denumerable Markov Chains*. Springer, 2nd edition, 1976.

Paths and probabilities – Example

- Paths where sending fails immediately

- $\omega = s_0s_1s_2$
- $\text{Cyl}(\omega) = \text{all paths starting } s_0s_1s_2\dots$
- $\mathbf{P}_{s_0}(\omega) = \mathbf{P}(s_0, s_1) \cdot \mathbf{P}(s_1, s_2)$
 $= \mathbf{1} \cdot \mathbf{0.01} = \mathbf{0.01}$
- $\mathbf{Pr}_{s_0}(\text{Cyl}(\omega)) = \mathbf{P}_{s_0}(\omega) = 0.01$



- Paths which are eventually successful and with no failures

- $\text{Cyl}(s_0s_1s_3) \cup \text{Cyl}(s_0s_1s_1s_3) \cup \text{Cyl}(s_0s_1s_1s_1s_3) \cup \dots$
- $\mathbf{Pr}_{s_0}(\text{Cyl}(s_0s_1s_3) \cup \text{Cyl}(s_0s_1s_1s_3) \cup \text{Cyl}(s_0s_1s_1s_1s_3) \cup \dots)$
 $= \mathbf{P}_{s_0}(s_0s_1s_3) + \mathbf{P}_{s_0}(s_0s_1s_1s_3) + \mathbf{P}_{s_0}(s_0s_1s_1s_1s_3) + \dots$
 $= 1 \cdot 0.98 + 1 \cdot 0.01 \cdot 0.98 + 1 \cdot 0.01 \cdot 0.01 \cdot 0.98 + \dots$
 $= 0.9898989898\dots$
 $= 98/99$

Final Remarks

- Discrete-time Markov chains (DTMCs)
 - State-transition systems augmented with probabilities
- Path-based properties of DTMCs
 - Formalized by defining a probability space over infinite paths
- The concept of a **path** is employed in all types of model supported by PRISM: DTMCs, CTMCs, MDPs and PTAs
- **PRISM** is a tool for quantitative modeling and analysis
 - It provides **probabilistic operators**
 - The parameters are state, **path** and **reward properties**
 - The tool can compute (mean or) **expected values of random variables**
 - Defined based on **reward structures**
 - Example: expected reward/cost cumulated in k steps – the number of steps is counted along (execution) **paths**



Modelare cu clase

Adapted from:
Timothy Lethbridge and Robert Laganieri,
Object-Oriented Software Engineering –
Practical Software Development using UML and Java, 2005
(chapter 5)

5.1 Ce este UML?

UML (engl. *Unified Modelling Language*) este un limbaj grafic pentru modelarea sistemelor software orientate pe obiecte.

- La sfarsitul anilor '80 si inceputul anilor '90 au aparut primele metodologii de dezvoltare orientata pe obiecte (OO).
 - Proliferarea metodelor si notatiilor OO a condus la confuzie in industria software.
 - In scurt timp s-a simtit necesitatea unei unificari si chiar a unei standardizari in acest domeniu.
- UML a fost dezvoltat incepand cu 1994, fiind in principal rezultatul eforturilor conjugate a trei persoane: Grady Booch, James Rumbaugh si Ivar Jacobson [BRJ99,JBR99,RJB99].
 - In 1997, OMG (Object Management Group) a inceput procesul de standardizare a UML.

Diagrame UML

- Diagrame de clase
 - Vizualizeaza clase si relatii intre clase
- Diagrame de obiecte
 - Vizualizeaza colectii de obiecte si conexiuni intre obiecte
- Diagrame de cazuri de utilizare
 - Vizualizeaza cazuri de utilizare, actori si relatii intre acestea
- Diagrame de interactiune (secventiere sau comunicare)
 - Vizualizeaza comportamentul aratand modul in care interactioneaza obiectele unui sistem
- Diagrame de stare si diagrame de activitate
 - Vizualizeaza comportamentul (intern al) unui sistem
- Diagrame de componente si diagrame de desfasurare
 - Vizualizeaza configuratia logica si fizica a componentelor de calcul

Modelare

Un model ar trebui

- Sa utilizeze o notatie standard
- Sa fie usor de inteles de catre toti participantii la proiect
- Sa poata fi exprimat la un nivel de abstractizare adecvat

Modelarea faciliteaza

- Vizualizarea si documentarea sistemului
- Specificarea structurii si comportamentului
- Construirea de schite de proiectare

Limbajul UML furnizeaza o notatie standard pentru reprezentarea structurii si comportamentului sistemelor OO.

5.2 Diagrame de clase UML

Principalele simboluri utilizate in diagramele de clase:

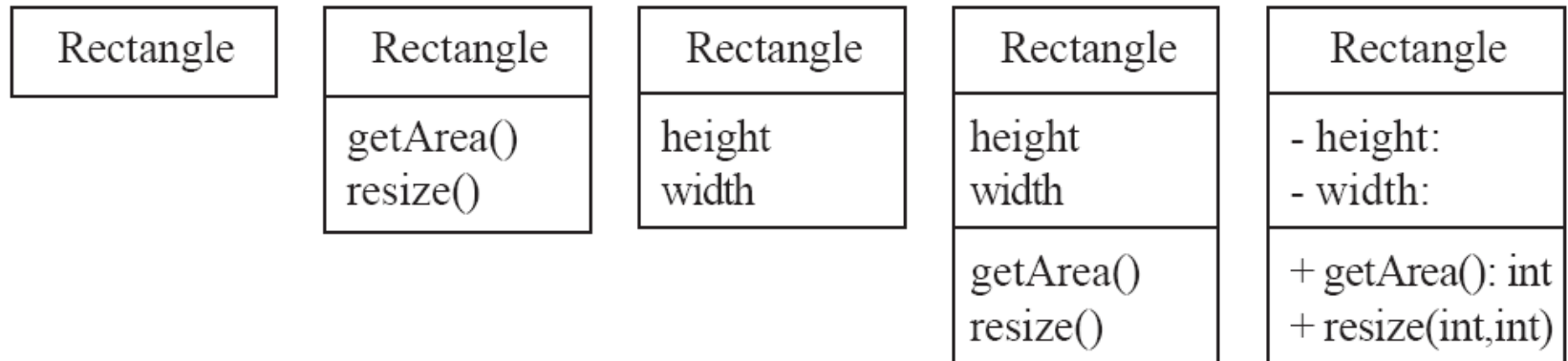
- Clase
 - Reprezinta tipurile obiectelor
- Asocieri
 - Descriu tipul conexiunilor intre obiecte
- Atribute
 - Reprezinta date simple din clase si obiecte
- Operatii
 - Reprezinta functii efectuate de clase si obiecte
- Generalizari
 - Grupeaza clasele in ierarhii de generalizare / specializare

Clase

Grafic, o clasa este reprezentata ca un dreptunghi, incluzand de obicei numele clasei

- Diagrama mai poate vizualiza attributele si operatiile clasei
- Semnatura completa a unei operatii este:

numeOperatie(numParametru : tipParametru ...): tipReturnat



Atribute si operatii

Optiunile de vizibilitate pentru atribute si operatii sunt urmatoarele:

- private (-)
- public (+)
- protected (#)
- pachet sau implementare (nici un simbol)

■ Numele claselor si operatiilor abstracte sunt redade in *italics*.

ClassName

- privateAttr : int = 0
protectedAttr : String
+ publicAttr : Long
implAttr : String

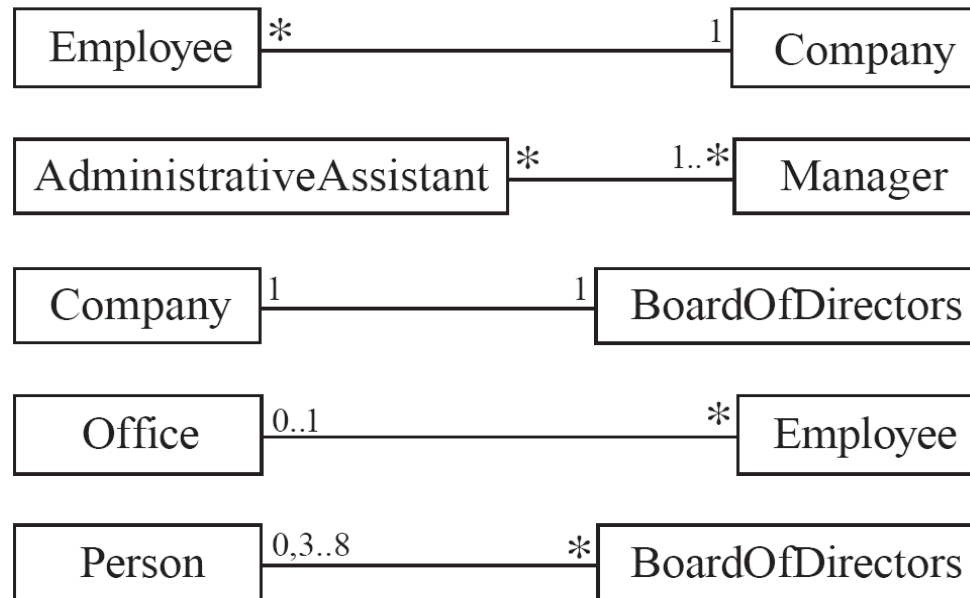
op1(param : String)
+ op2()

AbstractClassName

5.3 Asociere

O asociere este o relatie structurala, care specifica faptul ca obiecte de un anumit tip sunt conectate la obiecte de un alt tip [BRJ99].

- O asociere este reprezentata ca o linie, posibil orientata
 - La fiecare capat al unei asocieri se poate specifica multiplicitatea
 - Este posibil, desi *nu este recomandat*, ca multiplicitatea sa ramana nedefinita.



Etichetarea asocierilor

- Fiecare asociere poate fi etichetata, pentru a face explicita natura asocierii.

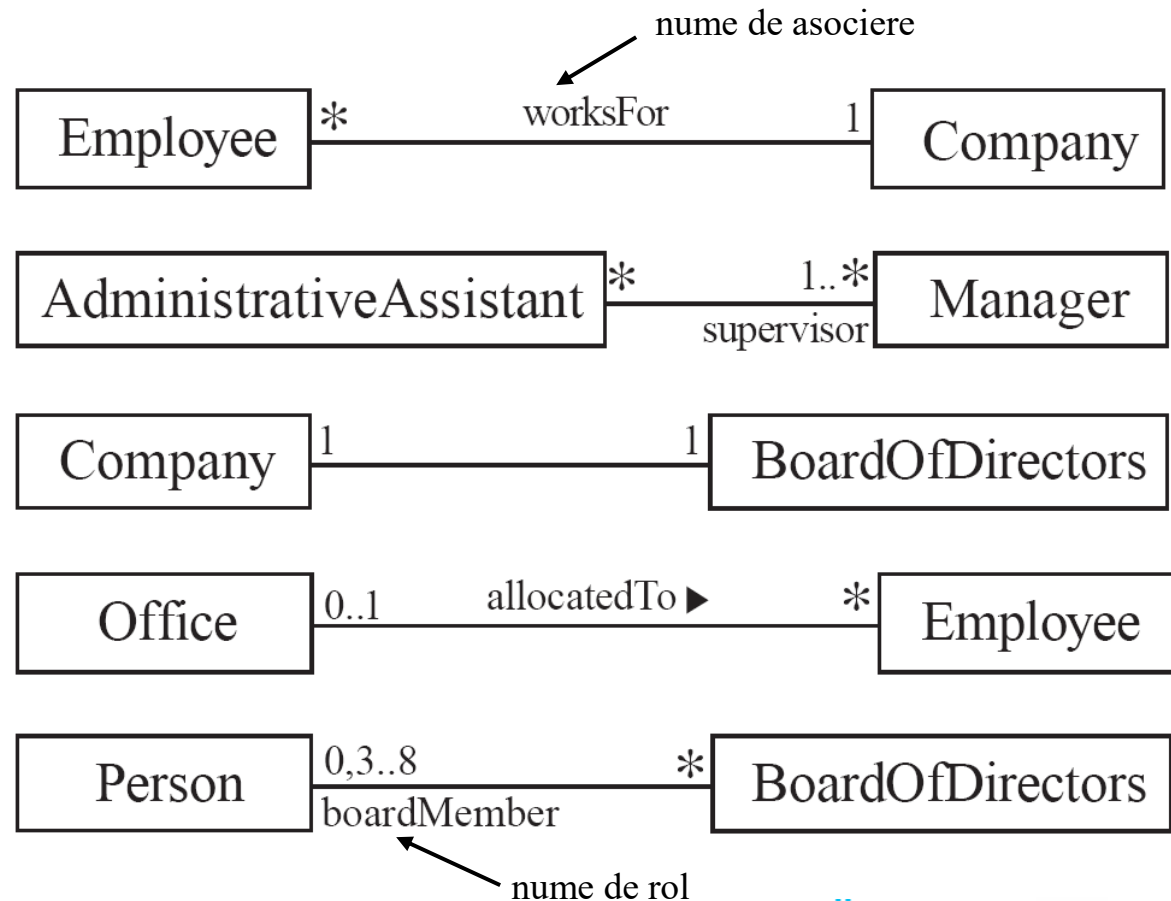
Exista doua tipuri de etichete (optionale) :

- nume de asocieri**

- plasate in mijlocul liniei care reprezinta asocierea
- exprimate prin verbe (sau expresii verbale)
- simbolul '►' poate clarifica 'directia' unei asocieri

- nume de roluri**

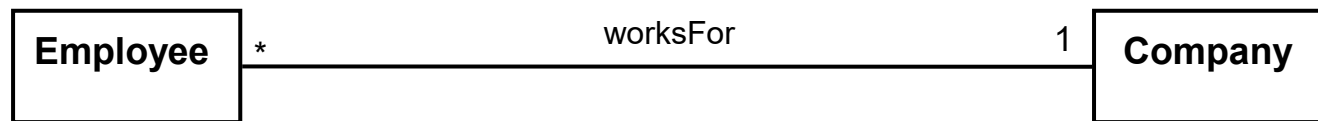
- plasate la unul sau la ambele capete ale unei asocieri



Analiza si validarea asocierilor

- **Relatii de tip 1-N**

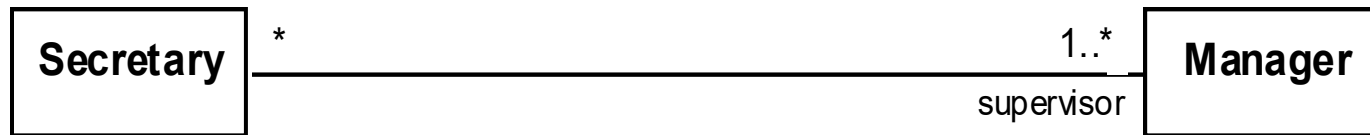
- O companie poate avea mai multi angajati
- Un angajat poate lucra pentru o singura companie
 - Nu se inregistreaza date despre alte activitati ale angajatilor



Analiza si validarea asocierilor

- **Relatii de tip M-N**

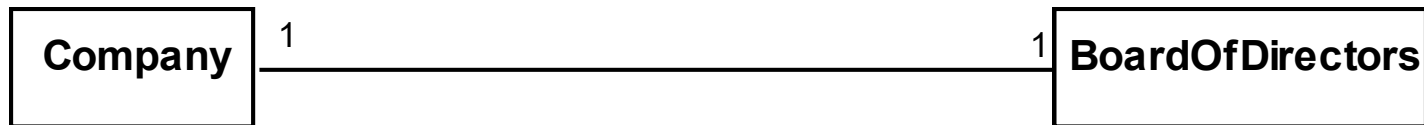
- O secretara poate lucra pentru mai multi manageri
 - Fiecare secretara are cel putin un manager
- Un manager poate avea zero sau mai multe secretare



Analiza si validarea asocierilor

- **Relatii de tip 1-1 (mai putin comune)**

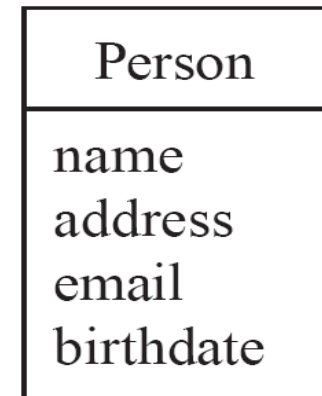
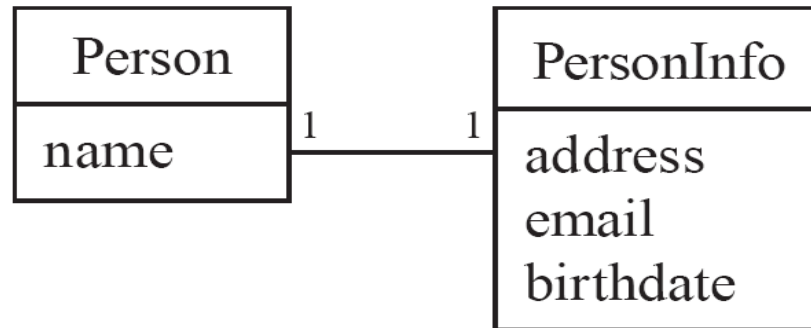
—Pentru fiecare companie (Company) exista exact un consiliu de administratie (BoardOfDirectors).



Analiza si validarea asocierilor

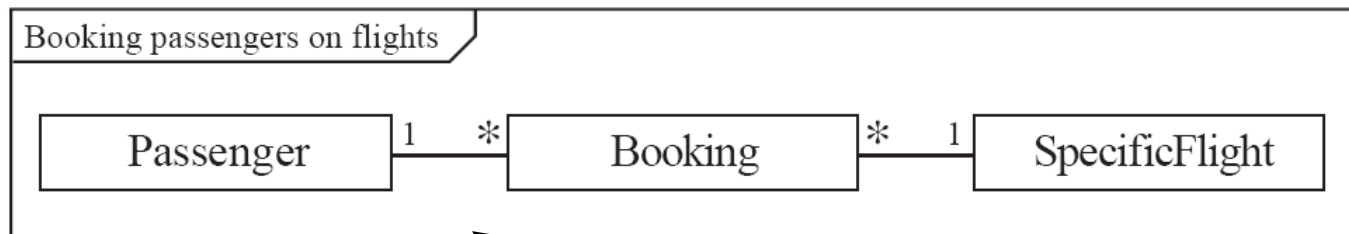
Evitarea asocierilor inutile de tip 1-1.

- Relatie 1-1 nepotrivita (in partea stanga)
- Solutia corecta este data in partea dreapta



Un exemplu mai complex

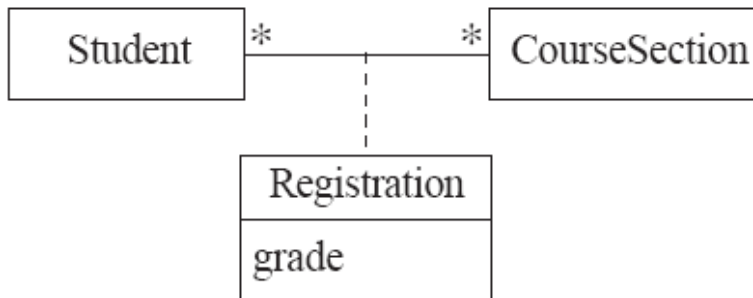
- O rezervare (Booking) este intotdeauna pentru exact un pasager.
- Un pasager poate avea zero sau mai multe rezervari.



- Aceasta diagrama este inclusa intr-un **cadru** (engl. *frame*) cu **eticheta** 'Booking passengers on flights'
 - Aceasta este o facilitate UML 2.0
 - Orice diagrama UML 2.0 poate utiliza cadre

Clase de asociere

- Uneori, un atribut care priveste doua clase asociate nu poate fi plasat in nici una dintre clase. Solutia este sa se creeze o *clasa de asociere* (de exemplu **Registration**) care sa pastreze respectivul atribut (de exemplu **grade**).
- Cele doua diagrame date mai jos sunt semantic echivalente:



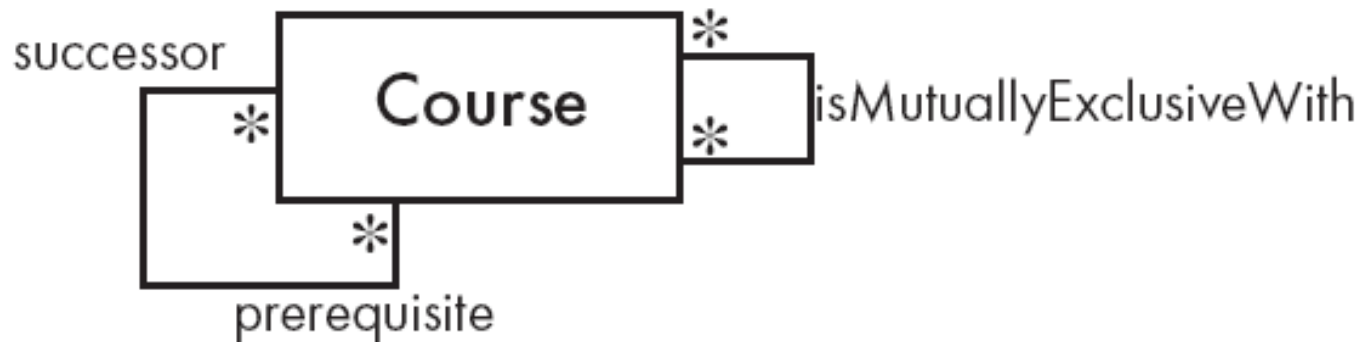
Registration este o *clasa de asociere*.

Exceptand faptul ca este atasata unei asocieri, o clasa de asociere nu difera cu nimic de orice alta clasa.



Asocieri reflexive

- O relatie de asociere poate fi reflexiva (se pot transmite mesaje intre instante ale aceleiasi clase).
 - Un curs poate necesita ca alte cursuri sa fi fost urmate inainte ca o conditie esentiala (engl. *prerequisite*).
 - Doua cursuri sunt mutual exclusive daca acopera (aproape) acelasi material.



Asocieri unidirectionale

- Asocierile sunt implicit *bidirectionale*.
- Este posibil sa se limiteze navigabilitatea unei asocieri prin adaugarea unei sageti la unul dintre capete; asocierea devine *unidirectionala*.
 - De exemplu, diagrama data mai jos este proiectata pe baza urmatoarelor presupuneri:
 - utilizatorul poate asocia oricarei zile oricate note;
 - nu este necesar sa se determine ziua careia ii apartine o anumita nota.



5.4 Generalizare

■ **Generalizarea** (relatia is-a / is-a-kind-of) indica o relatie de *mostenire*.

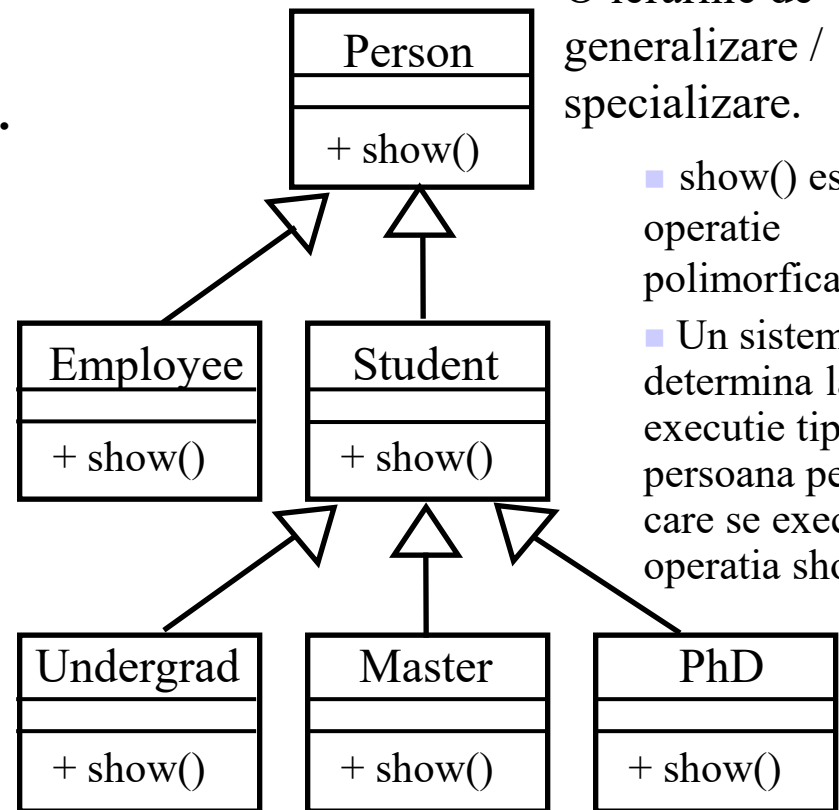
■ Prin mostenire

- se evita duplicarea de attribute si operatii prin crearea unei superclase separate care contine comportamentul comun.

- se poate adauga sau modifica comportament in clasele derivate mai specializate, care “mostenesc” comportament de la clasa de baza.

■ Relatia de generalizare este redată ca o linie terminată într-un mic triunghi gol care indică spre superclasa.

■ În ierarhiile de mostenire *comportamentul polimorfic* apare atunci când o metodă dintr-o clasă derivată are aceeași semnătură ca și o metodă din clasă părinte.



O ierarhie de generalizare / specializare.

- show() este o operație polimorfică

- Un sistem OO determină la execuție tipul de persoană pentru care se execută operația show().

Regula is-a:

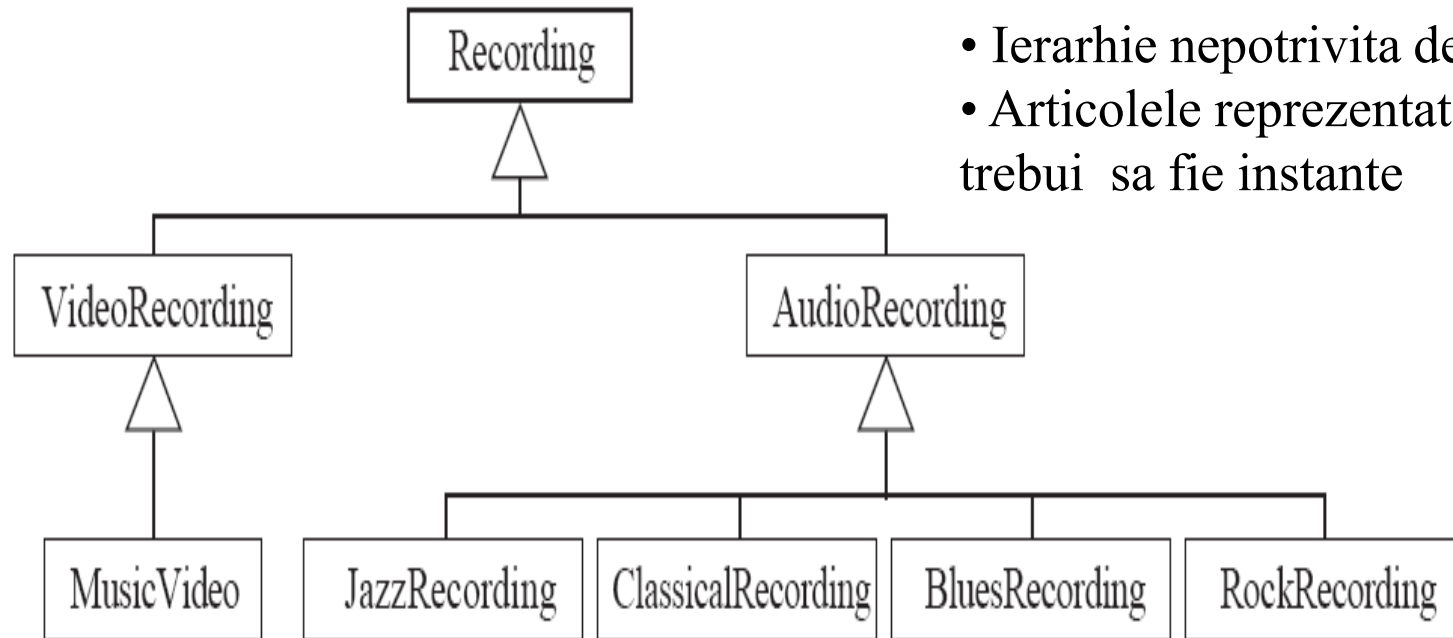
- Clasa A poate fi o subclasă validă a clasei B doar dacă în limba engleză are sens să se spună ‘an A is-a B’ (un A este-un B).

Seturi de generalizare

- Un set de generalizare este un grup etichetat de generalizari cu o superclasa comuna.
- Eticheta (numita discriminator in versiunile mai vechi ale UML) descrie criteriul utilizat la specializarea superclasei in doua sau mai multe subclase.
- In mod tipic, eticheta unui set de generalizare (de exemplu habitat, sau typeOfFood) va fi un atribut care are o valoare diferita in fiecare subclasa.



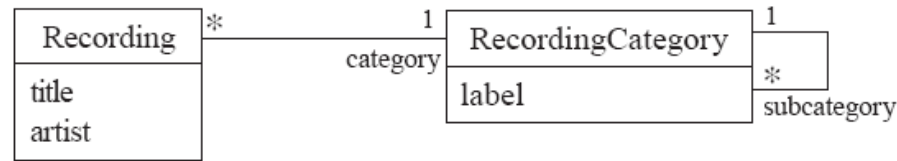
Evitarea generalizarilor inutile



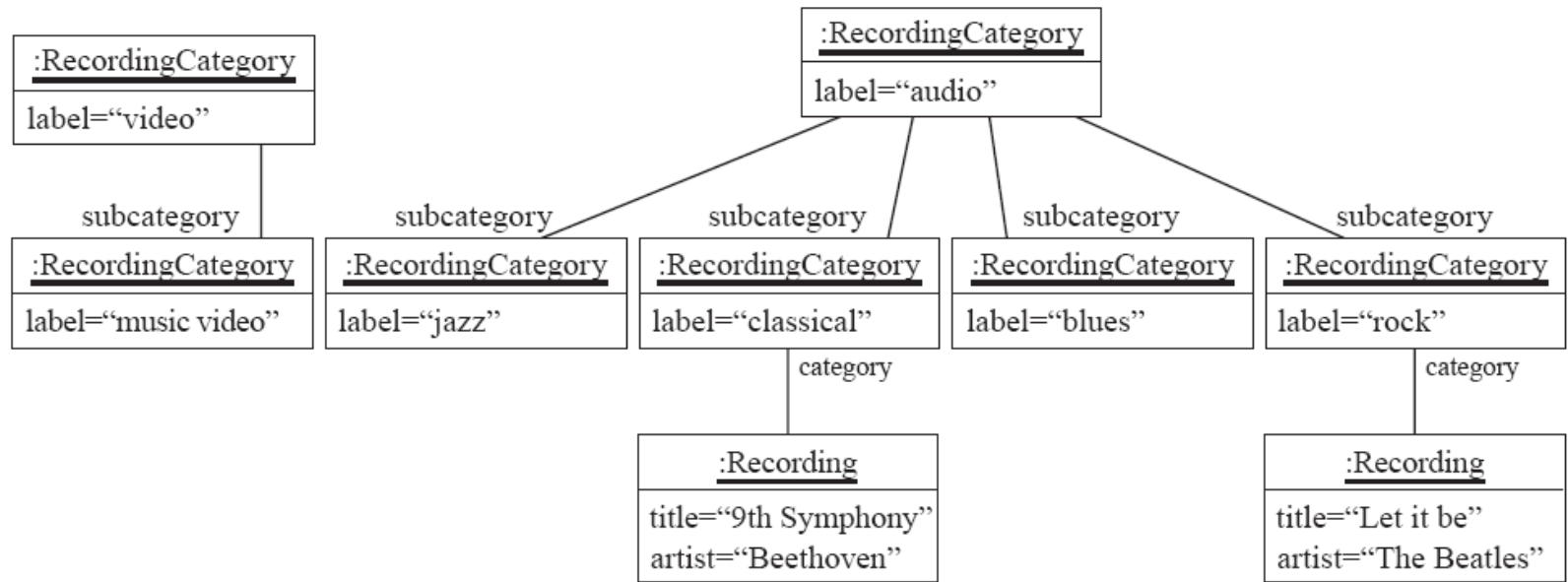
- Ierarhie nepotrivita de clase
- Articolele reprezentate ar trebui sa fie instante

- In limba engleza are sens sa spunem 'an AudioRecording is a Recording', etc
- Totusi, pentru justificarea existentei unei clase trebuie sa existe cel putin o operatie care se executa in mod diferit in clasa respectiva.
- Majoritatea claselor din aceasta ierarhie se comporta la fel.
 - De exemplu, articolele de tip JazzRecording, ClassicalRecording, BluesRecording si RockRecording nu difera nici prin modul in care pot fi vandute, nici prin tipul de informatie accesibil clientilor.

Evitarea generalizarilor inutile (continuare)



(a)



(b)

Diagrama de clasa imbunatatita (a), impreuna cu o diagrama corespunzatoare de obiecte (b)

Seturi de generalizare multiple

- **Problema de modelare:**

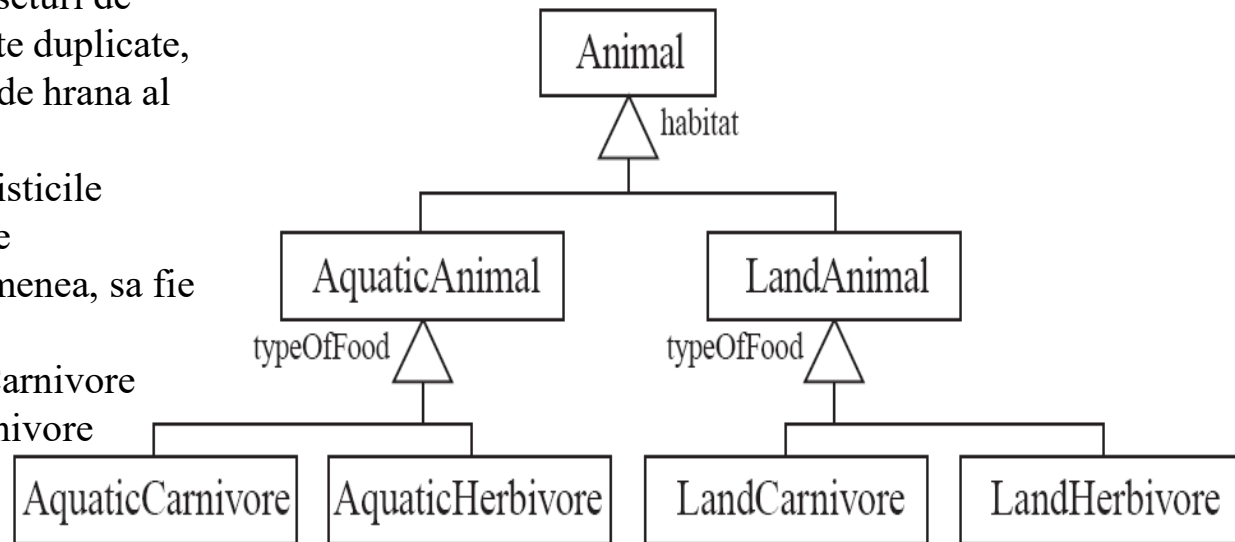
—Dorim sa construim o diagrama care sa permita reprezentarea tuturor combinatiilor posibile pentru habitatul si tipul de hrana al animalelor (de exemplu, dorim sa se poata reprezenta carnivore acvatice, cum sunt rechinii).

- **O solutie posibila** are la baza organizarea modelului pe doua niveluri:

- un prim nivel corespunzator habitatului,
- un al doilea nivel cu seturi de generalizare cu etichete duplicate, corespunzator tipului de hrana al animalelor.

- **Dezavantaj:** toate caracteristicile asociate cu al doilea nivel de generalizare trebuie, de asemenea, sa fie duplicate. De exemplu:

- prada unui AquaticCarnivore
- prada unui LandCarnivore



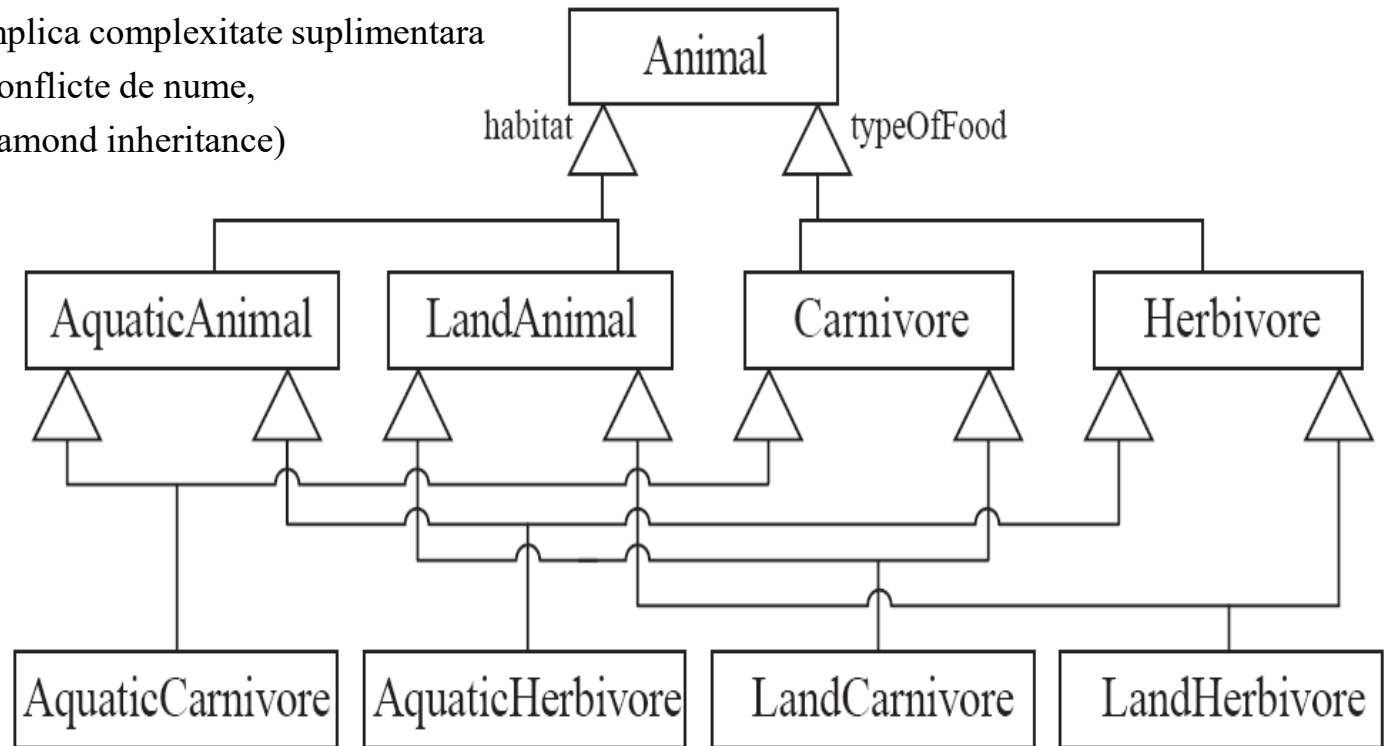
Seturi de generalizare multiple

- **Solutie bazata pe mostenire multipla**

- Avantaj: evita duplicarea caracteristicilor (ex.: prada este asociata numai clasei Carnivore)

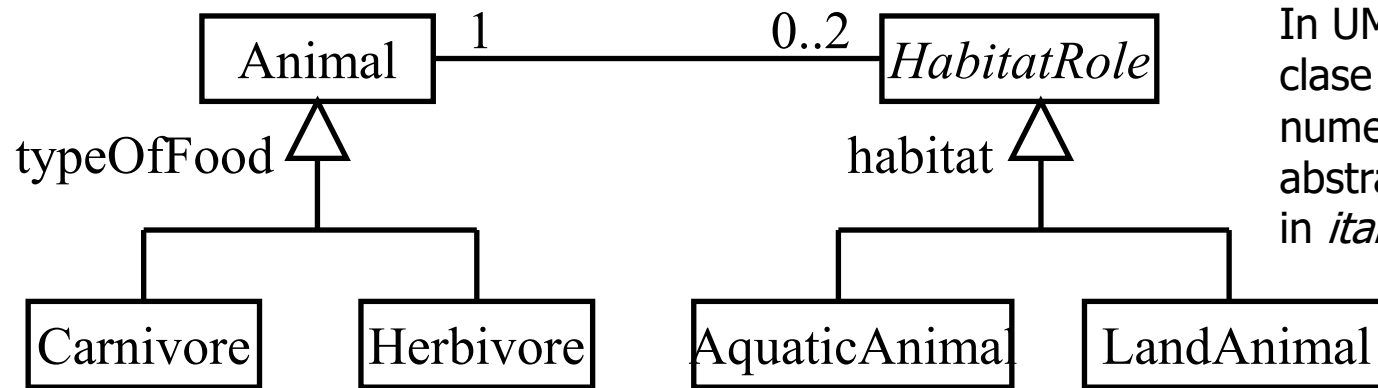
- Dezavantaje:

- Necesita chiar mai multe clase si generalizari
- In general, mostenirea multipla poate implica complexitate suplimentara (conflicte de nume, diamond inheritance)



Seturi de generalizare multiple

- O solutie superioara (bazata pe sablonul de design Player-Role)

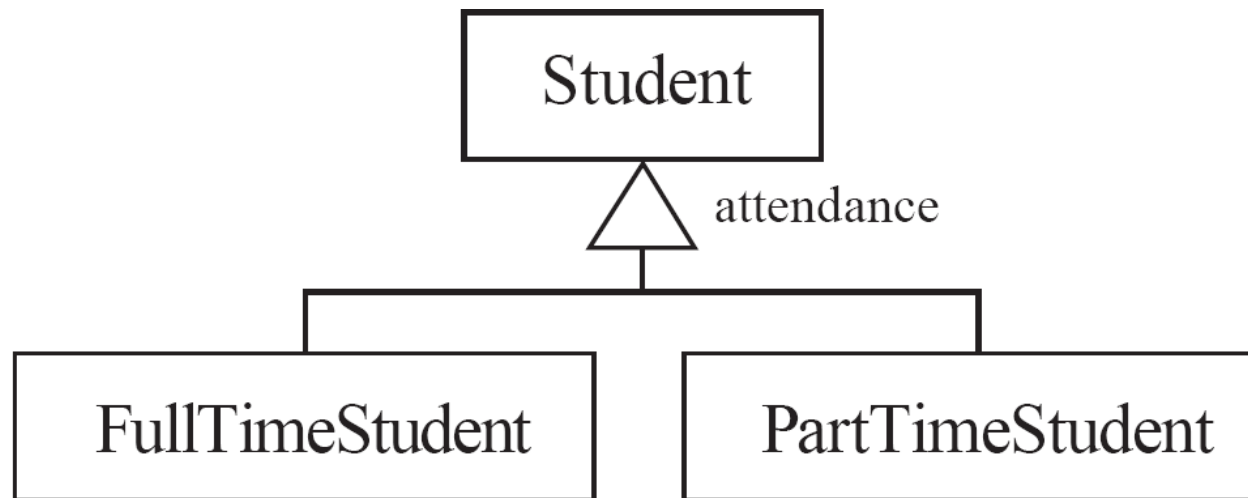


In UML numele de clase abstracte si numele de metode abstracte sunt redat in *italics*.

- Modelul sablonului de design Player-Role consta dintr-o clasa Player asociata cu o clasa abstracta Role; clasa Role este superclasa unui set de clase ce modeleaza roluri concrete posibile.
 - Un *rol* este un ansamblu particular de caracteristici asociate cu un obiect intr-un context particular.
 - Un obiect poate *juca* roluri diferite in contexte diferite.
- Desigur, in exemplul considerat este improbabil ca rolurile (**AquaticAnimal**, **LandAnimal**) sa se schimbe; in general insa, rolurile se pot modifica in functie de context.

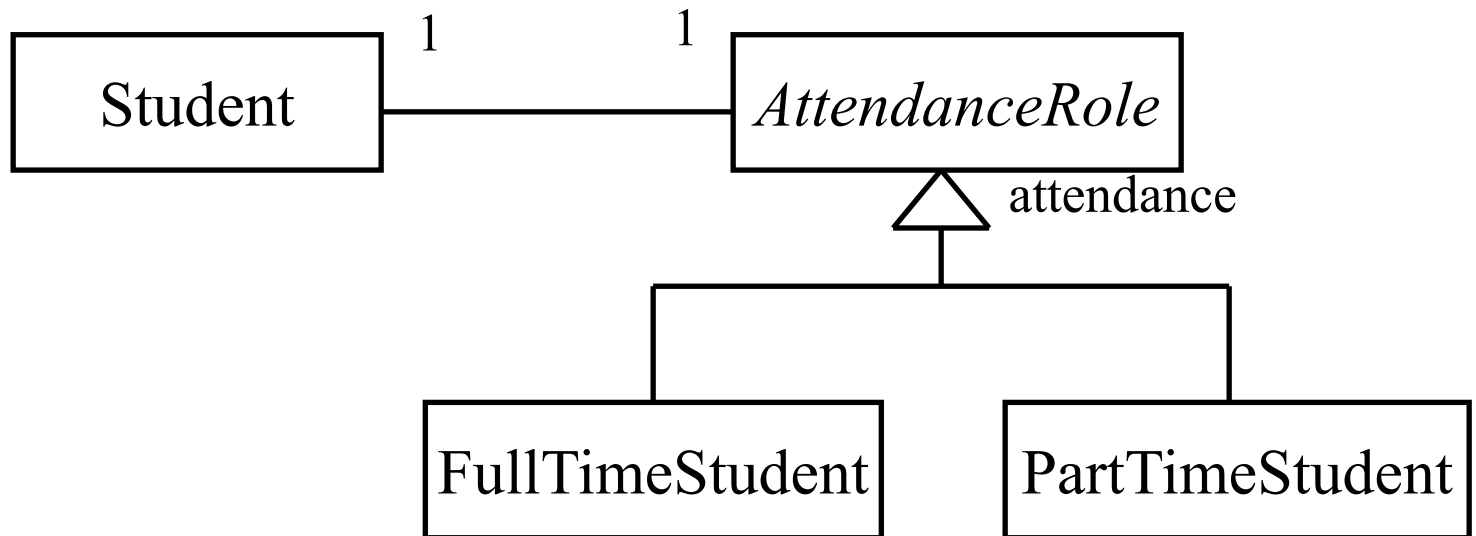
Clasa unui obiect nu trebuie sa se modifice

- Niciodata un obiect nu ar trebui sa isi modifice clasa!
 - In perioada studiilor, statutul unui student (in ceea ce priveste frecventa cu care audiaza cursurile) se poate modifica din FullTimeStudent in PartTimeStudent si invers.
 - Nu este de dorit ca o asemenea schimbare de statut sa fie modelata prin distrugerea unei instante FullTimeStudent (PartTimeStudent) si crearea unei instante PartTimeStudent (FullTimeStudent). **Diagrama de clase data mai jos este un model nepotrivit!**



Clasa unui obiect nu trebuie sa se modifice

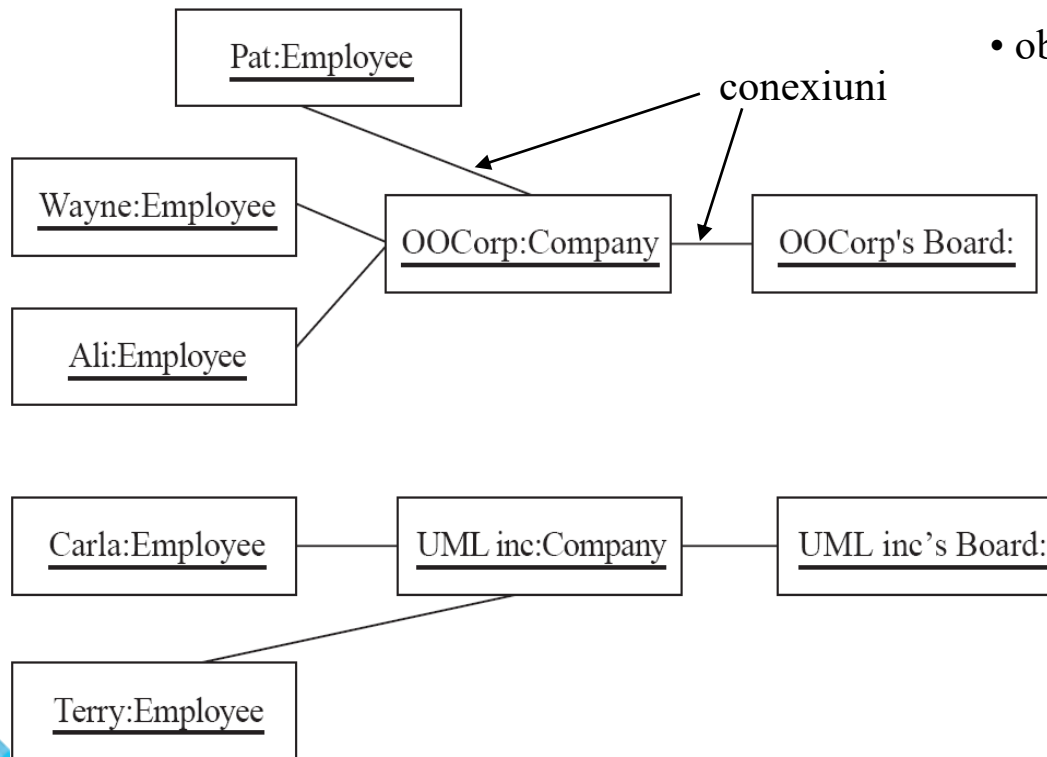
- O solutie superioara (bazata pe sablonul Player-Role)



5.5 Diagramme de objets

O **conexiune** (engl. *link*) este o instanta de asociere

- asa cum un obiect ca este instanta a unei clase



In UML un **obiect** este reprezentat sub forma unui dreptunghi (ca si o clasa, dar) cu numele obiectului subliniat. Se pot reprezenta:

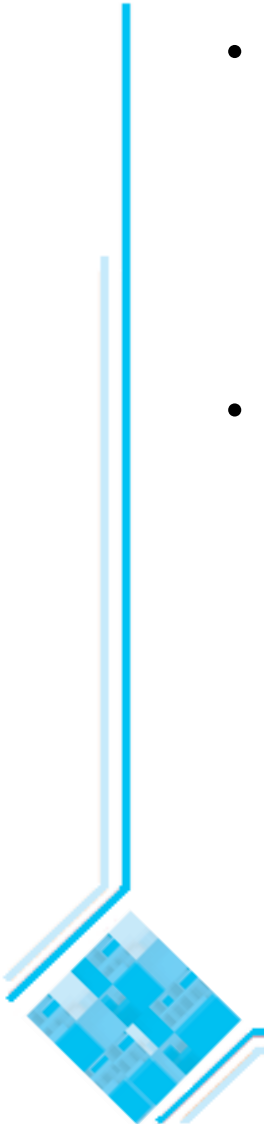
- obiecte cu nume:
 - numeObiect : NumeClasa
- obiecte anonime:
 - : NumeClasa

Diagramme de obiecte pentru

- asocierea de tip 1-N intre clasele Employee si Company, si
- asocierea de tip 1-1 intre clasele Company si BoardOfDirectors.

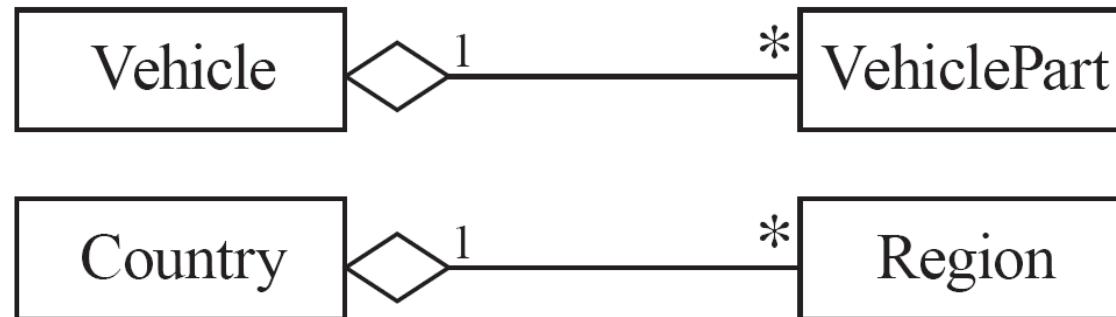
Asocieri, generalizari si diagrame de obiecte

- O asociere descrie relatii care exista intre instante la executie.
 - Intr-o diagrama de obiecte generata pe baza unei diagrame de clase, exista instante ale ambelor clase aflate intr-o relatie de asociere.
- Generalizarile descriu relatii intre clase (in diagrame de clase).
 - Generalizarile nu apar deloc in diagramele de obiecte.
 - O instanta a unei clase C trebuie sa fie in acelasi timp considerata a fi instanta a oricarei superclase a clasei C.



5.6 Agregare

- Agregarea este un tip particular de asociere, reprezentand o relatie structurala intre parte si intreg (relatie ‘has-a’).
 - ‘Intregul’ este adesea numit *ansamblu* sau *agregat*.
 - Grafic, agregarea este reprezentata ca o linie terminata intr-un romb care indica spre ‘intreg’.



Cand se utilizeaza relatia de agregare

In general, o asociere poate fi tratata drept agregare daca:

- relatia este de tip parte-intreg, putand fi exprimata prin expresii de genul 'is part of' (este parte a) sau 'is composed of' (este compus din)
- cineva sau ceva care posedea sau controleaza 'intregul' posedea sau controleaza si 'partile'.

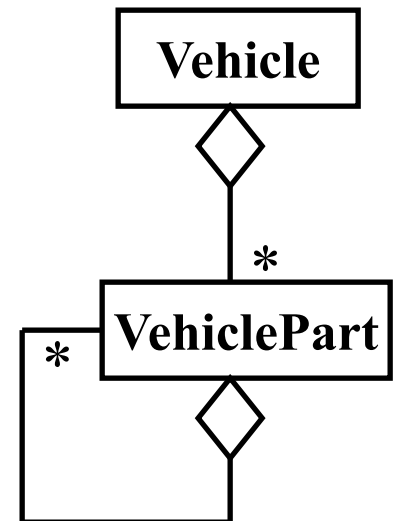
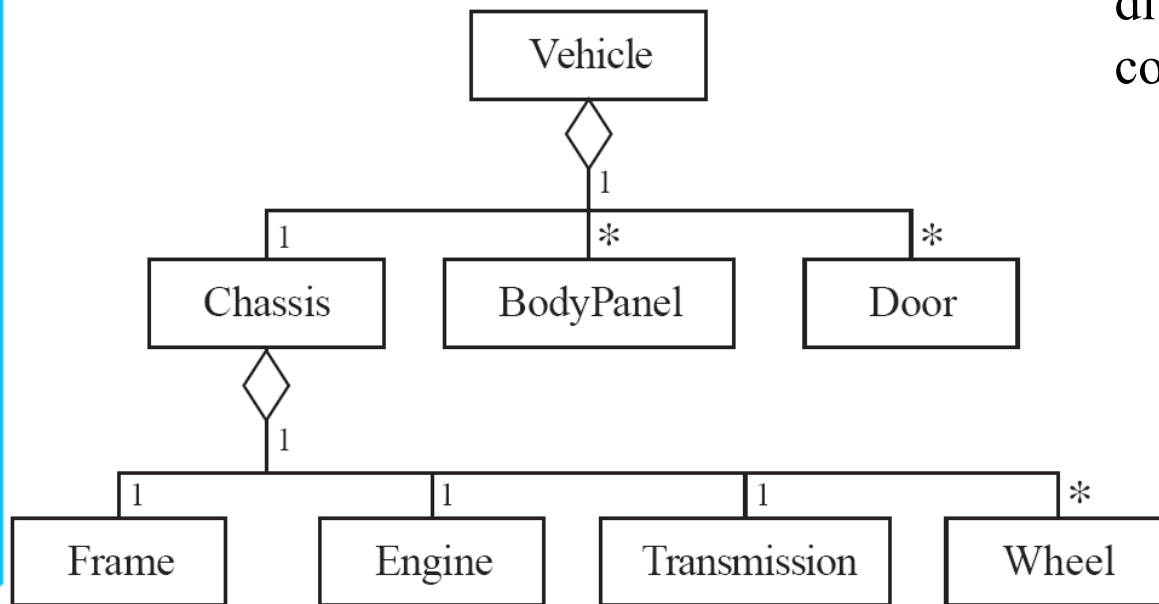
Compozitie

- **Compozitia este o forma de agregare in care**
 - o ‘parte’ poate apartine unui singur ‘intreg’ (in agregarea simpla poate exista partajare)
 - daca agregatul (intregul) este distrus atunci si partile se distrug.
- Grafic, compozitia este reprezentata ca si agregarea, dar rombul care indica spre ‘intreg’ este plin.



Ierarhie de agregare

Aceasta este o reprezentare mai flexibilă. Poate fi adaptată pentru diverse tipuri de vehicule, care au diverse configurații de componente (parti).



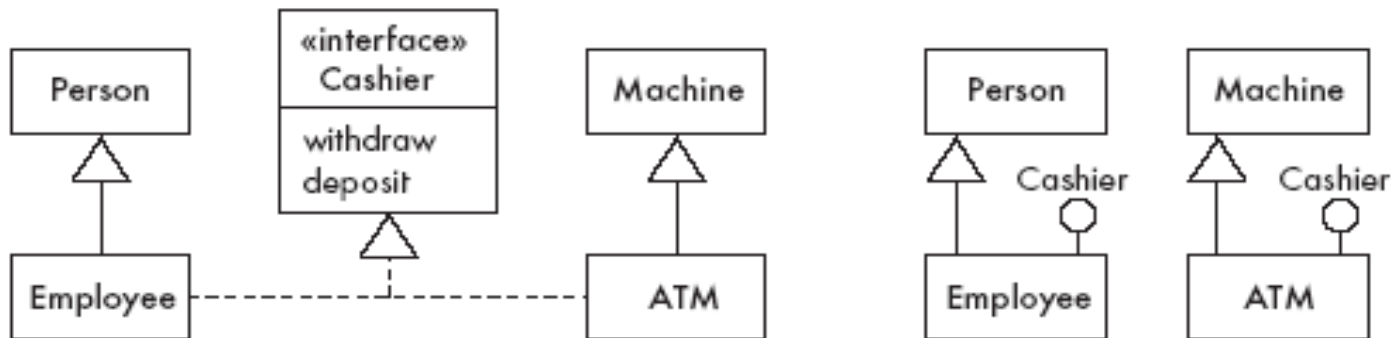
Propagare

- Un mecanism prin care o operatie asupra agregatului implica operatii efectuate asupra partilor.
- Proprietatile partilor sunt adesea propagate inapoi la agregat.
 - De exemplu, greutatea unei masini (agregatul) poate fi calculata prin insumarea greutatilor partilor componente.
- Se poate spune ca propagarea este pentru agregare ceea ce este mostenirea pentru generalizare.
 - Principala diferenta este urmatoarea:
 - Mostenirea este un mecanism implicit
 - Propagarea trebuie sa fie programata atunci cand este necesar

Interfete

O interfata descrie o parte a comportamentului vizibil al unui set de obiecte.

- O *interfata* este similara cu o clasa, dar nu include variabile de instanta si implementari de metode.
- In UML, relatia intre o interfata si clasa (sau componenta) care o implementeaza (realizeaza) se numeste *realizare*.
 - Grafic, relatia de realizare este reprezentata ca o linie intrerupta terminata cu un triunghi gol.
- In UML exista doua moduri de reprezentare a interfetelor, ambele ilustrate mai jos in specificarea interfetei Cashier (implementata de clasele Employee si ATM)



5.9 Construirea diagramelor de clasa

Se pot construi modele UML in diferite stadii ale procesului de dezvoltare

- **Model de explorare domeniu:**

- Dezvoltat in faza de analiza domeniu
- In mod normal contine unele clase, asocieri si attribute care nu fac parte din sistemul implementat (sunt necesare doar pentru intelegerea domeniului).
- In acest stadiu inginerul software nu este constrans sa aplice tehnici de optimizare a modelelor, cum este evitarea mostenirii multiple.

- **Model domeniu sistem:**

- Modeleaza aspecte ale domeniului care urmeaza sa fie reprezentate in sistem
- Clasele din acest model devin module software reale iar instantele a multe dintre aceste clase sunt memorate persistent in baze de date.
- Acest model poate include mai putin de jumatate din clasele modelului sistem (complet).
- Ar trebui construit astfel incat sa fie cat mai independent de interafata utilizator si de clasele arhitecturale.

- **Model sistem (complet):**

- Include modelul de domeniu sistem, impreuna cu
- Clase utilizate in constructia interfetei utilizator, clase necesare arhitecturii sistem (clienti, servere, protocoale pentru arhitecturi Layers, etc.), precum si clase utilitare (care fac sistemul mai reutilizabil, mai usor de intretinut, etc.)

O metodologie de analiza si proiectare

- Se identifica mai intai un set de **clase** candidate
- Se adauga **asocieri** si **attribute**
- Se identifica **generalizari**
- Se construiește lista **responsabilitatilor** pentru fiecare clasa
- Se decide asupra **operatiilor** specifice care sunt necesare in realizarea fiecărei responsabilitati
- Se **repetă** intregul proces pana cand se obtine un model satisfacator
 - Se adauga sau se elimina clase, asocieri, attribute, generalizari, responsabilitati sau operatii
 - Se identifica interfete, se aplica sabloane si principii de proiectare

Procesul de dezvoltare

- nu trebuie sa fie dezorganizat,*
- dar nici rigid.*

Identificarea claselor

- In dezvoltarea unui model de domeniu inginerul software tinde sa *descopere* clase (pe baza descrierii cerintelor, a notelor de interviu sau pe baza rezultatelor sesiunilor de brainstorming)
 - Uneori poate fi necesar sa se inventeze anumite elemente ale modelului de domeniu; de exemplu, poate fi necesar sa se inventeze o superclasa (al carei nume nu rezulta din analiza domeniului) pentru a se obtine o ierarhie de generalizare.
- Atunci cand se lucreaza la construirea interfetei utilizator sau la construirea arhitecturii este adesea necesar sa se *inventeze* clase
 - Asemenea clase sunt necesare pentru rezolvarea anumitor probleme de proiectare particulare
- *Reutilizarea* trebuie sa reprezinte o preocupare continua; se pot
 - Reutiliza cadre de aplicatie (engl. *frameworks*)
 - Extinde sisteme existente
 - Studia sisteme similare

O tehnica simpla pentru identificarea claselor in domeniul de aplicatie

- Se utilizeaza materiale sursa cum sunt descrierile de cerinte
- Se extrag substantivele si expresiile substantivele (engl. *noun phrases*).
- Se elimina substantivele care:
 - Sunt redundante (doua nume pentru aceeasi clasa)
 - Reprezinta instante
 - Sunt vagi sau prea generale (de exemplu: ‘scopul utilizatorului’ sau ‘informatia reprezentata in aplicatie’)
 - Nu sunt necesare in aplicatie
- Se separa clasele dintr-un model de domeniu care reprezinta *tipuri de utilizatori* sau alte categorii de *actori*
- Se face o distinctie clara intre clase si attribute de clase

O tehnica simpla pentru identificarea claselor in domeniul de aplicatie

Exemplu: Utilizand urmatoarea descriere de sistem informatic, se cere lista substantivelor si a expresiilor substantivale care ar putea reprezenta clase in modelul de domeniu sistem (vezi si descrierea in limba engleza, din carte, anexa C).

Liniile aeriene Ootumlia efectueaza curse aeriene din Java Valey, capitala Ootumlia. Sistemul informatic tine evidenta pasagerilor si a locurilor alocate acestora la diferitele curse, precum si a personalului care alcatuieste echipajul. In privinta echipajului, sistemul trebuie sa stie ce face fiecare si cine pe cine conduce sau supravegheaza. Liniile aeriene Ootumlia efectueaza zilnic curse aeriene numerotate pe baza unui program regulat. In viitor, liniile aeriene Ootumlia urmaresc sa se extinda, deci sistemul informatic trebuie sa fie flexibil; in particular, se doreste sa se adauge un serviciu de tip ‘frequent-flier plan’ (evidenta si facilitati pentru clientii care utilizeaza frecvent serviciile liniilor aeriene).

O tehnica simpla pentru identificarea claselor in domeniul de aplicatie

Solutie:

- Substantivele care sunt puse in lista initiala de clase sunt: Cursa (sau cursa aeriana; engl. Flight), Pasager (engl. Passenger) si Angajat (engl. Employee).
- Nu se includ celelalte substantive sau expresii substantivale deoarece:
 - ‘Liniile aeriene Ootumlia’, ‘Java Valey’, ‘Ootumlia’ reprezinta instante
 - ‘Sistem informatic’ este numele sistemului insusi
 - ‘Loc’ este mai degraba un atribut
 - ‘Echipaj’ implica intregul echipaj; s-ar putea crea o clasa ‘MembruEchipaj’, dar clasa Angajat (Employee) ofera o mai buna flexibilitate.
 - ‘Program’ reprezinta informatie complexa care este descrisa de attributele si relatiile unor clase, cum este Cursa.
 - ‘Viitor’ – acest concept nu este parte a sistemului care urmeaza sa fie construit
 - ‘Frequent-flier plan’ – acest serviciu este planificat a fi dezvoltat intr-o versiune ulterioara a sistemului

Identificare asocieri si attribute

- Se porneste de la clasele care sunt considerate a fi cele mai importante (centrale) pentru aplicatie
- Pentru fiecare din aceste clase se identifica attributele si asocierile cu alte clase.
- Se continua progresiv spre clase considerate a fi mai putin importante.
- Se evita orice complexitate inutila.
 - Un sistem este mai simplu daca manipuleaza mai putina informatie.
 - Nu se vor adauga decat attribute sau asocieri care sunt in mod cert relevante pentru aplicatie (sunt necesare pentru implementarea unor cerinte).

Identificare asocieri

- O relatie de asociere exista daca o clasa

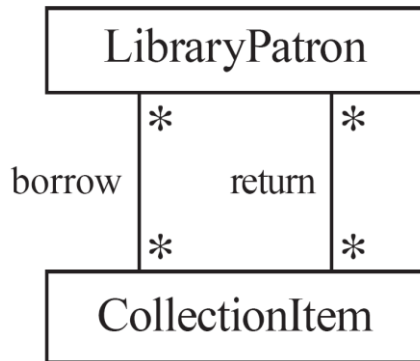
- *poseda*
- *controleaza*
- *este conectata la*
- *este in relatie cu*
- *este parte a*
- *are drept parte*
- *este un membru al*, sau
- *are drept membru*

o alta clasa din model (asemenea informatie poate fi adesea extrasa din documentele de definire sau de specificare a cerintelor)

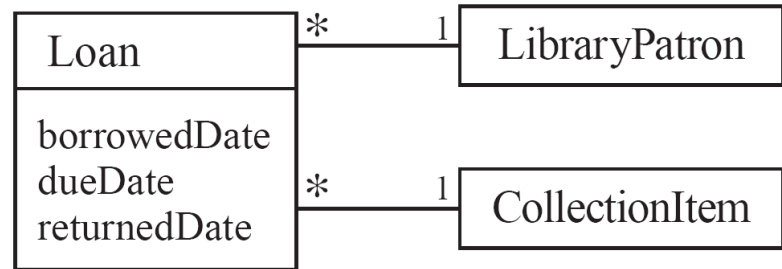
- Se specifica multiplicitatea la ambele capete
- Se utilizeaza nume sugestive pentru asociere sau roluri

Actiuni si asocieri

- O eroare comuna este sa se reprezinte *actiuni* (sau cazuri de utilizare) ca fiind asocieri



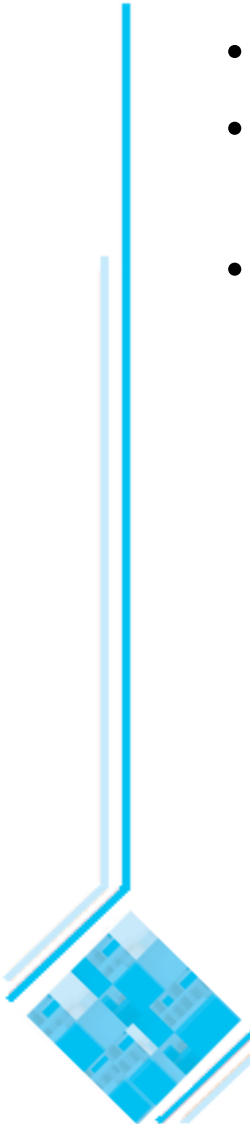
Model eronat: contine asocieri care reprezinta, de fapt, actiuni.



Solutie superioara: operatia de imprumut (engl. *borrow*) creeaza o instanta Loan, iar operatia de returnare (engl. *return*) modifica valoarea atributului returnDate

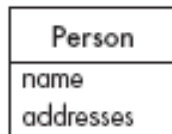
Identificare attribute

- Se identifica informatia care trebuie pastrata in fiecare clasa.
- Anumite substantive care nu au fost selectate initial spre a deveni clase in modelul de domeniu sistem pot deveni attribute.
- In general, un atribut contine o valoare simpla
 - un string
 - un numar



Identificare attribute

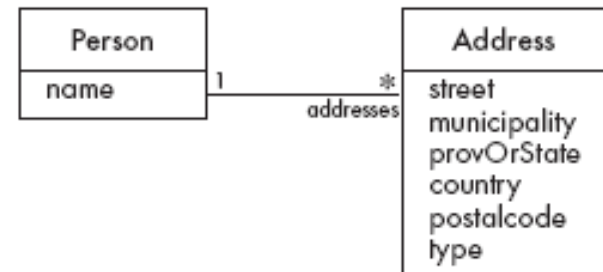
- Se va evita duplicarea atributelor
- Daca o parte dintre attributele unei clase formeaza un grup coerent, atunci se poate crea o clasa distincta care sa contina respectivele attribute (vezi partea dreapta a figurii de mai jos)



Bad, due to
a plural attribute



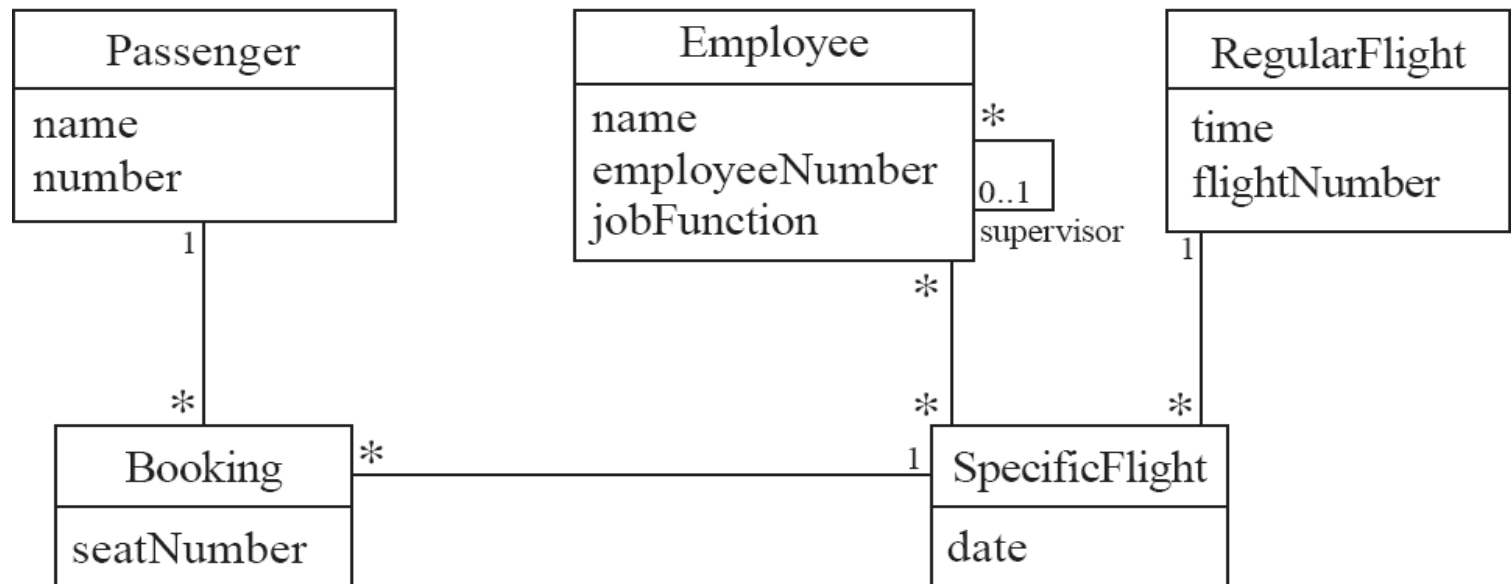
Bad, due to too many
attributes, and the
inability to add more
addresses



Good solution. The type indicates whether it
is a home address, business address etc.

Exemplu: attribute si asocieri in sistemul de rezervari pentru curse aeriene

- Clasa centrala de la care se porneste in identificarea asocierilor si atributelor este Flight (Cursa). Pentru evitarea redundanțelor aceasta este impartita in: RegularFlight (cursa regulata) si SpecificFlight (cursa specifica, sau zbor).
- In continuare se studiaza modul in care se fac rezervarile de locuri pentru pasageri (se introduce clasa Booking).
- Expresia 'cine pe cine conduce' implica existenta unei asocieri reflexive pentru clasa Employee (se utilizeaza numele de rol 'supervisor').

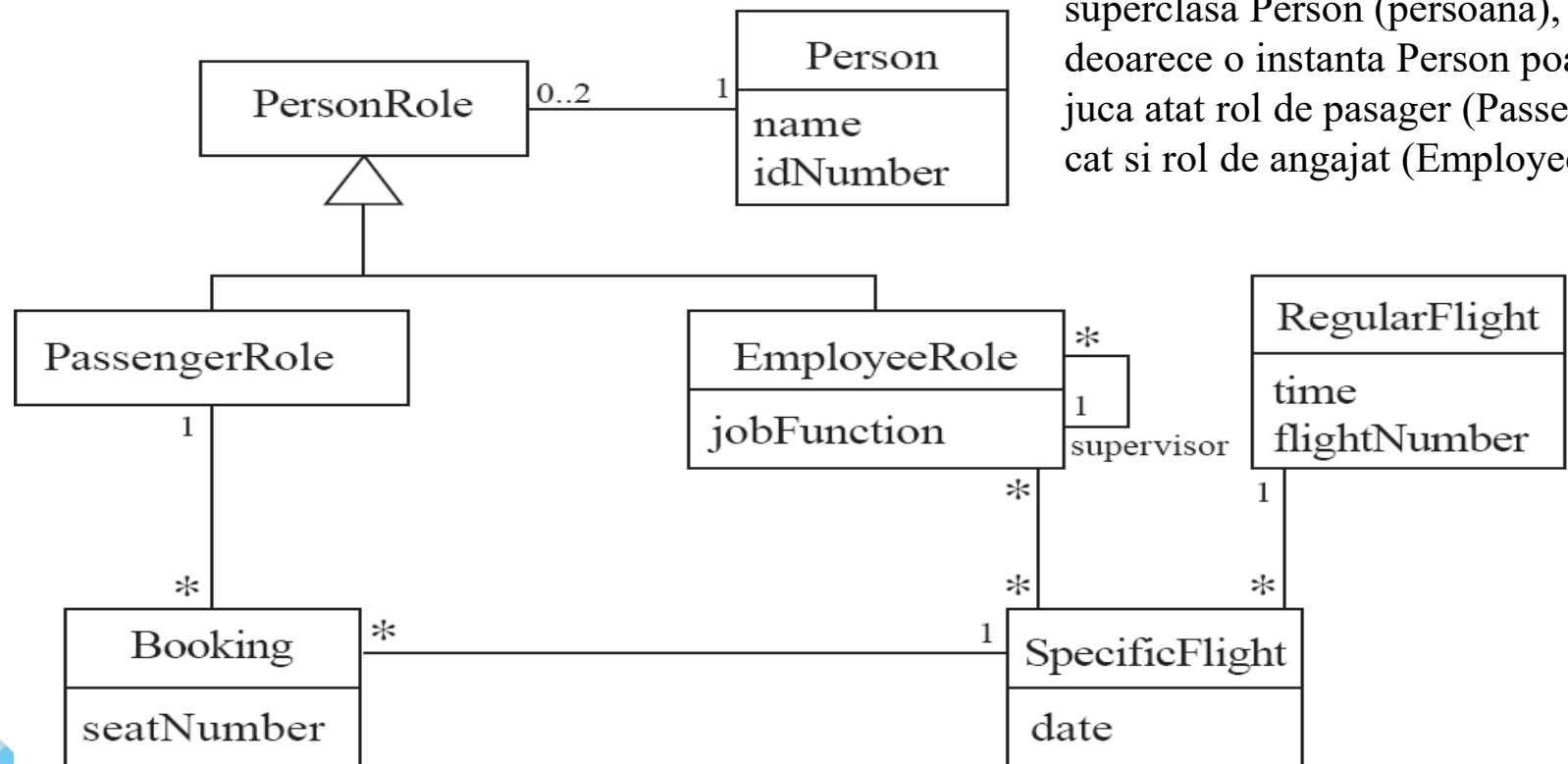


Identificare generalizari si interfete

- Exista doua moduri de a identifica generalizari:
 - bottom-up
 - Se grupeaza impreuna doua sau mai multe clase care partajeaza comportament (atribute, asocieri sau operatii) prin crearea unei noi superclase (care reprezinta comportamentul comun)
 - top-down
 - Se identifica mai intai clasele mai generale, care sunt apoi specializate daca este necesar
- In locul unei superclase se poate crea o *interfata* daca
 - Clasele sunt foarte diferite, exceptand cateva operatii comune
 - Una sau mai multe dintre clase au deja superclase
 - Se doreste limitarea operatiilor care pot fi efectuate asupra unei variabile doar la cele specificate de interfata

Exemplu: generalizare

- Clasele Passenger si Employee partajeaza attributele pentru nume si numar de identificare.
- Nu este suficient sa se creeze o superclasa Person (persoana), deoarece o instanta Person poate juca atat rol de pasager (Passenger) cat si rol de angajat (Employee).

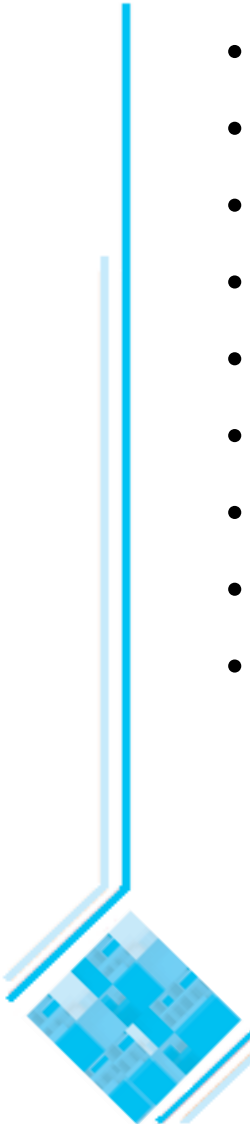


Alocarea de responsabilitati la clase

- Principalele responsabilitati sunt date de cerintele functionale.
- Fiecare responsabilitate trebuie sa fie atribuita unei clase (in general si alte clase vor colabora pentru realizarea respectivei sarcini de calcul).
 - Daca o clasa are prea multe responsabilitati atunci poate fi impartita in clase mai mici
 - Daca o clasa nu are nici o responsabilitate atunci, probabil, este *inutila*
 - Cand o responsabilitate nu poate fi atribuita nici uneia dintre clasele existente atunci trebuie sa se creeze o *noua clasa*
- Pentru determinarea responsabilitatilor
 - Se analizeaza cazurile de utilizare
 - Se cauta verbe sau substantive care descriu *actiuni* in descrierile sistem

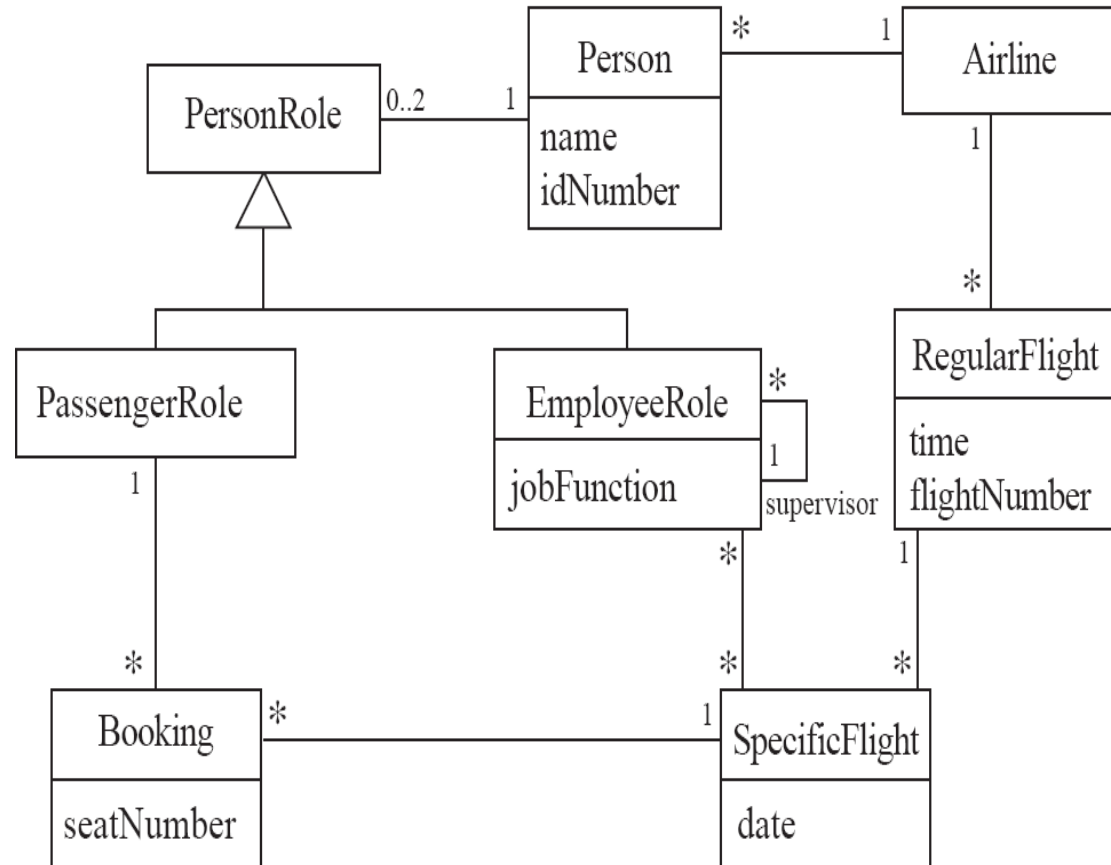
Categorii de responsabilitati

- Interogare sau modificare valori de atribute
- Creare si initializare obiecte (instante) noi
- Incarcare din baze de date sau salvare in baze de date
- Distrugere obiecte (instante)
- Adaugare sau stergere conexiuni intre obiecte
- Copiere, conversie, transformare, transmite sau formatare date
- Calcularea unor rezultate numerice
- Navigare si cautare
- Alte sarcini specifice



Exemple de responsabilitati

- Crearea instanta RegularFlight
- Modificare attribute pentru o instanta RegularFlight
- Crearea instanta SpecificFlight
- Rezervare loc pentru un pasager



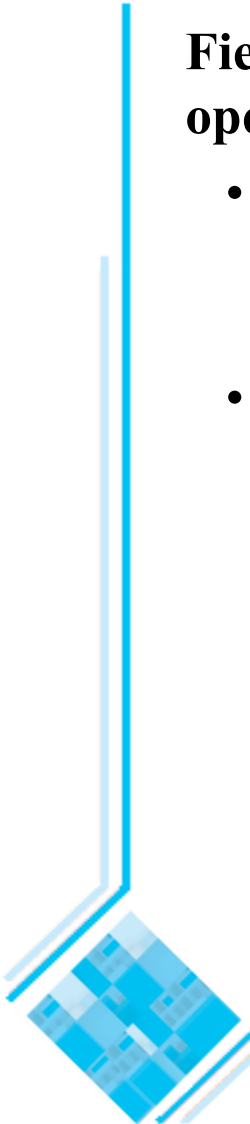
Prototipizare diagrame de clase prin analiza de tip Clasa-Responsabilitate-Colaborare (CRC)

- La identificarea unei clase se scrie numele acesteia pe un cartonas
- Pe masura ce sunt identificate, attributele si responsabilitatile se ataseaza claselor
 - Daca responsabilitatile nu incap pe cartonas
 - Asta sugereaza ca respectiva clasa ar trebui impartita in clase mai mici.
- Cartonasele se dispun pe o tabla de scris intr-o diagrama de clase.
 - Se deseneaza asocieri si generalizari pentru reprezentarea relatiilor dintre clase.

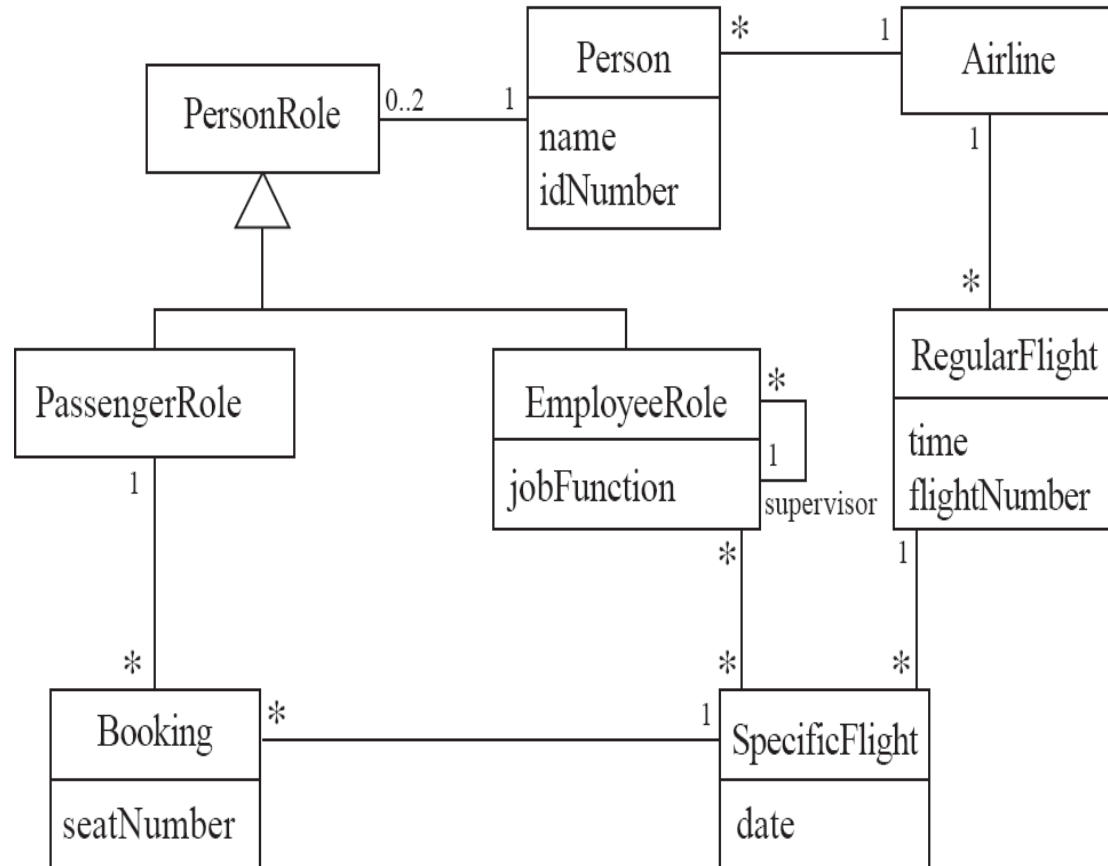
Identificare operatii

Fiecare responsabilitate este implementata prin una sau mai multe operatii

- Operatia principala care implementeaza responsabilitatea este in mod normal declarata **public**, devenind parte a interfetei intregului sistem.
- Celelalte metode care colaboreaza la implementarea unei responsabilitatii trebuie sa fie (cat mai) private.

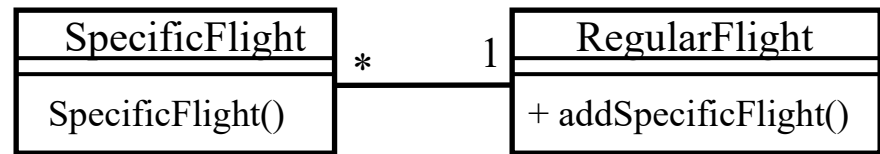


Exemple de operatii



Operatii pentru crearea unui obiect si conectarea sa la un obiect existent (conexiune bidirectionala)

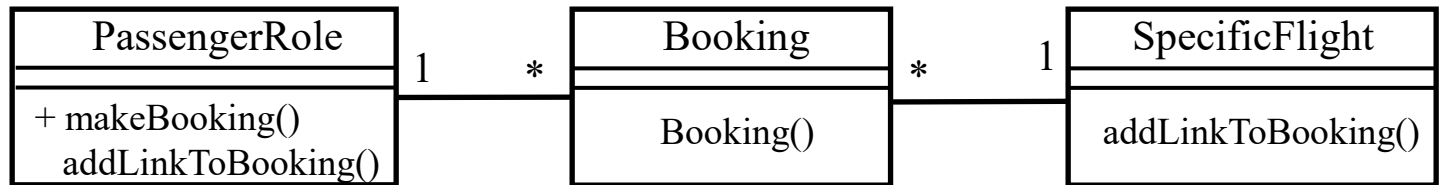
Exemplu: Crearea unui obiect SpecificFlight si conectarea sa la un obiect RegularFlight. Aceasta responsabilitate poate fi realizata cu ajutorul urmatoarelor operatii (implementarea Java este data in sectiunea 5.10):



1. (public) Instanta `RegularFlight`
 - Apeleaza constructorul `SpecificFlight`; apoi, efectueaza o conexiune unidirectionala catre noua instanta `SpecificFlight` (de fapt, fiecare instanta `RegularFlight` pastreaza o lista de obiecte `SpecificFlight`; lista este necesara pentru implementarea relatiei de tip 1-N).
2. (non-public) Constructorul clasei `SpecificFlight` (pe langa alte actiuni)
 - Realizeaza o conexiune unidirectionala inapoi catre instanta `RegularFlight`.

Operatii pentru crearea unei instante de clasa de asociere

Exemplu: crearea unui obiect Booking, care realizeaza legatura intre un obiect SpecificFlight si un obiect PassengerRole (existente dinainte). Aceasta responsabilitate poate fi realizata in patru pasi (se doreste o asociere bidirectionala):



1. (public) Instanta PassengerRole
 - Apeleaza constructorul clasei Booking (operatia 2)
2. (non-public) Constructorul clasei Booking (pe langa alte actiuni)
 - Face o conexiune unidirectionala catre instanta PassengerRole
 - Face o conexiune unidirectionala catre instanta SpecificFlight
 - Apeleaza operatiile 3 and 4.
3. (non-public) Instanta SpecificFlight
 - Face o conexiune unidirectionala catre instanta Booking
4. (non-public) Instanta PassengerRole
 - Face o conexiune unidirectionala catre instanta Booking.

5.10 Implementarea diagramelor de clase in Java

- Atributele sunt implementate ca variabile de instanta
- Generalizarile sunt implementate utilizand cuvantul cheie **extends**
- Realizarile de interfete sunt implementate utilizand cuvantul cheie **implements**
- Asocierile sunt implementate utilizand variabile de instanta
 - Intr-o asociere bidirectionala fiecare clasa are o variabila de instanta de tipul clasei de la celalalt capat al asocierii
 - Pentru o asociere unidirectionala in care multiplicitatea la celalalt capat este 'unu' sau 'optional'
 - Se declara o variabila de tipul clasei respective (o referinta)
 - Pentru o asociere de tip 1-N in care multiplicitatea la celalalt capat este 'mai multi' sau 'mai multe':
 - Se utilizeaza o clasa care implementeaza o colectie, cum este ArrayList sau LinkedList

Exemplu: SpecificFlight

```
class SpecificFlight
{
    private Calendar date;
    private RegularFlight regularFlight;
    ...
    // Constructor that should only be called from
    // addSpecificFlight
    SpecificFlight( Calendar aDate, RegularFlight aRegularFlight)
    {
        date = aDate;
        regularFlight = aRegularFlight;
    }
}
```

Exemplu: RegularFlight

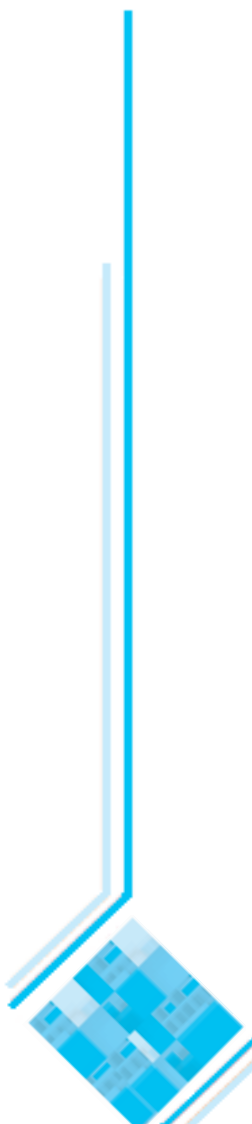
```
class RegularFlight
{
    private List specificFlights;
    ...
    // Method that has primary responsibility
    public void addSpecificFlight(Calendar aDate)
    {
        SpecificFlight newSpecificFlight;
        newSpecificFlight = new SpecificFlight(aDate, this);
        specificFlights.add(newSpecificFlight);
    }
    ...
}
```

Referinte suplimentare

[BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language user guide*. Addison-Wesley, 1999.

[JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.

[RJB99] J. Rumbaugh, I. Jacobson and G. Booch. *The Unified Modeling Language reference manual*. Addison-Wesley, 1999.



Using Design Patterns

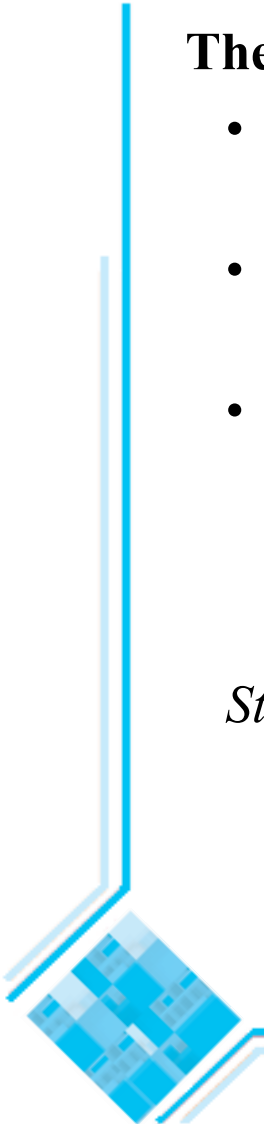
Adapted from:
Timothy Lethbridge and Robert Laganieri,
Object-Oriented Software Engineering –
Practical Software Development using UML and Java, 2005
(chapter 6)

6.1 Introduction to Patterns

The recurring aspects of designs are called *design patterns*.

- A *pattern* is the outline of a reusable solution to a commonly occurring problem encountered in a particular context
- Many of them have been systematically documented for all software developers to use
- A good pattern should
 - Be as general as possible
 - Contain a solution that has been proven to effectively solve the problem in the indicated context.

Studying patterns is an effective way to learn from the experience of others



Introduction to Patterns

- The notion of patterns was developed in an engineering context by the architect and design theorist **Christopher Alexander**
 - His work has influenced Computer Science
- Design patterns became popular in Software Engineering after the publication of the book:

[GoF94] E. Gamma, R. Helm, R. Johnson and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. (The four authors of this book are known as the “**Gang of Four**”, abbreviated as “**GoF**”)

- The GoF book [GoF94] presents a catalog of 23 design patterns .

Pattern description

Context:

- The general situation in which the pattern applies

Problem:

—A short sentence or two raising the main difficulty.

Forces:

- The issues or concerns to consider when solving the problem

Solution:

- The recommended way to solve the problem in the given context.
—‘to balance the forces’

Antipatterns: (Optional)

- Solutions that are inferior or do not work in this context.

Related patterns: (Optional)

- Patterns that are similar to this pattern.

References:

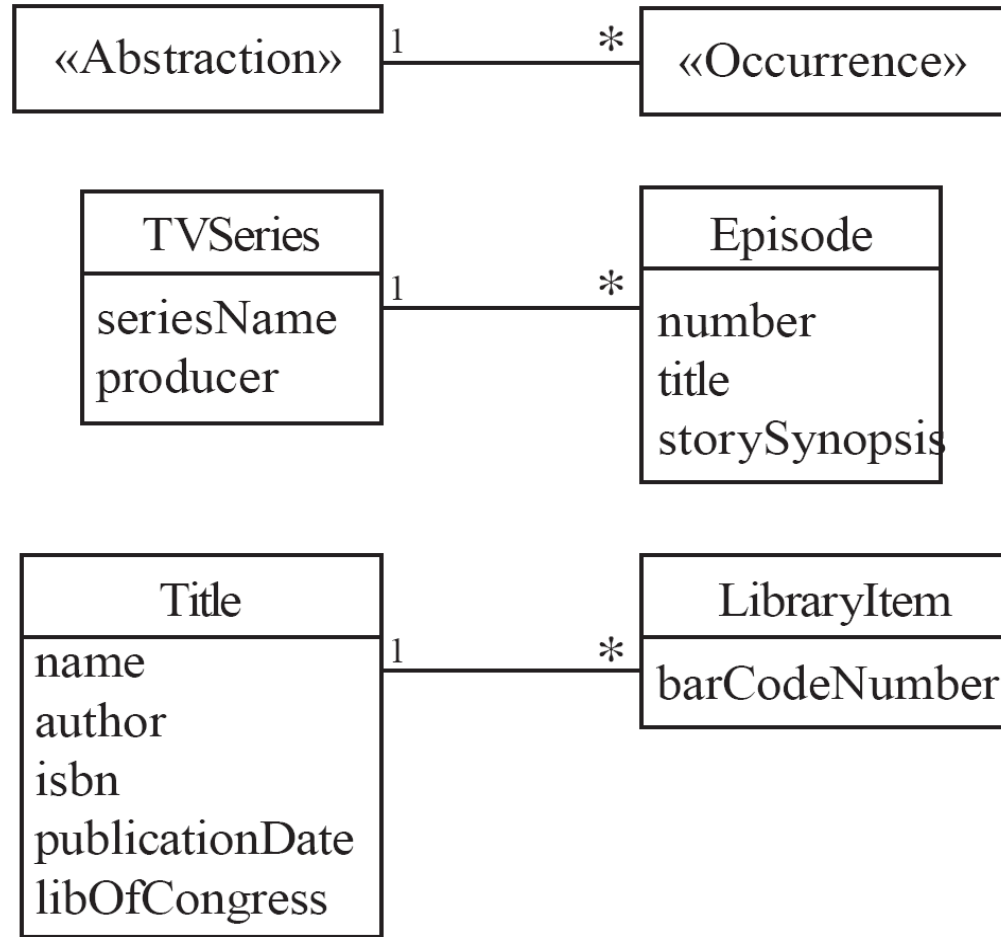
- Who developed or inspired the pattern.

6.2 The Abstraction-Occurrence Pattern

- **Context:**
 - Often in a domain model you find a set of related objects (*occurrences*).
 - The members of such a set share common information
 - but also differ from each other in important ways.
- **Problem:**
 - What is the best way to represent such sets of occurrences in a class diagram?
- **Forces:**
 - You want to represent the members of each set of occurrences without duplicating the common information
- **References:**
 - Abstraction-Occurrence is a generalization of the Title-Item pattern [EP00]

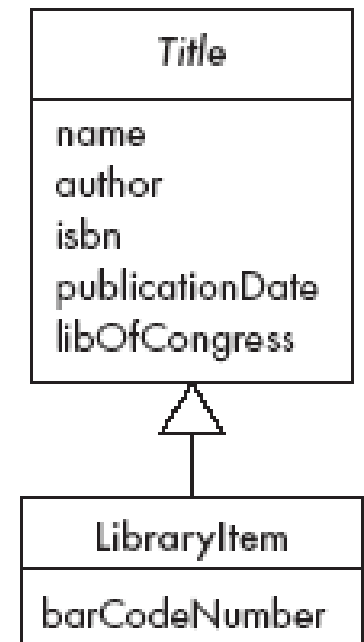
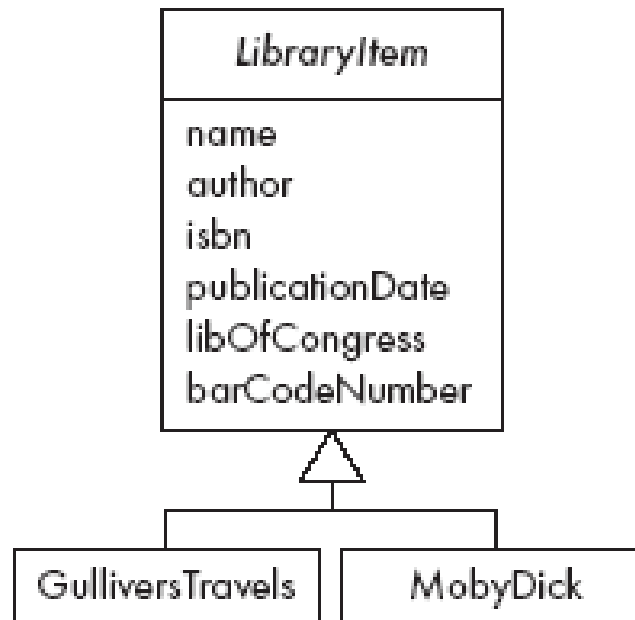
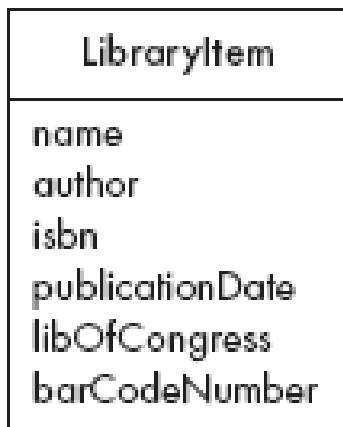
Abstraction-Occurrence

- *Solution:*



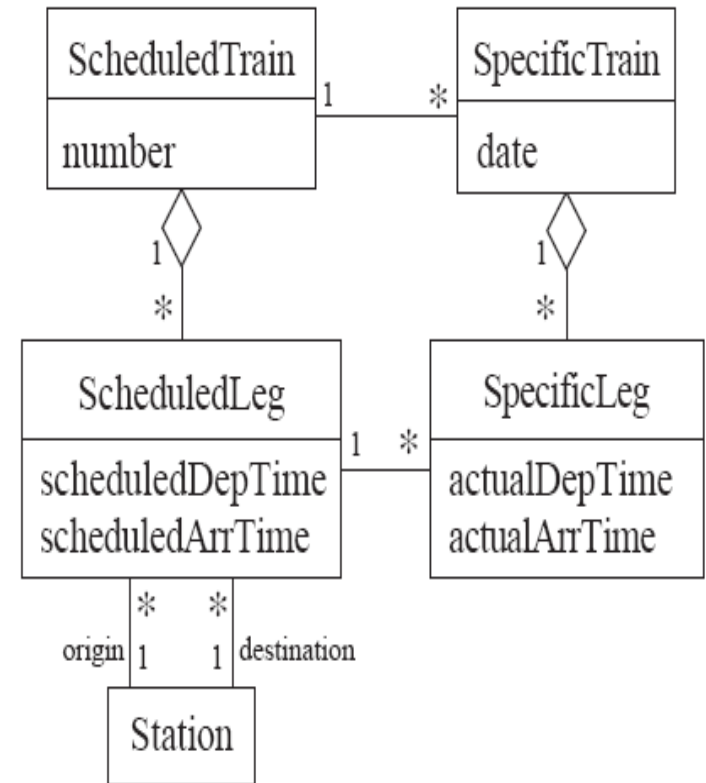
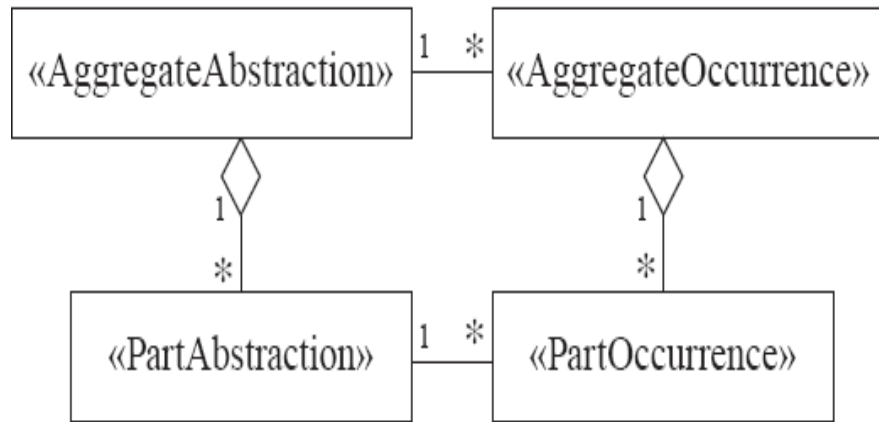
Abstraction-Occurrence

Antipatterns:



Abstraction-Occurrence

Square variant



6.3 The General Hierarchy Pattern

- **Context:**

- Objects in a hierarchy can have one or more objects above them (superiors), and one or more objects below them (subordinates).
- Some objects cannot have any subordinates

- **Problem:**

- How do you represent a hierarchy of objects, in which some objects cannot have subordinates?

- **Forces:**

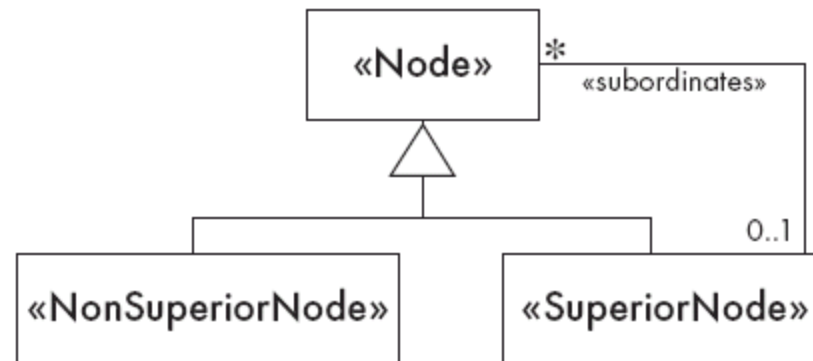
- You want a flexible way of representing the hierarchy
 - that prevents certain objects from having subordinates
- All the objects have many common properties and operations

- **References:**

- The Composite pattern described in [GoF94] is a specialization of the General Hierarchy pattern [LL05] (in the Composite pattern the association between an inner node and its subordinates is an aggregation). [GoF94] also includes Composite in the related patterns list of the Interpreter pattern.

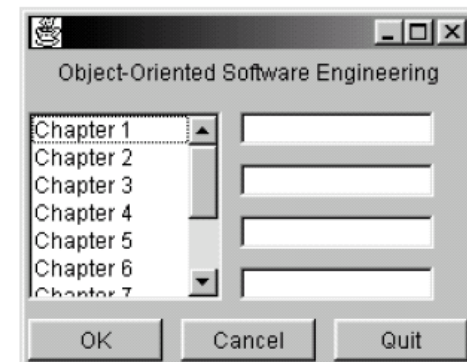
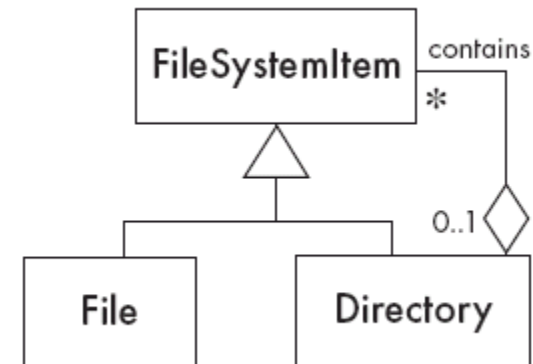
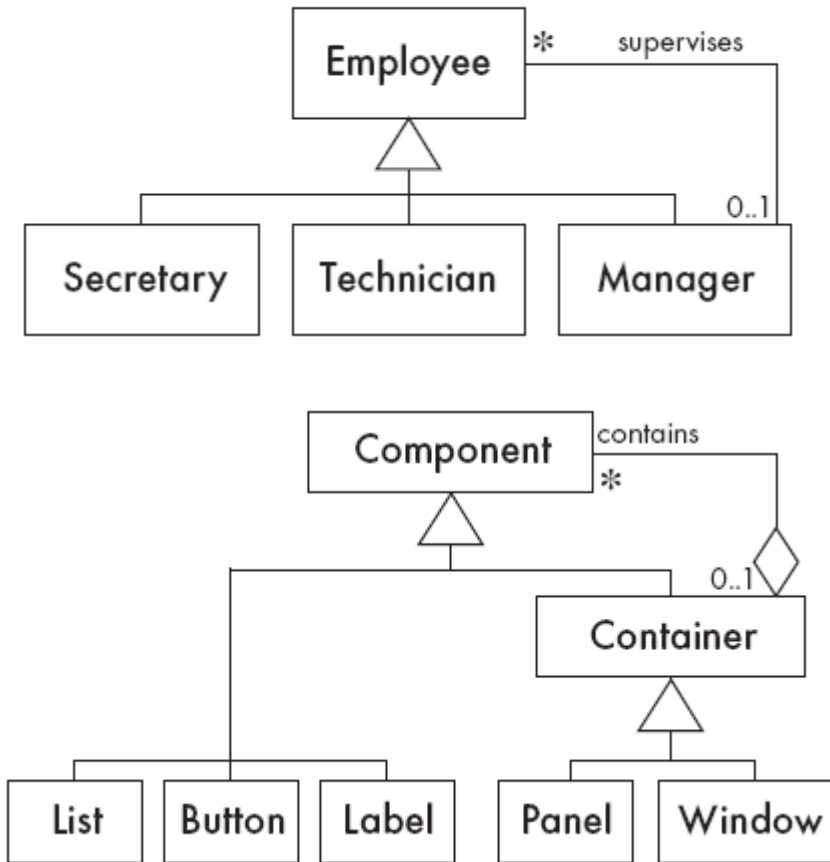
General Hierarchy

- *Solution:*



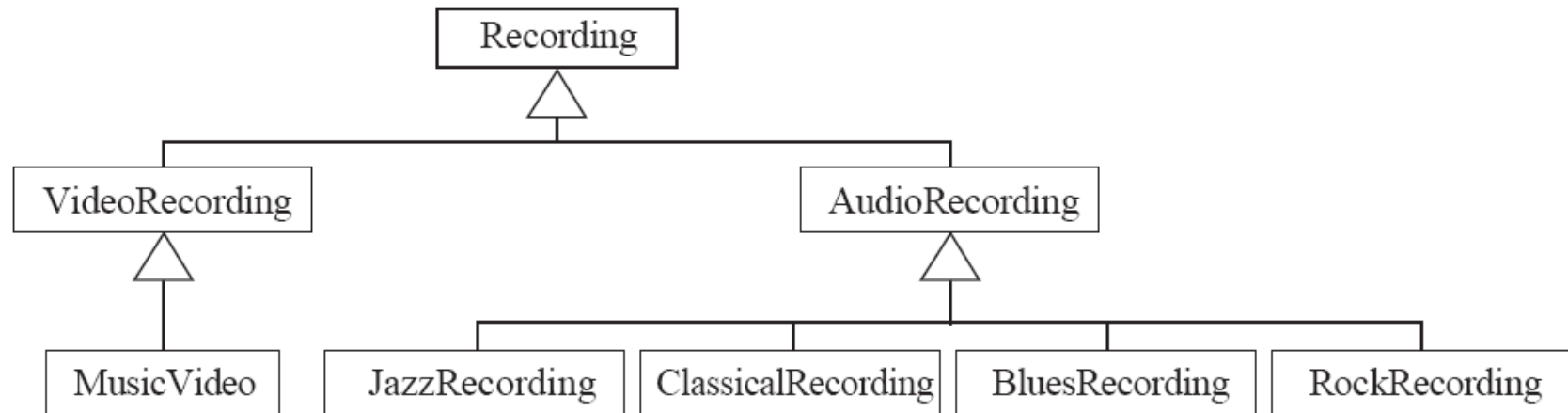
General Hierarchy

- *Solution:*



General Hierarchy

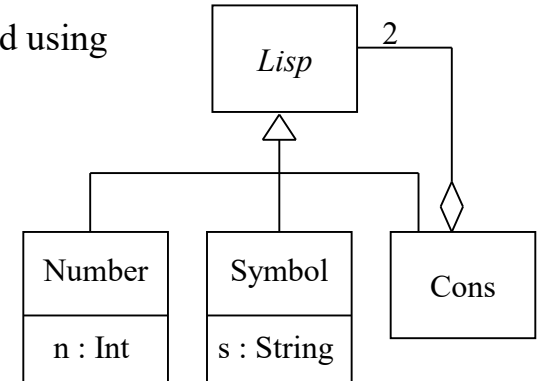
Antipattern:



Implementation Notes

- Consider a **Java implementation** of the Lisp data structure (designed using the **Composite pattern**):

```
abstract class Lisp { ... }  
class Number extends Lisp { int n; ... }  
class Symbol extends Lisp { String s; ... }  
class Cons extends Lisp {  
    Lisp car;  
    Lisp cdr;  
    ...  
}
```



- In Scala you can implement Composite as above (in Java style) [Nik16]. However, a **Scala implementation** of **Composite** could also use **case classes**:

```
abstract class Lisp  
case class Number (n: Int) extends Lisp  
case class Symbol (s: String) extends Lisp  
case class Cons (car: Lisp, cdr: Lisp) extends Lisp
```


Implementation Notes

abstract class Lisp

case class Number (n: Int) extends Lisp

case class Symbol (s: String) extends Lisp

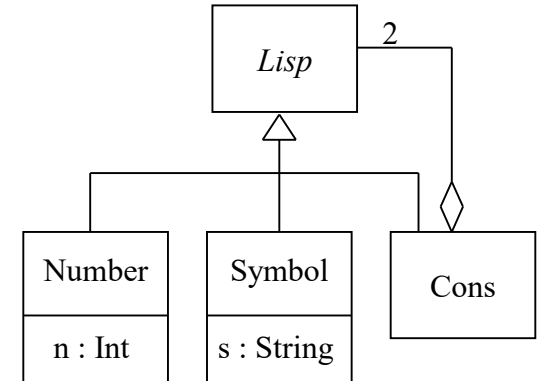
case class Cons (car: Lisp,cdr: Lisp) extends Lisp

- **Scala case classes** support **pattern matching**. The following Scala function implements a flattening operation:

```
def flat (lisp: Lisp): Lisp = lisp match {  
  case Cons (Number(n),cdr)      => Cons (Number (n),flat(cdr))  
  case Cons (Symbol(s),cdr)      => Cons (Symbol (s),flat(cdr))  
  case Cons (Cons(caar,cdar),cdr) => flat (Cons (caar,Cons (cdar,cdr)))  
  case lisp                      => lisp  
}
```

```
flat(Cons (Cons (Number(1),Number (2)),null)) →  
  Cons (Number(1),Cons(Number (2),null))
```

- The **Scala solution** is **concise**. Imagine implementing this function (flat) in Java style, using if statements !



6.4 The Player-Role Pattern

- ***Context:***

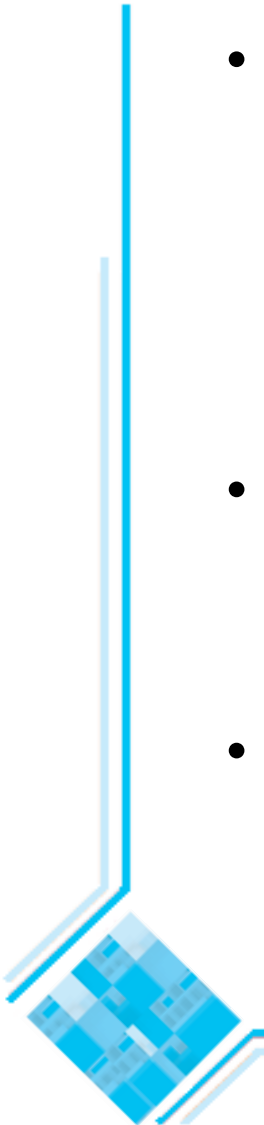
- A *role* is a particular set of properties associated with an object in a particular context.
- An object may *play* different roles in different contexts.

- ***Problem:***

- How do you best model players and roles so that a player can change roles or possess multiple roles?

- ***References:***

- The Player-Role pattern appears in [Rum91]

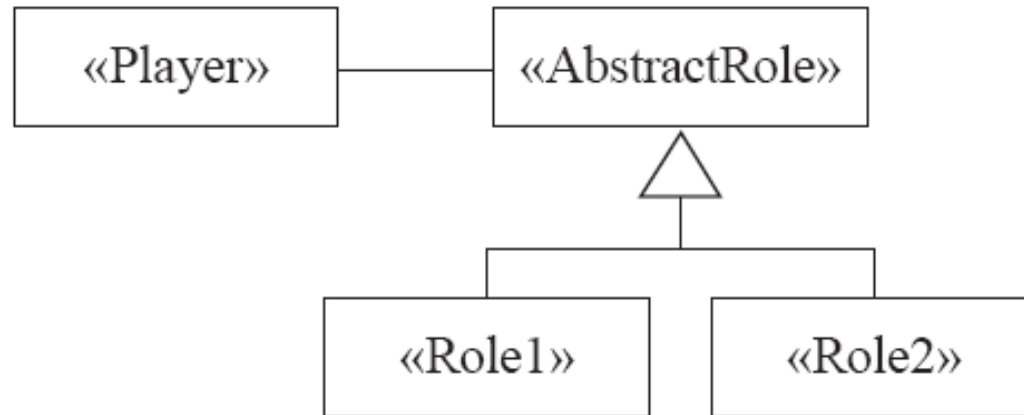


Player-Role

- ***Forces:***

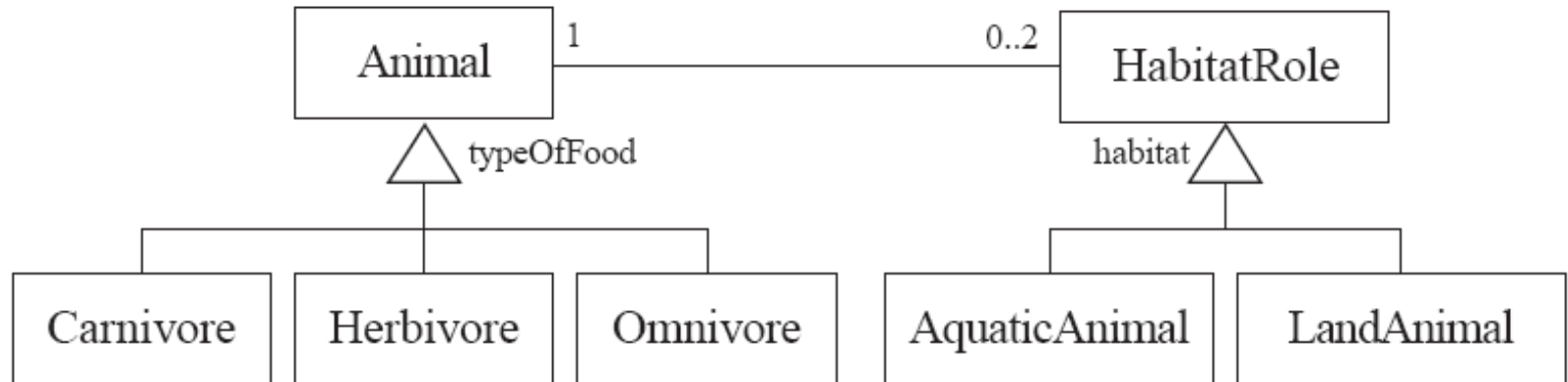
- It is desirable to improve encapsulation by capturing the information associated with each separate role in a class.
- You want to avoid multiple inheritance.
- You cannot allow an instance to change class

- ***Solution:***



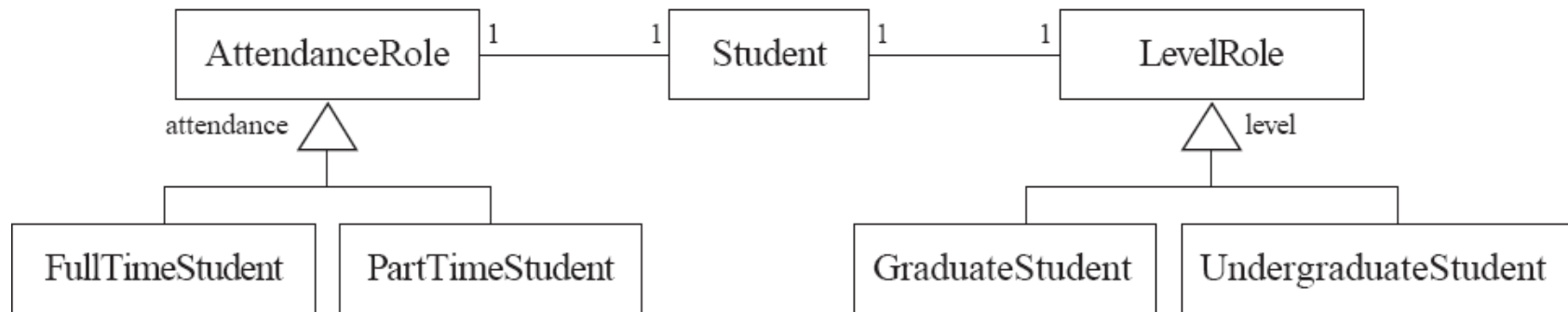
Player-Role

Example 1:



Player-Role

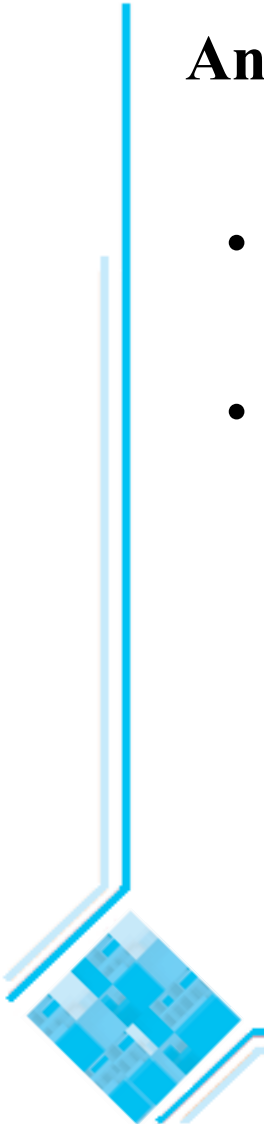
Example 2:



Player-Role

Antipatterns:

- Merge all the properties and behaviours into a single «Player» class and not have «Role» classes at all.
- Create roles as subclasses of the «Player» class.



6.5 The Singleton Pattern

- ***Context:***

- It is very common to find classes for which only one instance should exist (*singleton*)

- ***Problem:***

- How do you ensure that it is never possible to create more than one instance of a singleton class?

- ***Forces:***

- The use of a public constructor cannot guarantee that no more than one instance will be created.

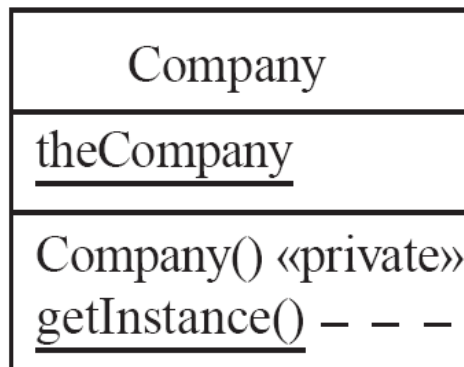
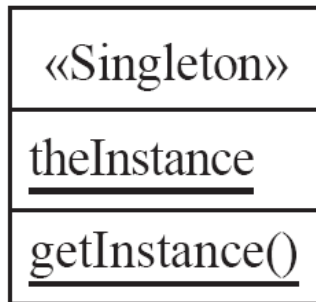
- The singleton instance must also be accessible to all classes that require it

- ***References:***

- Singleton is one of the GoF patterns

Singleton

- ***Solution:***



```
if (theCompany==null)
    theCompany= new Company();

return theCompany;
```


6.6 The Observer Pattern

- ***Context:***

- When an association is created between two classes, the code for the classes becomes inseparable.
- If you want to reuse one class, then you also have to reuse the other.

- ***Problem:***

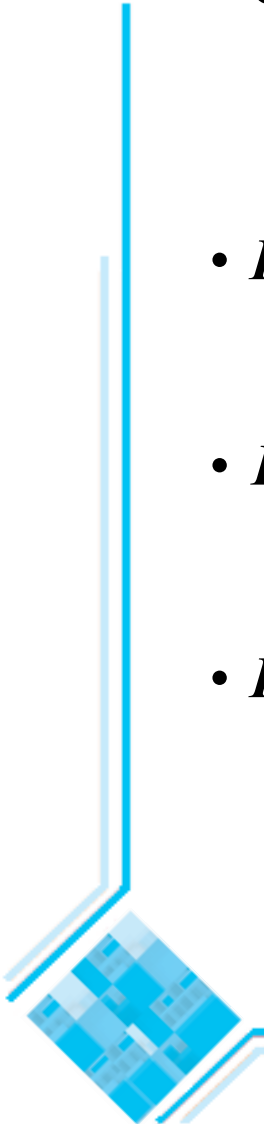
- How do you reduce the interconnection between classes, especially between classes that belong to different modules or subsystems?

- ***Forces:***

- You want to maximize the flexibility of the system to the greatest extent possible

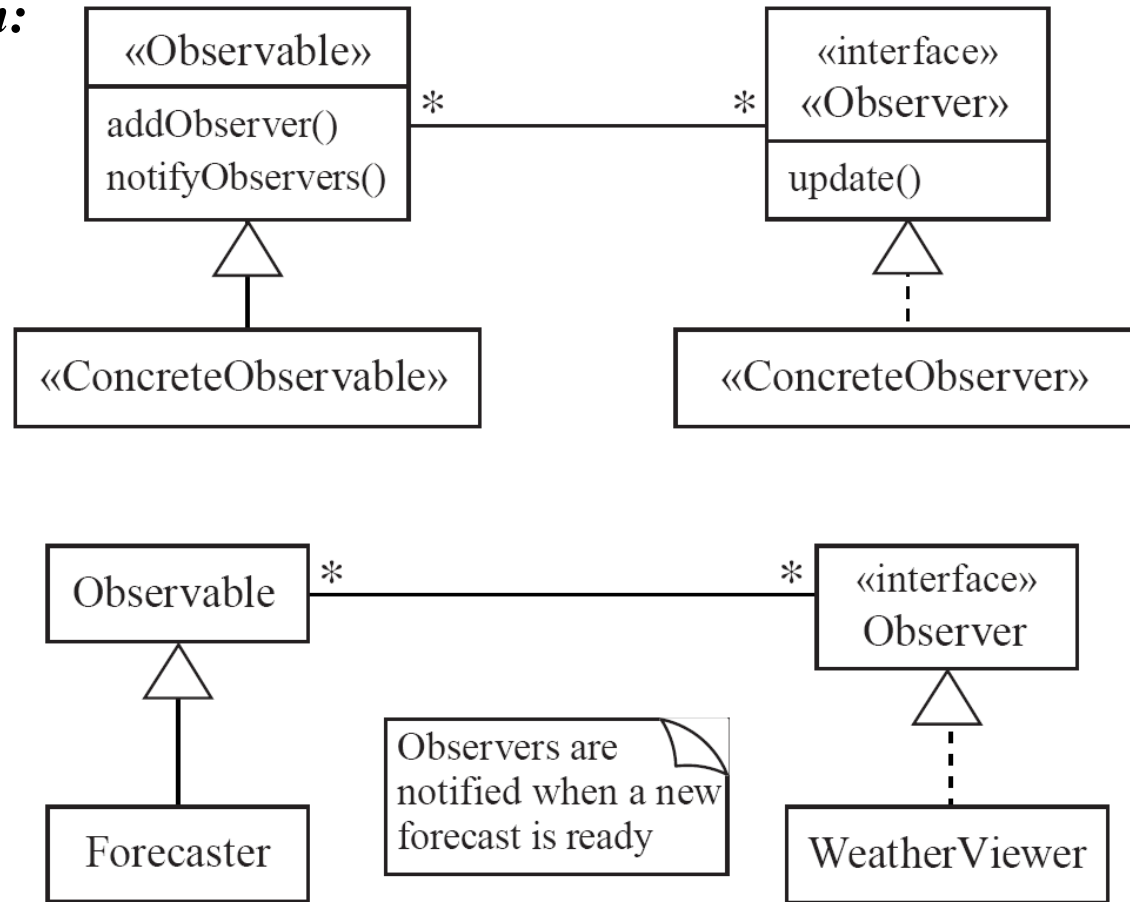
- ***References:***

- Observer is one of the GoF patterns



Observer

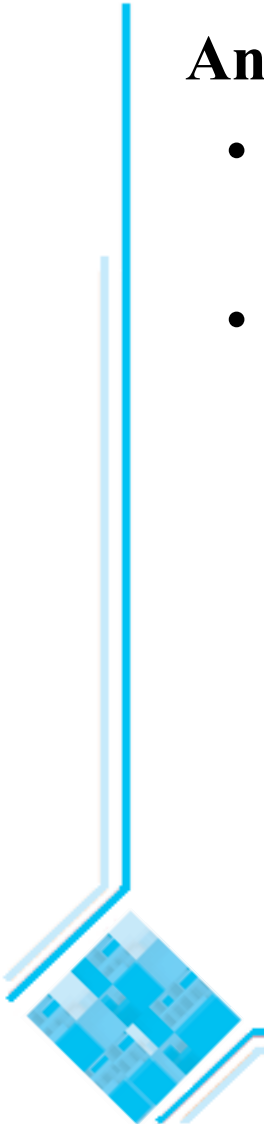
- ***Solution:***



Observer

Antipatterns:

- Connect an observer directly to an observable so that they both have references to each other.
- Make the observers *subclasses* of the observable.



6.7 The Delegation Pattern

- ***Context:***

- You are designing a method in a class
- You realize that another class has a method which provides the required service
- Inheritance is not appropriate
 - E.g. because the isa rule does not apply

- ***Problem:***

- How can you most effectively make use of a method that already exists in the other class?

- ***Forces:***

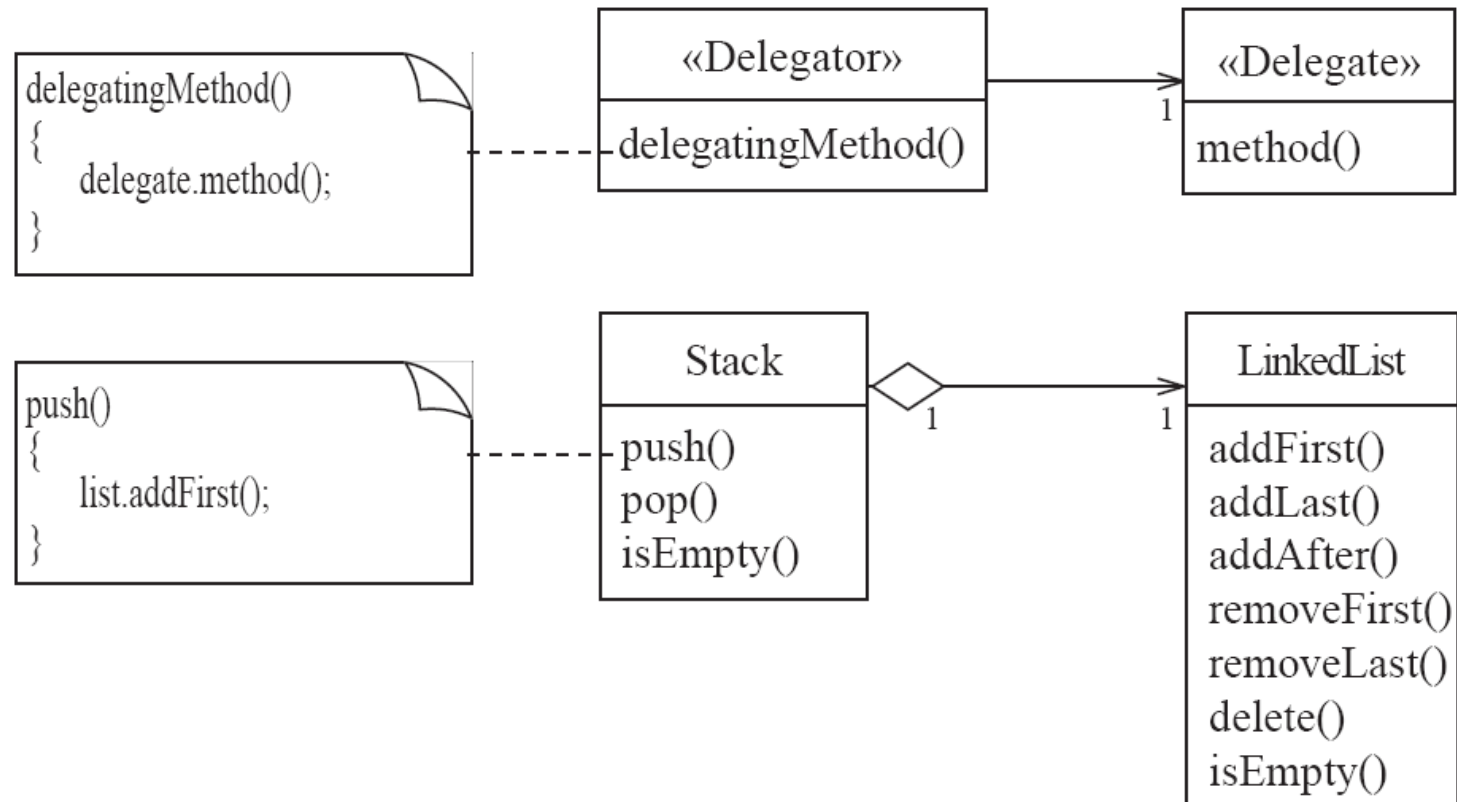
- You want to minimize development cost by reusing methods

- ***References:***

- The Delegation pattern is mentioned in [Gra02]

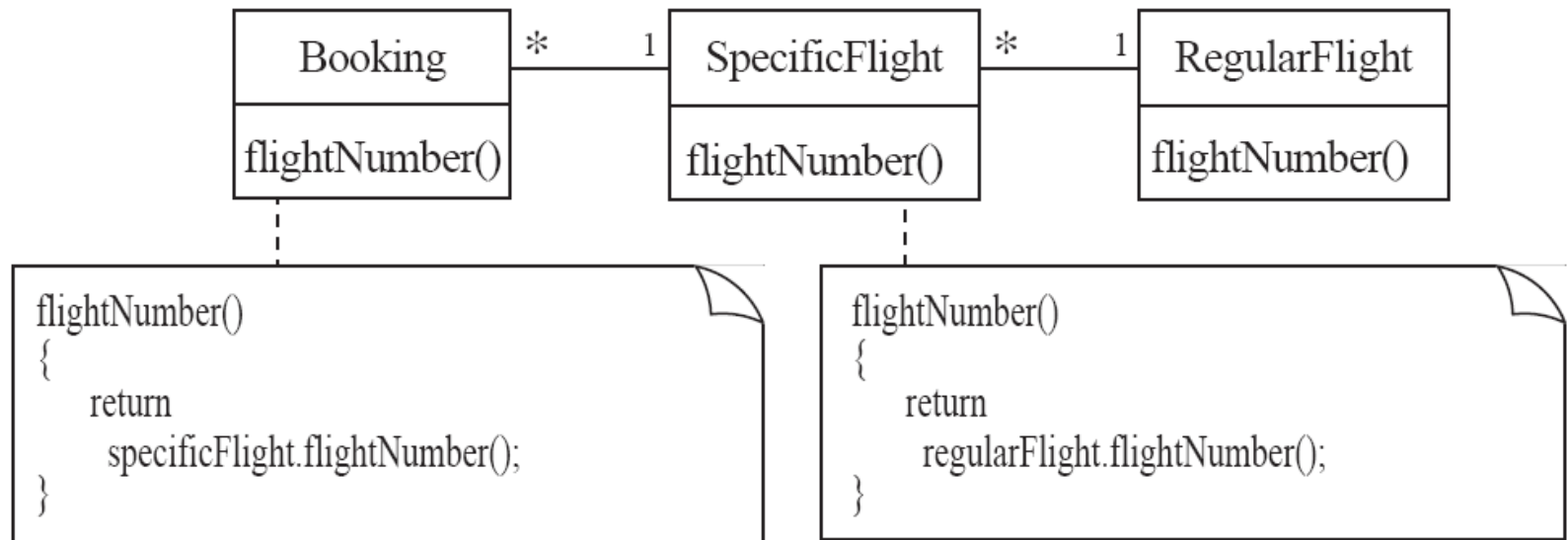
Delegation

- ***Solution:***



Delegation

Example:



Delegation

Antipatterns

- Overuse generalization and *inherit* the method that is to be reused
- Instead of creating a *single* method in the «Delegator» that does nothing other than call a method in the «Delegate»
 - consider having many different methods in the «Delegator» call the delegate's method
- Access non-neighboring classes

```
return specificFlight.regularFlight.flightNumber();

return getRegularFlight().flightNumber();
```

6.8 The Adapter Pattern

- **Context:**

- You are building an inheritance hierarchy and want to incorporate into it an existing class.
- The reused (existing) class is also often already part of its own inheritance hierarchy.

- **Problem:**

- How to obtain the power of polymorphism when reusing a class whose methods
 - have the same function
 - but *not* the same signatureas the other methods in the hierarchy?

- **Forces:**

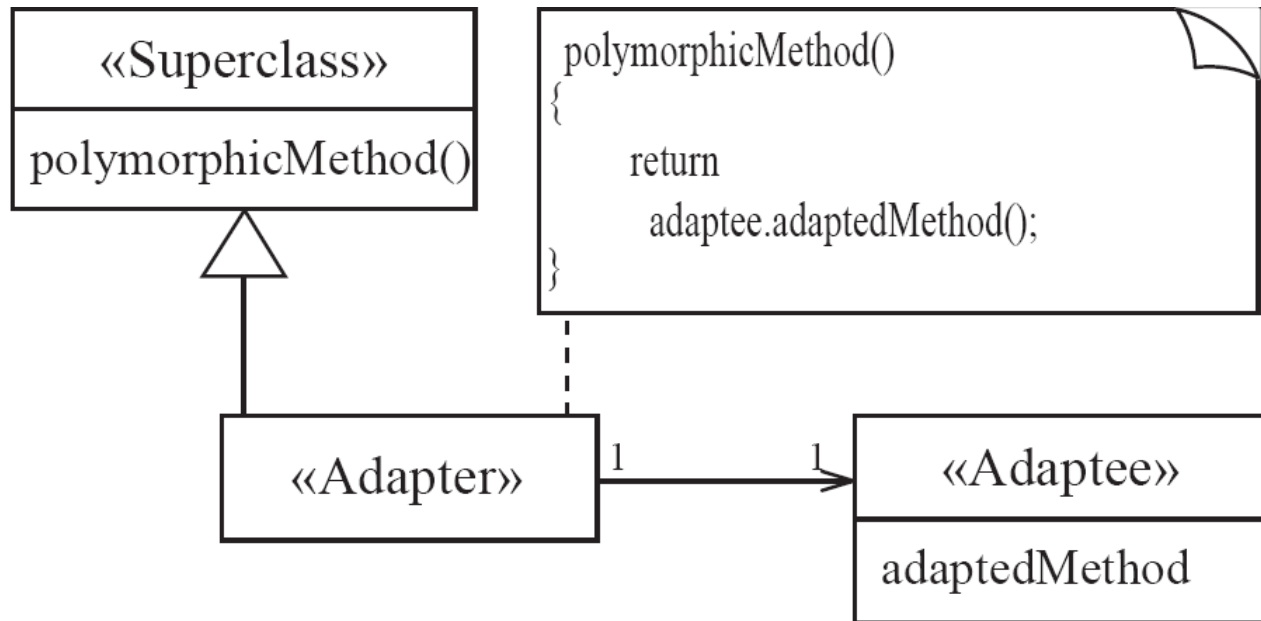
- You do not have access to multiple inheritance or you do not want to use it.

- **References:**

- Adapter is one of the GoF patterns

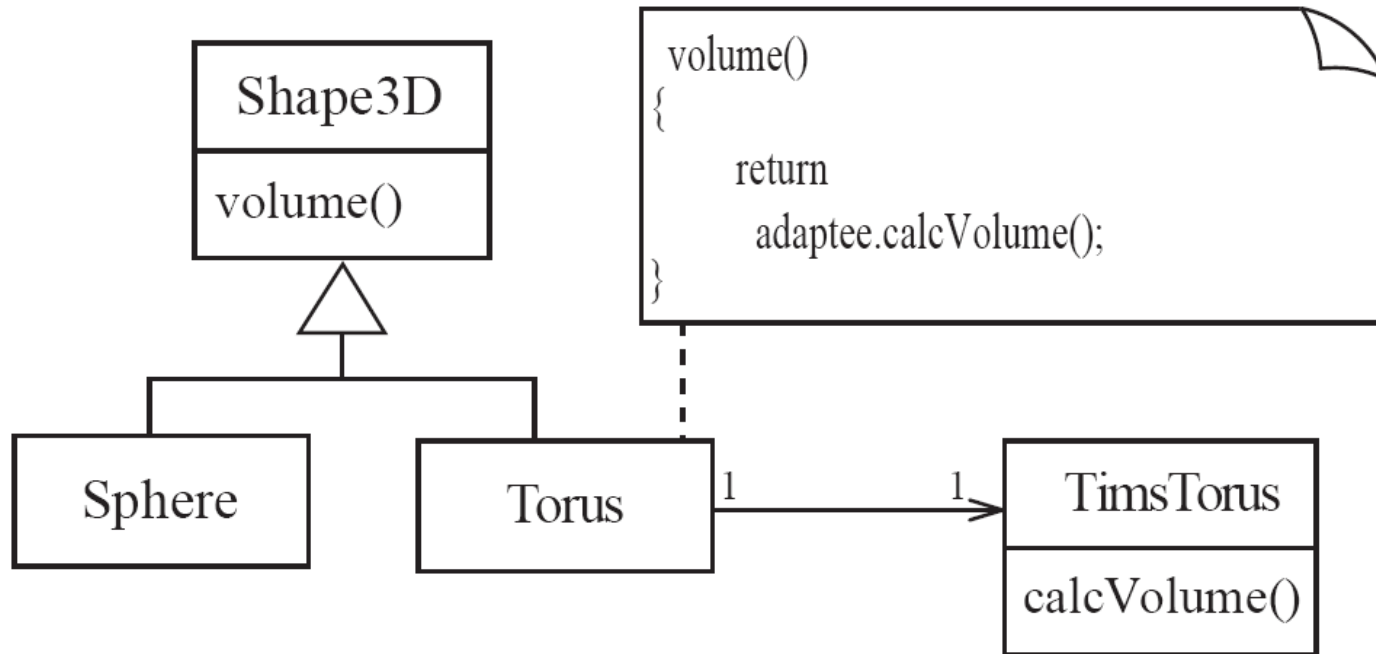
Adapter

- ***Solution:***



Adapter

Example:



6.9 The Façade Pattern

- **Context:**

- Often, an application contains several complex packages.
- A programmer working with such packages has to manipulate many different classes

- **Problem:**

- How do you simplify the view that programmers have of a complex package?

- **Forces:**

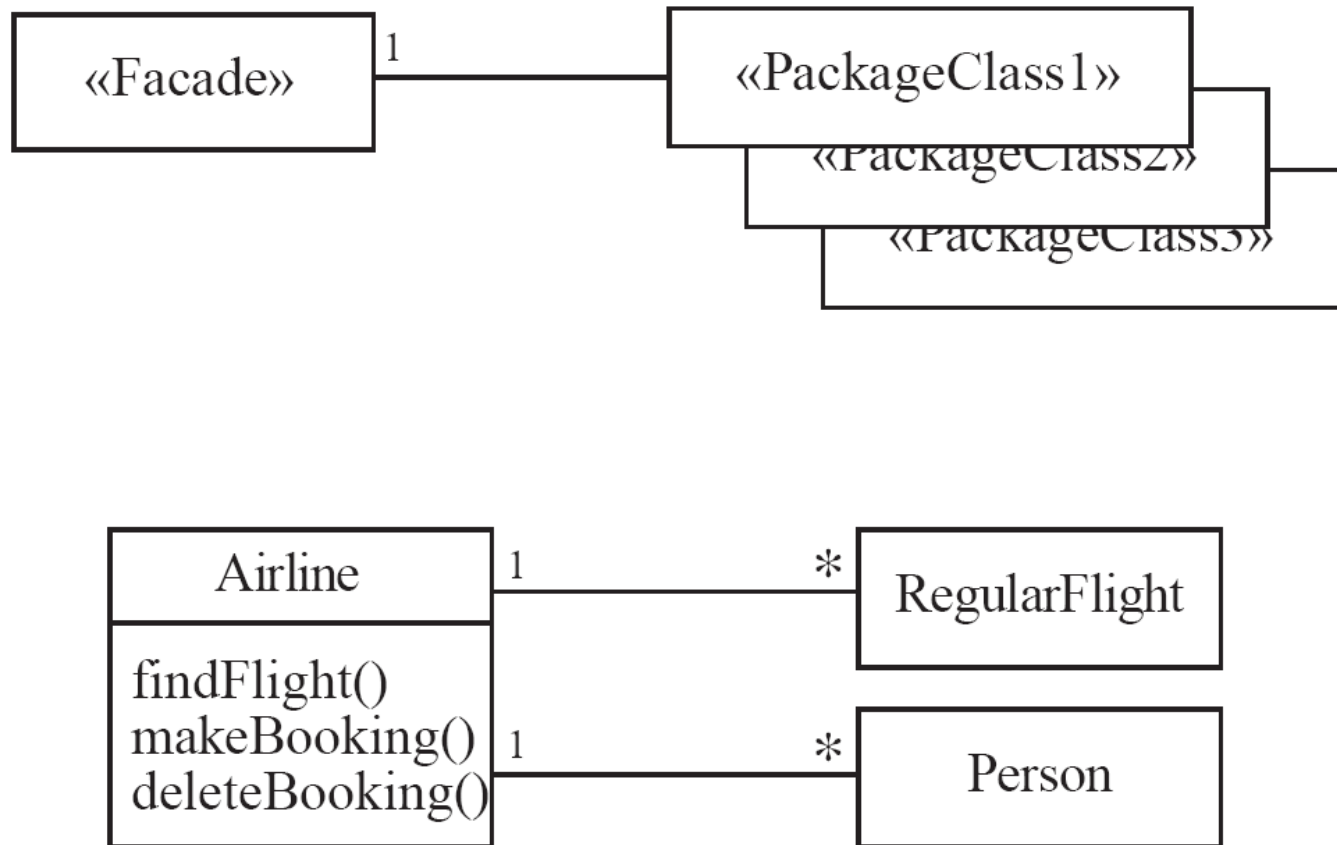
- It is hard for a programmer to understand and use an entire subsystem
- If several different application classes call methods of the complex package, then any modifications made to the package will necessitate a complete review of all these classes.

- **References:**

- Façade is one of the GoF patterns

Façade

- *Solution:*



6.10 The Immutable Pattern

- **Context:**
 - An immutable object is an object that has a state that never changes after creation
- **Problem:**
 - How do you create a class whose instances are immutable?
- **Forces:**
 - There must be no loopholes that would allow ‘illegal’ modification of an immutable object
- **Solution:**
 - Ensure that the constructor of the immutable class is the *only* place where the values of instance variables are set or modified.
 - Instance methods which access properties must not have side effects.
 - If a method that would otherwise modify an instance variable is required, then it has to return a *new* instance of the class.
- **References:**
 - This pattern was introduced in [Gra02]

6.11 The Read-only Interface Pattern

- *Context:*

- You sometimes want certain privileged classes to be able to modify attributes of objects that are otherwise immutable

- *Problem:*

- How do you create a situation where some classes see a class as read-only whereas others are able to make modifications?

- *Forces:*

- Restricting access by using the **public**, **protected** and **private** keywords is not adequately selective.

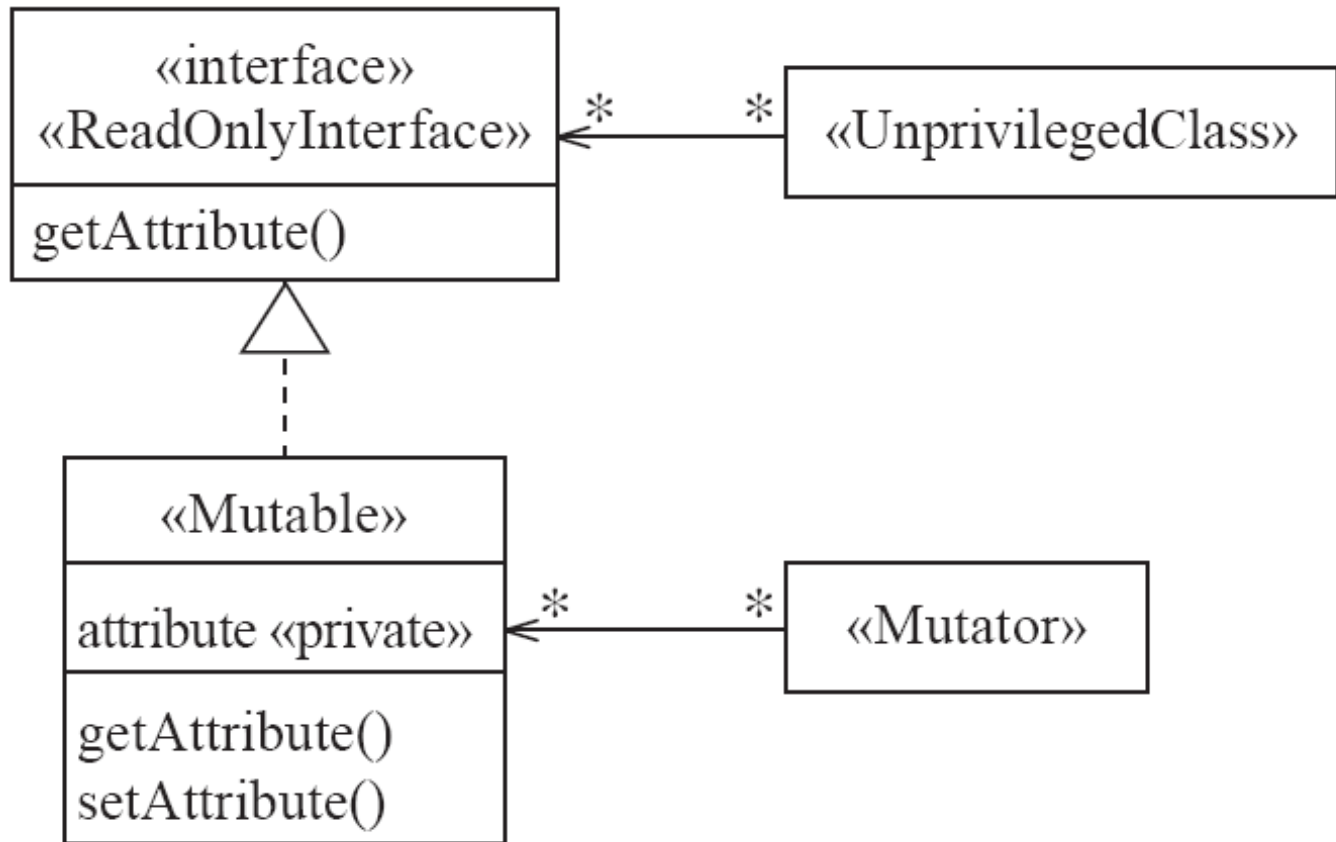
- Making access **public** makes it public for both reading and writing

- *References:*

- This pattern was introduced in [Gra02]

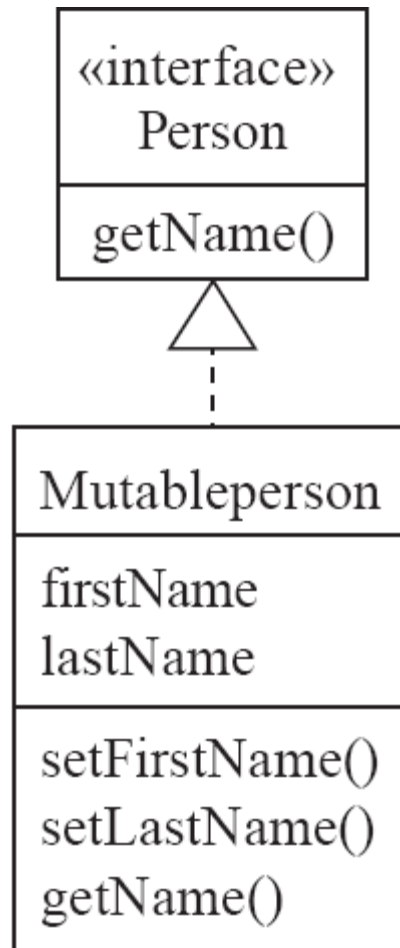
Read-only Interface

- *Solution:*



Read-only Interface

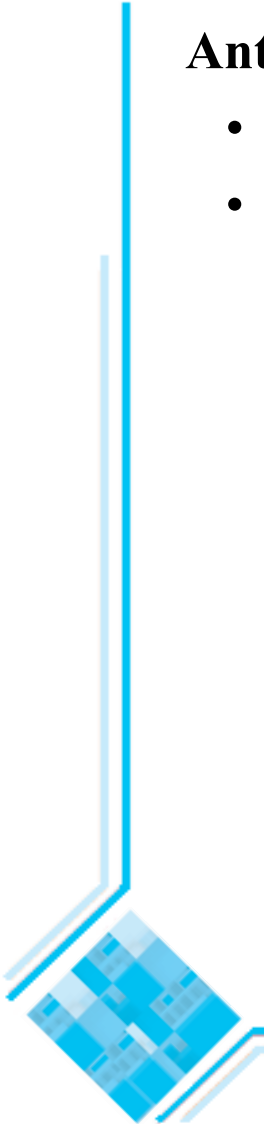
Example:



Read-only Interface

Antipatterns:

- Make the read-only class a *subclass* of the «Mutable» class
- Override all methods that modify properties
 - such that they throw an exception



6.12 The Proxy Pattern

- **Context:**

- Often, it is time-consuming and complicated to create instances of a class (*heavyweight* classes).
- There is a time delay and a complex mechanism involved in creating the object in memory

- **Problem:**

- How to reduce the need to create instances of a heavyweight class?

- **Forces:**

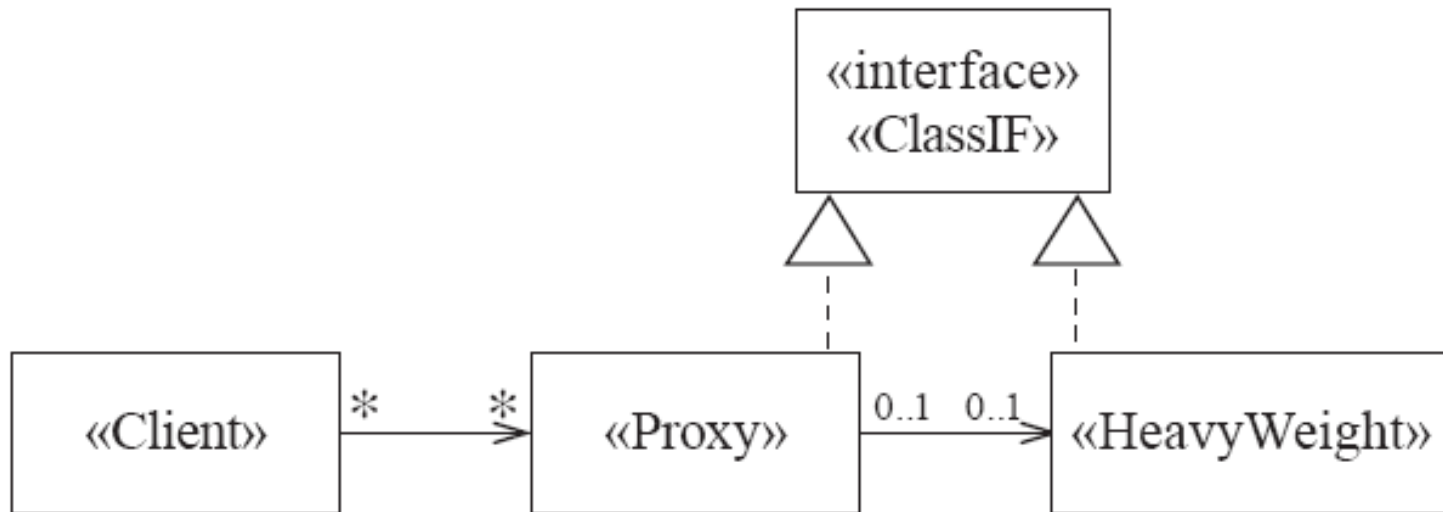
- We want all the objects in a domain model to be available for programs to use when they execute a system's various responsibilities.
- It is also important for many objects to persist from run to run of the same program

- **References:**

- Proxy is one of the GoF patterns

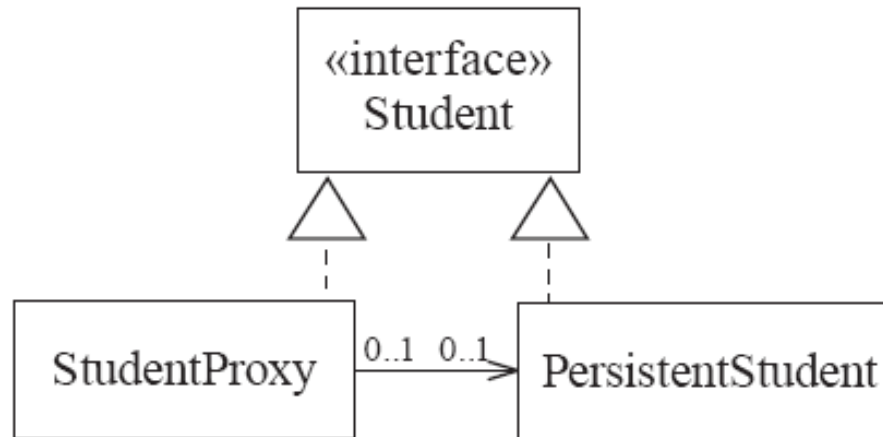
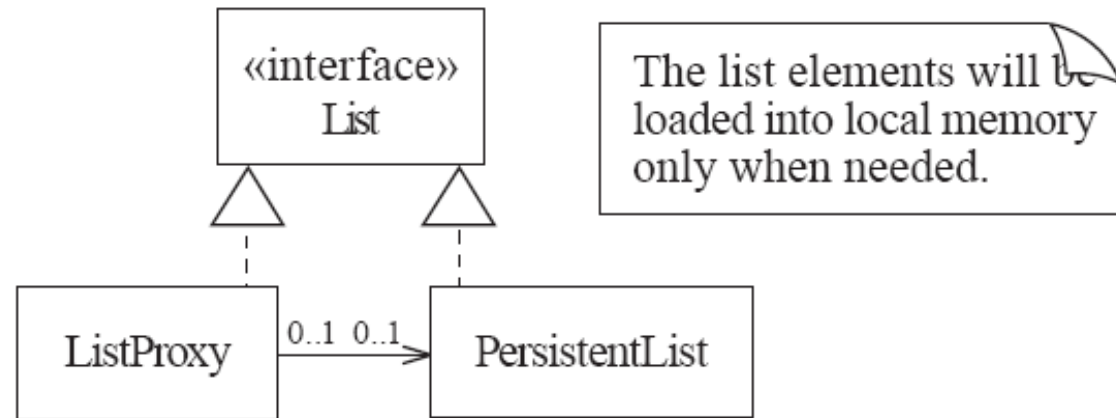
Proxy

- *Solution:*



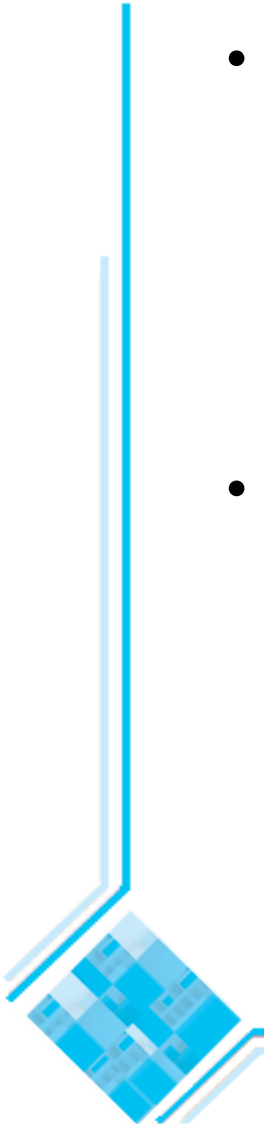
Proxy

Examples:



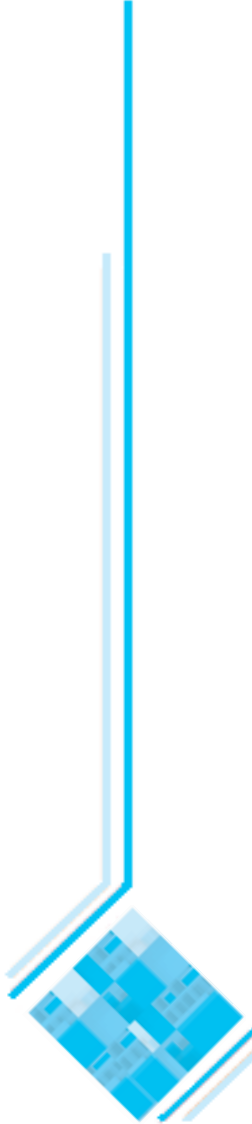
Final remarks

- **Patterns are not a panacea:**
 - Whenever you see an indication that a pattern should be applied, you might be tempted to blindly apply the pattern.
 - This can lead to unwise design decisions .
- *Resolution:*
 - *Always understand in depth the forces that need to be balanced, and when other patterns better balance the forces.*
 - *Make sure you justify each design decision carefully.*



References

- [GoF94] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994. (The four authors of this book are referred to as the “Gang of Four”, frequently abbreviated as “GoF”)
- [EP00] H.-E. Eriksson and M. Penker, *The Business Modeling with UML: Business Patterns at Work*, Wiley, 2000.
- [Gra02] M. Grand, *The Patterns in Java Volume 1: A Catalog of Reusable Design Patterns Illustrated with UML*, 2nd edition, Wiley, 1999.
- [LL05] T. Lethbridge and R. Laganriere, *Object-Oriented Software Engineering – Practical Software Development using UML and Java*, 2nd edition, McGraw-Hill, 2005
- [Rum91] J. Rumbaugh et al., *Object-Oriented Modeling and Design*, Prentice Hall, 1991.
- [Nik16] I. Nikolov. *Scala Design Patterns*, Packt Publishing, 2016.



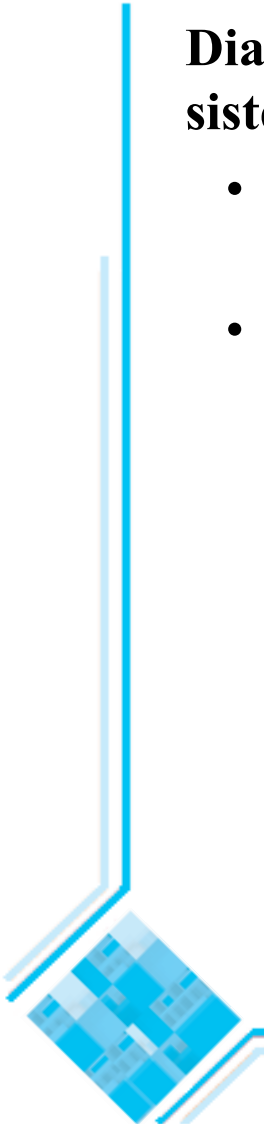
Modelare interactiuni si comportament

Adapted from:
Timothy Lethbridge and Robert Laganriere,
Object-Oriented Software Engineering –
Practical Software Development using UML and Java, 2005
(chapter 8)

8.1 Diagrame de interactiune

Diagramele de interactiune modeleaza aspecte dinamice ale sistemelor software

- O diagrama de interactiune vizualizeaza actori si obiecte care interactioneaza prin mesaje.
- In mod normal, o asemenea diagrama se construiește pe baza unui caz de utilizare si a unei diagrame de clase.

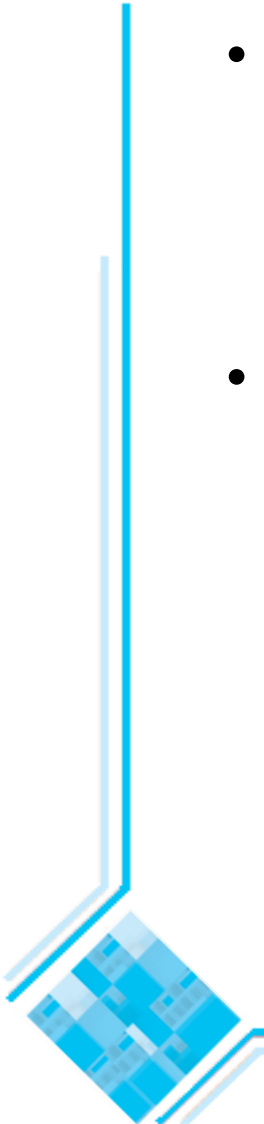


Interactiuni si mesaje

- Diagramele de interactiune vizualizeaza modul in care un grup de obiecte si actori comunica pentru a efectua actiunile corespunzatoare
 - unui caz de utilizare sau
 - unui alt fragment de functionalitate.
- Diagramele de interactiune pot sa vizualizeze diverse tipuri de *mesaje* ce pot fi schimbate intre obiecte:
 - » Simple apeluri de procedura
 - » Comenzi date de un actor prin interfata utilizator
 - » Mesaje schimbate in retea
- Diagramele de interactiune vizualizeaza secvente de mesaje.
 - Elementele diagramelor de interactiune:
 - Obiecte
 - Actori
 - Mesaje
 - » reprezentate sub forma de sageti (etichetate)

Crearea diagramelor de interactiune

- Deoarece trebuie sa fie cunoscuti actorii si obiectele implicate in interactiune, in mod normal se creeaza mai intai un model al cazurilor de utilizare si o diagrama de clase.
- Pentru vizualizarea interactiunilor se utilizeaza doua tipuri de diagrame:
 - Diagramme de secventiere*
 - Diagramme de comunicare*



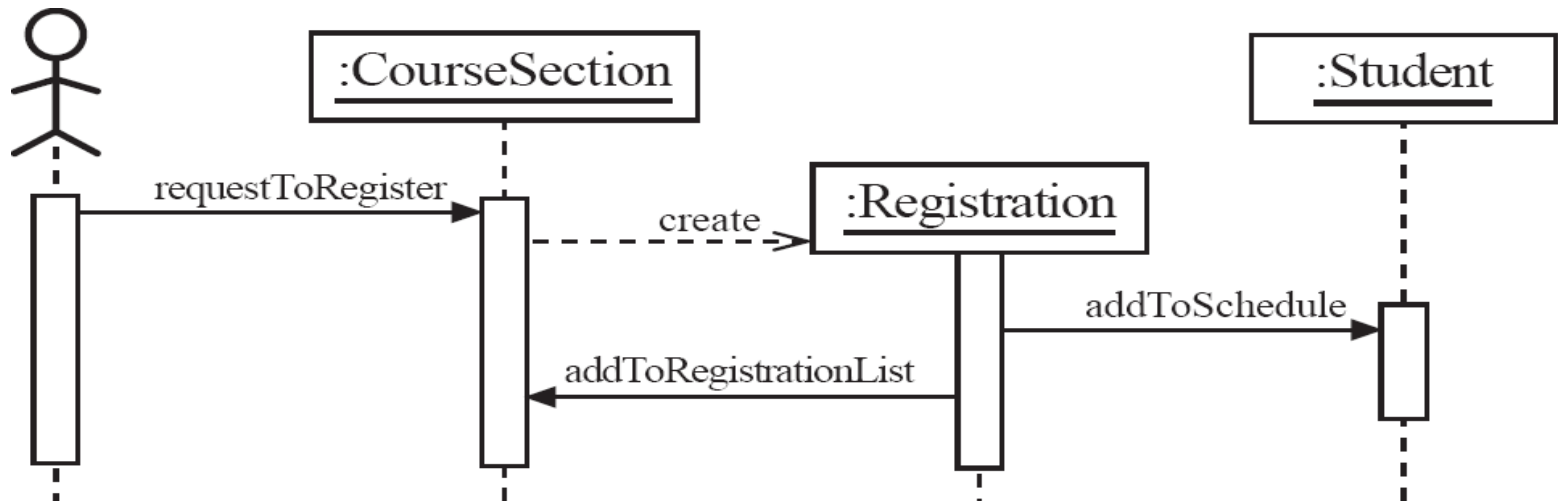
Diagrame de secventiere

O *diagrama de secventiere* vizualizeaza secventa de mesaje schimbate de un grup de obiecte cu scopul efectuării unei anumite sarcini de calcul.

- Obiectele sunt aranjate in diagrama pe orizontala.
- Adesea, in partea stanga este vizualizat un actor care initiaza interactiunea.
- Dimensiunea verticala reprezinta timpul; partea de sus a diagramei corespunde inceputului interactiunii, timpul progresand spre partea de jos a diagramei.
- O linie verticala intrerupta (engl. *lifeline*) este atasata fiecarui obiect sau actor.
- Aceasta devine un dreptunghi, numit *caseta de activare* (engl. *activation box*), pe perioada de *activare* (engl. *live activation*) in care obiectul efectueaza sarcini de calcul.
- Un *mesaj* este reprezentat ca o sageata intre casetele de activare ale obiectelor transmitator si receptor.
 - Mesajele sunt etichetate; optional, orice mesaj poate avea o lista de argumente si un raspuns (o valoare returnata). Sintaxa completa este:
$$\text{raspuns} := \text{mesaj}(\text{arg}, \dots)$$
 - Un *mesaj de creare* este reprezentat printr-o linie intrerupta cu eticheta ‘create’.

Exemplu de diagrama de secventiere

Diagrama de secventiere pentru o operatie de inregistrare student:



- Etichetele mesajelor din diagrama de secventiere corespund operatiilor din diagrama de clase (data mai jos).
- Receptionarea unui mesaj de catre un obiect implica executia uneia dintre metodele obiectului.
- Diagramele de secventiere permit identificarea operatiilor claselor.

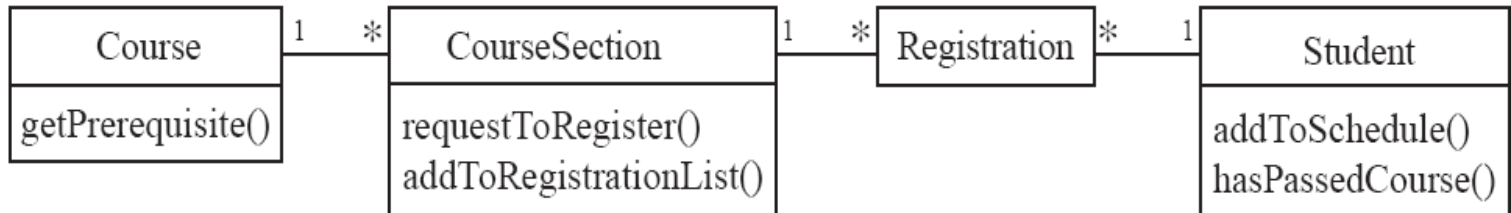
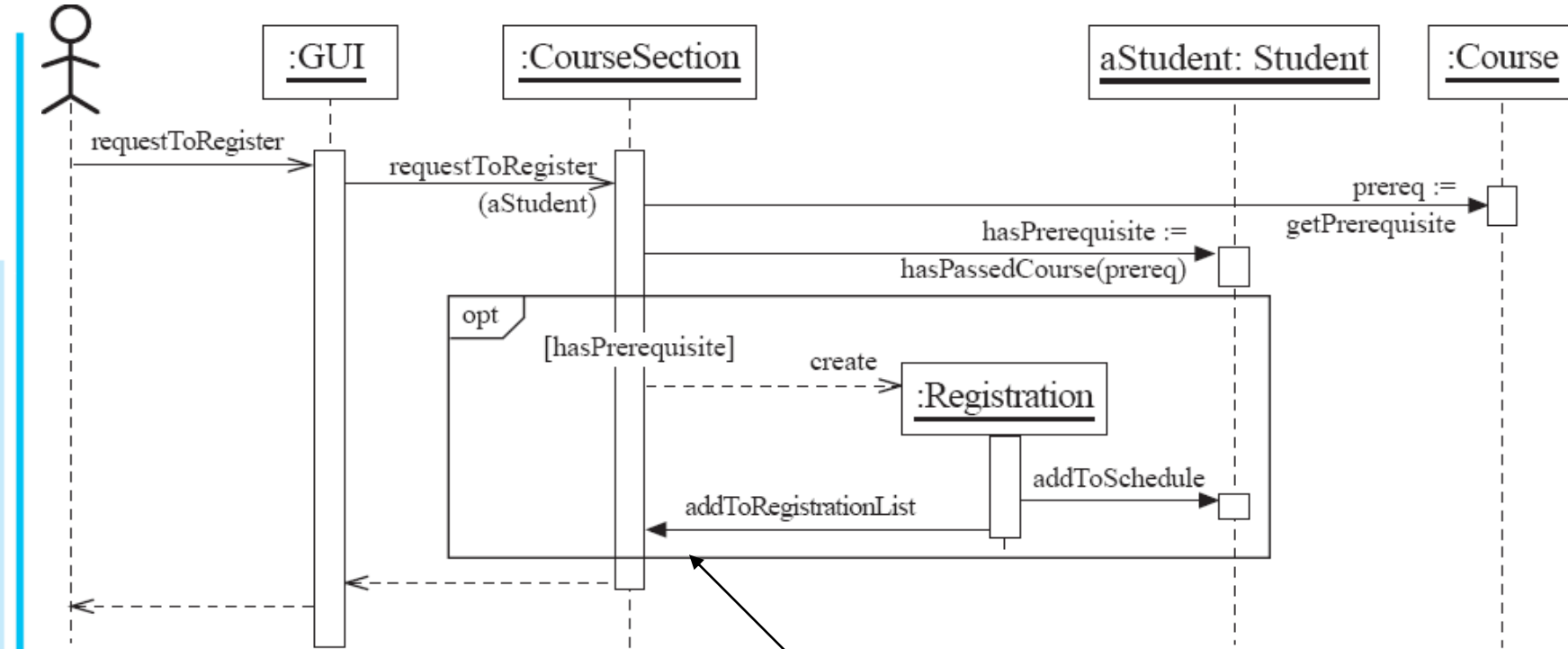


Diagrama de secventiere - acelasi exemplu, mai multe detalii



- Acesta este un **fragment combinat** (concept introdus in UML 2.0)
- Vizualizeaza o subsecventa a unei interactiuni.
 - `[hasPrerequisite]` este o conditie Booleana care trebuie sa fie satisfacuta pentru ca subsecventa sa se execute.
 - 'opt' este o eticheta care indica faptul ca executia subsecventei este optionala.

Diagrama de secventiere – fragment combinat pentru procese iterative

- Intr-o diagrama de secventiere se poate reprezenta un proces iterativ de calcul utilizand un fragment combinat cu eticheta 'loop'.
- Numarul de iteratii este specificat utilizand sintaxa: min .. max.

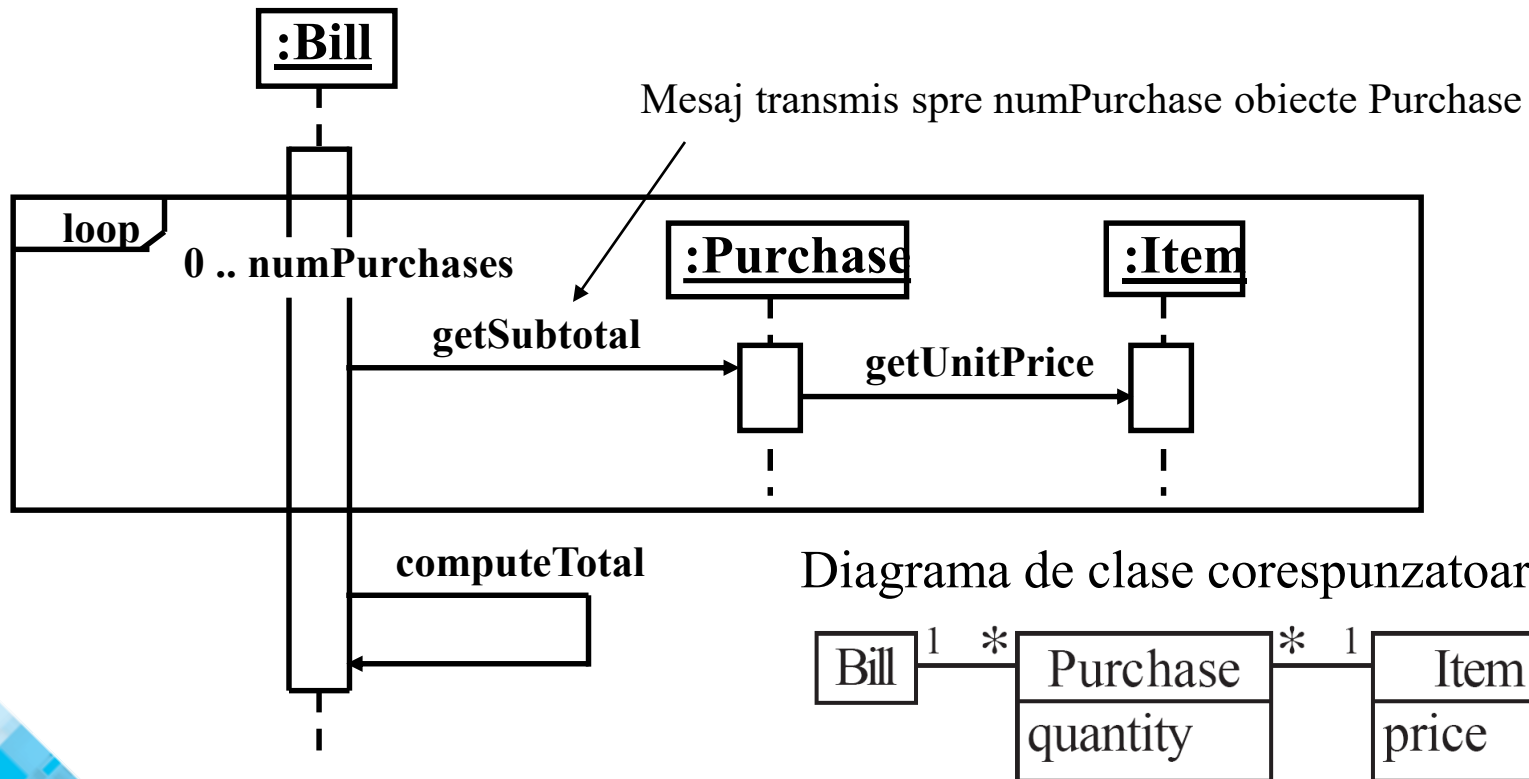
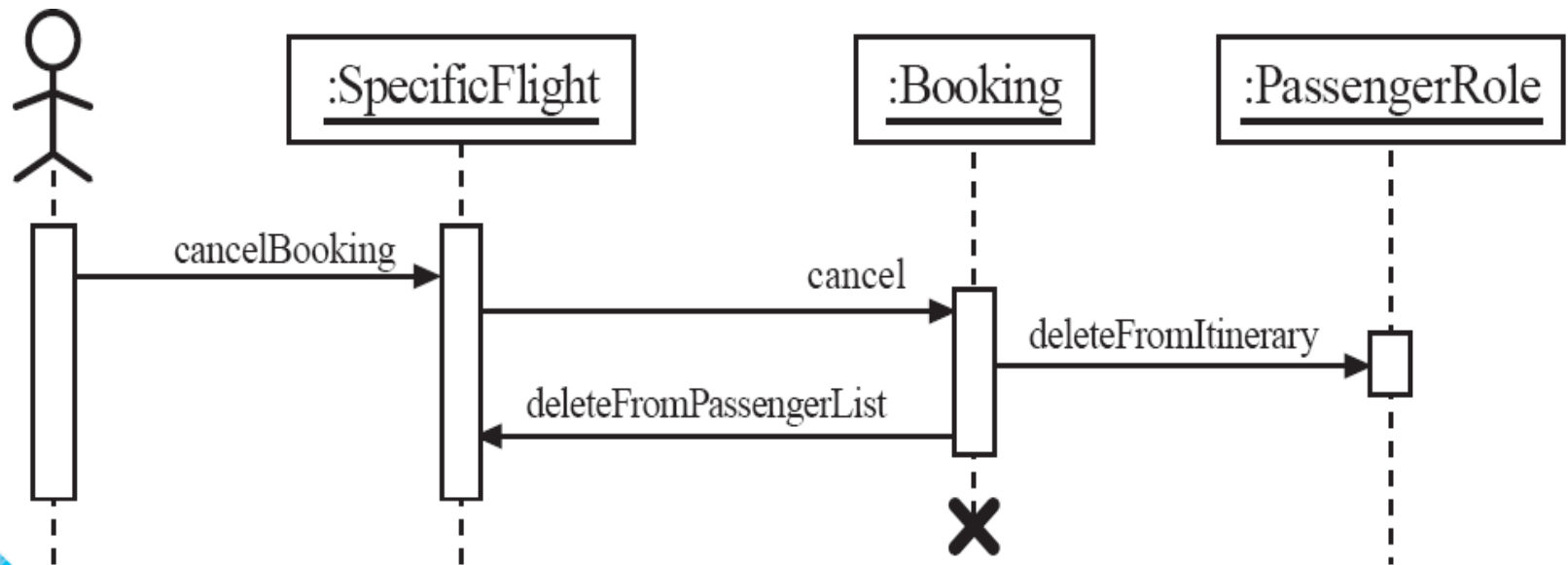


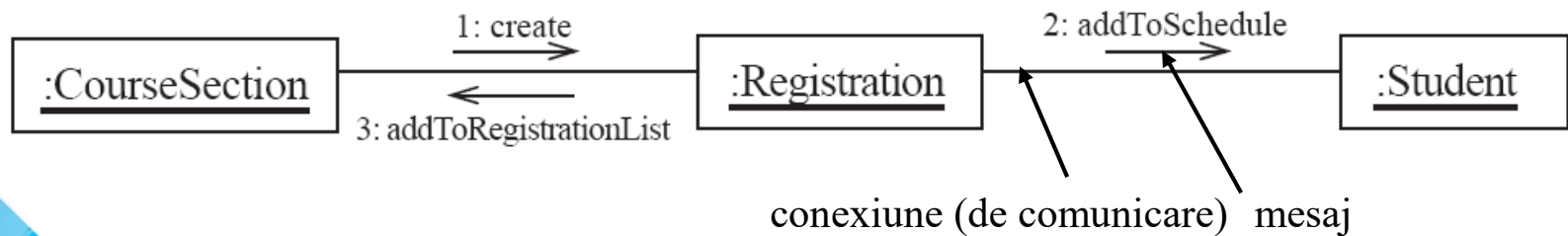
Diagrama de secventiere - un exemplu cu distrugere de instanta

- Se poate utiliza un 'X' pentru a se indica momentul in care un obiect este distrus.
- Aceasta diagrama descrie o posibila interactiune pentru anulara unei rezervari la o cursa aeriana (pentru sistemul introdus in capitolul 5).



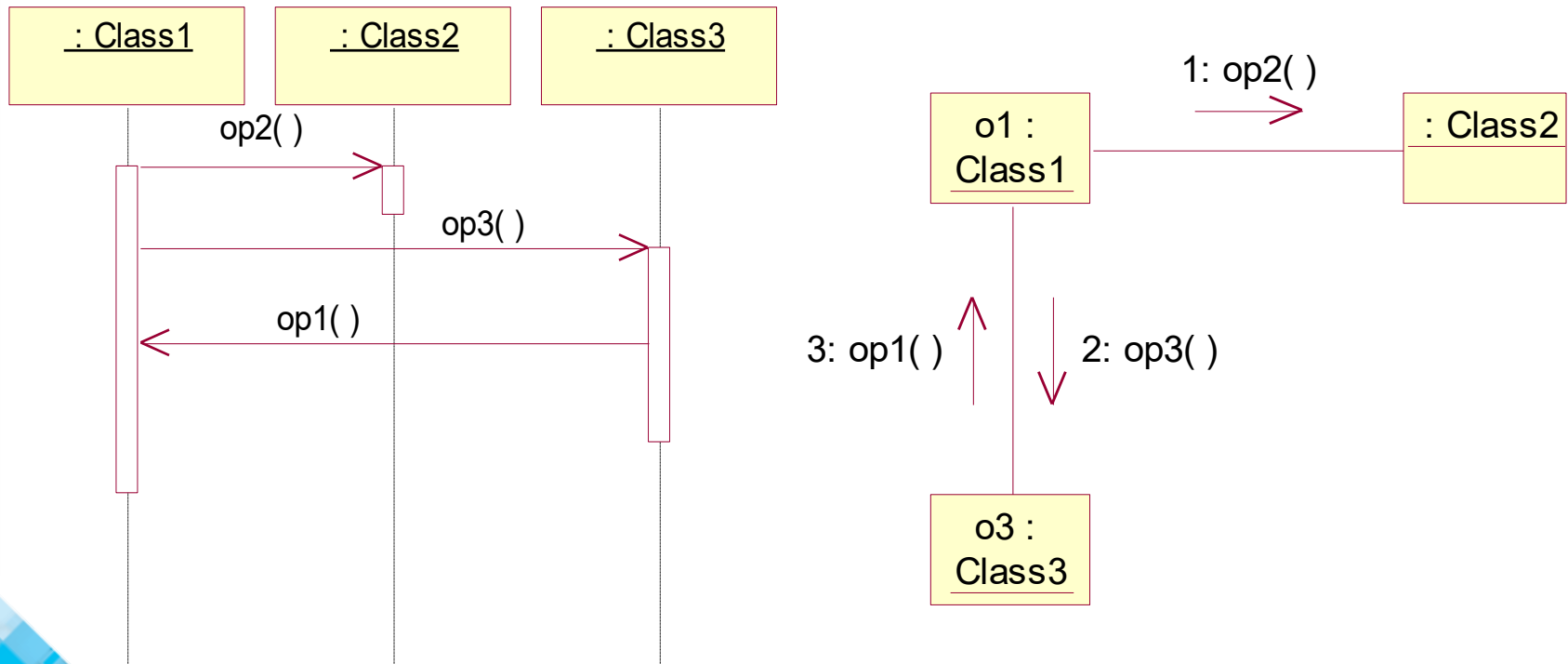
Diagrame de comunicare

- O *diagrama de comunicare* (numita *diagrama de colaborare* in UML 1.x) este o diagrama de interactiune care:
 - vizualizeaza obiectele in doua dimensiuni (un graf cu obiecte in noduri)
 - utilizeaza o schema de numerotare pentru a indica ordinea mesajelor.
- Diagramele de comunicare si diagramele de secventiere
 - utilizeaza scheme diferite de dispunere a obiectelor,
 - dar sunt foarte asemanatoare (in UML 1.x cele doua tipuri de diagrame sunt semantic echivalente).
- Diagramele de comunicare sunt ceva mai putin expresive decat diagramele de secventiere.
 - Fragmentele combinate - introduse in UML 2.0 (2004) – lipsesc in diagramele de comunicare.



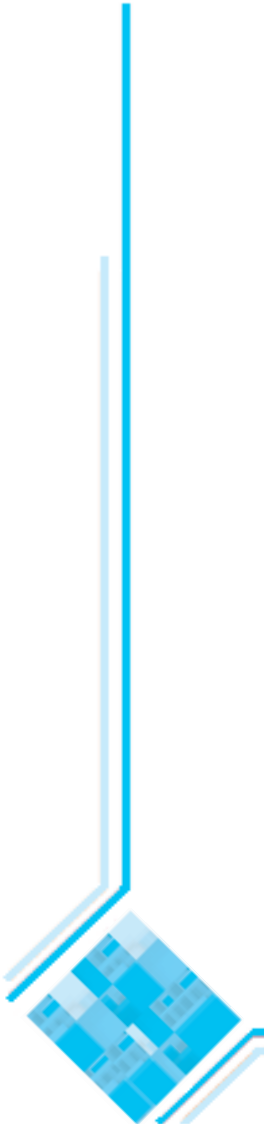
Diagrame de comunicare si diagrame de secventiere

Observatie: In Rational Rose 2002 (care utilizeaza UML 1.x), cu tasta F5 se poate comuta intre o diagrama de comunicare si o diagrama de secventiere echivalenta: cele doua reprezentari sunt izomorfe.



Conexiuni de comunicare

- Intre doua obiecte (dintr-o diagrama de comunicare) poate exista o conexiune de comunicare oricand exista posibilitatea ca un obiect sa transmita un mesaj celuiilalt obiect.
- Clasele celor doua obiecte care interactioneaza pot fi conectate
 - printr-o asociere, sau
 - printr-o dependenta.



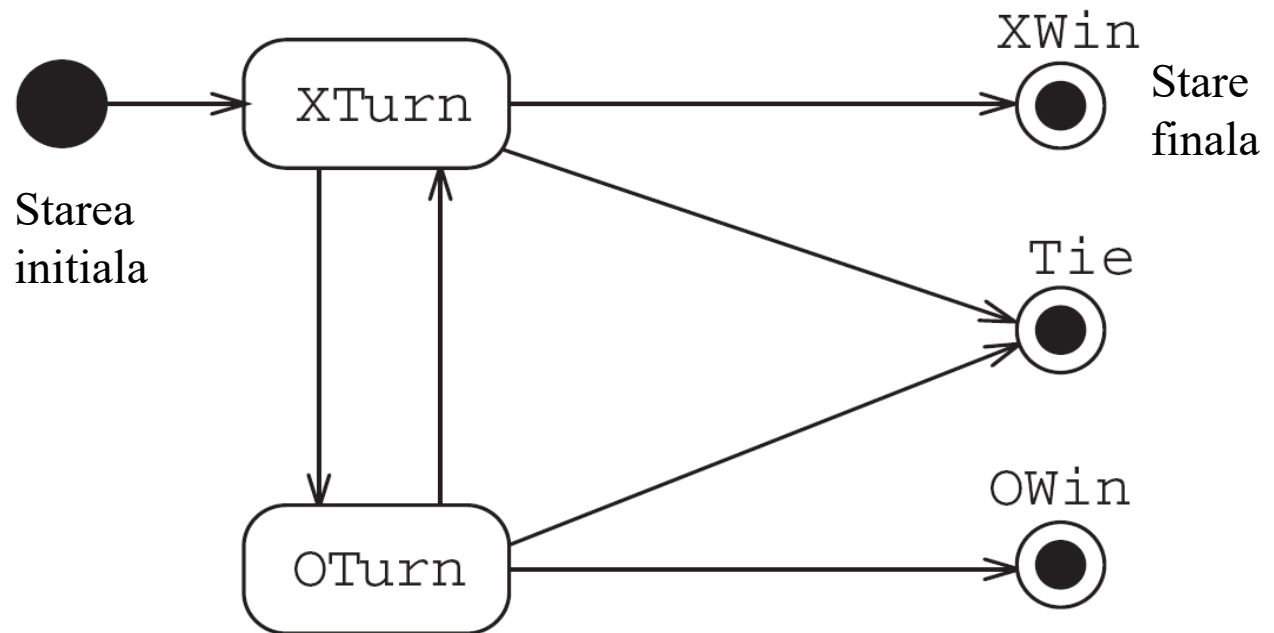
8.2 Diagrame de stare

O *diagrama de stare* (sau *diagrama de tranzitie de stare*) descrie comportamentul unui (sub)sistem, sau comportamentul unui *obiect individual*.

- In fiecare moment, sistemul sau obiectul se afla intr-o anumita stare.
 - Starea reflecta comportamentul sistemului ca reactie la evenimentele care apar.
- Unele evenimente pot determina sistemul sa isi modifice starea.
- O diagrama de stare este un graf etichetat in care nodurile sunt stari, iar arcele sunt tranzitii (etichetate).
- O tranzitie reprezinta o modificare a starii ca raspuns la un eveniment
 - Se considera ca tranzitiile se efectueaza instantaneu.
 - Eticheta unei tranzitii reprezinta evenimentul care determina modificarea starii.

Diagrame de stare – un prim exemplu

- Diagrama de stare pentru jocul tic-tac-toe (X si 0):



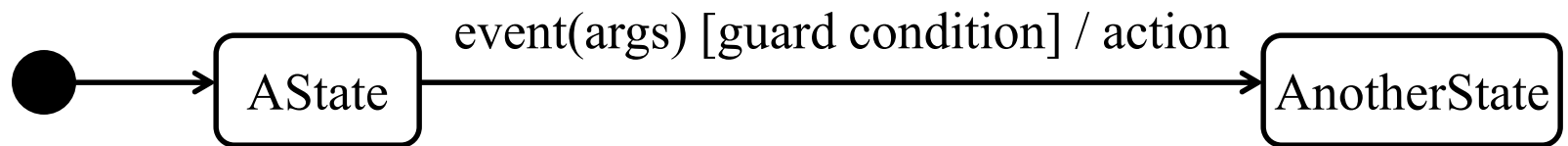
—In UML, o *stare* este reprezentata ca un dreptunghi cu colturi rotunjite, care contine numele starii (exista simboluri speciale pentru starea initiala si starile finale).

—O *tranzitie* este o sageata care leaga doua stari (in acest prim exemplu toate tranzitiile sunt neetichetate).

Elementele diagramelor de stare

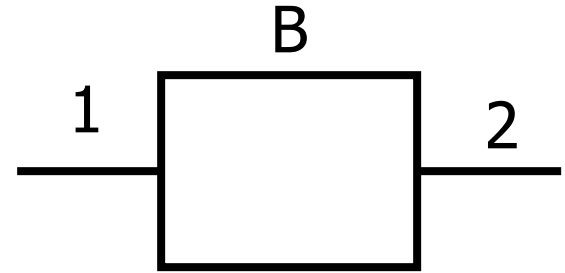
In general, o diagrama de tranzitie de stare (DTS) include:

- stari
- tranzitii; specificarea unei tranzitii poate include (optional):
 - un eveniment care cauzeaza tranzitia (cu argumente optionale),
 - o conditie de gardare data intre paranteze drepte (daca exista mai mult de o tranzitie care pleaca din starea respectiva atunci conditiile de gardare trebuie sa fie mutual exclusive) si
 - o actiune, care este un comportament atomic (care nu poate fi intrerupt) atasat tranzitiei.



Exemplu: Specificare DTS pentru un buffer de capacitate unu

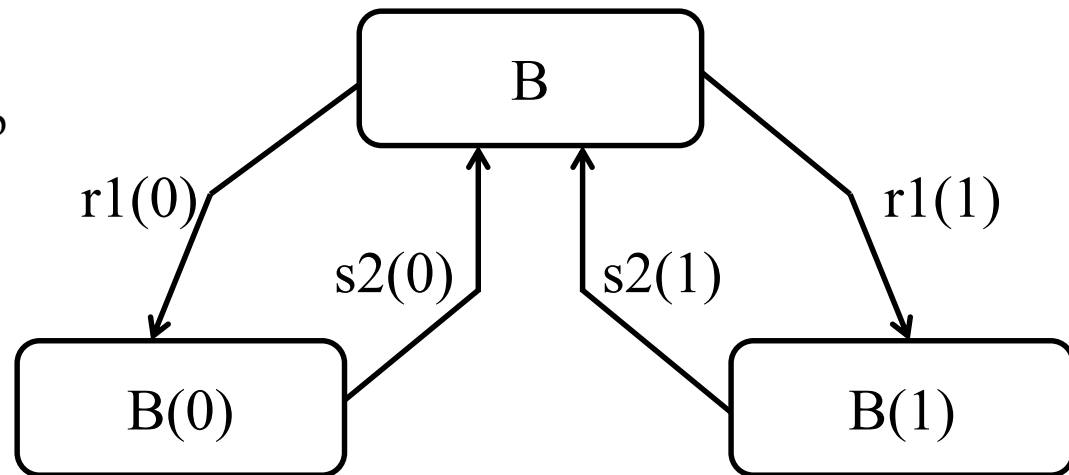
- Desenul alaturat sugereaza structura unui buffer cu capacitate de un bit.
 - Un asemenea buffer B poate accepta un bit la portul de intrare (1); atunci cand este incarcat cu un bit il poate transmite la portul de iesire (2).



- In DTS alaturata este utilizata urmatoarea conventie:

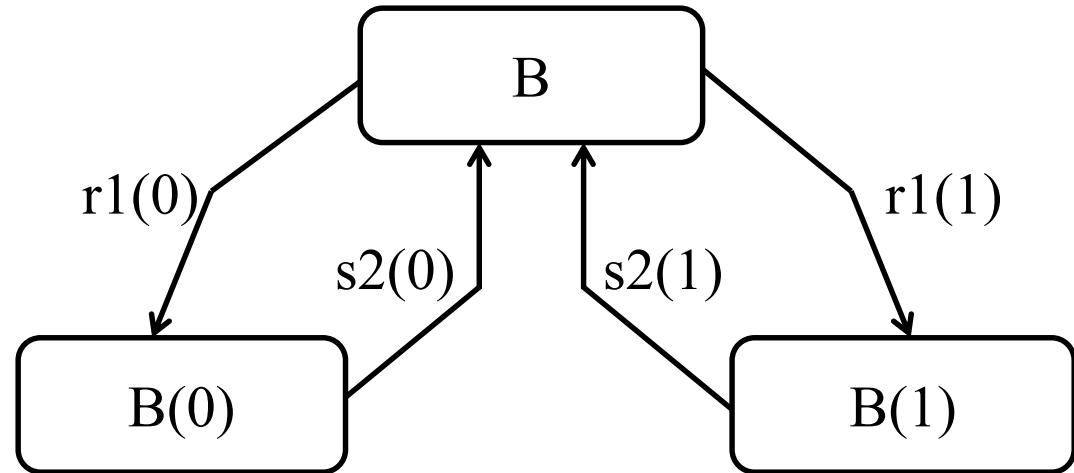
- $r1(b)$ = receptionare bit b la portul 1
- $s2(b)$ = transmitere bit b la portul 2

- Aceasta DTS descrie un automat cu stari finite.



Diagrame de tranzitie de stare si specificari formale

- O diagrama de tranzitie de stare (DTS) ofera o descriere precisa a comportamentului unui sistem.



- Utilizand o notatie de tip algebra de proces [BW90], comportamentul unui buffer de capacitate unu poate fi specificat prin urmatoarea ecuatie:

$$B = r1(0) . s2(0) . B + r1(1) . s2(1) . B$$

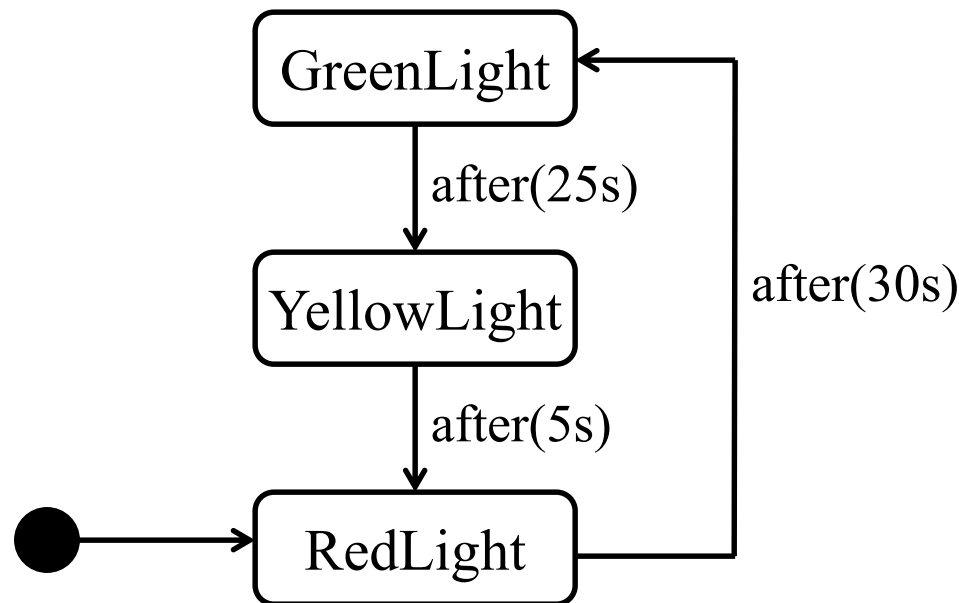
Aici

- ‘.’ este operatorul de compunere secventiala
- ‘+’ este operatorul de alegere nedeterminista
- In cazul in care componenta de tip buffer poate stoca nu doar biti, ci elemente ale unei multimi finite de date ($d \in D$), se obtine o ecuatie de forma:

$$B = \sum_{d \in D} r1(d) . s2(d) . B$$

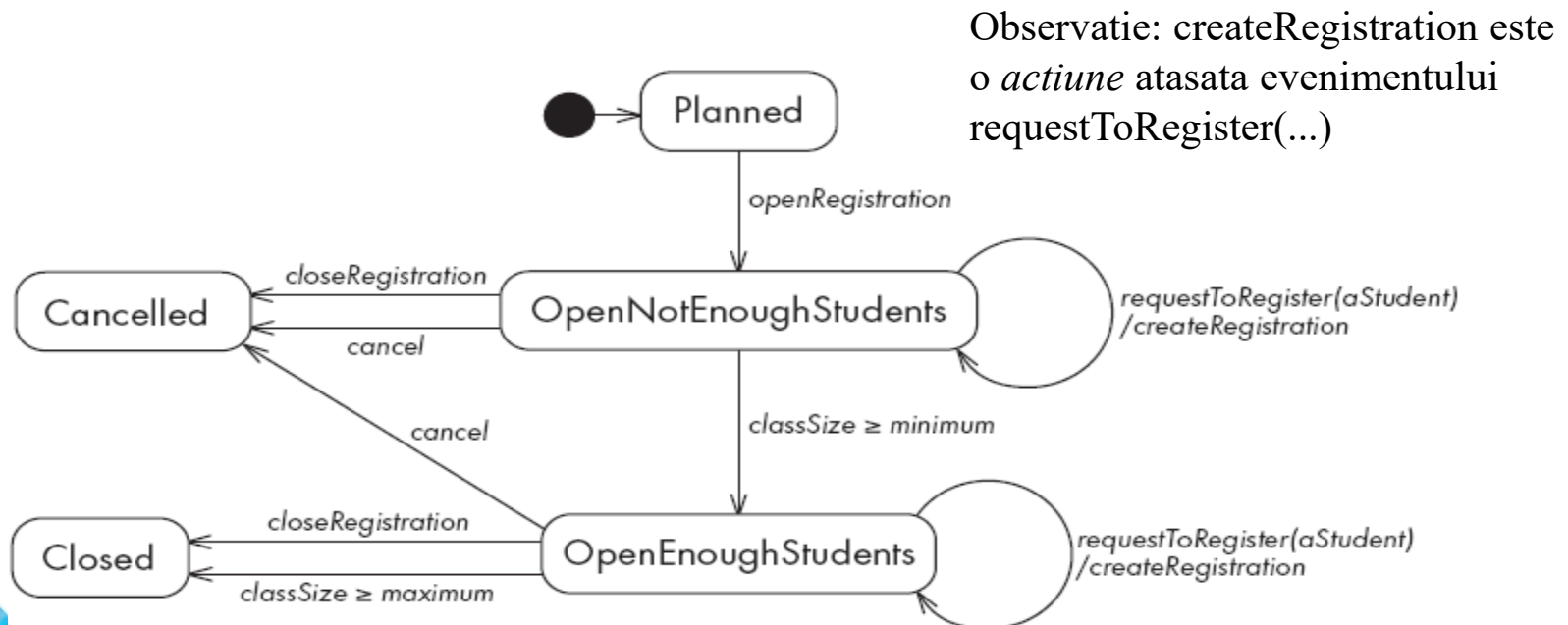
Tranzitii declansate de scurgerea unor durate de timp

- Evenimentul care declanseaza o tranzitie poate fi dat de scurgerea unei anumite durate de timp.
- DTS data mai jos ilustreaza acest concept in modelarea unui semafor simplu.



Diagrame de tranzitie de stare – exemplu cu tranzitii conditionale

- Aceasta DTS descrie comportamentul instantelor clasei CourseSection.
 - Include doua tranzitii declansate cand anumite conditii devin adevarate.
 - De exemplu, in starea OpenNotEnoughStudents un obiect CourseSection face o tranzitie in starea OpenEnoughStudents (in care cursul poate fi predat) de indata ce conditia ($classSize \geq minimum$) devine adevarata.



Activitati in diagrame de stare

- O *activitate* este un proces care se desfasoara in timp ce sistemul se gaseste intr-o anumita stare.
 - O activitate se desfasoara pe o anumita durata (care nu este atomica) de timp.
 - O activitate este reprezentata in interiorul unei stari precedata de “do /”.
 - Sistemul efectueaza o tranzitie prin care paraseste starea curenta ca raspuns la finalizarea activitatii (daca nici o alta tranzitie nu este declansata inainte de un alt eveniment).

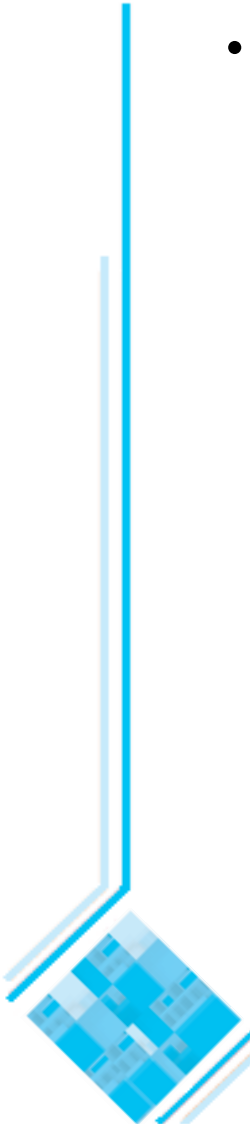
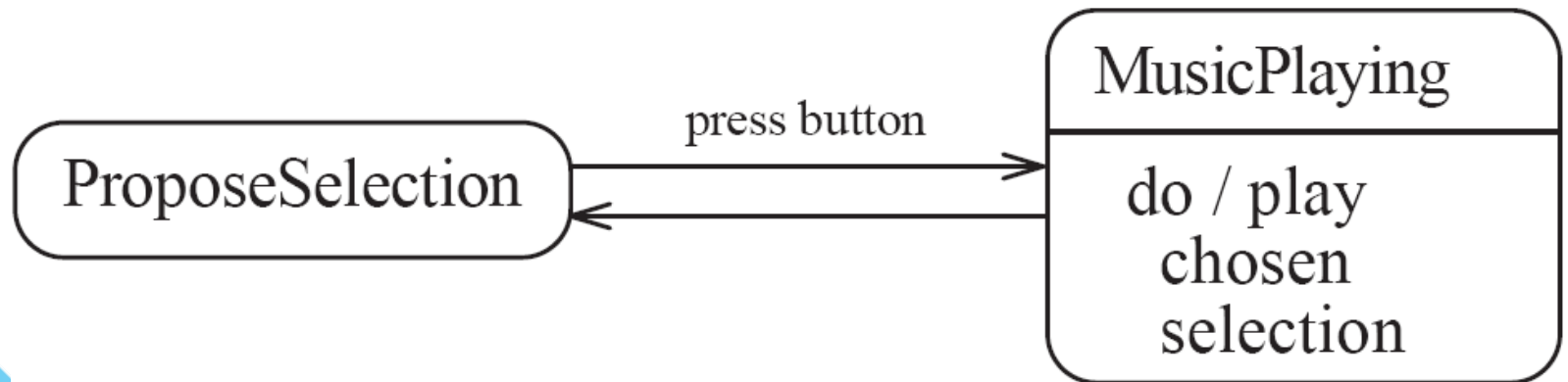


Diagrama de stare – un exemplu cu o activitate

Diagrama de stare pentru un tonomat:

- In starea ProposeSelection sistemul asteapta ca utilizatorul sa apese un buton (selectarea unei piese muzicale)
- In starea MusicPlay sistemul interpreteaza piesa muzicala aleasa pana la sfarsit (activitatea “do / play chosen selection”).
 - Apoi sistemul face o tranzitie inapoi in starea ProposeSelection.



Actiuni in diagrame de stare

- O *actiune* este o activitate atomica (instantanee) care are loc
 - Atunci cand se efectueaza o anumita tranzitie (in acest caz actiunea este atasata unui eveniment)
 - La intrarea intr-o stare
 - O actiune de intrare este reprezentata in interiorul starii, precedata de sirul “entry /”
 - La iesirea dintr-o stare
 - O actiune de iesire este reprezentata in interiorul starii, precedata de sirul “exit /”

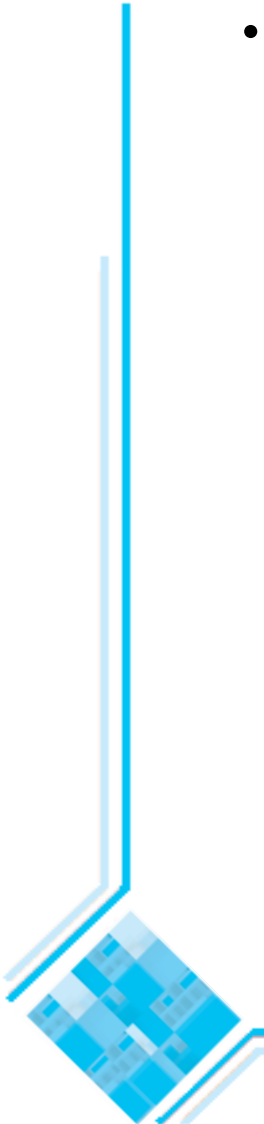


Diagrama de stare – un exemplu cu actiuni

Diagrama de stare pentru un sistem de deschidere si inchidere automata a unei usi de garaj

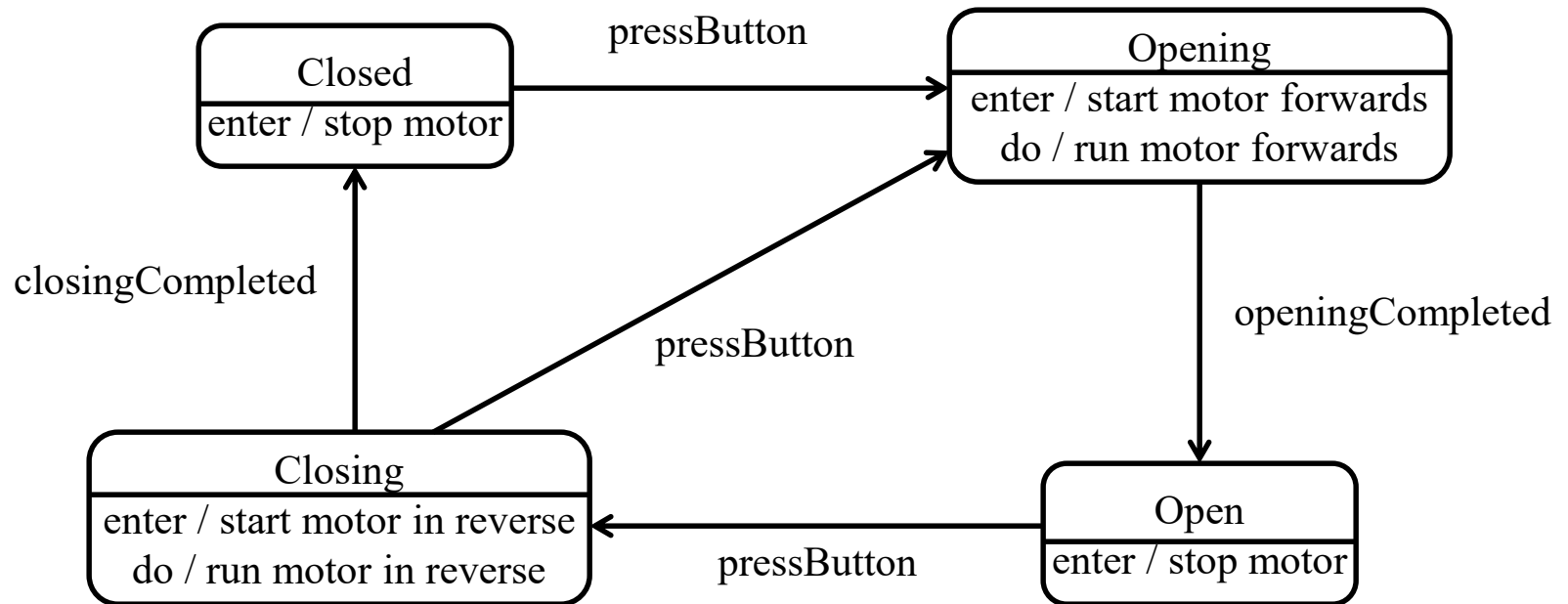
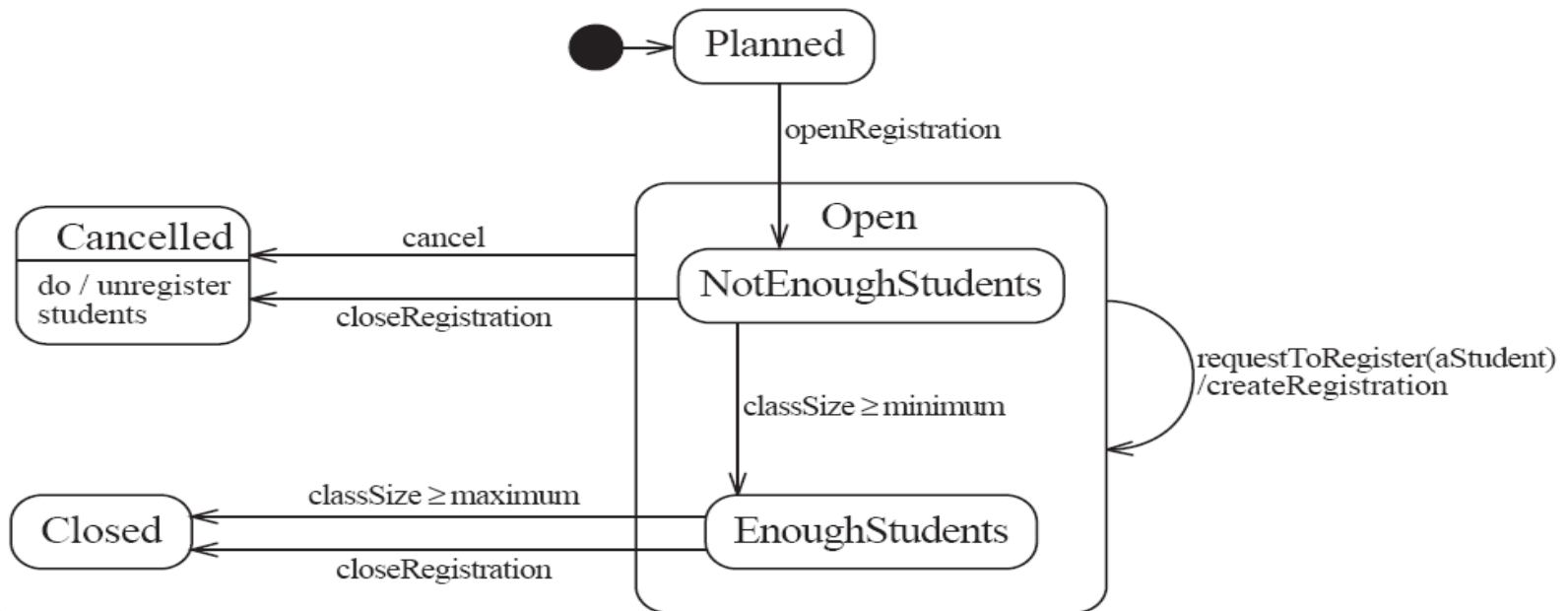


Diagrama de stare – un exemplu cu substari

- O diagrama de stare poate fi incuibata in interiorul unei stari.
 - Starile diagramei interioare se numesc *substari*.
 - Daca doua sau mai multe stari au o tranzitie identica, ele pot fi grupate intr-o *superstare*. In loc sa se pastreze doua tranzitii identice (cate una pentru fiecare substare) tranzitia poate fi atasata la superstare.
- Figura de mai jos vizualizeaza comportamentul instantelor clasei CourseSection, modificat pentru a utiliza stari interioare. Este acum suficient a se reprezinte o singura tranzitie ‘cancel’ si o singura tranzitie ‘requestToRegister’.

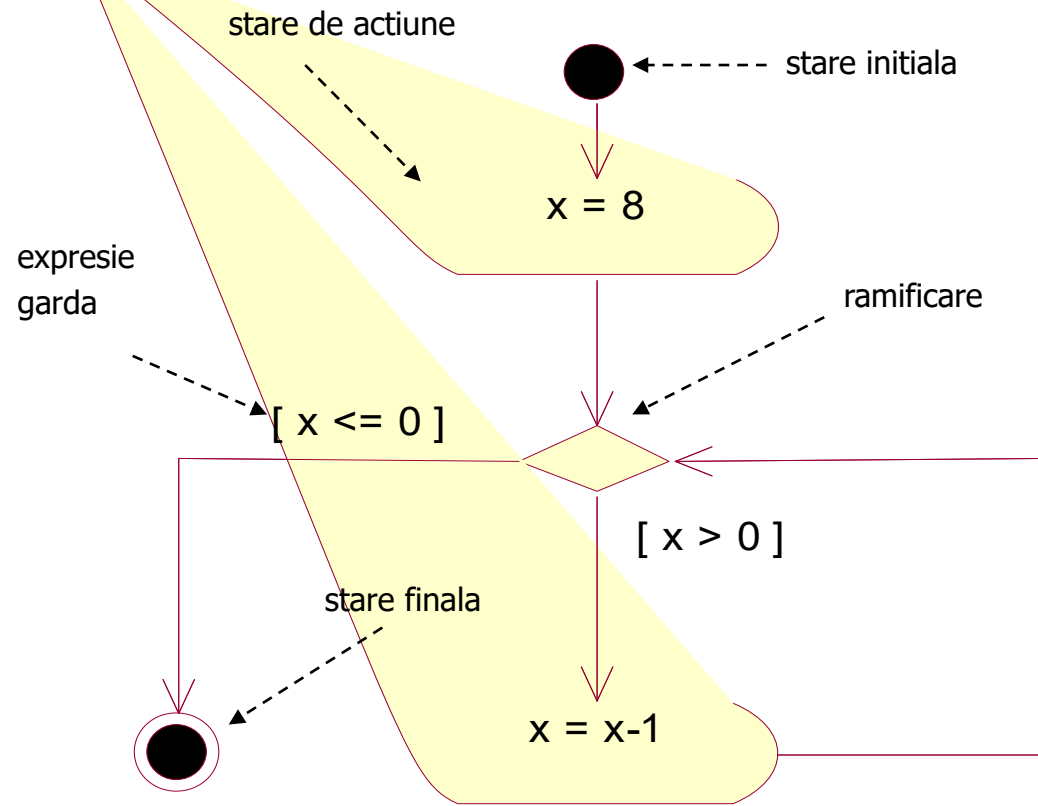


8.3. Diagrame de activitate

- O diagrama de activitate permite reprezentarea fluxului de activitati efectuate in cadrul unui sistem.
- O *diagrama de activitate* este un tip special de diagrama de tranzitie de stare, care vizualizeaza fluxul de la activitate la activitate in cadrul unui sistem. Intr-o diagrama de activitate starile sunt stari de activitate iar fluxul controlului trece imediat la urmatoarea stare cand activitatea starii curente se termina [BRJ99].
 - O *stare de activitate* reprezinta un proces desfasurat intr-o masina de stare.
 - O *stare de actiune* este un tip special de stare de activitate care nu poate fi descompusa in activitati mai simple (activitatea interna a unei stari de actiune este un proces atomic).

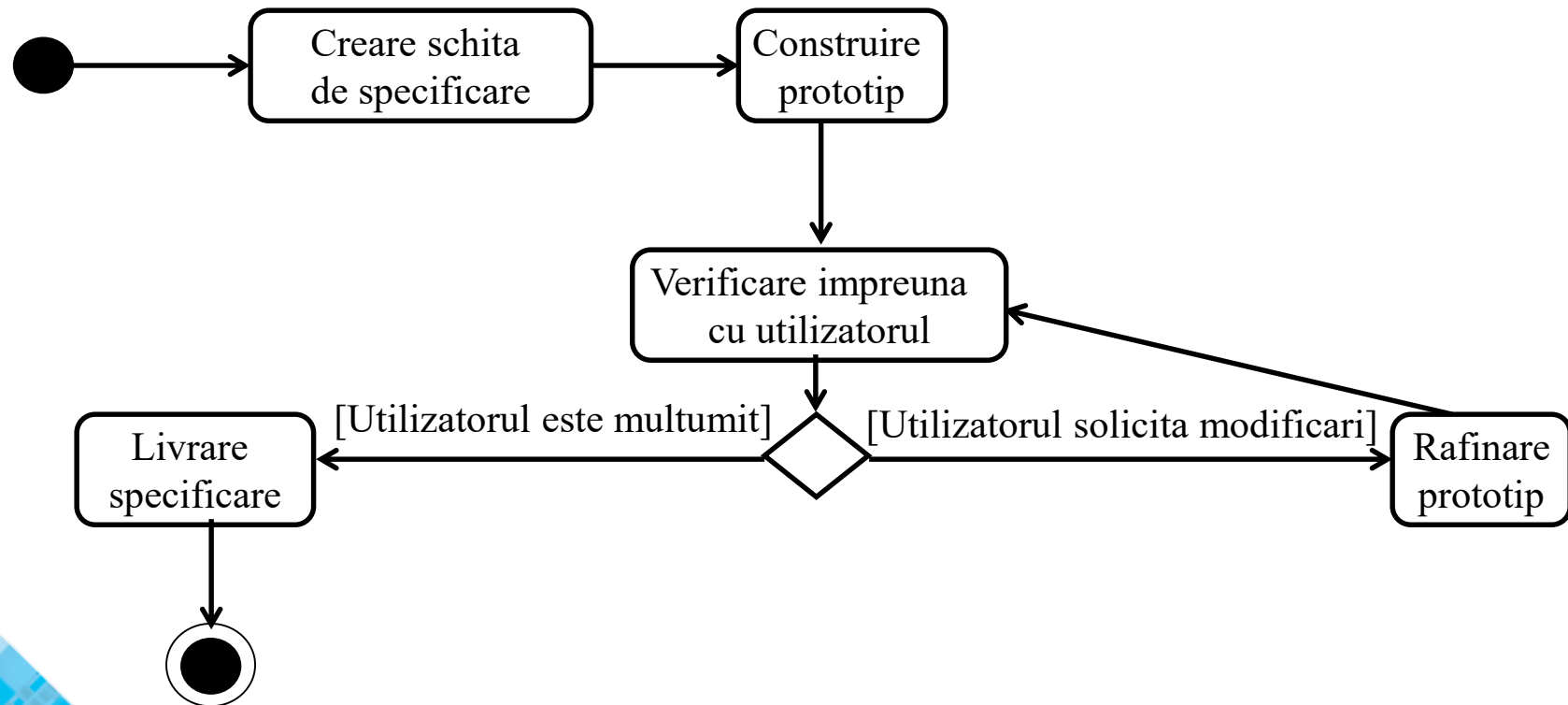
Diagrame de activitate – un exemplu cu stari de actiune

O diagrama de activitate poate utiliza stari de actiune pentru a vizualiza fluxul controlului intr-un program imperativ.



Diagrame de activitate – un exemplu cu stari de activitate

- O diagrama de activitate poate utiliza stari de activitate pentru a vizualiza activitatile umane din procesele software.
- Figura de mai jos prezinta diagrama de activitate pentru dezvoltare prin prototipizare [Bel05]:



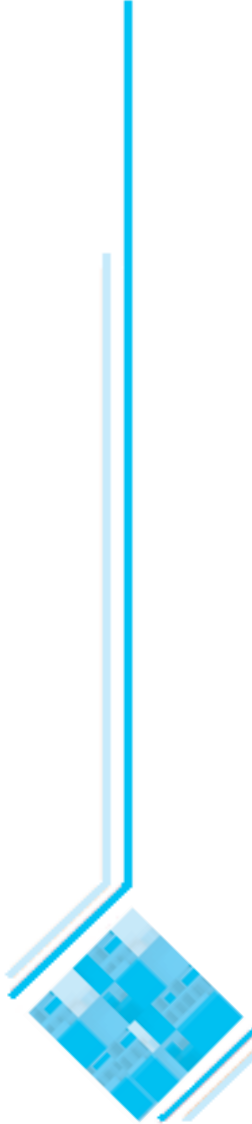
Referinte suplimentare

[BW90] J.C. Baeten, W.P. Weijland. *Process algebra*. Cambridge University Press, 1990.

[Bel05] D. Bell. *Software Engineering for Students – a Programming Approach* (4th edition). Addison-Wesley, 2005.

[BRJ99] G. Booch, J.Rumbaugh, I. Jacobson. *The Unified Modeling Language User Guide*. Addison Wesley, 1999.





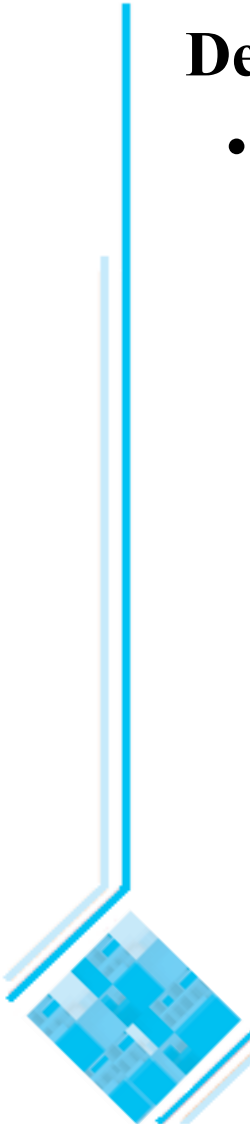
Projectare Software

Adapted from:
Timothy Lethbridge and Robert Laganieri,
Object-Oriented Software Engineering –
Practical Software Development using UML and Java, 2005
(chapter 9)

9.1 Procesul de proiectare

Definitie:

- In contextul ingineriei software, *proiectarea* este un proces de rezolvare probleme al carui obiectiv consta in descoperirea si descrierea unui mod de:
 - implementare a *cerintelor functionale*
 - cu respectarea constrangerilor impuse de *cerintele nonfunctionale*
 - si a principiilor generale de *calitate*.



Optiuni de proiectare

Inginerul software trebuie sa rezolve *probleme de proiectare*.

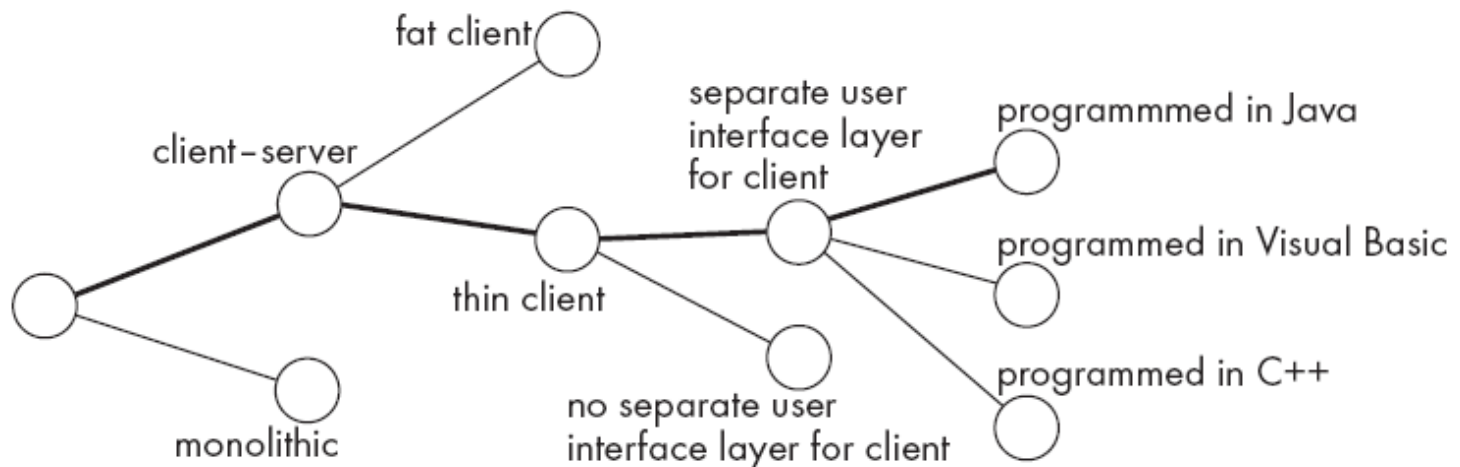
- Pentru fiecare problema de proiectare exista mai multe solutii alternative, sau *optiuni de proiectare*.
- Rezolvarea fiecărei probleme implica *decizii de proiectare*.
 - Este un proces de alegere a celei mai bune optiuni dintre mai multe alternative.

In luarea deciziilor de proiectare inginerul software ia in considerare:

- cerintele software,
- partea de proiect construita pana in acel moment,
- tehnologia disponibila,
- principii si practici de proiectare,
- experienta proiectelor anterioare.

Optiuni de proiectare

- Rareori se intampla ca doi ingineri software sa aleaga exact aceeasi solutie la o problema de proiectare.
- Figura de mai jos descrie optiunile (spatiul) de design pentru un posibil proiect software.



- Fiecare decizie tehnica ar trebui inregistrata, impreuna cu ratiunea de proiectare pe care se bazeaza.

Subsisteme, componente si module

- ***Modulul* reprezinta ‘caramida’ de baza in modularizarea sistemelor software. In general, un modul *nu* este un sistem independent.**
 - In cele ce urmeaza, notiunea de *modul* denota un element definit la nivelul limbajului de programare.
 - Exemple de module:
 - metode, clase si pachete in Java
 - functii si fisiere modul in C.
- **In sens strict, o *componenta* este o parte fizica a unui sistem, care poate fi inlocuita cu o componenta similara si care furnizeaza realizarea (si se conformeaza) unui set de interfete [BRJ99].**
- **Un *sistem* este un ansamblu de elemente si conexiuni care alcatuiesc un tot unitar.**
- **Un *subsistem* este o parte a unui sistem care apare de sine statator intr-un proces.**



Proiectare top-down si bottom-up

Proiectare top-down

- Se proiecteaza mai intai structura de ansamblu a sistemului.
- Se continua gradual prin decizii de proiectare tot mai detaliate.

Proiectare bottom-up

- Se incepe prin decizii de proiectare referitoare la componente reutilizabile de nivel coborat.
- Se decide apoi modul in care acestea pot fi combinate pentru construirea structurii de ansamblu a sistemului.

In practica, abordarile top-down si bottom-up sunt combinate:

- Abordarea top-down este aproape intotdeauna necesara pentru proiectarea structurii de ansamblu a sistemului
- Abordarea bottom-up permite favorizeaza reutilizarea componentelor.

9.2 Principiile activitatii de proiectare

Prin aplicarea **principiilor de proiectare** prezentate in continuare inginerul software urmareste sa obtina:

- Sporirea profitului (reducerea costurilor si cresterea veniturilor)
- Conformitatea proiectului cu cerintele
- Accelerarea procesului de dezvoltare
- Asigurarea calitatii
 - Fiabilitate
 - Mentenabilitate
 - Reutilizabilitate
 - Eficienta

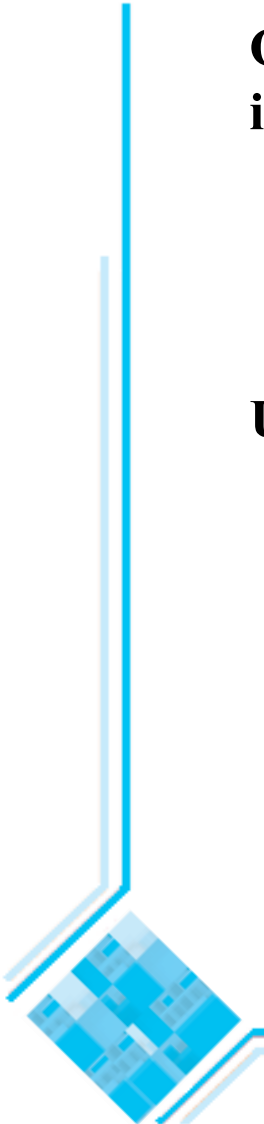
Principiul 1: Divide and conquer (divide et impera)

O problema complexa poate fi mai usor abordata daca este mai intai descompusa in probleme mai simple.

- Fiecare subproblema poate fi abordata de o echipa separata.
- Fiecare inginer software se poate specializa.
- Fiecare subproblema este mai usor de inteles si abordat.

Un sistem software poate fi descompus in mai multe moduri:

- Un sistem poate fi descompus in subsisteme
- Un subsistem poate fi descompus in pachete
- Un pachet este alcatuit din clase
- O clasa este alcatuita din metode



Principiul 2: Cresterea gradului de coeziune

Un subsistem sau un modul prezinta un grad ridicat de coeziune atunci cand manifesta unitate si legatura interna puternica intre elementele componente.

- Un sistem alcatuit din module coezive este mai usor de inteles si de modificat.
- Pot fi evidentiuate diverse tipuri de coeziune a modulelor software. In lista data mai jos tipurile de coeziune sunt ordonate descrescator dupa gradul de coeziune:
 - Coeziune functionala
 - Coeziune de nivel
 - Coeziune de comunicare
 - Coeziune de secventiere
 - Coeziune procedurala
 - Coeziune temporala
 - Coeziune de utilitate

Coeziune functionala

Modulul efectueaza o sarcina de calcul specifica (unica) si returneaza un rezultat, fara a produce efecte laterale.

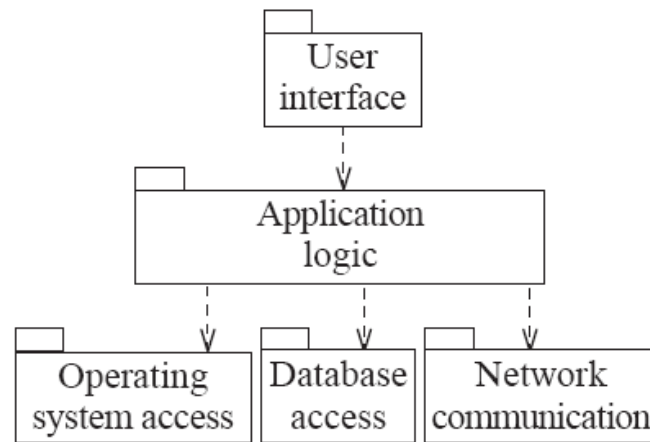
- Un modul este lipsit de efecte laterale daca nu modifica starea sistemului atunci cand efectueaza o sarcina de calcul.
 - Un modul care actualizeaza o baza de date, interactioneaza cu utilizatorul sau creaza un fisier nou nu este functional coeziv.
- Un modul functional coeziv este:
 - Usor de inteles
 - Usor de reutilizat
 - Usor de inlocuit

Coeziune de nivel (engl. *layer cohesion*)

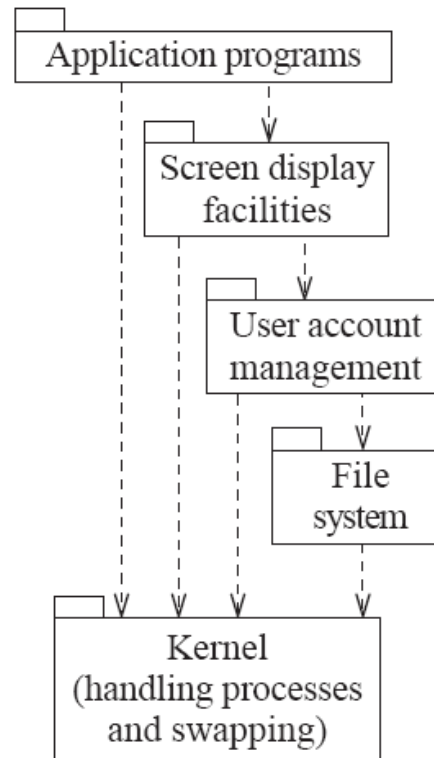
Toate serviciile furnizate utilizatorului sau unui nivel (strat) superior sunt grupate (functionalitatea nivelului) si orice alta functionalitate este plasata in alta parte a sistemului.

- Organizarea este ierarhica
 - Un nivel superior poate accesa serviciile furnizate de un nivel inferior
 - Un nivel inferior nu acceseaza serviciile furnizate de un nivel superior
- Efectele laterale sunt permise, chiar esentiale in unele aplicatii (de exemplu intr-un nivel de acces la baza de date).
- Orice nivel poate fi inlocuit fara vreun impact asupra celorlalte niveluri
 - Este suficient sa se inlocuiasca API (engl. *Application Programming Interface*) pentru nivelul respectiv
- API = setul de proceduri prin care un nivel isi furnizeaza serviciile

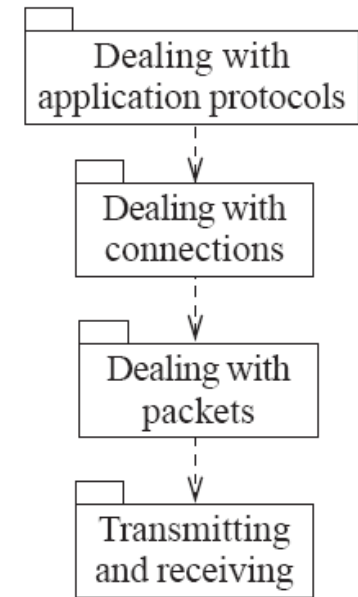
Example



(a) Typical layers in an application program



(b) Typical layers in an operating system



(c) Simplified view of layers in a communication system

Coeziune de comunicare

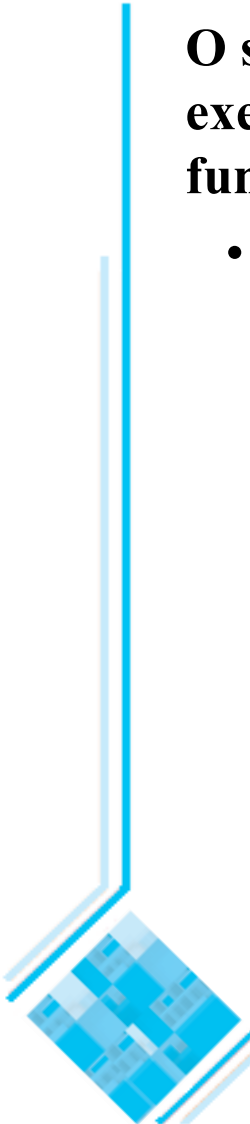
Toate elementele de procesare care acceseaza sau manipuleaza anumite date sunt pastrate impreuna (de exemplu, in aceeași clasă) și orice altă funcționalitate este plasată în altă parte a sistemului.

- Unul dintre punctele forte ale paradigmei OO constă tocmai în faptul că aceasta favorizează coeziunea de comunicare.
- Coeziunea de nivel nu trebuie sacrificată pentru obținerea coeziunii de comunicare.
 - De exemplu, chiar dacă anumite obiecte sunt stocate într-o bază de date sau pe un site la distanță (pe un nivel inferior), o clasă ar trebui să încarce sau să salveze obiecte doar utilizând serviciile API furnizate de nivelul inferior

Coeziune de secventiere

O serie de proceduri de calcul, care isi furnizeaza intrari si se executa in secventa, sunt pastrate impreuna si orice alta functionalitate este plasata in alta parte a sistemului.

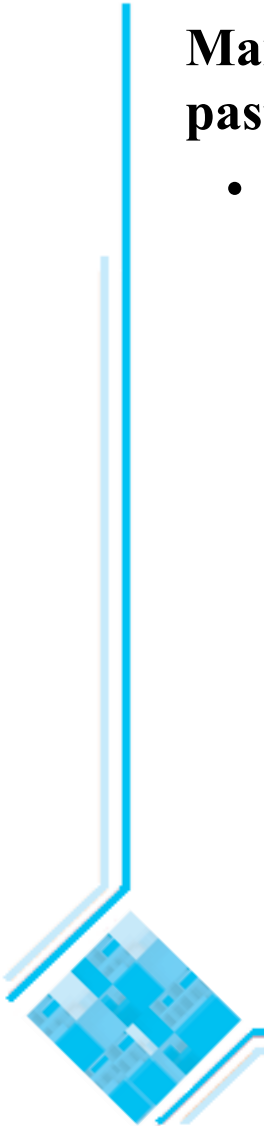
- Coeziunea de comunicare nu ar trebui sacrificata pentru obtinerea coeziunii de secventiere.
 - Metodele din clase diferite pot sa isi furnizeze (reciproc) intrari si pot fi apelate in secventa, dar este preferabil ca fiecare sa fie pastrata in clasa de care apartine.



Coeziune procedurala

Mai multe proceduri care se executa intr-o anumita ordine sunt pastrate impreuna, chiar daca nu isi furnizeaza intrari una alteia.

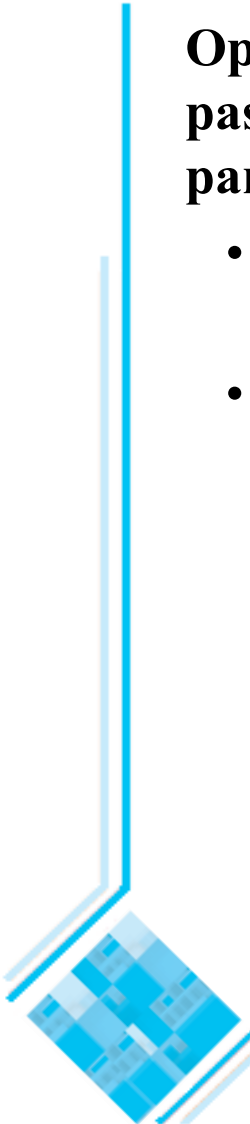
- Mai slaba decat coeziunea de secventiere



Coeziune temporală

Operatiile efectuate in aceeasi faza a executiei programului sunt pastrate impreuna si orice alta functionalitate este plasata in alta parte a sistemului.

- Exemplu: plasarea in aceeasi sectiune a codului executat in faza de initializare a sistemului
- Mai slaba decat coeziunea procedurala



Coeziune de utilitate

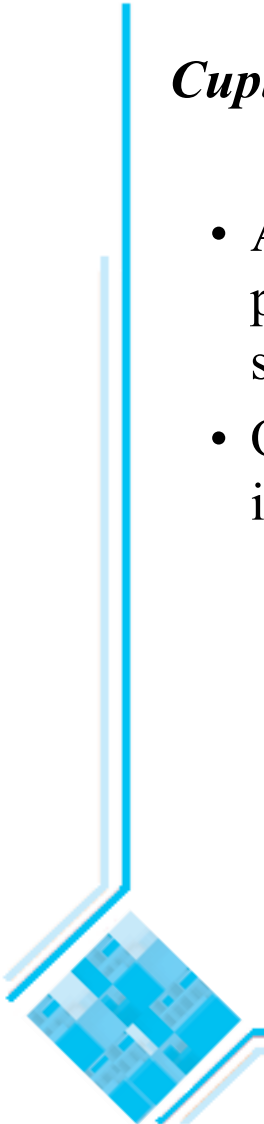
Utilitatile logic conectate sunt pastrate impreuna.

- Aici, o *utilitate* este o procedura sau o clasa care are aplicabilitate larga la mai multe subsisteme si este proiectata a fi reutilizabila.
- De exemplu clasa **`java.lang.Math`** are coeziune de utilitate.
 - De remarcat faptul ca functiile din clasa **`java.lang.Math`** nu manifesta coeziune de comunicare, de secventiere, procedurala sau temporală; sunt legate intre ele doar din punct de vedere logic.

Principiul 3: Reducerea gradului de cuplare

Cuplarea apare atunci cand exista interdependente intre module

- Atunci cand exista interdependente, modificarile efectuate intr-o parte a unui sistem pot sa atraga dupa sine modificari in alta parte a sistemului.
- O retea de interdependente ingreuneaza intelegerea componentelor individuale.

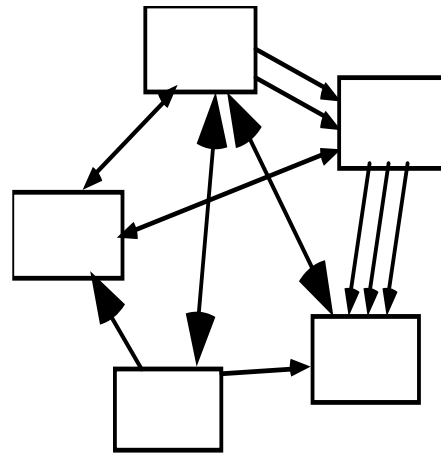


Reducerea gradului de cuplare

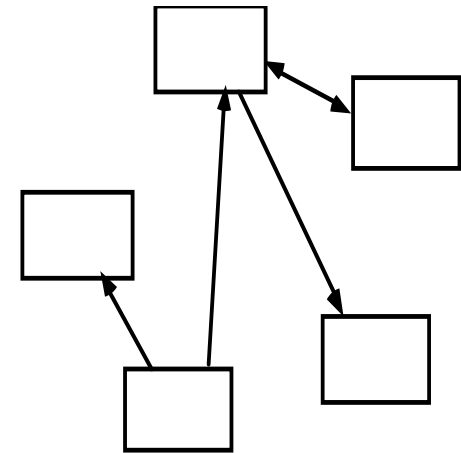
- Tipuri de cuplare (date în ordinea descrescătoare a gradelor de cuplare):

- De continut
- Prin date globale
- De control
- De marca
- Prin date
- Prin apel rutina
- Prin utilizare tip
- Prin import
- Externa

Sistem cu cuplare intensa



Sistem cu grad redus de cuplare



- Pentru reducerea gradului de cuplare trebuie sa se reduca:
 - numarul* conexiunilor intre module si
 - taria* conexiunilor (sugerata in figura prin grosimea sagetilor)

Cuplare de continut

Apare atunci cand un modul modifica *pe ascuns* date *interne* ale unui alt modul

- Cuplarea de continut ar trebui sa fie intotdeauna evitata, deoarece orice element de procesare a datelor ar trebui sa fie usor de localizat si usor de inteles.
- Pentru reducerea cuplarii de continut toate variabilele de instanta ar trebui sa fie *incapsulate*:
 - declarate `private`
 - accesate numai prin metode `set` si `get`
- O forma mai subtila de cuplare de continut apare atunci cand se modifica direct o variabila de instanta a unei variabile de instanta (chiar daca toate variabilele de instanta sunt private), ca in exemplul urmator:

Exemplu de cuplare de continut

```
public class Line
{
    private Point start, end;
    ...
    public Point getStart() { return start; }
    public Point getEnd() { return end; }
}

public class Arch
{
    private Line baseline;
    ...
    void slant(int newY)
    // Modifica coordonata y a variabilei thEnd (punct)
    // a variabilei baseline (linie)
    {
        Point theEnd = baseline.getEnd();
        theEnd.setLocation(theEnd.getX(),newY);
    }
}
```

Cuplare prin date globale

Apare la utilizarea de variabile globale – toate modulele care utilizeaza o variabila globala devin cuplate intre ele si cu modulul care declara respectiva variabila

- In Java, variabilele `public static` servesc drept variabile globale.
- O forma mai slaba de cuplare prin date globale apare atunci cand o variabila poate fi accesata dintr-o *submultime* a claselor sistem
 - De exemplu clasele dintr-un pachet Java
- Cuplarea prin date globale este acceptabila atunci cand:
 - Se utilizeaza constante globale partajate de intregul sistem
 - Se utilizeaza variabile globale deoarece este mai dificil sa se forteze ca un numar mare de rutine sa transmita anumite date sub forma de parametri.

Cuplare de control

Apare atunci cand o procedura apeleaza o alta procedura utilizand un 'flag' care controleaza in mod explicit logica (comportamentul) celei de a doua proceduri

—Exemplu:

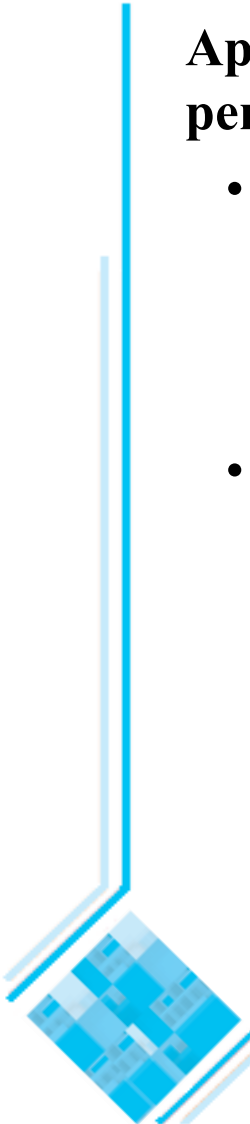
```
public routineX(String command)
{
    if (command.equals("drawCircle"))
    {
        drawCircle();
    }
    else
    {
        drawRectangle();
    }
}
```

- Pentru realizarea unei modificari trebuie modificata atat metoda apelanta cat si metoda apelata.
- Acolo unde este posibil, utilizarea operatiilor polimorifice reprezinta cea mai buna modalitate de reducere a cuplarii de control.

Cuplare de marca (engl. *stamp coupling*)

Apare atunci cand una dintre clasele aplicatiei este utilizata ca *tip* pentru argumentul unei metode

- Orice modificare este ingreunata de faptul ca o clasa utilizeaza o alta clasa (exista o dependenta semantica)
 - Reutilizarea unei clase impune reutilizarea celeilalte
- Doua modalitati de reducere a cuplarii de marca:
 - Utilizarea unei interfete ca tip al argumentului
 - Transmiterea de date simple



Exemplu de cuplare de marca

```
public class Emlaler
{
    public void sendEmail(Employee e, String text) {...}
    ...
}
```

- Problema este ca metoda `sendMail()` *nu are nevoie* sa acceseze intreg obiectul `Employee`; este suficient sa acceseze numele si adresa de email a unui `Employee`.

Doua moduri de reducere a cuplarii de marca

Utilizarea unei interfete ca tip al argumentului metodei:

```
public interface Addressee {  
    public abstract String getName();  
    public abstract String getEmail();  
}  
  
public class Employee implements Addressee {...}  
  
public class Emler {  
    public void sendEmail(Addressee e, String text) {...}  
    ...  
}
```

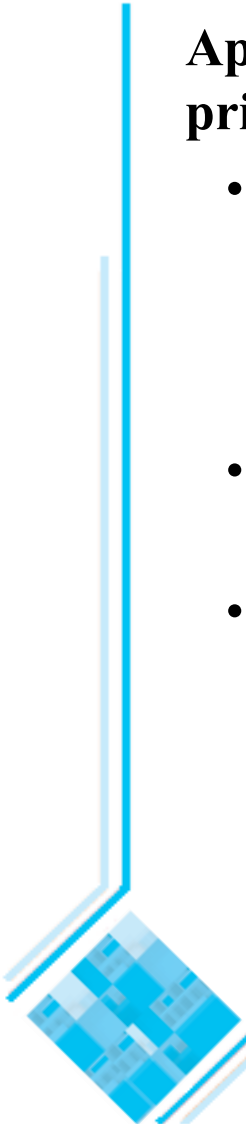
Transmiterea de date simple (inlocuieste cuplarea de marca cu cuplare prin date):

```
public class Emler {  
    public void sendEmail(String name, String email, String text) {...}  
    ...  
}
```

Cuplare prin date

Apare atunci cand tipurile argumentelor unei metode sunt fie primitive fie clase de biblioteca (nu ale aplicatiei)

- Cu cat o metoda are mai multe argumente cu atat cuplarea este mai tare
 - Toate metodele care o utilizeaza trebuie sa transmita toate argumentele
- Cuplarea prin date ar trebui redusa evitand transmiterea de argumente inutile catre metode.
- Evident, metodele trebuie sa aiba argumente, deci un anumit grad de cuplare de marca sau prin date exista in orice aplicatie netriviala.



Cuplare prin apel rutina

Apare atunci cand o rutina (o metoda intr-un sistem OO) apeleaza o alta rutina

- Rutinele sunt cuplate deoarece comportamentul uneia depinde de comportamentul celeilalte.
- Cuplarea prin apel de rutina este prezenta in orice sistem netrivial.
- Intr-un program in care o secventa de doua sau mai multe metode este utilizata pentru un anumit calcul in mod repetat, cuplarea prin apel de rutina poate fi redusa prin scrierea unei singure rutine care incapsuleaza respectiva secventa.

Cuplare prin utilizare tip

Apare atunci cand un modul utilizeaza un tip de date definit intr-un alt modul

- Apare oricand o clasa declara o variabila de instanta sau o variabila locala avand ca tip o alta clasa.
- In scopul reducerii acestui gen de cuplare, pentru tipul unei variabile se alege cea mai generala clasa sau interfata care contine operatiile necesare.



Cuplare prin import

Apare atunci cand un modul importa un alt modul (de exemplu un pachet in Java)

- Modulul care importa depinde semantic de modulul importat.
- Daca in modulul importat se modifica sau se adauga ceva, atunci poate aparea un conflict (de exemplu de nume) cu modulul care realizeaza operatia de import; acesta din urma trebuie la randul sau sa fie modificat pentru solutionarea conflictului.
 - In general, cuplarea prin import nu poate fi evitata, deoarece aceasta permite utilizarea bibliotecilor sistem.
 - Desigur, este indicat sa se evite importarea de pachete sau clase care nu sunt necesare in aplicatie.



Cuplare externa

Apare atunci cand un modul are o dependenta semantica fata de sistemul de operare, pachete software produse de o alta firma, componente hardware, etc.

- Corespunde unui nivel relativ ridicat de cuplare (mai redus decat cuplarea prin date globale, dar mai ridicat decat cuplarea de control, cf. [Pre97]).
- Este indicat sa se reduca numarul de locuri din cod in care exista asemenea dependente.
- Sablonul de design Façade poate reduce acest tip de cuplare prin simplificarea interfetei spre functiile externe.

Principiul 4: Mentinerea unui nivel inalt de abstractizare

Pentru reducerea complexitatii, este important ca in fiecare etapa de proiectare sa se mentina un nivel inalt de abstractizare, prin ascunderea sau amanarea tratarii detaliilor.

- Mentinerea unui nivel ridicat de abstractizare permite inginerului software
 - Sa se concentreze asupra fiecărei probleme la un anumit nivel de generalitate
 - Sa ignore detaliile irelevante
- Abstractizarea este necesara deoarece creierul uman poate procesa o cantitate limitata de informatie in fiecare moment.
 - Limbajul UML poate fi utilizat pentru modelarea sistemelor software la un nivel de abstractizare adecvat [BRJ99].
 - Diagramele UML pot vizualiza modele cu diferite grade de detaliere.
 - Abstractiile procedurale sau de date sunt de obicei suportate direct la nivelul limbajului de programare.

Principiul 5: Sporirea gradului de reutilizabilitate

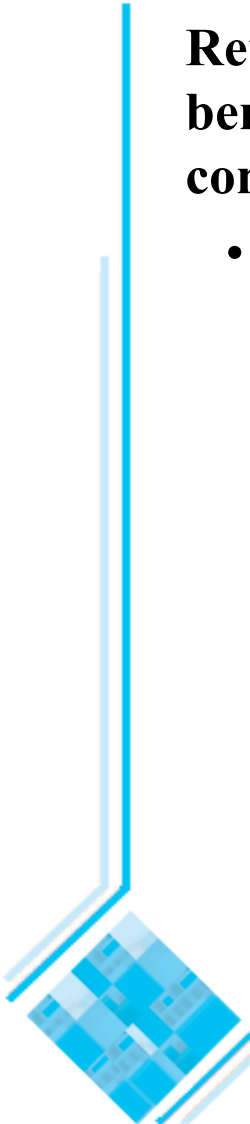
Diferitele aspecte ale unui sistem software ar trebui proiectate astfel incat sa poata fi utilizate din nou in diferite contexte, in cadrul sistemului curent sau in cadrul altor sisteme.

- Reutilizabilitatea poate fi incorporata la nivel de procedura, clasa, framework
 - Un *framework* este un sablon arhitectural, extensibil, specific unui domeniu de aplicatie [BRJ99, JBR99].
- Strategii de baza pentru sporirea gradului de reutilizabilitate (construirea de componente reutilizabile):
 - Construirea unui proiect cat mai simplu si cat mai general
 - Aplicarea precedentelor trei principii de proiectare (cresterea gradului de coeziune, reducerea gradului de cuplare, sporirea nivelului de abstractizare)

Principiul 6: Reutilizare design sau cod

Reutilizarea anumitor parti de proiect sau de cod reprezinta beneficiul obtinut in urma efortului investit in construirea de componente reutilizabile.

- *Clonarea codului* (adica copierea anumitor segmente de cod dintr-o parte in alta) nu ar trebui sa fie considerata o forma de reutilizare.



Principiul 7: Proiectare pentru flexibilitate

Proiectarea pentru *flexibilitate* (sau *adaptabilitate*) este anticiparea eventualelor modificari ale proiectului si pregatirea pentru acestea.

- Moduri de realizare a unui proiect flexibil:
 - Reducerea gradului de cuplare si cresterea gradului de coeziune a modulelor
 - Crearea de entitati abstracte
 - Pastrarea optiunilor de proiectare
 - Nu ar trebui limitate optiunile de proiectare pentru cei ce vor modifica ulterior sistemul
 - Construirea de cod reutilizabil si reutilizarea codului



Principiul 8: Anticiparea uzurii

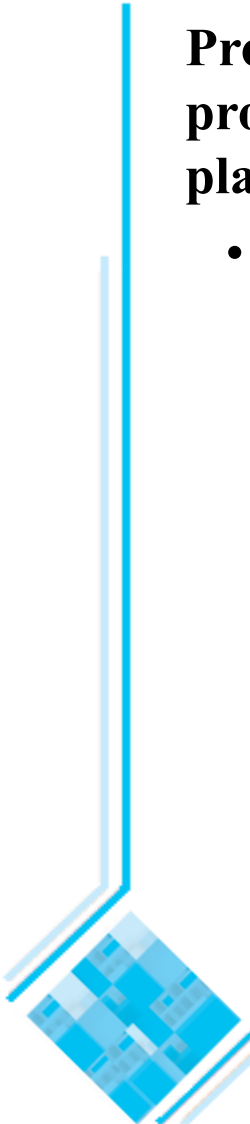
Anticiparea uzurii unui produs software se realizeaza prin masuri de planificare care tin cont de evolutia tehnologiei sau a mediului de lucru astfel incat produsul software sa continue si in viitor sa poata fi utilizat sau sa poata fi modificat cu usurinta.

- Pentru anticiparea uzurii unui produs software:
 - Se vor evita versiunile mai vechi ale tehnologiei software / hardware
 - Se va evita utilizarea de biblioteci specifice unor medii de lucru particulare
 - Se vor evita serviciile nedocumentate sau putin utilizate ale bibliotecilor software
 - Se va evita utilizarea de software sau hardware specializat furnizat de companii despre care se crede ca nu manifesta stabilitate pe termen lung
 - Se vor utiliza limbaje si instrumente standard disponibile prin mai multi furnizori

Principiul 9: Proiectare pentru portabilitate

Proiectarea pentru portabilitate are drept obiectiv realizarea de produse software care sa poata fi utilizate pe cat mai multe platforme de calcul.

- Pentru realizarea unui produs portabil se va evita utilizarea de facilitati specifice unei platforme particulare
 - De exemplu, o biblioteca software disponibila numai sub Microsoft Windows.



Principiul 10: Proiectare pentru testabilitate

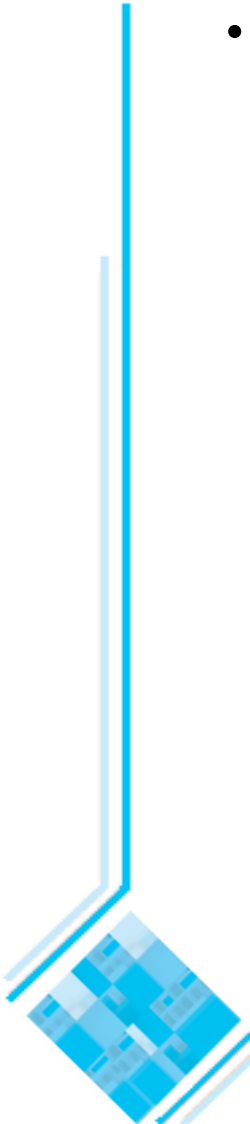
In faza de proiectare a unui produs software se pot lua masuri pentru inlesnirea activitatii de testare.

- Este avantajos ca proiectul sa fie realizat astfel incat sa faciliteze *testarea automata* a produsului software.
 - Intreaga functionalitate ar trebui sa poata fi executata fara trecere printr-o interfata utilizator (IU) grafica.
 - In acest scop se separa IU de nucleul functional al aplicatiei
 - » O alta solutie ar fi sa se construiasca o versiune de tip linie de comanda a sistemului.
 - Se pot apoi elabora componente de testare care apeleaza serviciile API ale nivelului functional.



Proiectare pentru testabilitate

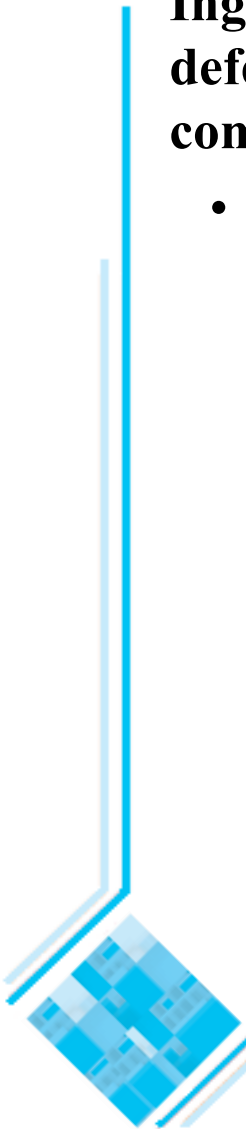
- In fiecare clasa se pot scrie metode utilizate ca drivere de test pentru celelalte metode ale clasei.
 - In Java, se poate crea o metoda `main()` (driver de test) pentru verificarea celorlalte metode ale clasei.
- Pentru asigurarea faptului ca o subclasa se comporta in mod consistent cu superclasa sa, driverele de test din superclasa trebuie sa fie executate in fiecare subclasa.



Principiul 11: Proiectare defensivă

Inginerii software trebuie să ia măsuri de precauție (să ‘proiecteze defensiv’) pentru a preîntâmpina utilizarea necorespunzătoare a componentelor pe care le construiesc.

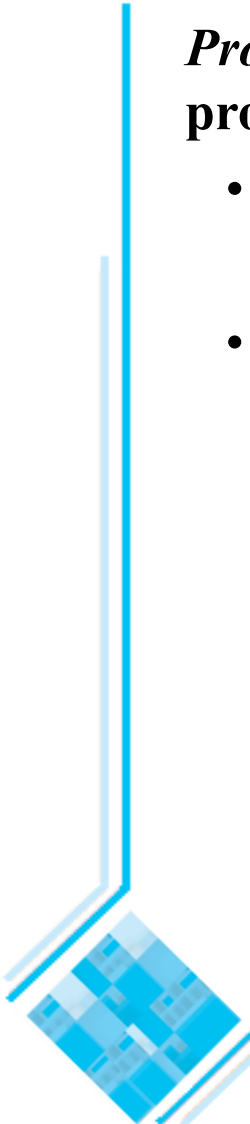
- Asta înseamnă că trebuie verificată validitatea intrărilor componentelor.
 - Practic, trebuie verificate *preconditiile* fiecărei componente.
 - În proiectarea defensivă ar trebui să se evite verificările repetate în mod inutil.



Proiectare prin contract

***Proiectarea prin contract* este o tehnica ce ofera un cadru pentru proiectarea defensiva sistematica si eficienta.**

- Ideea de baza
 - Fiecare metoda are un *contract* explicit cu utilizatorii sai
- Pentru fiecare metoda, contractul consta din asertiuni care stabilesc:
 - *preconditiile* (solicitate de metoda inaintea executiei),
 - *postconditiile* (pe care metoda le garanteaza la sfarsitul executiei) si
 - *invariantii* (pe care metoda se angajeaza sa nu ii modifice in timpul executiei).



9.5 Arhitectura software

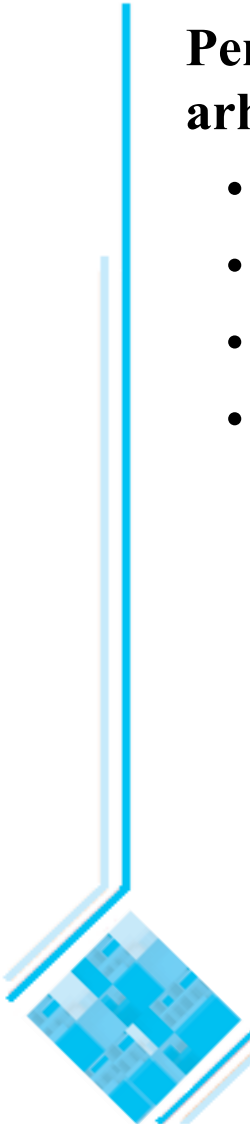
***Arhitectura software* se refera la structura de ansamblu a sistemului software si la modul in care aceasta structura furnizeaza sistemului integritate semantica [SG96].**

- Procesul de proiectare arhitecturala consta in:
 - Descompunerea sistemului software in subsisteme
 - Determinarea interfetelor subsistemelor
 - Decizii privind modul de interactiune intre subsisteme
- Documentatia produsa ca rezultat al procesului de proiectare arhitecturala este adesea numita *model arhitectural*.
- Arhitectura este nucleul proiectului software; fiecare inginer software trebuie sa o inteleaga.
 - Orice decizie nepotrivita luata in faza de proiectare a arhitecturii software poate avea implicatii negative asupra eficientei, reutilizabilitatii si mentenabilitatii sistemului.

Importanta arhitecturii software

Pentru participantii la un proiect software, dezvoltarea unui model arhitectural inlesneste:

- Intelegerea sistemului
- Organizarea procesului de dezvoltare
- Sporirea gradului de reutilizabilitate
- Evolutia sistemului



Continutul unui model arhitectural

Arhitectura unui sistem este adesea exprimata prin mai multe *vederi*

- Descompunerea logica a sistemului in subsisteme cu interfetele lor
- Dinamica interactiunii intre subsisteme si componente (la executie)
- Datele partajate intre subsisteme
- Componentele si masinile de calcul pe care componentele sunt desfasurate la executie

Observatii:

- O descriere de arhitectura arata ca o descriere completa de sistem (poate include orice tip de model) dar este mai mica: contine numai modelele si vederile sistem relevante din punct de vedere arhitectural [JBR99].
- Un model arhitectural descrie atat aspecte structurale cat si aspecte comportamentale ale sistemului.

Stabilitatea arhitecturii software

Pentru asigurarea fiabilitatii si mentenabilitatii unui sistem software, modelul arhitectural trebuie sa fie stabil.

- O arhitectura software este *stabila* daca permite adaugarea de noi servicii si facilitati la sistem cu modificari minime la nivelul arhitecturii.
- De exemplu, in UP (the Unified Process [JBR99]), prin definitie, la sfarsitul fazei de *elaborare* arhitectura software ar trebui sa fie stabila.
 - Succesul unui proiect UP depinde de capacitatea unei mici echipe de arhitecti si dezvoltatori (de valoare profesionala ridicata) de a produce o arhitectura stabila cu resurse limitate.
 - Conform [JBR99], este posibila dezvoltarea unei arhitecturi stabile cu doar aprox. 30% din resursele alocate proiectului.

Dezvoltarea unui model arhitectural

- **Mai intai se construiește o arhitectura tentativa (inainte de considerarea detaliilor legate de cazurile de utilizare).**
 - In acest stadiu se utilizeaza:
 - Cunostinte despre *domeniul de aplicatie*,
 - *Sabloane arhitecturale* generale (urmeaza a fi discutate).
 - *Sugestie*: mai multe echipe diferite pot sa lucreze in mod independent la conceperea de schite ale arhitecturii, iar apoi cele mai bune idei pot sa fie combinate.
- **Apoi, se alege si se implementeaza un set de cazuri de utilizare, construindu-se o arhitectura imbunatatita, s.a.m.d.**
 - In fiecare iteratie
 - Se elaboreaza elementele arhitecturale specifice aplicatiei, care pot include orice aspecte ale sistemului considerate a fi relevante d.p.d.v. arhitectural
 - Se considera fiecare caz de utilizare si se ajusteaza arhitectura pentru permite implementarea cazului de utilizare
 - Se maturizeaza arhitectura
 - Ar trebui inceput cu *cazurile de utilizare relevante din punct de vedere arhitectural*.

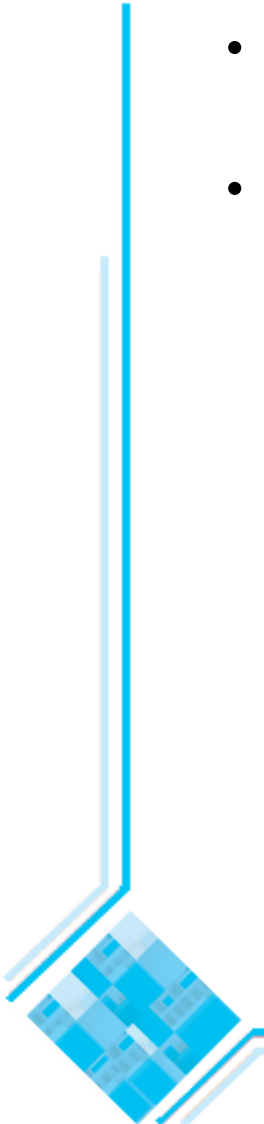
Dezvoltarea unui model arhitectural

Cazurile de utilizare relevante din punct de vedere arhitectural sunt cele care [JBR99]:

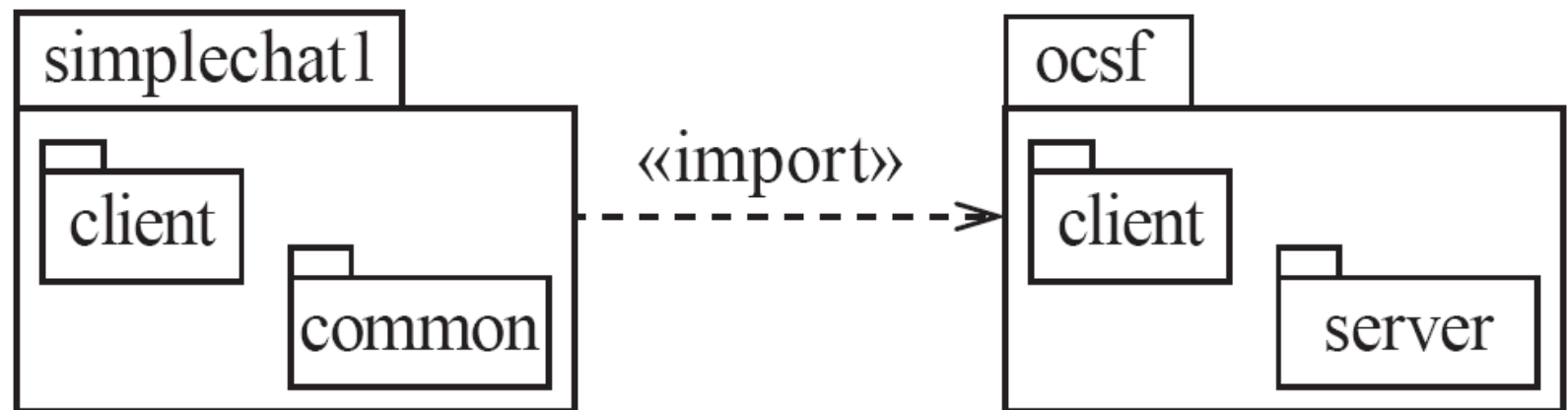
- Ajuta la diminuarea celor mai semnificative riscuri tehnice sau manageriale
- Sunt cele mai importante pentru utilizatorii sistemului
 - Exemplu
 - Se considera un sistem de tip ATM (Automated Teller Machine – automat bancar) care implementeaza patru cazuri de utilizare: Retragerre Bani, Depunere Bani, Transfer Bani intre Conturi, si Vizualizare Cont.
 - Arhitectul software decide ca unicul caz de utilizare relevant din punct de vedere arhitectural este Retragerre Bani [JBR99], deoarece fara acest serviciu sistemul ATM este lipsit de sens.
- Ajuta la implementarea oricarei functionalitati semnificative.

Descrierea unei arhitecturi cu UML

- Toate diagramele UML pot fi utile in descrierea diferitelor aspecte ale modelului arhitectural.
- Urmatoarele diagrame UML sunt in particular importante pentru modelarea arhitecturii:
 - Diagrame de pachete
 - Diagrame de componente
 - Diagrame de desfasurare



Diagrame de pachete

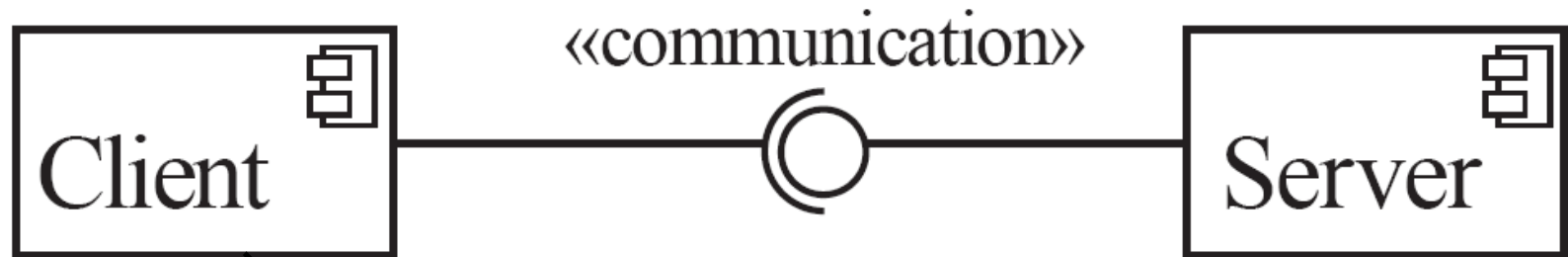


Un *pachet* este un mecanism general de organizare a elementelor de model in grupuri [BRJ99].

- O *nota* este un simbol grafic care permite exprimarea de constrangeri sau de comentarii atasate unui element (de model) sau unei colectii de elemente.

www.lloseng.com

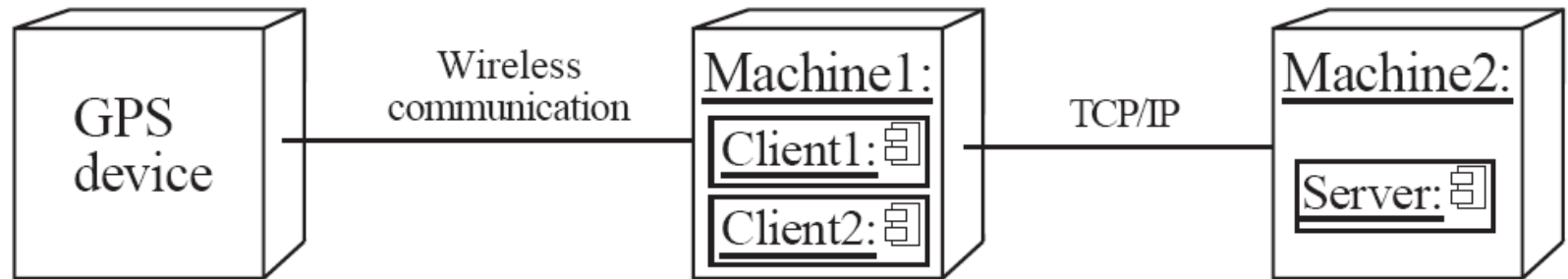
Diagrame de componente



O *componenta* este o parte fizică a unui sistem, care poate fi înlocuită cu o componentă similară și care furnizează realizarea (și se conformează) unui set de interfețe [BRJ99].

- O componentă *furnizează* una sau mai multe interfețe care pot fi utilizate de alte componente.
- Pentru vizualizarea unei componente care *utilizează* o interfață furnizată de o altă componentă se utilizează un semicerc (la capatul unei linii).

Diagrame de desfasurare



Fiecare nod dintr-o diagrama de desfasurare poate gazdui una sau mai multe componente software

Un *nod* este un element fizic care exista la executie si reprezinta o resursa de calcul (furnizand cel putin capacitate de memorare si, adesea, de procesare) [BRJ99].

9.6 Sabloane arhitecturale

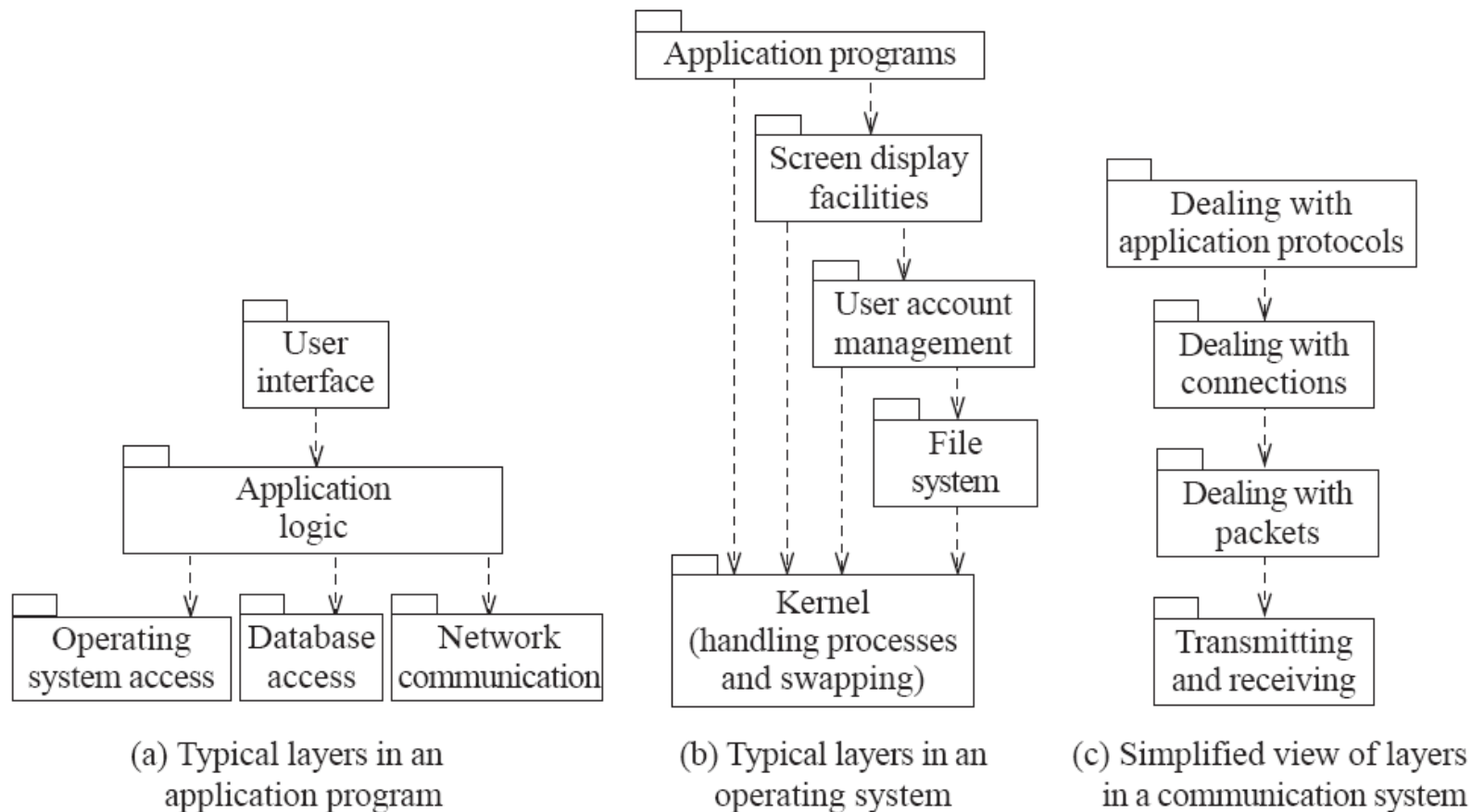
- **Notiunea de sablon (engl. *pattern*) este utilizata si in contextul arhitecturilor software**
 - *Sabloanele arhitecturale* (numite si *stiluri arhitecturale*) inlesnesc proiectarea de sisteme software alcatuite din subsisteme si componente cat mai independente cu putinta.
 - Un *sablon* este schita unei solutii reutilizabile la o problema generala intalnita intr-un anumit context. Notiunea de sablon este utilizata la diferite niveluri de abstractizare:
 - Sabloanele de proiectare* – cum sunt Observer, Façade sau Proxy – sunt larg utilizate in modelarea si proiectarea software
 - Sabloanele arhitecturale*– cum sunt Layers, Client-Server sau Pipe-and-Filter – reprezinta solutii standard pentru diferite probleme arhitecturale.

Sablonul arhitectural Multi-Layer

Intr-un sistem organizat pe niveluri (Multi-Layer), fiecare nivel utilizeaza doar serviciile furnizate de nivelurile inferioare – adesea numai serviciile nivelului imediat inferior.

- Fiecare nivel are o interfata bine definita (structura API care defineste serviciile pe care le furnizeaza) utilizata de nivelurile superioare
 - Un nivel superior vede un nivel inferior ca un set de *servicii*
 - Un nivel inferior nu cunoaste detaliile sau interfetele furnizate de un nivel superior
- De exemplu, intr-un program de aplicatie
 - Este important sa existe un nivel separat pentru interfata utilizator (independenta acestui nivel permite aplicatiei sa functioneze cu diferite interfete utilizator)
 - Nivelurile imediat inferioare interfetei utilizator furnizeaza functiile determinate de cazurile de utilizare
 - Nivelurile inferioare furnizeaza servicii pentru acces la baze de date, comunicare in retea, etc.

Exemple de système Multi-Layer



Sablonul arhitectural Client-Server

- Exista cel putin o componenta cu rol de *server*, care asteapta si apoi trateaza (sau serveste) conexiunile.
- Exista cel putin o componenta cu rol de *client*, care initiaza conexiuni cu scopul de a obtine anumite servicii.

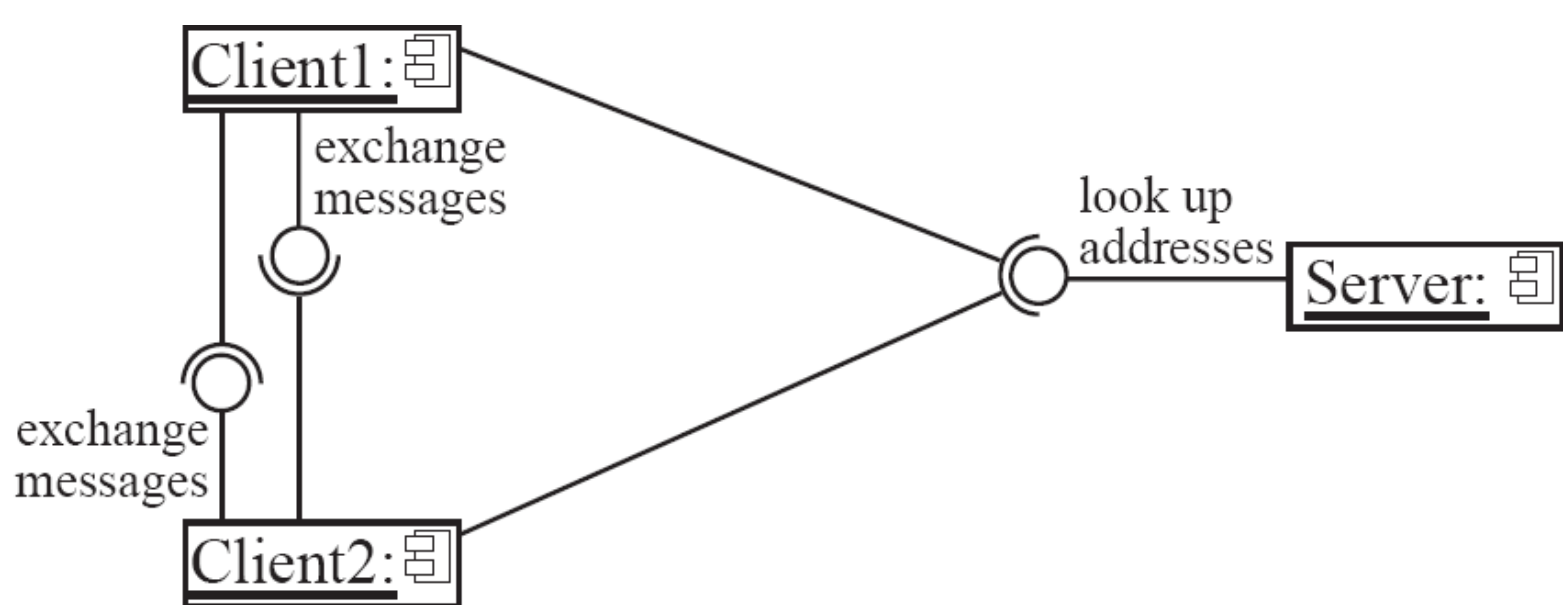
—O varianta importanta a arhitecturii (two-tier) descrisa mai sus este reprezentata de modelul **three-tier**, care include trei tipuri de noduri: clienti, servere de aplicatie si servere (de baze) de date.

» Un server de aplicatie se comporta ca si un client atunci cand acceseaza un server de date.

—O alta extensie este data de sablonul **Peer-to-Peer**. Acesta descrie o arhitectura de sistem distribuit compusa din diverse componente software.

- Fiecare dintre aceste componente poate fi atat client cat si server
- Oricare doua componente (engl. *peers*) pot stabili un canal de comunicare

O arhitectura peer-to-peer

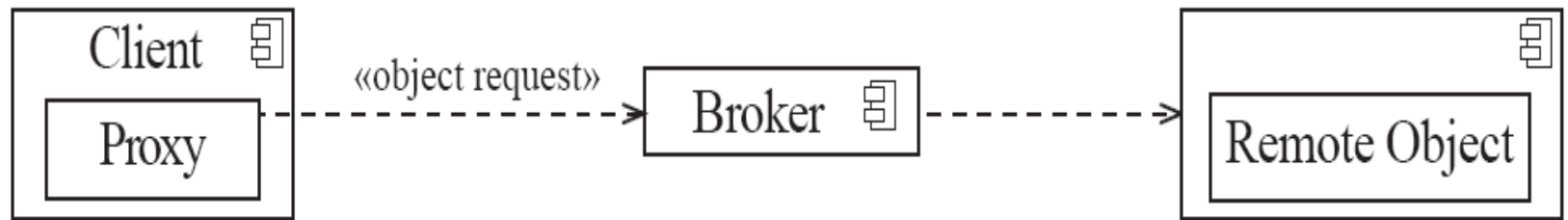


Un server central poate fi utilizat pentru stabilirea conexiunilor intre perechi (engl. *peers*)

Sablonul arhitectural Broker

- **Sablonul arhitectural Broker inlesneste distribuirea obiectelor unui sistem software in mod *transparent* pe diferite noduri.**
 - Utilizand arhitectura Broker, un obiect poate apela metode ale unui alt obiect fara sa stie ca acesta din urma este situat pe un alt calculator.
 - Sablonul de proiectare Proxy poate fi de folos in acest scop.
 - O componenta Broker (care trateaza toate invocarile locale de obiecte) poate fi atasata fiecarui calculator care gazduieste obiecte distribuite.
 - CORBA (Common Object Request Broker Architecture) este un standard bine cunoscut care permite construirea acestui tip de arhitectura.

Exemplu de sistem Broker

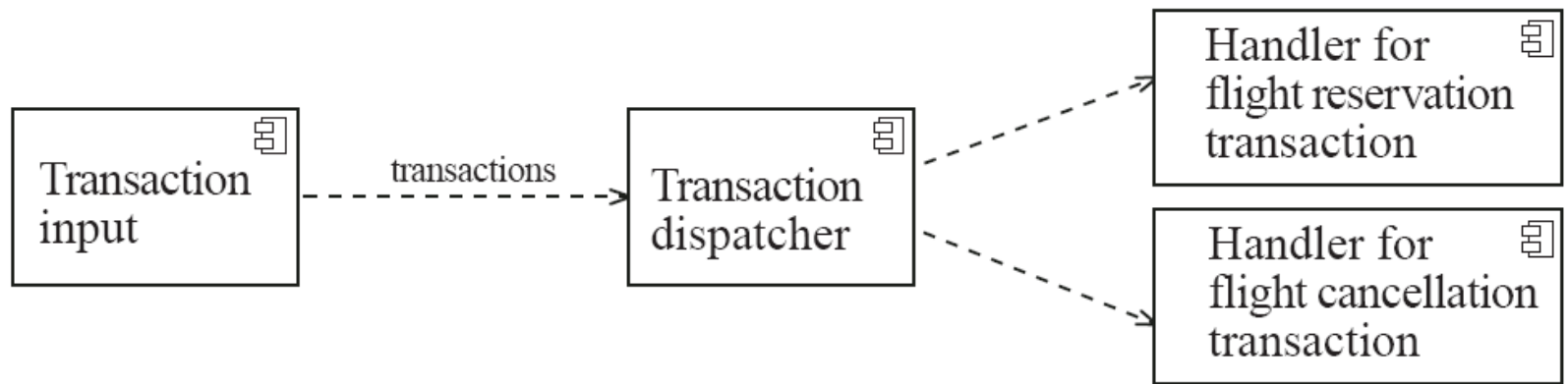


Sablonul arhitectural Transaction Processing

In sablonul arhitectural Transaction Processing:

- Un proces citește o serie de intrări în secvență.
 - Fiecare intrare descrie o *tranzacție* – o comandă care în mod tipic efectuează anumite modificări asupra datelor stocate de sistem.
- Există o componentă *dispecer* care decide acțiunea corespunzătoare fiecărei tranzacții.
- Aceasta expediază un mesaj spre una dintr-o serie de componente care intermediază tranzacția (engl. *transaction handler*).
 - În fiecare caz, este esențial ca tranzacția să fie realizată complet sau deloc.

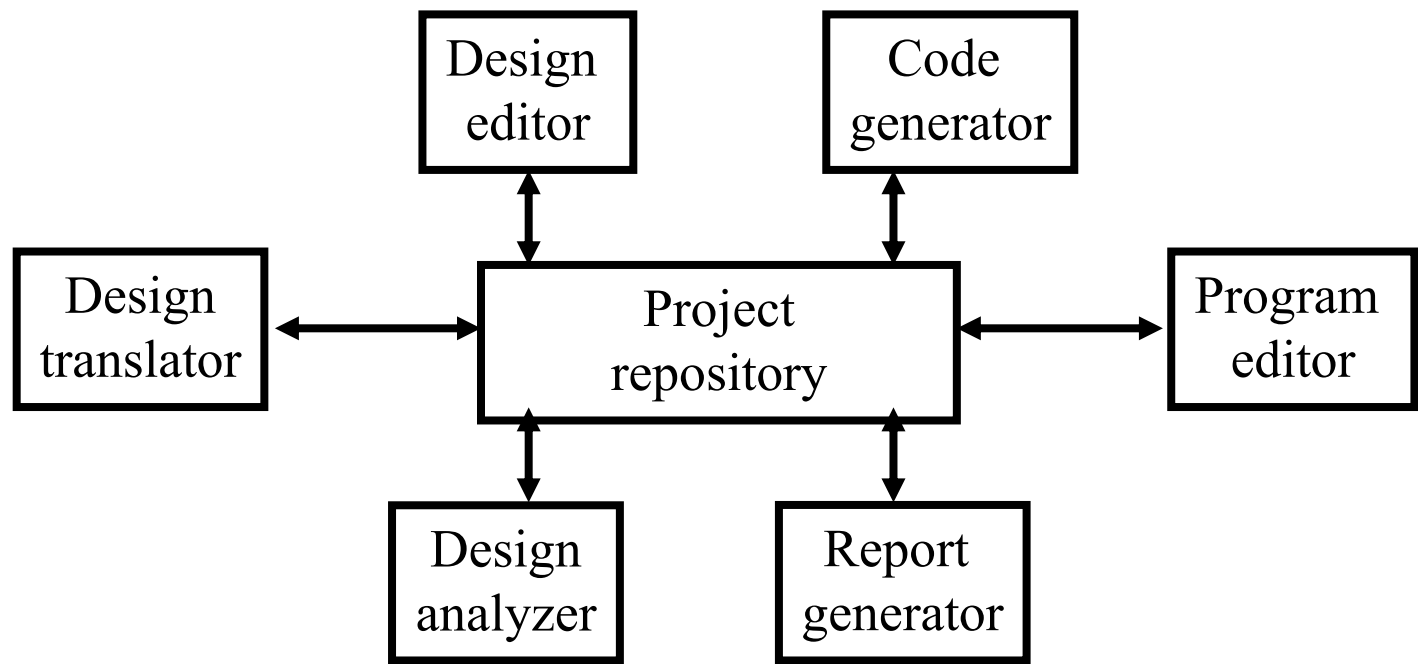
Exemplu de sistem bazat pe sablonul Transaction Processing



Arhitectura Transaction Processing ce poate fi utilizata in cazul sistemului de rezervari pentru curse aeriene (capitolul 5)

Sablonul arhitectural Repository

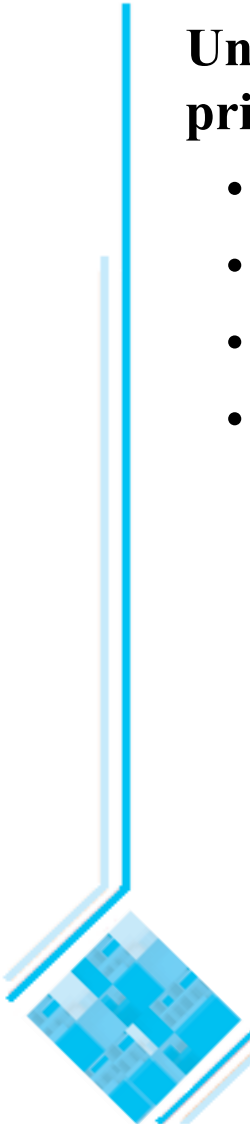
- **Sablonul Repository utilizeaza un depozit (sau baza) de date (engl. *repository*) partajat de toate subsistemele.**
 - De exemplu, acest model poate fi utilizat in proiectarea pachetelor CASE [Som04]:



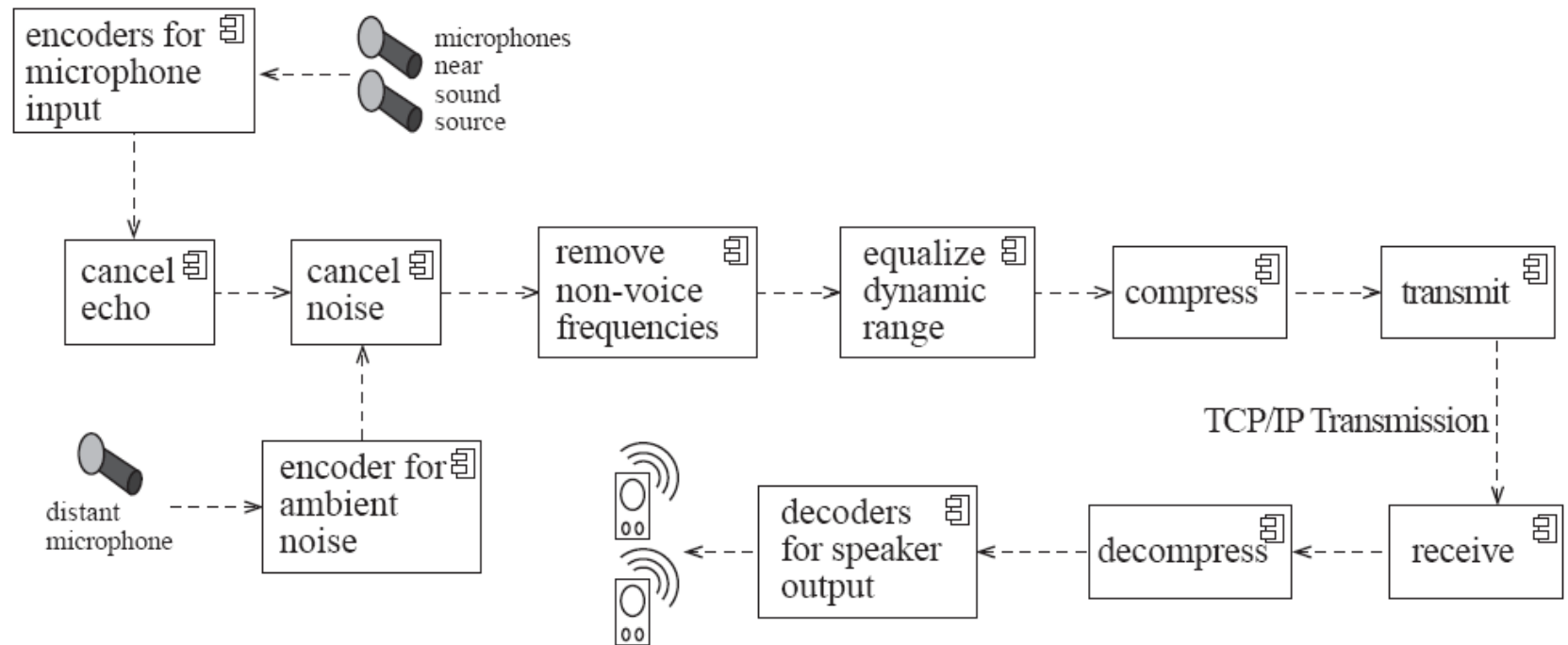
Sablonul arhitectural Pipe-and-Filter

Un flux de date, intr-un format relativ simplu, este transmis printr-o serie de procese:

- Fiecare proces transforma fluxul intr-un anumit mod.
- Fluxul de date alimenteaza arhitectura in mod continuuu.
- (Conceptual) Procesele lucreaza in mod concurent.
- Aceasta arhitectura este foarte flexibila:
 - Aproape toate componentele ar putea fi eliminate
 - Componentele pot fi inlocuite
 - Se pot insera noi componente
 - Anumite componente pot fi reordonate.



Exemplu de arhitectura Pipe-and-Filter

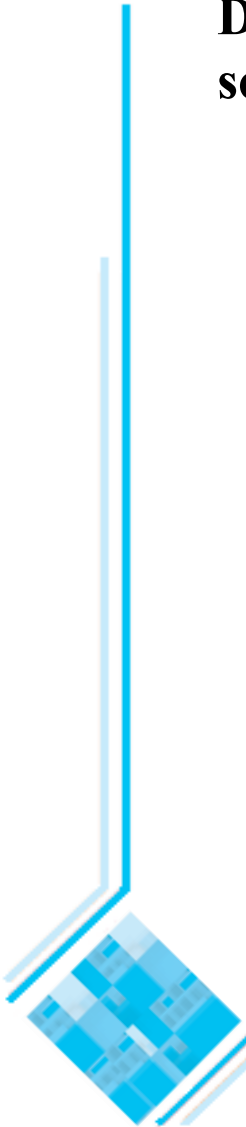


O arhitectura Pipe-and-Filter pentru procesare sunet

9.7 Documentul de design

Documentul de design (sau proiectare) poate sprijini inginerii software in:

- Luarea unor decizii de proiectare
- Comunicarea diverselor aspecte ale proiectului (proiectul se adreseaza celor care implementeaza, modifica sau utilizeaza sistemul)
- Revizuirea si imbunatatirea proiectului



Structura unui document de design

A. Obiective:

- Identificarea sistemului descris in documentul de design
- Referinte catre cerintele software proiectate (*traceability*)

B. Prioritati:

- Descrierea prioritatilor (de exemplu, attribute de calitate) ce ghideaza procesul de proiectare

C. Descriere de ansamblu:

- Ofera cititorului o vedere generala asupra proiectului (de exemplu, sub forma unei diagrame)

D. Probleme majore:

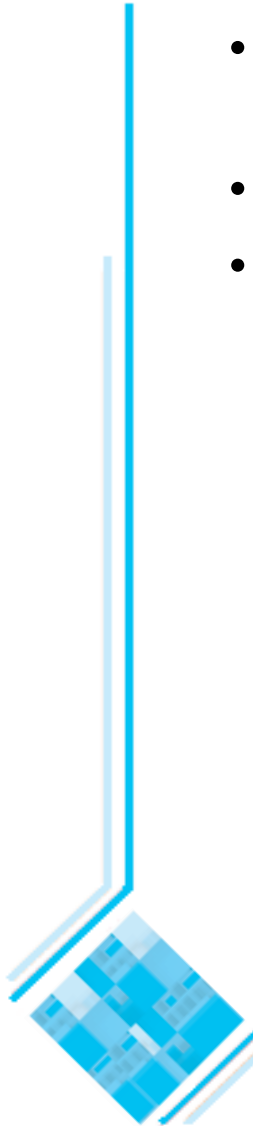
- Probleme majore tratate in proiect
- Solutii alternative considerate, decizii si ratiunile pe care se bazeaza

E. Detalii:

- Orice alte detalii de interes pentru cititor, care nu sunt mentionate in sectiunile precedente

In elaborarea unui document de design

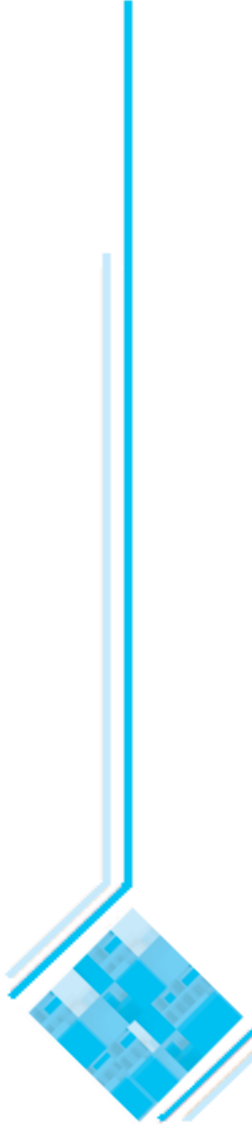
- Se vor evita informatiile *evidente* pentru proiectantii sau pentru programatorii experientati.
- Se vor evita detaliile ce pot fi inserate sub forma de *comentarii* in cod.
- Se vor evita detaliile ce pot fi *extrase automat* din cod (de exemplu lista metodelor `public`).



Referinte suplimentare

- [BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [Pre97] R. Pressman. *Software Engineering: A Practitioner's Approach*, (4th edition). McGraw-Hill, 1997.
- [SG96] M. Shaw and D. Garlan. *Software Architecture*. Prentice-Hall, 1996.
- [Som04] I. Sommerville. *Software Engineering*, (7th edition). Addison-Wesley, 2004.





Testing and Inspecting to Ensure High Quality

Adapted from :

Timothy Lethbridge and Robert Laganriere,

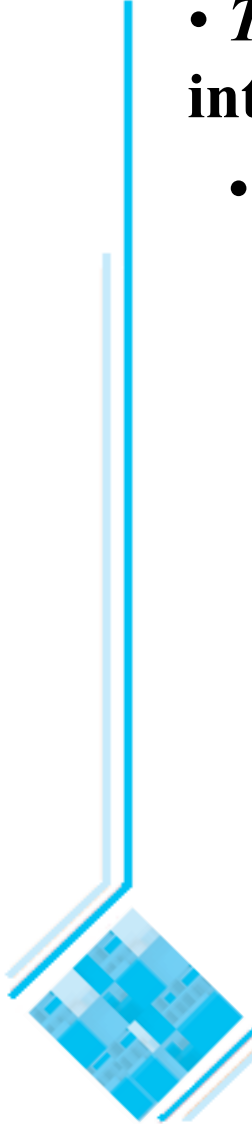
Object-Oriented Software Engineering –

Practical Software Development using UML and Java, 2005

(chapter 10)

10.1 Basic definitions

- A *failure* is an unacceptable behaviour exhibited by a system
 - Any violation of a *functional* or a *non-functional* (*explicit* or *implicit*) requirement should be considered a failure.
 - Examples:
 - » an outright crash,
 - » the production of incorrect output,
 - » the system running too slowly,
 - » the user having trouble finding online help, etc.
 - The frequency of failures measures the *reliability*.
 - Important objective: maximize reliability $\Leftrightarrow$ minimize the failure rate
- A *defect* is a flaw in any aspect of the system that contributes, or may potentially contribute, to the occurrence of one or more failures
 - could be in the requirements, the design or the code
 - It might take several defects to cause a particular failure
- An *error* is an omission or an inappropriate decision by a software developer that leads to the introduction of a defect

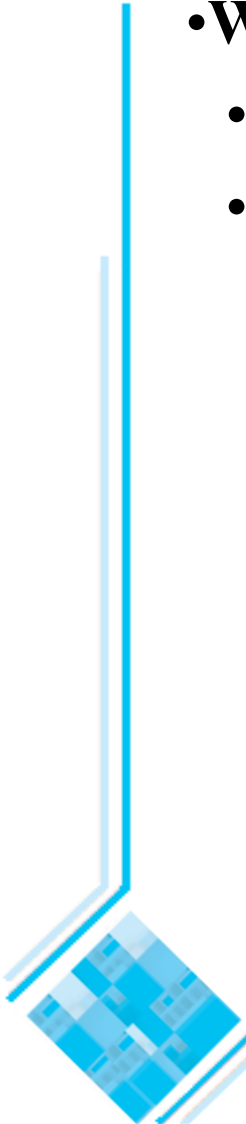
- 
- ***Testing*** is the process of executing a program with the intent of finding any defect that might be present.
 - We emphasize that
 - program testing can only demonstrate the presence of defects not their absence!*

10.2 Effective and efficient testing

- To test *effectively*, you must use a strategy that uncovers as many defects as possible
- To test *efficiently*, you must find the largest possible number of defects using the fewest possible tests
 - Testing is like detective work:
 - The tester must try to understand how programmers and designers think, so as to better find defects.
 - The tester must not leave anything uncovered, and must be suspicious of everything.
 - It does not pay to take an excessive amount of time; the tester has to be *efficient*.

Basic testing techniques

- **We consider two basic testing techniques:**
 - Path testing (applicable in ‘white-box’ testing)
 - Equivalence partitioning (applicable in both ‘white-box’ and ‘black-box’ testing)



White-box testing

Also called ‘glass-box’ or ‘structural’ testing

Testers have access to the system design

- They can
 - Examine the design documents
 - View the code
 - Observe at run time the steps taken by algorithms and their internal data
- Individual programmers often informally employ white-box testing to verify their own code

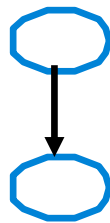
Control flow graph for white-box testing

- A well-known structural testing technique is *path testing*.
 - Path testing is effective at method (procedure, function) level and it is based on the control flow graph (CFG) of the method.
- It helps the programmer to systematically test the code
 - Each branch in the code (such as an **if** or a **while** statement) creates a node in the CFG
 - The testing strategy has to reach a targeted coverage of statements and branches. The objective can be to design tests for:
 - covering all possible paths (most often unfeasible)
 - covering all possible edges** (each statement is executed at least once)
 - The **cyclomatic complexity** (cc) of the CFG gives the maximum number of independent paths that must be tested in order to cover all possible edges [MCC76]:
 - » $cc(CFG) = noEdges - noNodes + 2$
 $= noBinaryDecisionPredicates + 1$
 - It is preferable to limit cc to 10 (11-20 moderate risk, 21-50 high risk, > 50 untestable / very high risk)

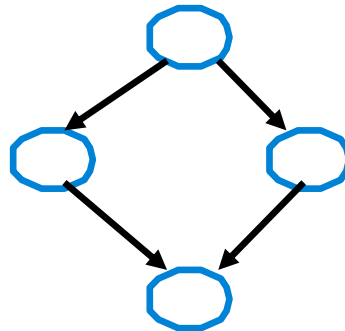
Control flow graph for white-box testing

- **The CFG of a structured program can be built as a combination of CFGs for the basic structured constructs.**

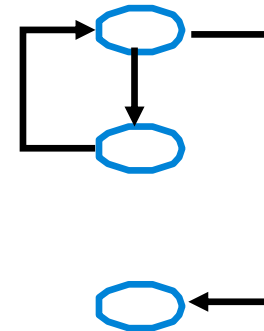
Sequence



If



While



- **The CFG**

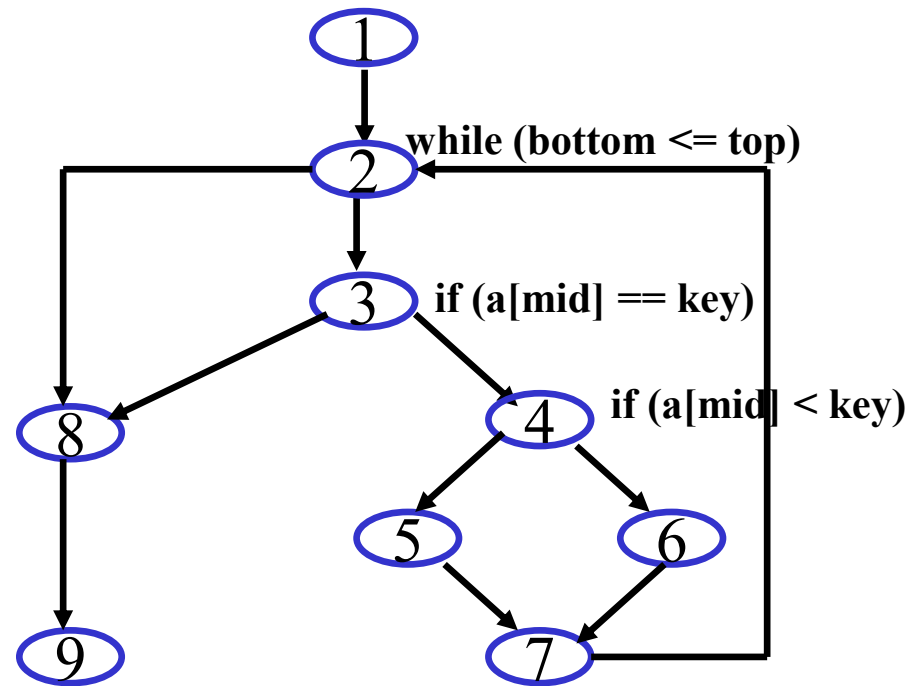
- is a directed graph
- with **single entry & single exit points**

Example: CFG of a binary search routine

[Som00]

```
class BinSearch {  
    ...  
    public static void search (int key, int[] a, Rez r)  
    {  
        int mid; int bottom = 0;  
        int top = a.length - 1;  
        r.found = false; r.index = -1;  
        while (bottom <= top) {  
            mid = (top + bottom) / 2;  
            if (a[mid] == key) {  
                r.index = mid;  
                r.found = true;  
                return;  
            } else {  
                if (a[mid] < key)  
                    bottom = mid + 1;  
                else  
                    top = mid - 1;  
            }  
        }  
    }  
}
```

cc(CFG)
= noEdges - noNodes + 2
= noBinaryDecisionPredicates + 1
= 4



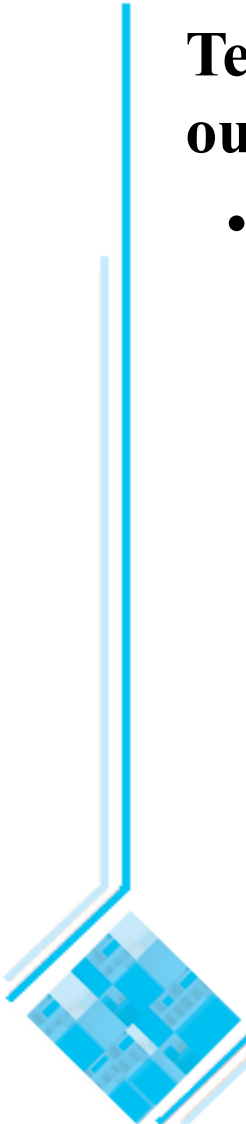
A set of independent paths:

- 1,2,8,9
- 1,2,3,8,9
- 1,2,3,4,5,7,2,8,9
- 1,2,3,4,6,7,2,8,9

Black-box testing

Testers provide the system with inputs and observe the outputs

- They can see none of:
 - The source code
 - The internal data
 - Any of the design documentation describing the system's internals

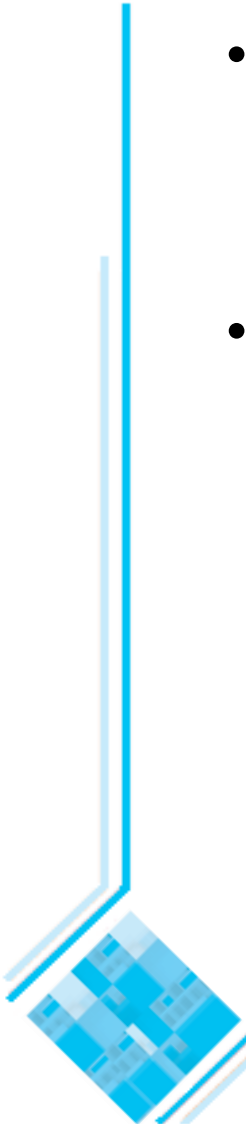


Equivalence classes

- It is inappropriate to test by *brute force*, using *every possible* input value
 - Takes a huge amount of time
 - Is impractical
 - Is pointless!
- You should divide the possible inputs into groups which you believe will be treated similarly by all algorithms.
 - Such a group is called an **equivalence class (EC)**
 - A tester needs only to run **one test per EC**
 - The tester has to
 - understand the required input,
 - appreciate how the software may have been designed

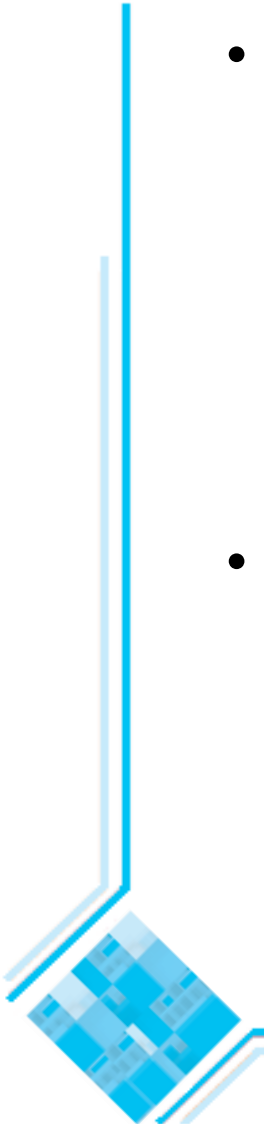
Examples of equivalence classes

- Valid input is a month number (1-12)
 - 3 ECs: $[-\infty..0]$, $[1..12]$, $[13..\infty]$
- Valid input is one of ten (10) strings representing a type of fuel
 - 11 ECs:
 - 10 classes, one for each string
 - 1 class representing all other strings



Combinations of equivalence classes

- **Combinatorial explosion** means that you cannot realistically test every possible system-wide EC.
 - For example, if there are 4 different inputs, each with 5 ECs, then there are $5 \times 5 \times 5 \times 5 = 5^4 (=625)$ possible system-wide ECs.
 - It is impractical to design 625 different test cases.
- In practice, the number of tests must be kept in reasonable limits !



Example equivalence class combinations

- **Example: a system that contains information about all kinds of land vehicles (e.g. passenger, racing, etc). Five types of input:**
 1. Type of unit: 'Metric' (km/h), or 'Imperial or US' (mph)
 - By using 'radio buttons' there is no possibility of an invalid input
 - 2 ECs: Metric, US/Imperial
 2. Maximum speed (integer): 1-750 km/h, or 1-500 mph
 - Validity depends on the type of unit (metric or US/Imperial)
 - 4 ECs: $[-\infty..0]$, $[1..500]$, $[501..750]$, $[751.. \infty]$
 3. Type of fuel (string): one of 10 possible strings
 - 11 ECs (10 plus 1 class for any invalid string)
 4. Time to accelerate to 100km/h or 60 mph (integer): 1-100s
 - 3 ECs: $[-\infty..0]$, $[1..100]$, $[101..\infty]$
 5. Range (integer): 1-5000 km, or 1-3000 miles.
 - Validity depends on the the type of unit (metric or US/Imperial)
 - 4 ECs: $[-\infty..0]$, $[1..3000]$, $[3001..5000]$, $[5001.. \infty]$

Example equivalence class combinations

- **Total number of ECs: $2 \times 4 \times 11 \times 3 \times 4 = 1056$ (!!)**
- **A reasonable strategy for reducing the number of tests:**
 - 1. You should first make sure that at least one test is run with every EC of every individual input.**
 - 2. You should test all combinations where an input is likely to *affect the interpretation* of another.**
 - You should also test a few other random combinations of ECs.
- **By using this strategy the number of tests for the vehicles example can be reduced from 1056 to 11:**
 - **You run one test for each EC of input 3 (11 tests)**
 - **While doing this you vary (in parallel)**
 - **the 8 possible combinations of ECs of inputs 1 and 2**
 - **the 8 possible combinations of ECs of inputs 1 and 5**
 - **the 3 possible ECs of input 4**

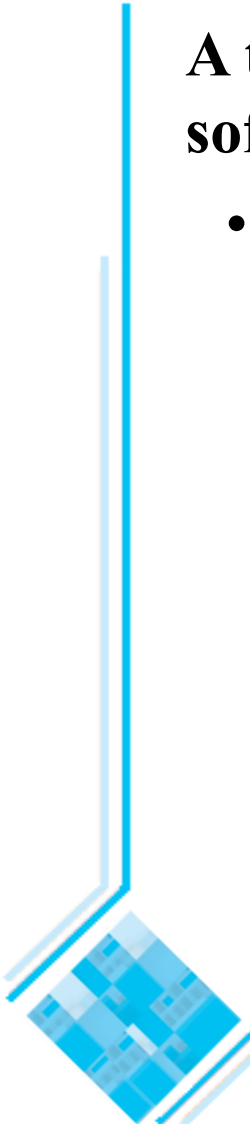
Testing at the boundaries of equivalence classes

- More errors in software occur at **the boundaries of equivalence classes**
- The idea of ECs testing should be expanded to specifically test values at the extremes of each EC.
 - E.g. The number 0 often causes problems
- *E.g.:* If the valid input is a month number (1-12)
 - Test the ECs as before
 - Test 0, 1, 12 and 13 as well as very large positive and negative values

Detecting specific categories of defects

A tester must try to uncover any defects the other software engineers might have introduced.

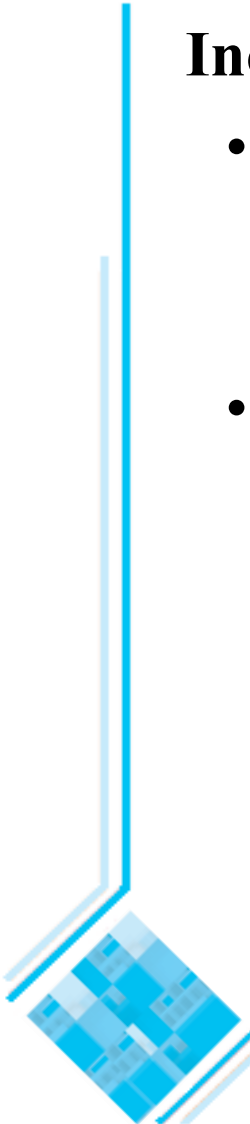
- This means designing tests that explicitly try to catch a range of specific types of defects that commonly occur



10.3 Defects in Ordinary Algorithms

Incorrect logical conditions

- *Defect:*
 - The logical conditions that govern looping and if-then-else statements are wrongly formulated.
- *Testing strategy:*
 - Use equivalence class and boundary testing.
 - Consider as an input each variable used in a rule or logical condition.



Example of incorrect logical conditions defect (in software for an aircraft landing gear system)

- **Specification:**

—The landing gear must be deployed whenever the plane is within 2 minutes from landing or takeoff, or within 2000 feet from the ground. If visibility is less than 1000 feet, then the landing gear must be deployed whenever the plane is within 3 minutes from landing or lower than 2500 feet.

- (Otherwise, the aircraft's **alarm** is supposed to sound !)

- What is the **hard-to-find defect** in the following **implementation** ?

```
if (!landingGearDeployed &&
    (min(now-takeoffTime, estLandTime-now) <
      (visibility < 1000 ? 180 : 120) ||
      relativeAltitude <
        (visibility < 1000 ? 2500 : 2000)
    )
{
    throw
        new LandingGearException();
}
```

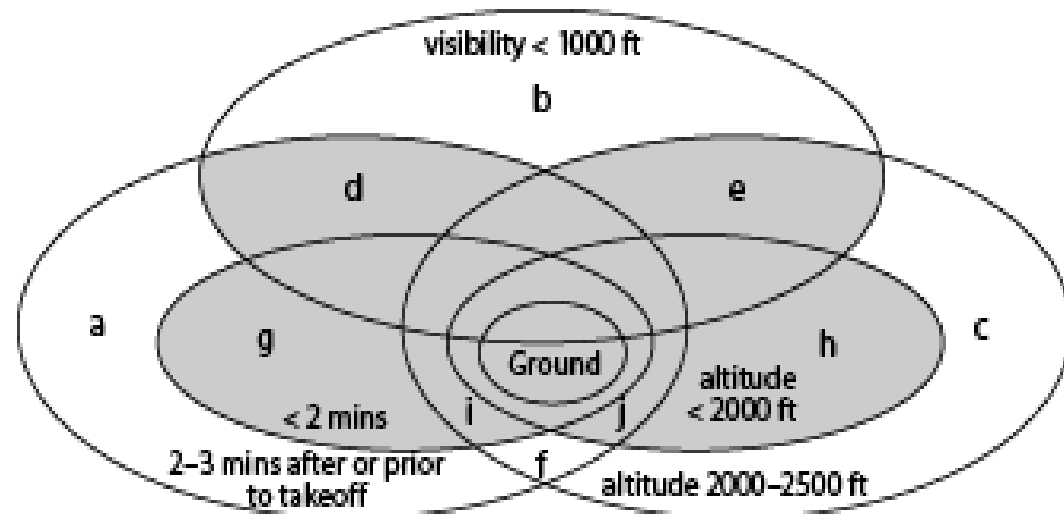

Example of incorrect logical conditions defect

—Total number of system ECs: 108

—For such a **critical system** all the 108 ECs (plus their boundaries) should be tested (in a simulated environment).

—Venn diagram: the gray area indicates the ECs where the alarm should sound if the landing gear is not deployed.

<i>Variable affecting condition</i>	<i>Equivalence classes</i>
Time since take-off	3: Within 2 minutes after take-off, 2–3 minutes after take-off, more than 3 minutes after takeoff
Time to landing	3: Within 2 minutes prior to landing, 2–3 minutes prior to landing, more than 3 minutes prior to landing
Relative altitude	3: < 2000 feet, 2000 feet to 2500 feet, 2500 feet
Visibility	2: < 1000 feet, 1000 feet
Landing gear deployed	2: true, false



Defects in Ordinary Algorithms

Not setting up the correct preconditions for an algorithm

- *Defect:*
 - *Preconditions* state what must be true before the algorithm should be executed.
 - A defect would exist if a program proceeds to do its work, even when the preconditions are not satisfied.
- *Testing strategy:*
 - Run test cases in which each precondition is not satisfied.

Defects in Ordinary Algorithms

Not handling null conditions

- *Defect:*
 - A *null condition* is a situation where there normally are one or more data items to process, but sometimes there are none.
 - It is a defect when a program behaves abnormally when a null condition is encountered.
- *Testing strategy:*
 - Determine the null conditions and run appropriate tests.
- *Example:*
 - A program that calculates the average sales of the members of each division in an organization.
 - Null condition: a division that has no members
 - Typical defect: the program attempts to divide by zero (zero sales divided by zero members)
 - Test the behavior of the program for a division with zero members.

Defects in Ordinary Algorithms

Not handling singleton or non-singleton conditions

- *Defect:*
 - A *singleton condition* is similar to a null condition. It occurs when there is normally *more than one* of something, but sometimes there is only one.
 - A *non-singleton condition* is the inverse (there is normally exactly *one*, but occasionally there is more than one).
 - Defects occur when the unusual case is not properly handled.
- *Testing strategy:*
 - Determine unusual conditions and run appropriate tests.



Defects in Ordinary Algorithms

Off-by-one errors

- *Defect:*
 - A program loops one too many times or one too few times.
 - Or a program inappropriately adds or subtracts one
 - E.g. by using ++v instead of v++ (or vice versa)
 - This is a particularly common type of defect.
- *Testing strategy:*
 - Develop **boundary tests** in which you verify that the program:
 - performs the correct number of iterations
 - computes the correct numerical answer

Defects in Ordinary Algorithms

- *Example:* Assuming Java-like 0-based indexing, the following loop always skips the first element:

```
for (i=1; i < arrayname.length; i++)  
{  
    /* do something */  
}
```

- The problem can easily be avoided by the designer by using the concept of an Iterator (in Java the `Iterator` class).
- But the tester must assume the worst and design boundary tests.

Defects in Ordinary Algorithms

Not terminating a loop or recursion

- *Defect:*

- A loop or a recursion does not always terminate, i.e. in some conditions it is ‘infinite’.

- It is the responsibility of the programmer to avoid (undesirable) infinite behaviour, but the tester must assume the worst and design appropriate test cases.

- *Testing strategies:*

- Analyse what causes a repetitive action to be stopped.

- Run test cases that you anticipate might not be handled correctly.

Defects in Ordinary Algorithms

Use of inappropriate standard algorithms

- *Defect:*
 - An inappropriate standard algorithm is one that is unnecessarily inefficient or has some other property that is widely recognized as being bad.
- *Testing strategies:*
 - The tester has to know the properties of algorithms and design tests that will determine whether any undesirable algorithms have been implemented.
- *Example:* The designer uses a ‘bubble sort’ instead of ‘quick sort’.
 - The tester can increase (e.g. *double*) the number of items being sorted and observe how execution time is affected.

10.4 Defects in Numerical Algorithms

- **Not using enough bits or digits to store maximum values**
 - *Testing strategies:* Test using very large numbers
 - *Example:* In 5 June 1996 the first test flight of the French Ariane-5 Launcher ended in failure.
 - The failure was caused by a *software error*: an attempt to convert a (too large) 64-bit floating point number to a 16-bit signed integer.
 - It was one of the most expensive computer bugs in history (causing a loss of approx. \$500 million).



Defects in Numerical Algorithms

Assuming a floating point value will be exactly equal to some other value

- *Defect:*

- If you perform an arithmetic calculation on a floating point value, then the result will very rarely be computed exactly.
- It is a defect to test if a floating point value is exactly equal to some other value.
- To test equality, you should always test if it is within a small range around that value.

- *Testing strategies:*

- Standard **boundary testing** should detect this type of defect.

Defects in Numerical Algorithms

Example of defect in testing floating value equality:

Bad:

```
for (double d = 0.0; d != 10.0; d+=2.0) { ... }
```

Better:

```
for (double d = 0.0; d < 10.0; d+=2.0) { ... }
```

10.5 Defects in Timing and Co-ordination

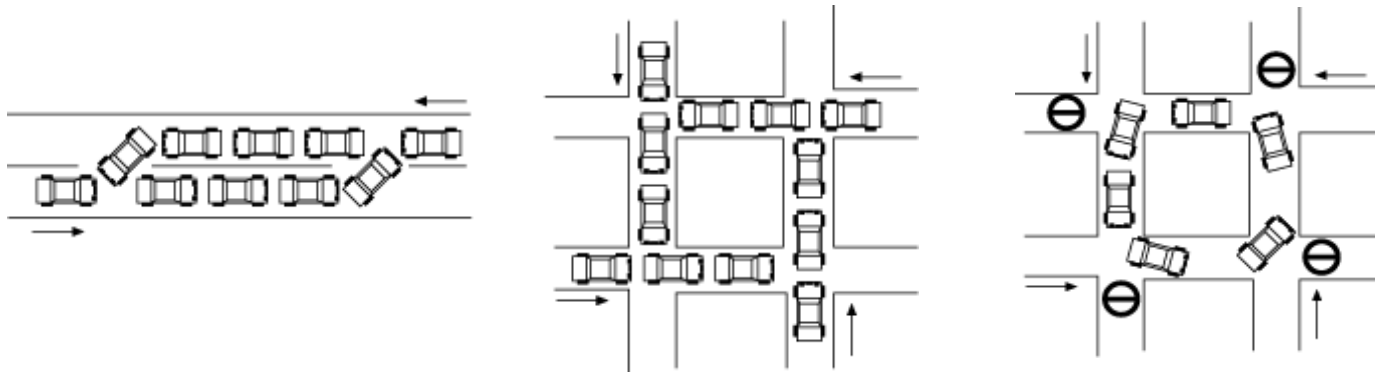
Deadlock and livelock

- *Defects:*

- A *deadlock* is a situation where two or more threads are stopped, waiting for each other to do something before either can proceed.

- The system is hung

- Livelock* is similar, but now the system can do some computations, but can never get out of some states.



Defects in Timing and Co-ordination

Deadlock and livelock

- *Testing strategies:*

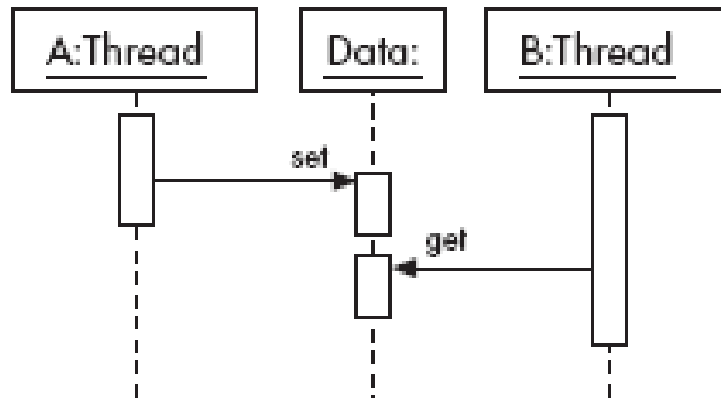
- Deadlocks and livelocks occur due to unusual combinations of conditions that are hard to anticipate or reproduce.
- If black box testing is the only possibility you can:
 - Run a large number of threads concurrently.
 - Deliberately deny resources to one or more threads (e.g. by cutting network connections, or by locking input files).
 - Vary the relative speed of the different threads
- However, the deadlock problem is difficult. Expensive white-box testing and *inspections* may be necessary.
 - *Formal verification techniques* have been successful in producing concurrent systems that have a high level of reliability.

Defects in Timing and Co-ordination

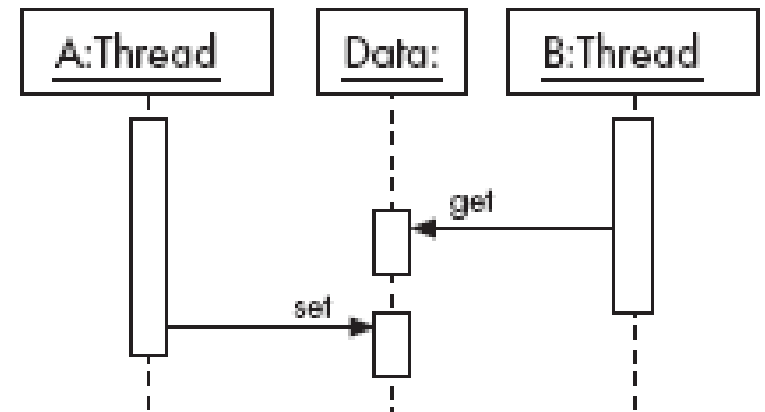
Critical races

- *Defect:*
 - One thread experiences a failure because another thread interferes with the ‘normal’ (or ‘expected’) sequence of events. The defect is not that the other thread tries to do something, but that the program allows the (‘abnormal’) interference to occur.
- *Testing strategies:*
 - Same as for deadlock and livelock.
 - *Testing for deadlock or critical races is generally ineffective, since they are produced by particular chains of events.*
 - It is particularly hard to test for critical races by using black-box testing alone.
 - Once again, *inspection* is often a better solution.

Example of critical race



a) Normal

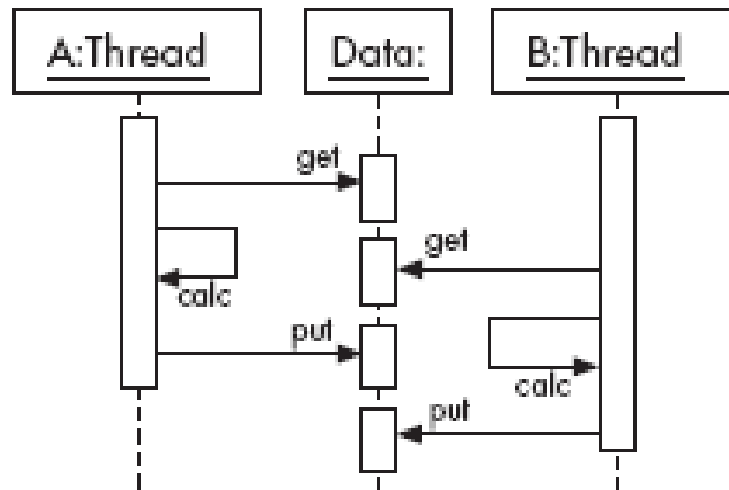


b) Abnormal due to delay in thread A

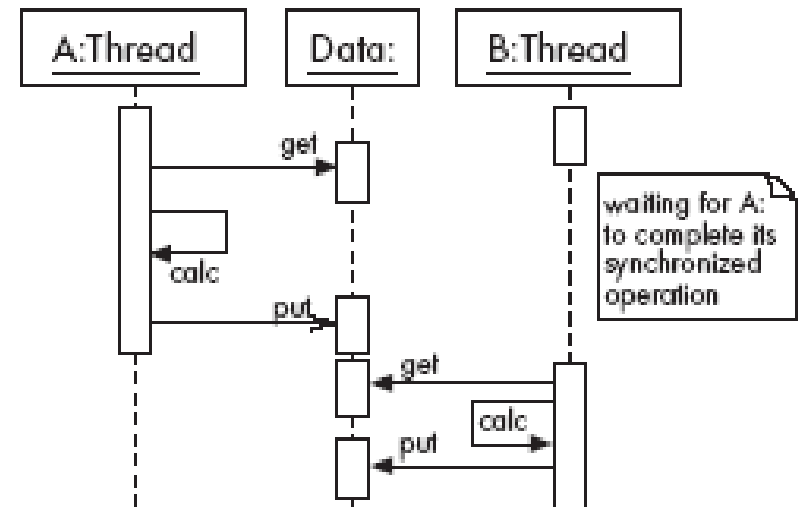
Synchronization

- **The easiest way to deal with deadlocks and critical races is to avoid them at design time !**
- Critical races can be prevented by *locking* data so that they cannot be accessed by other threads when they are not ready.
 - In Java, the **synchronized** keyword can be used.
 - It ensures that no other thread can access an object until the synchronized method terminates.

Example of a synchronized method



a) Abnormal: The value put by thread A is immediately overwritten by the value put by thread B.



b) The problem has been solved by accessing the data using synchronized methods.

10.9 Strategies for Testing Large Systems

Integration testing: big bang versus incremental testing

- In *big bang* testing, you take the entire system and test it all at once.
 - Impractical for most nontrivial systems.
 - Errors resulting from modules interactions are difficult to locate.
- A better strategy in most cases is *incremental* testing.
 - When you encounter a failure you can *locate* the defect more easily.
 - Unfortunately, there is a high probability that efforts to remove defects will actually add new ones. This phenomenon is called **the ripple effect**.
 - After a defect is located and fixed you may have to perform regression testing.
 - **Regression testing** is the process of retesting (parts of) a system that was previously built and tested when new functionality is added.

Strategies for Testing Large Systems

Incremental testing can be performed *horizontally* or *vertically*.

- Horizontal testing can only be applied when the system is divided into *separate* subsystems.
- The basic strategies for *vertical incremental testing* are:

—Top-down

- In this approach you have to design **stubs** (i.e. modules with the same interface but very limited functionality) that simulate lower-level modules.

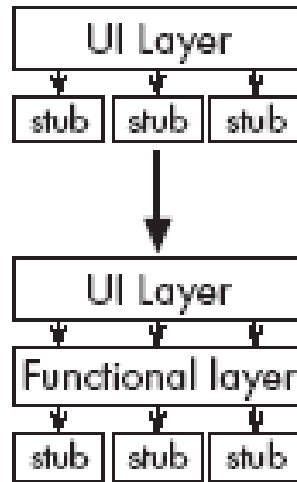
—Bottom-up

- In this approach you have to design **test drivers** for the lower-level modules.
 - » A test driver is a module designed specifically for testing by making calls to the lower layers.

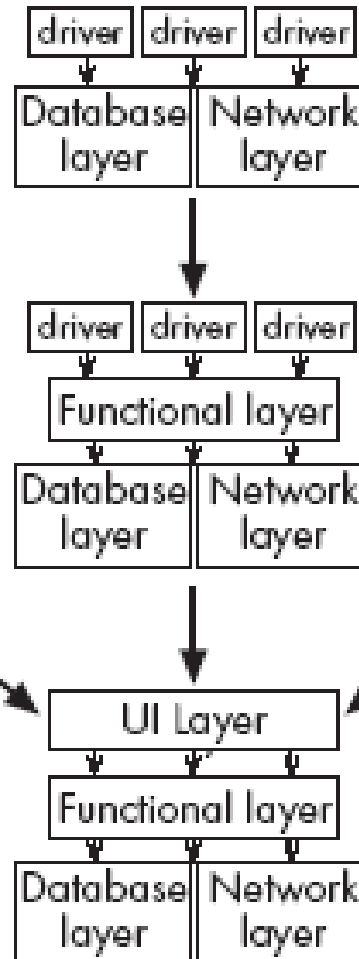
—Sandwich

Vertical strategies for incremental integration testing

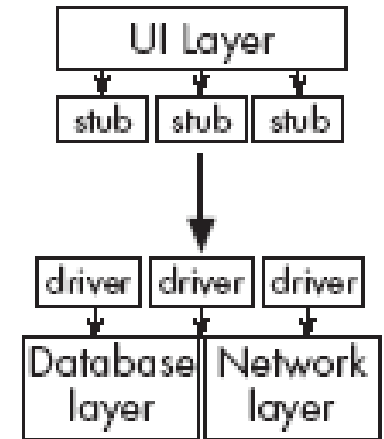
Top-down testing



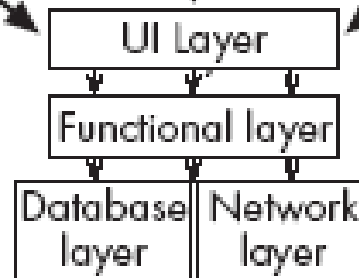
Bottom-up testing



Sandwich testing



Fully
integrated
system



- Most of the testing techniques and strategies given in this chapter work just as well for OO software as for non-OO software.
- In an OO system method and class testing correspond to *unit testing*.
 - An OO class is a unit of protection.
 - You can write special methods within each class to act as test drivers for the methods of that class.
 - In Java, you can create a main() method (to act as a test driver) in each class in order to exercise the other methods.
 - JUnit [Bec04] is a professional unit testing tool for Java classes:
<http://www.junit.org>
- However, in an OO system there is no obvious ‘top’ to the system.
 - Neither top-down, nor bottom-up integration are really appropriate for OO systems [Som00].



Use case integration testing

- **Use-case or scenario-based testing** is often the most effective *integration testing strategy* for OO systems [JBR99,Som00].
 - Black-box tests can be derived directly from the use case model.
 - Each use case can give rise to a number of *test cases*.
 - White-box tests are based on the use case realizations in the design model.
 - Interaction diagrams (e.g. *sequence diagrams*) provide essential information for test cases design.

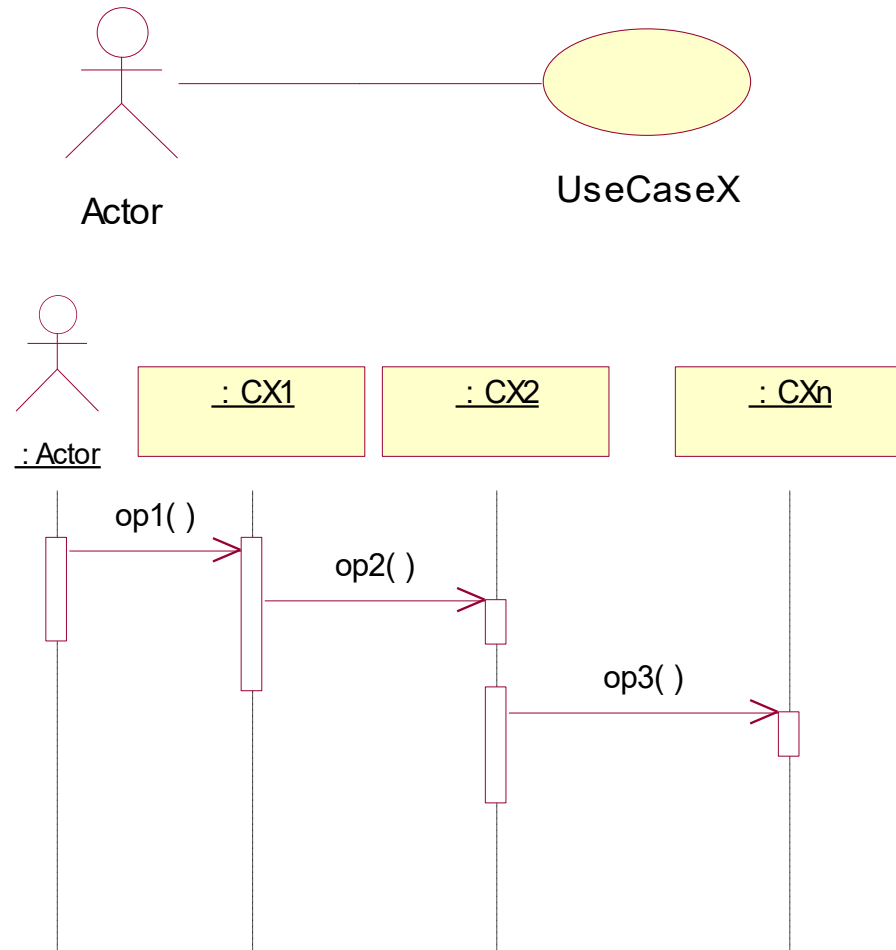


Test Cases

- A *test case* is an explicit set of instructions designed to detect a particular class of defect in a software system.
- Each test case should include the following information:
 - what to test (identification information),
 - with which input,
 - (expected) result,
 - and under which conditions.
- The specification may also include
 - detailed instructions on how to perform the test case (the *test procedure*)
 - cleanup instructions (when needed)
 - Example: instructions to remove erroneous test data (added during the execution of the test case) from some external database.

Designing integration test cases based on sequence diagrams

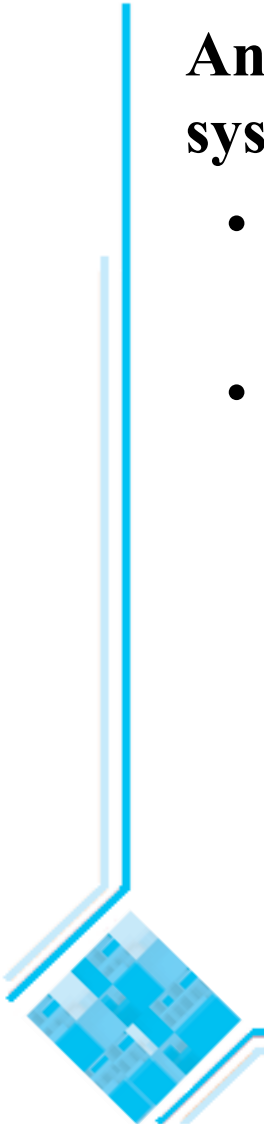
- Assume that:
 - CX1, CX2, ..., CXn are the classes that participate the design of UseCaseX.
 - The **sequence diagram** describes a use scenario for UseCaseX.
- Based on such an interaction (sequence) diagram, you can design an **integration test case** by specifying the conditions (start state, input from the actor, etc) that make the sequence happen [JBR99].
 - A **defect** is identified if the actual interaction (captured during the execution of the test case) and the interaction specified by the sequence diagram are not the same.
- The **actual interaction** can be captured
 - by running a debugger
 - by using print statements (e.g. in Java `System.out.println()`), etc.



10.10 Inspections

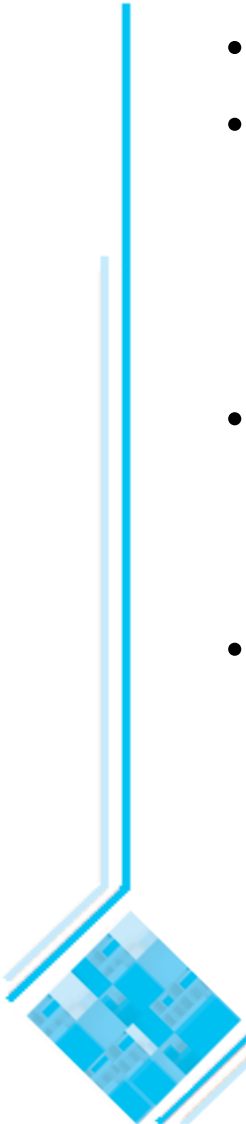
An inspection is an activity in which one or more people systematically

- Examine source code or documentation, looking for defects.
- Normally, inspection involves a meeting...
 - Although participants can also inspect alone at their desks.



Roles on inspection teams

- The *author*
- The *moderator*.
 - Calls and runs the meeting.
 - Makes sure that the general principles of inspection are adhered to.
- The *secretary*.
 - Responsible for recording the defects when they are found.
 - Must have a thorough knowledge of software engineering.
- *Paraphrasers*.
 - Step through the document explaining it in their own words.

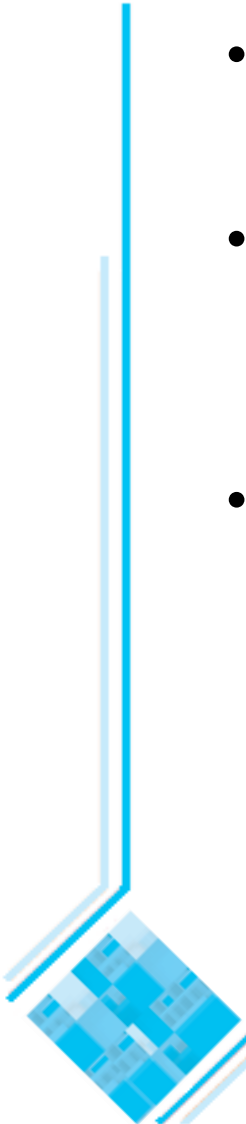


Principles of inspecting

- Inspect the most important documents of all types
 - code, design documents, test plans and requirements
- Choose an effective and efficient inspection team
 - between two and five people
 - Including experienced software engineers
- Require that participants prepare for inspections
 - They should study the documents prior to the meeting and come prepared with a list of defects
- Only inspect documents that are ready
 - Attempting to inspect a very poor document will result in defects being missed

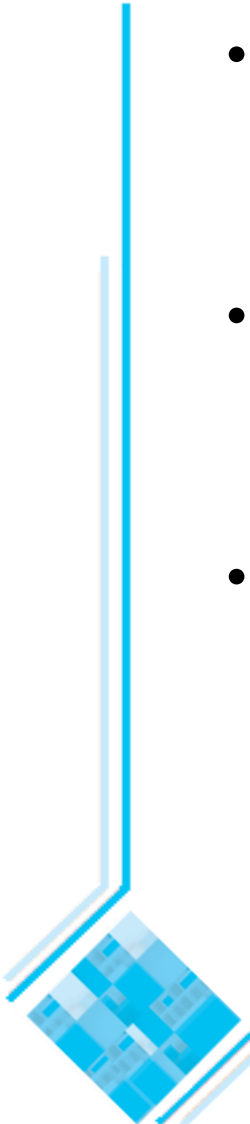
Principles of inspecting

- Avoid discussing how to fix defects
 - Fixing defects can be left to the author
- Avoid discussing style issues
 - Issues like are important, but should be discussed separately
- Do not rush the inspection process
 - A good speed to inspect is
 - 200 lines of code per hour (including comments)
 - or ten pages of text per hour



Principles of inspecting

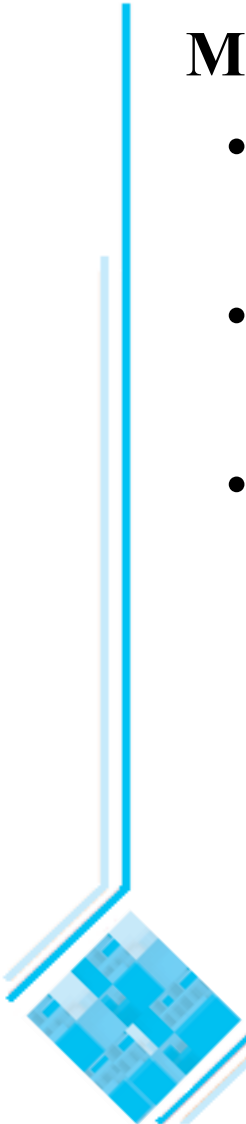
- Avoid making participants tired
 - It is best not to inspect for more than two hours at a time, or for more than four hours a day
- Keep and use logs of inspections
 - You can also use the logs to track the quality of the design process
- Re-inspect when changes are made
 - You should re-inspect any document or code that is changed more than 20%



A peer-review process

Managers are normally not involved

- This allows the participants to express their criticisms more openly, not fearing repercussions
- The members of an inspection team should feel they are all working together to create a better document
- Nobody should be blamed

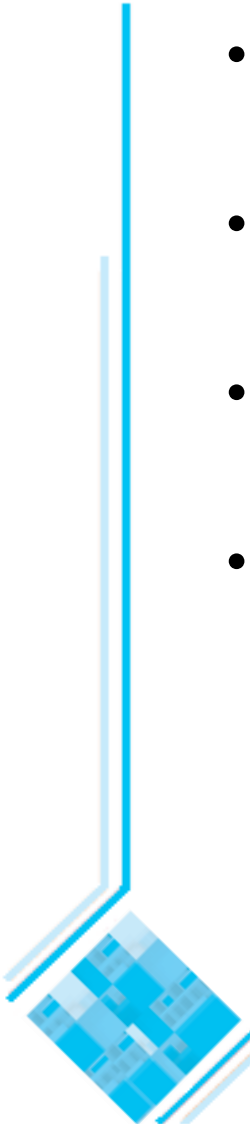


Conducting an inspection meeting

1. The moderator calls the meeting and distributes the documents.
2. The participants prepare for the meeting in advance.
3. At the start of the meeting, the moderator explains the procedures and verifies that everybody has prepared.
4. Paraphraser take turns explaining the contents of the document or code, without reading it verbatim.
 - Requiring that the paraphraser not be the author ensures that the paraphraser say what he or she *sees*, not what the author *intended* to say.
5. Everybody speaks up when they notice a defect.

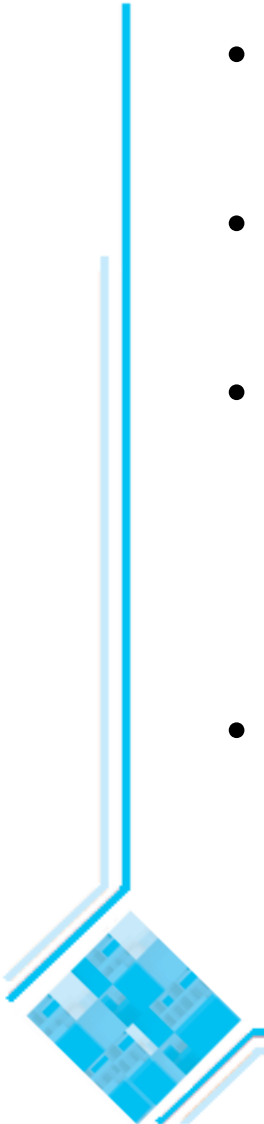
Inspecting compared to testing

- Both testing and inspection rely on different aspects of human intelligence.
- Testing can find defects whose consequences are obvious but which are buried in complex code.
- Inspecting can find defects that relate to maintainability or efficiency.
- The chances of mistakes are reduced if both activities are performed.



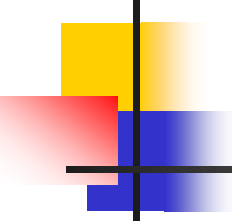
Testing or inspecting, which comes first?

- It is important to inspect software *before* extensively testing it.
- The reason for this is that inspecting allows you to quickly get rid of many defects.
- If you test first, and inspectors recommend that redesign is needed, the testing work has been wasted.
 - There is a growing consensus that it is most efficient to inspect software *before* any testing is done.
- Even before developer testing



Additional references

- [Bec04] K. Beck. *JUnit pocket guide*. O'Reilly, 2004.
- [BB99] W. Boggs, M. Boggs. *Mastering UML with Rational Rose* (1st edition), Sybex, 1999.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [MCC76] T. McCabe. A software complexity measure. *IEEE Trans. Software Engineering*, 2:308-320, 1976.
- [Som00] I. Sommerville. *Software Engineering* (6th edition), Addison-Wesley, 2000.
- [Som04] I. Sommerville. *Software Engineering* (7th edition), Addison-Wesley, 2004.
- [Som16] I. Sommerville. *Software Engineering* (10th edition), Addison-Wesley, 2016.

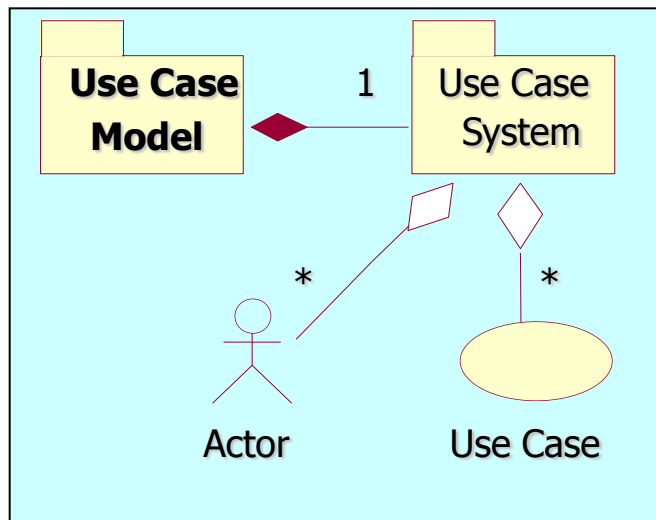


Use-case-driven development

- Use-case-driven development is a key feature of the unified process [BRJ99,JBR99,Kru03].
 - For large projects, it is usually employed as part of an iterative process.
- The use-case-driven development methodology comprises the following basic (activities or) workflows, each producing a corresponding model described by using the UML:
 - Requirements
 - Analysis
 - Design
 - Implementation
 - Test

Use-case-driven development

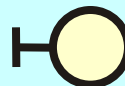
Workflows & models [BRJ99,JBR99,Kru03]:



Requirements

Analysis stereotypes

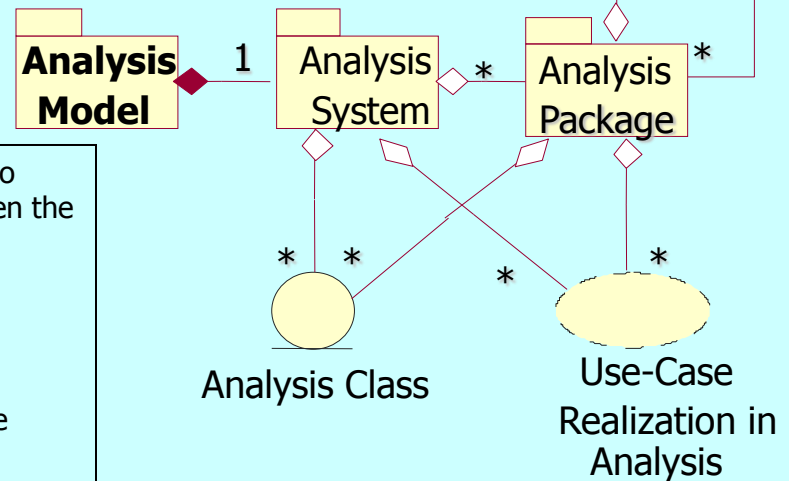
- Boundary class – used to model interactions between the system and its actors



- Control class – used to coordinate (or control) the activities of other classes



- Entity class – used to model long-lived (often persistent) information



- Typically, there is one control class per use case realization.
- Communication diagrams are primarily used to model the interactions between analysis objects.

Analysis

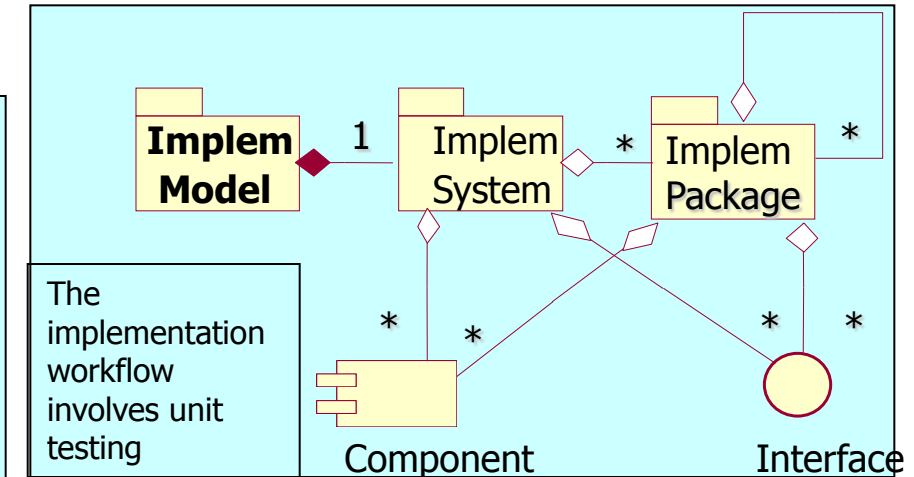
```

classDiagram
    class DesignModel
    class DesignSystem
    class DesignPackage
    class DeploymentModel
    class Node
    class DesignClass
    class UseCase
    class Interface

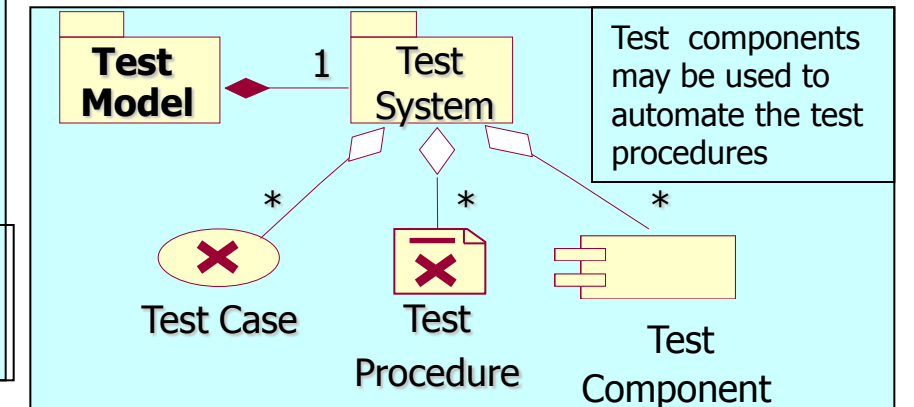
    DesignModel "1" *-- "*" DesignSystem
    DesignSystem "1" *-- "*" DesignPackage
    DesignPackage "1" *-- "*" DesignPackage
    DeploymentModel "1" *-- "*" Node
    DesignSystem "1" *-- "*" DesignClass
    DesignSystem "1" *-- "*" UseCase
    DesignSystem "1" *-- "*" Interface
    DesignPackage "1" *-- "*" UseCase
    DesignPackage "1" *-- "*" Interface
    
```

Sequence diagrams are primarily used to model the interactions between design objects.

Design



Implementation

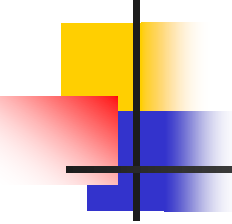


Test



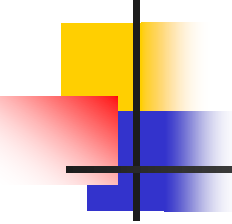
Use-case-driven development

Workflow	Description
Requirements	Actors who interact with the system are identified and use cases are developed to model the system requirements. The use-case model captures the functional requirements. Specific non-functional requirements can be attached to each use case.
Analysis	A conceptual model of the system based on analysis classes and use-case realizations is created. Conceptual classes are analyzed based on the roles (responsibilities, attributes) they play in the various use-case realizations. An analysis class is an abstraction of a number of design classes.
Design	The design model consists of design classes and sub-systems, use-case realizations, and deployment architectural models. The design model is developed based on the analysis model, but it is also adapted to the implementation environment (GUI kits, architectural frameworks, etc.).
Implementation	The components in the system (that implement design classes and may refine the deployment model) are structured into implementation sub-systems.
Test	System testing follows the completion of the implementation. The test model comprises test cases that can be derived directly from the use case model, test procedures and test components.



References

- [BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [Kru03] P. Kruchten. *The Rational Unified Process: An Introduction* (3rd edition). Addison-Wesley, 2003.
- [RS13] D. Rosenberg, M. Stephens, *Use Case Driven Object Modeling with UML: Theory and Practice*, Apress, 2013.
- [Som16] I. Sommerville. *Software Engineering*, (10th edition). Addison-Wesley, 2016.



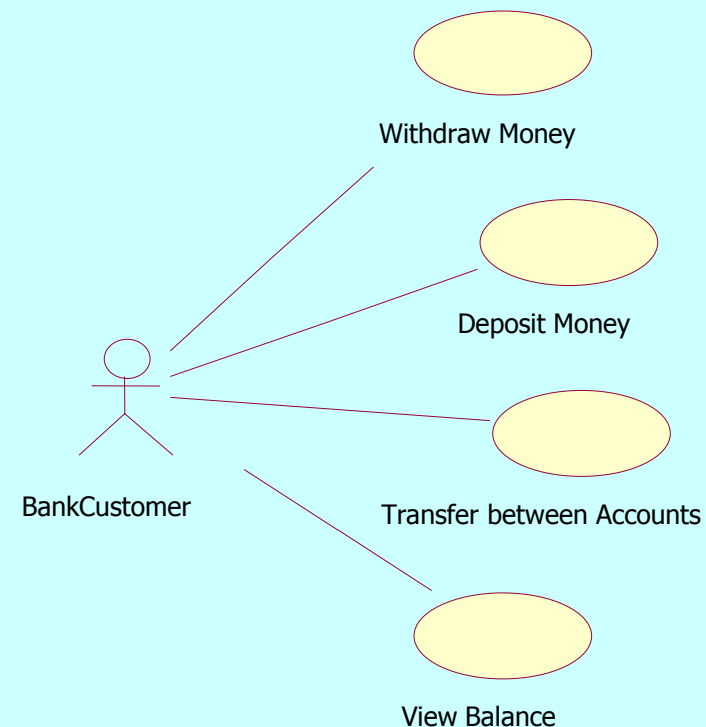
Use-case-driven development

- We illustrate the **use case driven development** approach (specific of the **Unified Process** [JBR99]) by means of a **case study**
 - An **Automated Teller Machine (ATM)** system based on examples from [JBR99,BB99,BB02,Kru03]
- In the **use-case driven** approach **each basic workflow produces a corresponding model:**
 - Use case model (developed in requirements workflow)
 - Analysis model
 - Design model
 - Implementation model
 - Test model

Case study: an ATM system

Use Case Model

- A **Use Case Model** is produced in the requirements workflow
- One can attach (natural language) descriptions to each use case, including descriptions of
 - Representative use case scenarios
 - E.g., the steps of a successful transfer between two accounts
 - Non-functional requirements (e.g., expected response time)
- The realization of the Withdraw Money use case is described in [JBR99]
- In this presentation we focus on the **Transfer between Accounts use case** realization
 - The other use cases must be handled in a similar manner

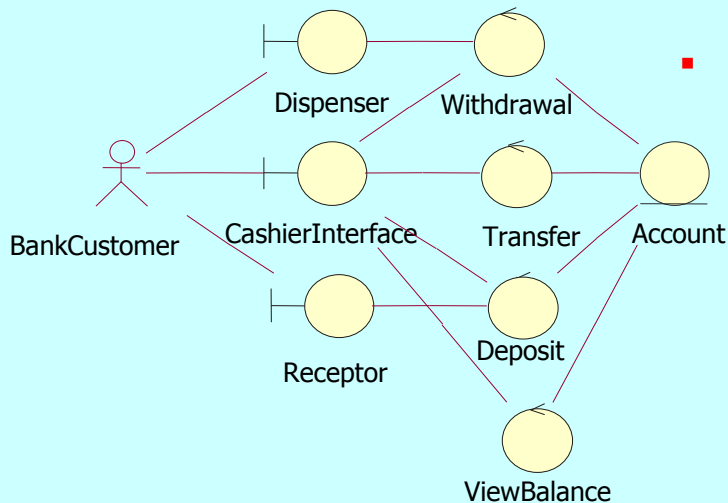


Case study: an ATM system

Analysis Model

- The **Analysis Model** works as a first cut at the design model [JBR99]
- Engineers develop an analysis model in order to better understand the problem

- ATM Analysis structure
- There is one control class per use case realization

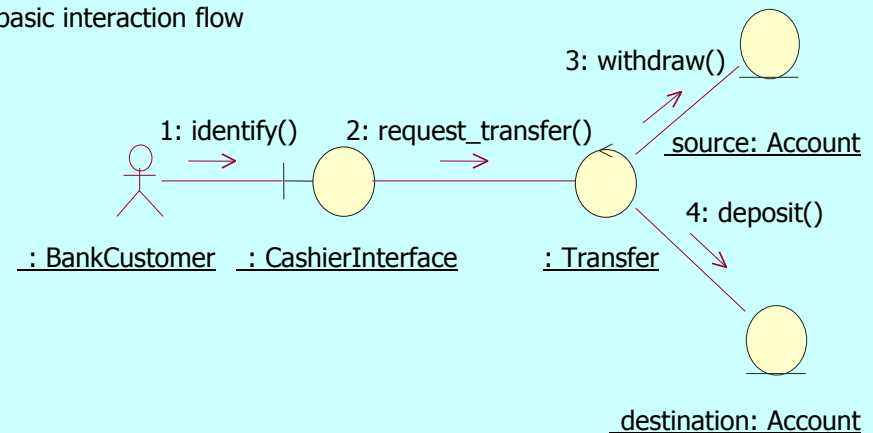
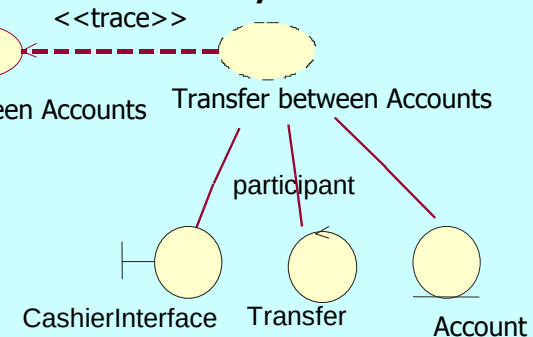


- Transfer between Accounts use case realization in the analysis model comprises
 - Analyses classes that participate in the realization of the use case (each use case is realized by a corresponding collection of analysis classes)
 - Communication (interaction) diagrams
 - E.g., for the basic interaction flow

Use Case Model



Analysis Model



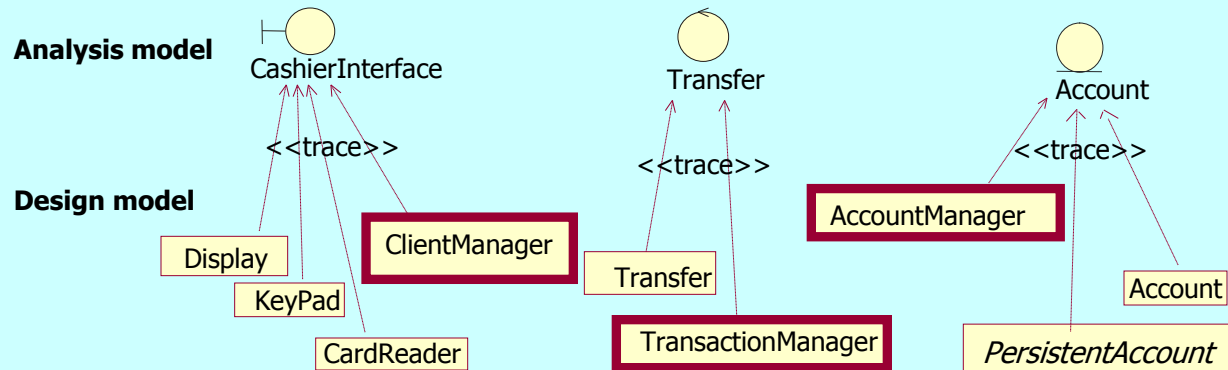
Case study: an ATM system

Design Model

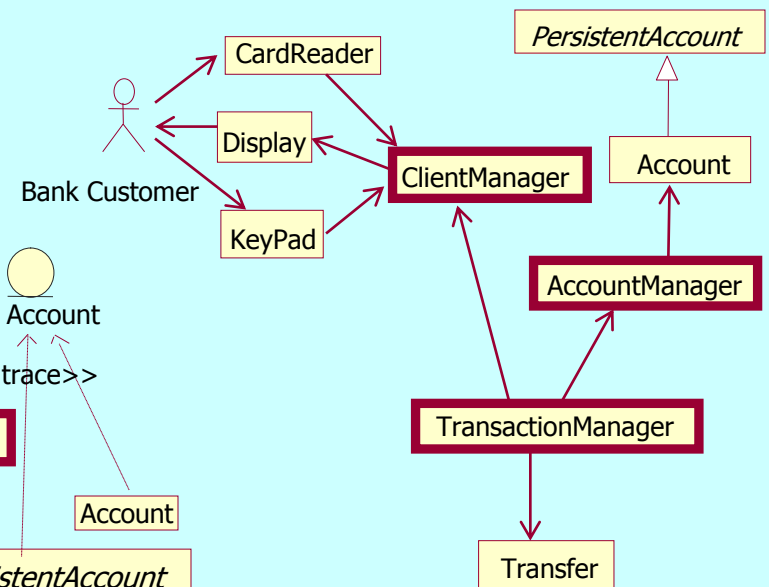
- The **Design Model** works as a blueprint for the implementation [JBR99]

- An analysis class is an abstraction of a number of design classes

- Design classes that participate in the realization of the Transfer between Accounts use case (tracing to corresponding analyses classes)



- Class diagram showing the relationships between the design classes that participate in the realization of the Transfer between Accounts use case



- ClientManager, TransactionManager, AccountManager are **active classes** [BRJ99] (designed to support a distributed implementation)
 - In general, a design class may participate in the design of several analysis classes; for example, ClientManager (TransactionManager) participates in the design of all interface (control) analysis classes
 - All the analyses classes specify and give rise to more refined design classes

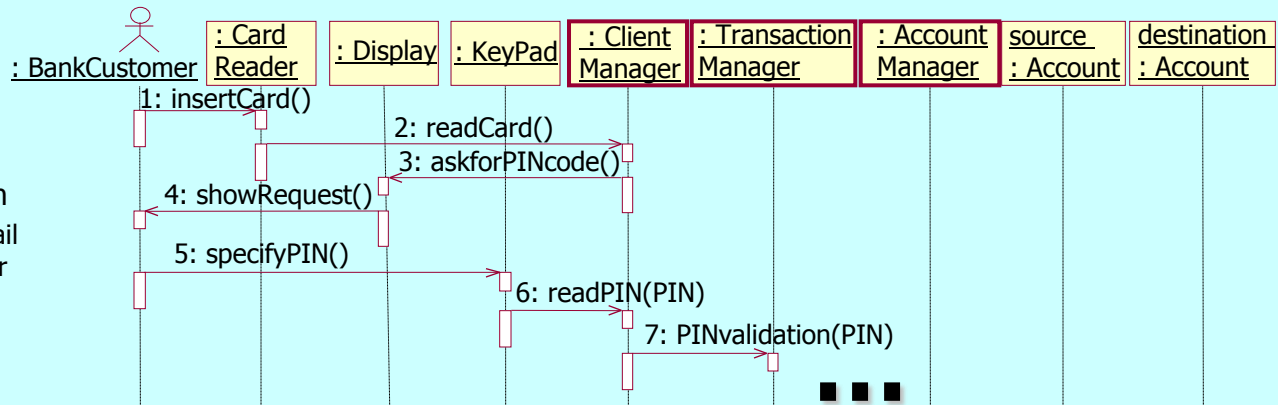
Case study: an ATM system

Design Model

Sequence Diagram

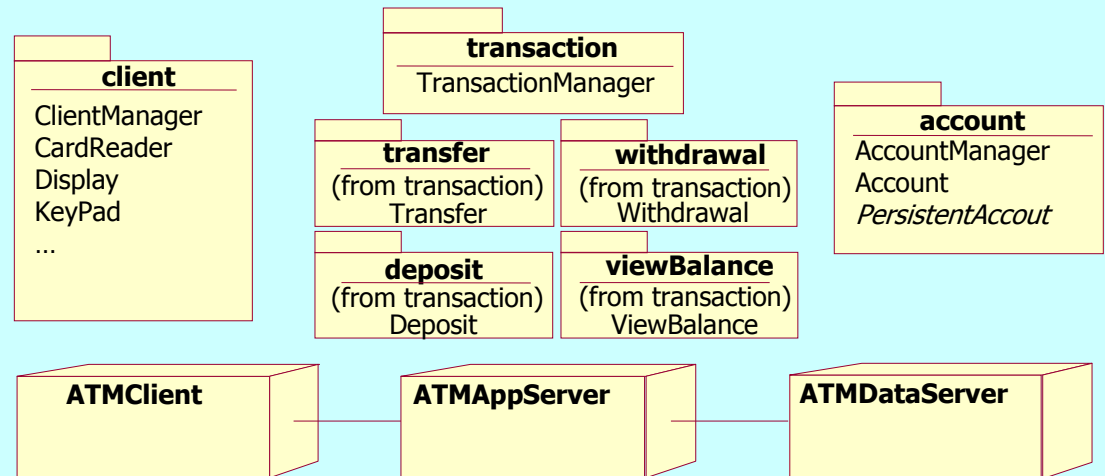
for the basic flow of the Transfer between Accounts use case realization in design

The amount of detail is significantly greater than in the (communication diagram from the) analysis model



Consider a software decomposition solution based on three main **packages** (subsystems): client, transaction, and account

The components that implement the three subsystems can be deployed on a **Three-Tier** architecture



Case study: an ATM system

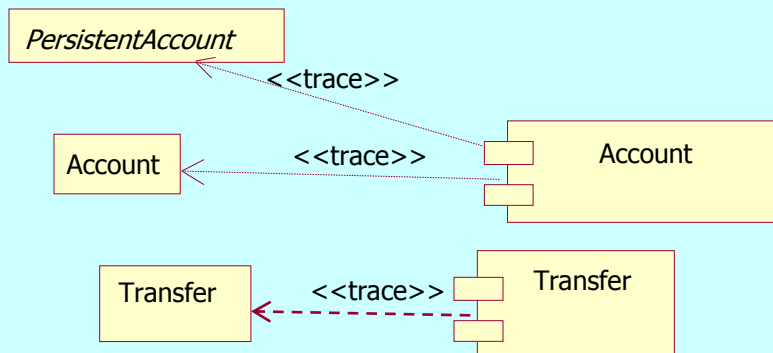
Implementation Model

- The implementation model shows **components** (that implement the design classes) and their dependencies

- (Some) Components that participate in the implementation of the Transfer between Accounts use case (tracing to design classes)

Design model

Implementation model



- You can also model compilation dependencies (to show which components are required to compile a specific component)

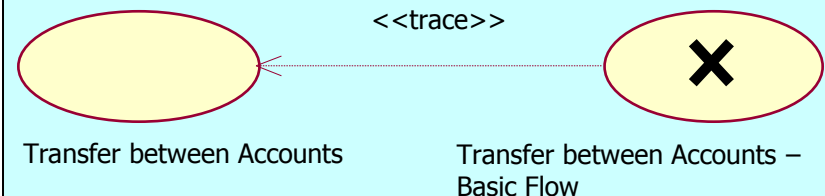
Test Model

- A **set of test cases** is derived **for each use case**

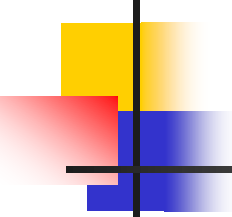
- Transfer between Accounts – Basic Flow test case (corresponds to Transfer between Accounts use case)

Use case model

Test model

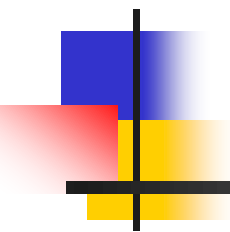


- Each test case should include the following information [JBR99,LL05]:
 - What to test (identification information)
 - With which input
 - (Expected) result
 - Under which conditions



References

- [BB99] W. Boggs, M. Boggs. *Mastering UML with Rational Rose* (1st edition), Sybex, 1999.
- [BB02] W. Boggs, M. Boggs. *Mastering UML with Rational Rose* (2st edition), Sybex, 2002.
- [BRJ99] G. Booch, J. Rumbaugh and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 1999.
- [JBR99] I. Jacobson, G. Booch and J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley, 1999.
- [Kru03] P. Kruchten. *The Rational Unified Process: An Introduction* (3rd edition). Addison-Wesley, 2003.
- [LL05] T. Lethbridge, R. Laganriere, *Object-Oriented Software Engineering: Practical Software Development using UML and Java*, 2nd edition, McGraw-Hill, 2005.
- [RS13] D. Rosenberg, M. Stephens, *Use Case Driven Object Modeling with UML: Theory and Practice*, Apress, 2013.
- [Som16] I. Sommerville. *Software Engineering*, (10th edition). Addison-Wesley, 2016.



Declarative prototyping



Declarative prototyping

- We present a simple programs development methodology based on mathematical induction, declarative prototyping, procedural design and implementation (the references for this chapter include [Boe84,Mit96,Pie20,RL99,Som15,Zav89]).
- We use the functional programming language Haskell [PJH99] for declarative prototyping, and C as a language for procedural implementation.
- For comparison purposes, we also present Scala [OSV16] solutions.



Declarative prototyping

- Haskell is a lazy purely functional programming language named after the famous logician Haskell Curry, whose contributions to lambda calculus and combinatory logic are well-known [CF58]
- Classic examples of high-level languages that can be used for prototyping purposes include: Lisp, Prolog and Smaltalk.



Declarative prototyping

- Haskell is a modern strongly typed functional programming language, appropriate for prototypes development.
- Advantages of Haskell as a prototyping tool:
 - Declarative specifications
 - Referential transparency (it provides support for equational reasoning)
 - Polymorphism and higher-order functions
 - A Haskell specification is typically much shorter than a corresponding C implementation



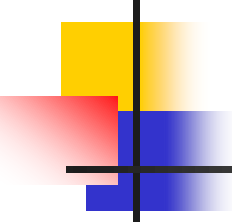
Declarative prototyping

- Haskell programs can be seen as 'executable mathematics' [RL99].
- Alternatively, we could adopt Z [Spi92] and develop formal specifications. Z specifications are more abstract, but are not executable.
- Haskell prototypes are executable and, therefore, can easily be evaluated and tested.
- It is generally accepted that prototyping reduces the number of problems with the requirements specifications [Boe84,Som15].
- The approach considered in this chapter is useful when the problems are novel or difficult.



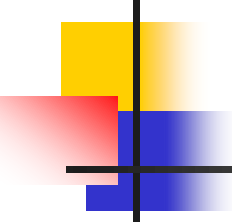
Declarative prototyping

- Scala [OSV2016] is a strongly typed multiparadigm language providing support for functional, object oriented (OO), imperative and concurrent programming
 - Data structures can be designed as classes in OO style
 - Computation can be expressed both in imperative (Java like) style and in functional style



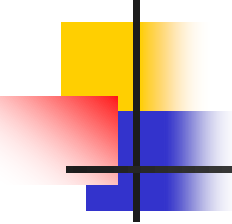
Declarative prototyping

- Scala is an OO language in pure form
 - Every value is an object
 - Every computation is implemented as a method
- With full support for functional programming, including
 - First class and higher order functions
 - Pattern matching (with case classes)
- Scala programs compile to JVM bytecodes
 - The performance of imperative style Scala programs is on par with corresponding Java programs
 - The two languages interoperate seamlessly



Declarative prototyping

- We present a **methodology** involving the following steps:
 1. *Build a Haskell declarative specification (prototype)*
The prototype is built by an inductive reasoning, which proves the correctness of the specification. For some examples we also present Scala solutions
 2. *Design a procedural solution* This step involves procedural design decisions, decisions concerning data structures representation, memory allocation policies, etc.
 3. *Accomplish the procedural implementation* We will use C for procedural implementation.



Declarative prototyping

- **Mathematical induction** is a convenient tool for recursive functions design (for the functions defined on finite structures).
- The most common forms of induction are
 - Induction on natural numbers
 - Structural induction
- They can be treated as instances of a general form of induction, called *well-founded induction* (see e.g. [Mit96]).



Declarative prototyping

- A *well-founded relation* on a set A is a binary relation $\prec$ on A with the property that there is no infinite descending sequence $a_0 \succ a_1 \succ a_2 \succ \dots$
 - A well-founded relation need not be transitive (example: $i \prec j$ if $j = i+1$, on the natural numbers).
 - A well-founded relation can not be reflexive (if $a \prec a$ then there is an infinite descending sequence $a \succ a \succ a \succ \dots$)
- An equivalent definition is that a binary relation $\prec$ on A is well-founded iff every nonempty subset B of A has a minimal element, where $a \in B$ is *minimal* if there is no $a' \in B$ with $a' \prec a$.



Declarative prototyping

- (Generalized or) Well-founded induction principle
 - Let $\prec$ be a well-founded binary relation on set A and let P be some property on A . If $P(a)$ holds whenever we have $P(b)$ for all $b \prec a$, then $P(a)$ is true for all $a \in A$.
- More familiar forms of induction can be obtained by using the following well-founded relations:
 - $m \prec n$ if $m+1=n$, for natural number induction
 - $e \prec e'$ if e is an immediate sub-expression of e' , for structural induction



Declarative prototyping

- In the sequel we will use mathematical induction to prove the correctness of recursive definitions.
- In each case, we will define a *complexity measure*: a function that maps the concrete structures in the problem domain to a set equipped with a well-founded relation.
 - The complexity measure must be chosen so that it decreases upon any recursive call.
- We will present various kinds of inductive reasoning.



Declarative prototyping

- For simplicity, we do not consider Haskell specifications based on higher-order mappings, and we only give recursive C implementations.
- Haskell is polymorphic. C is monomorphic. A Haskell prototype can specify an entire class of C implementations.
 - For simplicity, we ignore this aspect, and we only consider data structures containing primitive types (numeric values).



Declarative prototyping

- **Haskell → C transcription**

- Each Haskell function in the declarative specification is translated to a corresponding C function in the procedural implementation (using auxiliary C functions if necessary).
- Haskell functions defined by multiple equations are implemented using conditional statements in C.
- For each recursive call in the Haskell specification there is a corresponding recursive call in the C implementation.



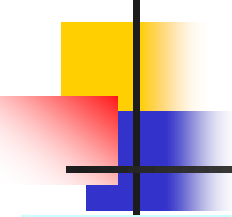
Declarative prototyping

- **Example Set union**
- Haskell specification:

```
member :: (Int,[Int]) -> Bool
member (e,[]) = False
member (e,x:xs) = if (e == x) then True else member (e,xs)

union :: ([Int],[Int]) -> [Int]
union ([],ys) = ys
union (x:xs,ys) = if member(x,ys) then union(xs,ys)
                  else x:union(xs,ys)
```

- The specification is correct. This follows by induction on a **simple complexity measure**:
 - `member (e, xs)` - by induction on `length(xs)` (/ structural induction)
 - `union (xs, ys)` – by induction on the `length(xs)`, assuming that `xs` and `ys` are lists without duplicated elements

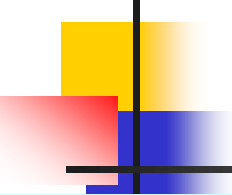


Declarative prototyping

- The Haskell prototype behaves as follows (experiments performed using the Hugs interpreter):

```
Main> union([], [1,2,3])  
[1,2,3]
```

```
Main> union([6,7,5,3], [5,6,9,1,2,7])  
[3,5,6,9,1,2,7]
```



Declarative prototyping

- Haskell also provides
 - Higher order functions (which enable partial application, or currying)
 - Polymorphic types [PJH99]
 - The following function can compute the union of two sets
 - Implemented as (homogeneous) polymorphic Haskell lists
 - Containing elements of any type which (is an instance of class Eq)

```
member :: (Eq a) => a -> [a] -> Bool
member e []      = False
member e (x:xs) = if (e == x) then True else member e xs

union :: (Eq a) => [a] -> [a] -> [a]
union []      ys = ys
union (x:xs)  ys =
    if member x ys then union xs ys else x:union xs ys
```



Declarative prototyping

- Actually, Haskell provides powerful libraries for list and set manipulation (including a library set union function)
- Similarly, Scala provides a library solution for the set union operation, comprising a Set class and a set union method

```
object Test {  
    def main(args: Array[String]) {  
        println(Set(1,2,3,4).union(Set(1,2,7,8,5)))  
        println(Set(1,2,3,4) union Set(1,2,7,8,5))  
    }  
}
```

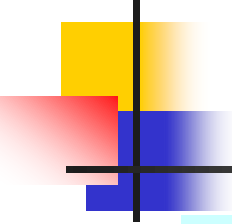
```
> scala Test  
Set(5,1,6,9,2,7,3)  
Set(5,1,6,9,2,7,3)
```




Declarative prototyping

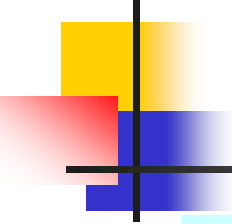
- **Scala provides support for** classes parameterized with types (generic classes) and **pattern matching** (case classes)
 - In the sequel we only consider Scala **immutable (functional) structures**

```
object Test {  
  def member1[T] (e:T,xs:List[T]) : Boolean = xs match {  
    case Nil      => false  
    case (x::xs1) => if (e==x) true else member1(e,xs1)  
  }  
  def union1[T] (xs:List[T],ys:List[T]):List[T] = xs match {  
    case Nil      => ys  
    case (x::xs1) => if (member1(x,ys)) union1(xs1,ys)  
                     else x::union1(xs1,ys)  
  }  
  def main(args: Array[String]) {  
    println(union1(List(6,7,5,3),List(5,6,9,1,2,7)))  
  }  
}  
  
> scala Test  
List(3,5,6,9,1,2,7)
```



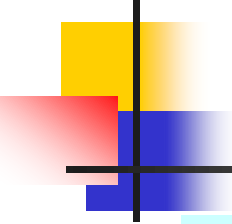
Declarative prototyping

- Both C and Scala (as well C++ or Java) provide support for improved performance based on
 - Mutable (static and dynamic) data structures
 - Imperative style programming
- In the sequel we only consider such options in the imperative C implementations



Declarative prototyping

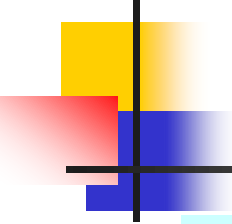
- **Designing the procedural implementation**
 - There are various options:
 - Recursive implementation
 - Implementation as WHILE program
 - Result produced
 - By the normal function return mechanism
 - By using an additional parameter transmitted by reference
 - There are also various options concerning the memory allocation policy
 - Use static structures (arrays)
 - Use dynamic structures (lists)
 - Allocate / not allocate space for the result
 - Alter / not alter the (input) parameters



Declarative prototyping

- We use the following type declaration for the C implementation

```
typedef struct elem {  
    int info;  
    struct elem* next;  
} ELEM, *LIST;
```

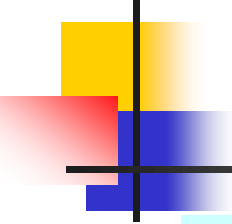


Declarative prototyping

- C implementation of `member`

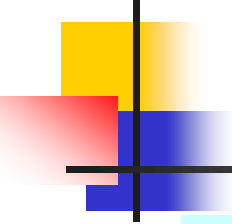
```
typedef enum {false,true} BOOL;
```

```
BOOL member(int e,LIST l)
{
    if (l == 0) return(false);
    else if (e == l->info) return(true);
    else return (member(e,l->next));
}
```



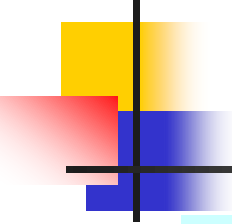
Declarative prototyping

- For `union` we consider four different implementations:
 - The first two variants
 - Alter the input parameters
 - Do not allocate space for the result.
 - The last two variants
 - Do not alter the input parameters
 - Allocate space for the result



Declarative prototyping

```
LIST union(LIST x, LIST y)
{ LIST z;
  if (x == 0) return(y);
  else if (member(x->info, y)) {
    z = union(x->next, y);
    free(x);
    return(z);
  } else {
    z = x;
    z -> next = union(x->next, y);
    return(z);
  }
}
```



Declarative prototyping

- The function can be used as follows:

```
LIST x,y,x;
```

```
...
```

```
/* Create the 'sets' x and y */
```

```
...
```

```
z = union(x,y);
```

```
/* The 'set' z is the union of x and y */
```




Declarative prototyping

- Alternatively, we can implement `union` as a C function of type `void`; the function returns its result by using an additional parameter transmitted by reference.

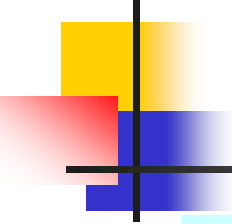
```
void union(LIST x, LIST y, LIST *z)
```

- In the sequel, we find convenient to use the term *procedure* to refer to such a C function of type `void`.



Declarative prototyping

```
void union(LIST x, LIST y, LIST *z)
{
    if (x == 0)    (*z) = y;
    else if (member(x->info, y)) {
        union(x->next, y, z);
        free(x);
    } else {
        (*z) = x;
        union(x->next, y, &((*z)->next));
    }
}
```



Declarative prototyping

- The procedure can be used as follows:

```
LIST x,y,x;
```

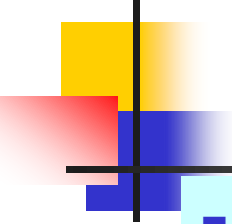
```
...
```

```
/* Create the 'sets' x and y */
```

```
...
```

```
union(x,y,&z);
```

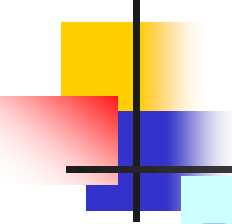
```
/* The 'set' z is the union of x and y */
```



Declarative prototyping

- The C function given below allocates space for the result and does not alter the input parameters.

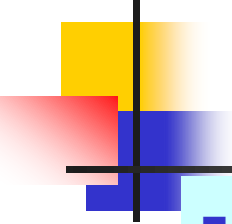
```
LIST union(LIST x, LIST y)
{ LIST z;
  if (x == 0) return(copy(y));
  else if (member(x->info, y)) {
    return (union(x->next, y));
  } else {
    z = (LIST)malloc(sizeof(ELEM));
    z->info = x->info;
    z->next = union(x->next, y);
    return(z);
  }
}
```



Declarative prototyping

- The implementation uses an auxiliary function that makes a physical copy of its parameter.

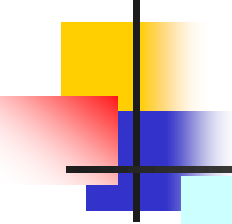
```
LIST copy (LIST l)
{ LIST r;
  if (l == 0) return(0);
  else {
    r = (LIST)malloc(sizeof(ELEM));
    r->info = l->info;
    r->next = copy(l->next);
    return(r);
  }
}
```



Declarative prototyping

- The last implementation solution uses an additional parameter transmitted by reference. It allocates space for the result and does not alter the input parameters.

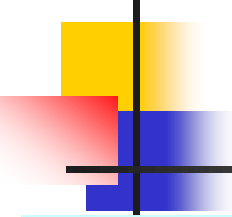
```
void union(LIST x, LIST y, LIST *z)
{
    if (x == 0)    copy(y, z);
    else if (member(x->info, y)) {
        union(x->next, y, z);
    } else {
        (*z) = (LIST)malloc(sizeof(ELEM));
        (*z)->info = x->info;
        union(x->next, y, &((*z)->next));
    }
}
```



Declarative prototyping

- In this case we use the following auxiliary procedure to make a physical copy of a list.

```
void copy(LIST l, LIST *r)
{
    if (l == 0)    (*r)=0;
    else {
        (*r) = (LIST)malloc(sizeof(ELEM));
        (*r)->info = l->info;
        copy(l->next, &((*r)->next));
    }
}
```



Declarative prototyping

- **Example Merging**
- Haskell specification:

```
merge :: ([Int],[Int]) -> [Int]
merge([],ys) = ys
merge(xs,[]) = xs
merge(x:xs,y:ys) = if (x<y) then x:merge(xs,y:ys)
                  else y:merge(x:xs,ys)
```

- The correctness proof for `merge(xs,ys)` can proceed by induction on the following **computed complexity measure**: `(length(xs) + length(ys))`. The sequences `xs` and `ys` are assumed to be ordered.



Declarative prototyping

- The Haskell prototype behaves as follows:

```
Main> merge ([1,3,5,7], [2,4,6])  
[1,2,3,4,5,6,7]
```



Declarative prototyping

- In Scala you can implement a concrete merge function as follows:

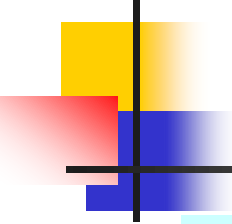
```
object Test {  
  def main(args: Array[String]) {  
    println(merge(List(1,3,5,7),List(2,3,4,6,8)))  
  }  
  def merge (xs:List[Int],ys:List[Int]):List[Int] =  
    (xs,ys) match {  
      case (Nil,ys)           => ys  
      case (xs,Nil)          => xs  
      case (x::xs1,y::ys1) => if (x<y) (x::merge(xs1,y::ys1))  
                               else (y::merge(x::xs1,ys1))  
    }  
}  
  
> scala Test  
List(1,2,3,3,4,5,6,7,8)
```



Declarative prototyping

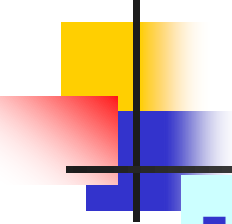
- A **more abstract (generic and) curried version** of the list merging algorithm can be implemented in **Scala** as follows:

```
object Test {  
  def main(args: Array[String]) {  
    println(mergeho(List(1,3,5,7),List(2,3,4,6,8))((x:Int,y:Int) => x<y))  
  }  
  def mergeho[T] (xs:List[T],ys:List[T])(less : (T,T) => Boolean):List[T] =  
    (xs,ys) match {  
      case (Nil,ys)      => ys  
      case (xs,Nil)      => xs  
      case (x::xs1,y::ys1) =>  
        if (less(x,y)) (x::mergeho(xs1,y::ys1)(less))  
        else (y::mergeho(x::xs1,ys1)(less))  
    }  
}  
  
> scala Test  
List(1,2,3,3,4,5,6,7,8)
```



Declarative prototyping

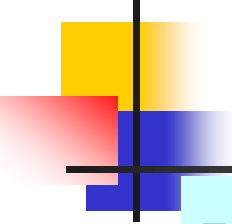
- For `merge` we only design two C implementation solutions (as function / procedure).
- In the both cases the input parameters are altered and no memory is allocated for the result.



Declarative prototyping

- C implementation as a function

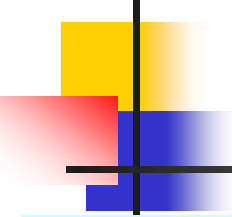
```
LIST merge(LIST x, LIST y)
{ LIST z;
  if (x == 0) return(y);
  else if (y == 0) return(x);
  else if ((x->info) < (y->info)) {
    z=x;
    z->next = merge(x->next, y);
    return(z);
  } else {
    z = y;
    z->next = merge(x, y->next);
    return(z);
  }
}
```



Declarative prototyping

- C implementation as a procedure

```
void merge(LIST x, LIST y, LIST *z)
{
    if (x == 0)    (*z)=y;
    else if (y == 0) (*z)=x;
    else if ((x->info) < (y->info)) {
        (*z)=x;
        merge(x->next, y, &((*z)->next));
    } else {
        (*z)=y;
        merge(x, y->next, &((*z)->next));
    }
}
```



Declarative prototyping

- **Example Tree flattening using difference lists**
- Haskell specification:

```
data Tree = Nil | T(Tree, Int, Tree)
flat(Nil, ys) = ys
flat(T(l, n, r), ys) = flat(l, n:flat(r, ys))
```

- Difference lists notation: if $xs = e1 : \dots : en : ys$ then
 $xs - ys = [e1, \dots, en]$
- The correctness proof for $flat(t, ys)$ can proceed by induction on the structure of t (by **structural induction**).
 $flat(t, ys) - ys =$ the list of nodes in t (obtained by a left-node-right inorder traversal)



Declarative prototyping

- The Haskell prototype behaves as follows:

```
Main> flat(T(T(Nil,2,T(Nil,4,Nil)),1,T(Nil,3,Nil)), [100,100])  
[2,4,1,3,100,100]
```




Declarative prototyping

- In the Scala implementation you can use **case classes** and **pattern matching**

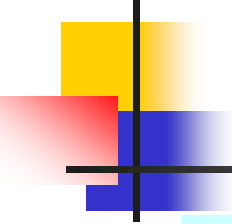
```
object Test {  
  def main(args: Array[String]) {  
    println(flat(T(T(Nil,2,T(Nil,4,Nil)),1,T(Nil,3,Nil)),List(100,100)))  
  }  
  def flat (t:Tree,ys:List[Int]):List[Int] = t match {  
    case Nil => ys  
    case T(l,n,r)    => flat(l,n::flat(r,ys))  
  }  
}  
  
abstract class Tree  
case object Nil extends Tree  
case class T (l:Tree,n:Int,r:Tree) extends Tree  
  
> scala Test  
List(2,4,1,3,100,100)
```



Declarative prototyping

- The flat method can also be allocated to the class Tree

```
object Test {  
  def main(args: Array[String]) {  
    println((T(T(Nil,2,T(Nil,4,Nil)),1,T(Nil,3,Nil))).flat(List(100,100)))  
    println((T(T(Nil,2,T(Nil,4,Nil)),1,T(Nil,3,Nil))) flat List(100,100))  
  }  
}  
  
class Tree {  
  def flat (ys:List[Int]):List[Int] =  
    this match {  
      case Nil => ys  
      case T(l,n,r)  => l.flat(n::r.flat(ys))  
    }  
}  
  
case object Nil extends Tree  
case class T (l:Tree,n:Int,r:Tree) extends Tree  
  
> scala Test  
List(2,4,1,3,100,100)  
List(2,4,1,3,100,100)
```

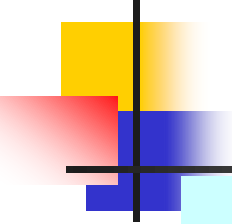


Declarative prototyping

- Apart from the type declaration for lists, in the C implementation we use the following type declaration for trees

```
typedef struct node {  
    int info;  
    struct node *left, *right;  
} NODE, *TREE;
```

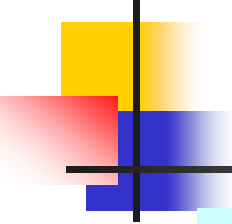
- We offer two implementation solutions.



Declarative prototyping

- C implementation as a function

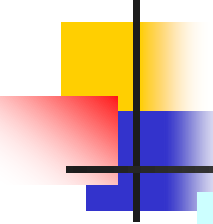
```
LIST flat(TREE t, LIST y)
{ LIST x;
  if (t == 0) return(y);
  else {
    x = (LIST)malloc(sizeof(ELEM));
    x->info = t->info;
    x->next = flat(t->right, y);
    return(flat(t->left, x));
  }
}
```



Declarative prototyping

- C implementation as a procedure

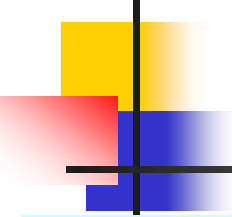
```
void flat(TREE t, LIST *x, LIST y)
{ LIST z;
  if (t == 0) (*x)=y;
  else {
    z = (LIST)malloc(sizeof(ELEM));
    z->info = t->info;
    flat(t->right, &(z->next), y);
    flat(t->left, x, z);
  }
}
```



Declarative prototyping

Rewriting techniques

- Many computations can be described using rewriting techniques.
- Sometimes, a data structure must be prepared before performing some calculations or some transformations on it.
- We want to transform a binary tree in a list.
 - We use a rewriting operation to reduce the complexity of the left sub-tree until it becomes `Nil`.
 - Next, the transformation is applied recursively on the right sub-tree.



Declarative prototyping

- **Example Tree flattening using a rewriting transformation**
- Haskell specification:

```
data Tree = Nil | T(Tree,Int,Tree)    deriving Show

transf :: Tree -> Tree
transf Nil = Nil
transf (T(Nil,n,r)) = T(Nil,n,transf(r))
transf (T(T(ll,nl,rl),n,r)) =
    transf(T(ll,nl,T(rl,n,r)))
```



Declarative prototyping

- The Haskell prototype behaves as follows:

```
Main> transf (T (T (T (Nil, 3, Nil), 2, Nil), 1, Nil))  
T (Nil, 3, T (Nil, 2, T (Nil, 1, Nil)))
```

- The result is a degenerate tree, rather than a list
 - (It is straightforward to design a version of this function with signature: `transf :: Tree -> [Int]`)



Declarative prototyping

- To prove the correctness of `transf` we use a more complex measure.
- The support set is $\mathbf{N} \times \mathbf{N}$, and we use the so-called lexicographic ordering (that we denote here by $\blacktriangleleft$) over $\mathbf{N} \times \mathbf{N}$. The lexicographic ordering is defined as follows:
$$(n_1, m_1) \blacktriangleleft (n_2, m_2) \text{ if } (n_1 < n_2) \text{ or } (n_1 = n_2 \text{ and } m_1 < m_2)$$
- It is easy to check that $\blacktriangleleft$ is a well founded relation over $\mathbf{N} \times \mathbf{N}$. Also, for each $(n, m) \in \mathbf{N} \times \mathbf{N}$ either $(n=0, m=0)$ or $(0,0) \blacktriangleleft (n,m)$.



Declarative prototyping

- The correctness of `transf` can be proved by induction on the following **composed complexity measure**:

$$\begin{aligned} c: \text{Tree} &\rightarrow \mathbf{N} \times \mathbf{N} & u, v: \text{Tree} &\rightarrow \mathbf{N} \\ c(t) &= (u(t), v(t)) & \text{for any } t &:: \text{Tree} \end{aligned}$$

- Here, $u(t)$ is the number of nodes in t and $v(t)$ is a measure of the complexity of the left sub-tree:

$$u(\text{Nil}) = 0$$

$$u(T(l, n, r)) = 1 + u(l) + u(r)$$

$$v(\text{Nil}) = 0$$

$$v(T(l, n, r)) = 1 + v(l)$$

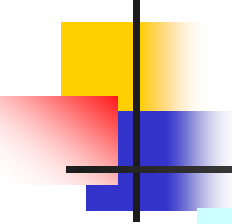
- Remark that $c(t) = (0, 0)$ iff $t = \text{Nil}$.
- We present two different implementation solutions.



Declarative prototyping

■ A Scala implementation

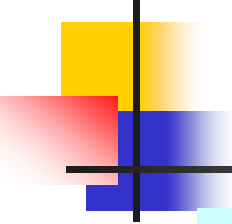
```
object Test {  
  def main(args: Array[String]) {  
    println(transf(T(T(T(Nil,3,Nil),2,Nil),1,Nil)))  
  }  
  def transf (t:Tree):Tree = t match {  
    case Nil                => Nil  
    case T(Nil,n,r)         => T(Nil,n,transf(r))  
    case T(T(l1,nl,r1),n,r) => transf (T(l1,nl,T(r1,n,r)))  
  }  
}  
abstract class Tree  
case object Nil extends Tree  
case class T (l:Tree,n:Int,r:Tree) extends Tree  
  
> scala Test  
T(Nil,3,T(Nil,2,T(Nil,1,Nil)))
```



Declarative prototyping

- C implementation as a function

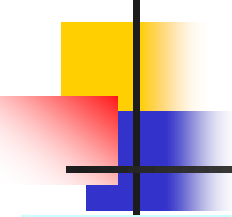
```
TREE transf(TREE t)
{
    TREE p;
    if (t == 0)    return(0);
    else if (t->left == 0){
        t->right = transf(t->right);
        return(t);
    } else {
        p = t;
        t = p->left;  p->left = t->right;
        t->right = p;
        return(transf(t));
    }
}
```



Declarative prototyping

- C procedure with **inout** parameter

```
void transf(TREE *t)
{
    TREE p;
    if ((*t) != 0) {
        if (((*t)->left) == 0)
            transf(&((*t)->right));
        else {
            p = (*t); (*t) = p->left;
            p->left = (*t)->right;
            (*t)->right = p;
            transf(t);
        }
    }
}
```



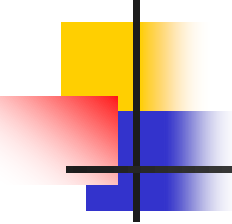
Declarative prototyping

- **Example Mutual recursion and simultaneous induction**
- Haskell specification:

```
data Btree = NilB | B(Int,Ttree,Ttree)
data Ttree = NilT | T(Int,Btree,Btree,Btree)

flatB :: (Btree,[Int]) -> [Int]
flatB (NilB,ys) = ys
flatB (B(n,t1,tr),ys) = n:flatT(t1,flatT(tr,ys))

flatT :: (Ttree,[Int]) -> [Int]
flatT (NilT,ys) = ys
flatT (T(n,b1,bm,br),ys) = n:flatB(b1,flatB(bm,flatB(br,ys)))
```



Declarative prototyping

- Let

```
t :: Ttree; b :: Btree
t = T(2,NilB,B(3,NilT,T(4,NilB,NilB,NilB)),NilB)
b = B(1,t,T(5,B(6,NilT,NilT),NilB,B(7,NilT,NilT)))
```

- The Haskell prototype behaves as follows:

```
Main> flatT (t,[0,0,0,0])
[2,3,4,0,0,0,0]
Main> flatB (b,[])
[1,2,3,4,5,6,7]
```



Declarative prototyping

■ Claim

- $\text{flatB}(b, ys) - ys$ = the list of nodes in b (obtained by a node-left-right traversal)
- $\text{flatT}(t, ys) - ys$ = the list of nodes in t (obtained by a node-left-mid-right traversal)

■ **Proof** By **simultaneous induction** on the number of nodes in the tree structure (the first parameter of each function):

- **Base case** For trees with zero nodes the specification is: $\text{flatB}(\text{NilB}, ys) = ys$, $\text{flatT}(\text{NilT}, ys) = ys$; this is correct since $ys - ys = []$. The both functions behave correctly for trees with zero nodes.
- **Induction step** For the induction step each function uses the induction hypothesis of the other function.

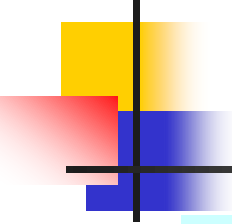


Declarative prototyping

■ Scala implementation with case classes

```
object Test {  
  def main(args: Array[String]) {  
    val t=T(2,NilB,B(3,NilT,T(4,NilB,NilB,NilB)),NilB)  
    val b=B(1,t,T(5,B(6,NilT,NilT),NilB,B(7,NilT,NilT)))  
    println(flatT(t,List(0,0,0,0)))  
    println(flatB(b,List()))  
  }  
  def flatB (b:Btree,ys>List[Int]):List[Int] = b match {  
    case NilB          => ys  
    case B(n,tl,tr)    => n::flatT(tl,flatT(tr,ys))  
  }  
  def flatT (t:Ttree,ys>List[Int]):List[Int] = t match {  
    case NilT          => ys  
    case T(n,bl,bm,br) => n::flatB(bl,flatB(bm,flatB(br,ys)))  
  }  
}  
  
abstract class Btree  
case object NilB extends Btree  
case class B (n:Int,l:Ttree,r:Ttree) extends Btree  
  
abstract class Ttree  
case object NilT extends Ttree  
case class T (n:Int,l:Btree,m:Btree,r:Btree) extends Ttree
```

```
> scala Test  
List(2,3,4,0,0,0,0)  
List(1,2,3,4,5,6,7)
```



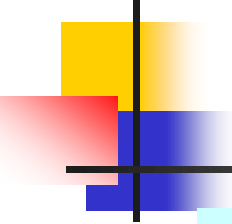
Declarative prototyping

- For the C implementation we use the following type declarations:

```
typedef struct Bnode {  
    int info;  
    struct Tnode *l, *r;  
} BNODE, *BTREE;
```

```
typedef struct Tnode {  
    int info;  
    struct Bnode *l, *m, *r;  
} TNODE, *TTREE;
```

- We give implementations as functions and procedures.

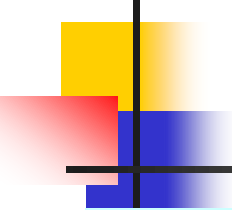


Declarative prototyping

- A pair of functions

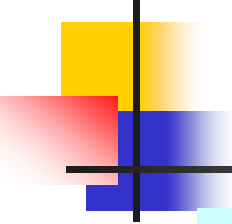
```
LIST flatT(TTREE, LIST);
```

```
LIST flatB(BTREE b, LIST y)
{ LIST x;
  if (b == 0) return(y);
  else {
    x = (LIST)malloc(sizeof(ELEM));
    x->info = b->info;
    x->next = flatT(b->l, flatT(b->r, y));
    return(x);
  }
}
```



Declarative prototyping

```
LIST flatT(TTREE t,LIST y)
{ LIST x;
  if (t == 0) return(y);
  else {
    x = (LIST)malloc(sizeof(ELEM));
    x->info = t->info;
    x->next = flatB(t->l,flatB(t->m,flatB(t->r,y)));
    return(x);
  }
}
```

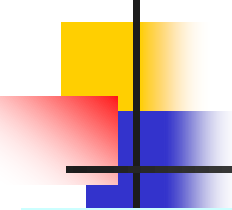


Declarative prototyping

- A pair of procedures

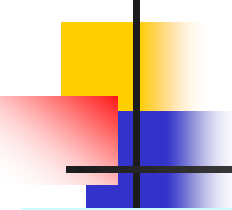
```
void flatT(TTREE, LIST *, LIST);
```

```
void flatB(BTREE b, LIST *x, LIST y)
{ LIST z;
  if (b == 0) (*x)=y;
  else {
    (*x) = (LIST)malloc(sizeof(ELEM));
    (*x)->info = b->info;
    flatT(b->r, &z, y);
    flatT(b->l, &((*x)->next), z);
  }
}
```



Declarative prototyping

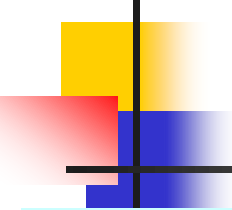
```
void flatT(TTREE t,LIST *x,LIST y)
{ LIST z,w;
  if (t == 0) (*x) = y;
  else {
    (*x) = (LIST)malloc(sizeof(ELEM));
    (*x)->info = t->info;
    flatB(t->r,&z,y);
    flatB(t->m,&w,z);
    flatB(t->l,&((*x)->next),w);
  }
}
```



Declarative prototyping

- **Remark** In C you may need to employ unions in order to implement Haskell user-defined types with multiple variants.
- **Example** Haskell type:

```
data Lisp = Nil | Atom Int | Cons (Lisp,Lisp)
```



Declarative prototyping

- The above Haskell definition can be implemented in C as follows:

```
typedef enum {atom,cons} SEL;
typedef struct Lisp {
    SEL sel;    // selector field
    union {
        int atom;
        struct {
            struct Lisp *car;
            struct Lisp *cdr;
        } cons;
    } lisp;
} CEL, *LISP;
```




Declarative prototyping

- In the Scala implementation you can use the Composite design pattern

```
abstract class Lisp
case object Nil extends Lisp
case class Atom (n:Int) extends Lisp
case class Cons (car:Lisp,cdr:Lisp) extends Lisp
```

- You can make the following experiment:

```
object Test {
  def main(args: Array[String]) {
    println(Cons(Cons(Atom(1),Cons(Atom(2),Nil)),Atom(10)))
  }
}

> scala Test
Cons(Cons(Atom(1),Cons(Atom(2),Nil)),Atom(10))
```



References

- [Boe84] B. Boehm, et al, Prototyping versus specifying: a multi-project experiment, *IEEE Transactions on Software Engineering*, vol. 10(3), pp. 290-303, 1984.
- [CF58] H. Curry, R. Feys. *Combinatory logic*, North Holland, 1958.
- [Mit96] J.C. Mitchell, *Foundations for programming languages*, MIT Press, 1996.



References

- [Mur96] T. Muresan, Software Engineering – Lecture Notes, Technical University of Cluj-Napoca, 1996.
- [OSV16] M. Odersky, L. Spoon, B. Venners, *Programming in Scala (Third Edition)*, Artima, 2016.
- [PJH99] S. Peyton-Jones, R.J.M. Hughes (eds), Report on the programming language Haskell 98; available at <http://www.haskell.org>, 1999.



References

- [Pie20] B. Pierce, *Software Foundations*, vol. 1 (*Logical Foundations*), 2020; available at <https://softwarefoundations.cis.upenn.edu/>
- [Som15] I. Sommerville, *Software Engineering*, (10th edition), Addison-Wesley, 2015.
- [Zav89] P. Zave, A compositional approach to multiparadigm programming, *IEEE Software*, vol. 6(5), pp. 15-27, 1989.